

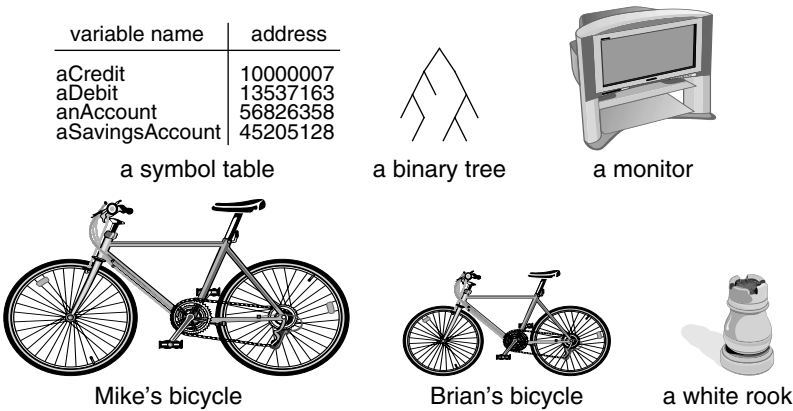


Figure 1.1 Objects. Objects lie at the heart of object-oriented technology.

Classification means that objects with the same data structure (**attributes**) and behavior (**operations**) are grouped into a class. *Paragraph*, *Monitor*, and *ChessPiece* are examples of classes. A **class** is an abstraction that describes properties important to an application and ignores the rest. Any choice of classes is arbitrary and depends on the application.

Each class describes a possibly infinite set of individual objects. Each object is said to be an **instance** of its class. An object has its own value for each attribute but shares the attribute names and operations with other instances of the class. Figure 1.2 shows two classes and some of their respective instances. An object contains an implicit reference to its own class; it “knows what kind of thing it is.”

Inheritance is the sharing of attributes and operations (**features**) among classes based on a hierarchical relationship. A **superclass** has general information that **subclasses** refine and elaborate. Each subclass incorporates, or inherits, all the features of its superclass and adds its own unique features. Subclasses need not repeat the features of the superclass. For example, *ScrollingWindow* and *FixedWindow* are subclasses of *Window*. Both subclasses inherit the features of *Window*, such as a visible region on the screen. *ScrollingWindow* adds a scroll bar and an offset. The ability to factor out common features of several classes into a superclass can greatly reduce repetition within designs and programs and is one of the main advantages of OO technology.

Polymorphism means that the same operation may behave differently for different classes. The *move* operation, for example, behaves differently for a pawn than for the queen in a chess game. An **operation** is a procedure or transformation that an object performs or is subject to. *RightJustify*, *display*, and *move* are examples of operations. An implementation of an operation by a specific class is called a **method**. Because an OO operator is polymorphic, it may have more than one method implementing it, each for a different class of object.

In the real world, an operation is simply an abstraction of analogous behavior across different kinds of objects. Each object “knows how” to perform its own operations. In an OO programming language, however, the language automatically selects the correct method to

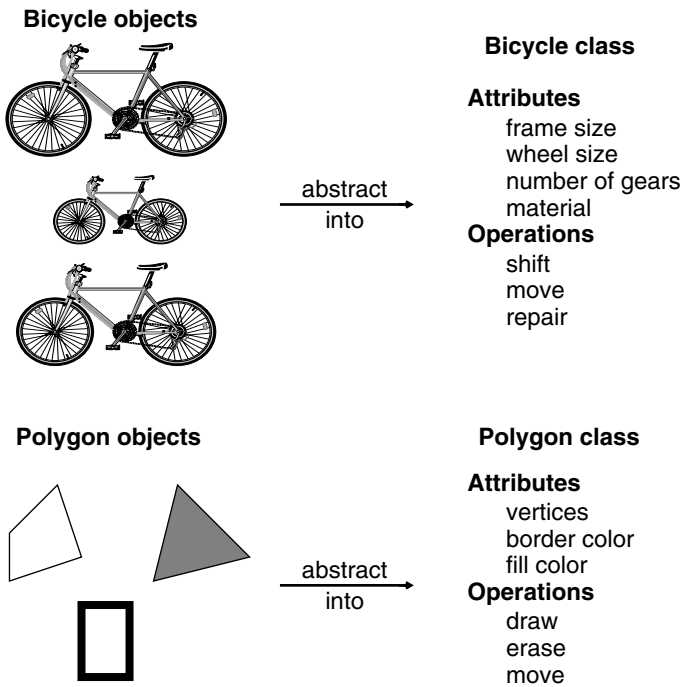


Figure 1.2 Objects and classes. Each class describes a possibly infinite set of individual objects.

implement an operation based on the name of the operation and the class of the object being operated on. The user of an operation need not be aware of how many methods exist to implement a given polymorphic operation. Developers can add new classes without changing existing code, as long as they provide methods for each applicable operation.

1.2 What Is OO Development?

This book is about OO development as a way of thinking about software based on abstractions that exist in the real world as well as in the program. In this context *development* refers to the software life cycle: analysis, design, and implementation. The essence of OO development is the identification and organization of application concepts, rather than their final representation in a programming language. Brooks observes that the hard part of software development is the manipulation of its *essence*, owing to the inherent complexity of the problem, rather than the *accidents* of its mapping into a particular language [Brooks-95].

This book does not explicitly address integration, maintenance, and enhancement, but a clean design in a precise notation facilitates the entire software life cycle. The OO concepts and notation used to express a design also provide useful documentation.

1.2.1 Modeling Concepts, Not Implementation

In the past, much of the OO community focused on programming languages, with the literature emphasizing implementation rather than analysis and design. OO programming languages were first useful in alleviating the inflexibility of traditional programming languages. In a sense, however, this emphasis was a step backward for software engineering—it focuses excessively on implementation mechanisms, rather than the underlying thought process that they support.

The real payoff comes from addressing front-end conceptual issues, rather than back-end implementation details. Design flaws that surface during implementation are more costly to fix than those that are found earlier. A premature focus on implementation restricts design choices and often leads to an inferior product. An OO development approach encourages software developers to work and think in terms of the application throughout the software life cycle. It is only when the inherent concepts of the application are identified, organized, and understood that the details of data structures and functions can be addressed effectively.

OO development is a conceptual process independent of a programming language until the final stages. OO development is fundamentally a way of thinking and not a programming technique. Its greatest benefits come from helping specifiers, developers, and customers express abstract concepts clearly and communicate them to each other. It can serve as a medium for specification, analysis, documentation, and interfacing, as well as for programming.

1.2.2 OO Methodology

We present a process for OO development and a graphical notation for representing OO concepts. The process consists of building a model of an application and then adding details to it during design. The same seamless notation is used from analysis to design to implementation, so that information added in one stage of development need not be lost or translated for the next stage. The methodology has the following stages.

- **System conception.** Software development begins with business analysts or users conceiving an application and formulating tentative requirements.
- **Analysis.** The analyst scrutinizes and rigorously restates the requirements from system conception by constructing models. The analyst must work with the requestor to understand the problem, because problem statements are rarely complete or correct. The analysis model is a concise, precise abstraction of *what* the desired system must do, not *how* it will be done. The analysis model should not contain implementation decisions. For example, a *Window* class in a workstation windowing system would be described in terms of its visible attributes and operations.

The analysis model has two parts: the **domain model**, a description of the real-world objects reflected within the system; and the **application model**, a description of the parts of the application system itself that are visible to the user. For example, domain objects for a stockbroker application might include stock, bond, trade, and commission. Application objects might control the execution of trades and present the results. Application experts who are not programmers can understand and criticize a good model.

- **System design.** The development team devise a high-level strategy—the *system architecture*—for solving the application problem. They also establish policies that will serve as a default for the subsequent, more detailed portions of design. The system designer must decide what performance characteristics to optimize, choose a strategy of attacking the problem, and make tentative resource allocations. For example, the system designer might decide that changes to the workstation screen must be fast and smooth, even when windows are moved or erased, and choose an appropriate communications protocol and memory buffering strategy.
- **Class design.** The class designer adds details to the analysis model in accordance with the system design strategy. The class designer elaborates both domain and application objects using the same OO concepts and notation, although they exist on different conceptual planes. The focus of class design is the data structures and algorithms needed to implement each class. For example, the class designer now determines data structures and algorithms for each of the operations of the *Window* class.
- **Implementation.** Implementers translate the classes and relationships developed during class design into a particular programming language, database, or hardware. Programming should be straightforward, because all of the hard decisions should have already been made. During implementation, it is important to follow good software engineering practice so that traceability to the design is apparent and so that the system remains flexible and extensible. For example, implementers would code the *Window* class in a programming language, using calls to the underlying graphics system on the workstation.

OO concepts apply throughout the system development life cycle, from analysis through design to implementation. You can carry the same classes from stage to stage without a change of notation, although they gain additional details in the later stages. The analysis and implementation models of *Window* are both correct, but they serve different purposes and represent a different level of abstraction. The same OO concepts of identity, classification, polymorphism, and inheritance apply throughout development.

Note that we are not suggesting a waterfall development process—first capturing requirements, then analyzing, then designing, and finally implementing. For any particular part of a system, developers must perform each stage in order, but they need not develop each part of the system in tandem. We advocate an iterative process—developing part of the system through several stages and then adding capability.

Some classes are not part of analysis but are introduced during design or implementation. For example, data structures such as *trees*, *hash tables*, and *linked lists* are rarely present in the real world and are not visible to users. Designers introduce them to support particular algorithms. Such data structure objects exist within a computer and are not directly observable.

We do not consider testing as a distinct step. Testing is important, but it must be part of an overall philosophy of quality control that occurs throughout the life cycle. Developers must check analysis models against reality. They must verify design models against various kinds of errors, in addition to testing implementations for correctness. Confining quality control to a separate step is more expensive and less effective.

1.2.3 Three Models

We use three kinds of models to describe a system from different viewpoints: the class model for the objects in the system and their relationships; the state model for the life history of objects; and the interaction model for the interactions among objects. Each model applies during all stages of development and acquires detail as development progresses. A complete description of a system requires models from all three viewpoints.

The **class model** describes the static structure of the objects in a system and their relationships. The class model defines the context for software development—the universe of discourse. The class model contains class diagrams. A **class diagram** is a graph whose nodes are classes and whose arcs are relationships among classes.

The **state model** describes the aspects of an object that change over time. The state model specifies and implements control with state diagrams. A **state diagram** is a graph whose nodes are states and whose arcs are transitions between states caused by events.

The **interaction model** describes how the objects in a system cooperate to achieve broader results. The interaction model starts with use cases that are then elaborated with sequence and activity diagrams. A **use case** focuses on the functionality of a system—that is, what a system does for users. A **sequence diagram** shows the objects that interact and the time sequence of their interactions. An **activity diagram** elaborates important processing steps.

The three models are separate parts of the description of a complete system but are cross-linked. The class model is most fundamental, because it is necessary to describe *what* is changing or transforming before describing *when* or *how* it changes.

1.3 OO Themes

Several themes pervade OO technology. Although these themes are not unique to OO systems, they are particularly well supported.

1.3.1 Abstraction

Abstraction lets you focus on essential aspects of an application while ignoring details. This means focusing on what an object is and does, before deciding how to implement it. Use of abstraction preserves the freedom to make decisions as long as possible by avoiding premature commitments to details. Most modern languages provide data abstraction, but inheritance and polymorphism add power. The ability to abstract is probably the most important skill required for OO development.

1.3.2 Encapsulation

Encapsulation (also **information hiding**) separates the external aspects of an object, that are accessible to other objects, from the internal implementation details, that are hidden from other objects. Encapsulation prevents portions of a program from becoming so interdependent that a small change has massive ripple effects. You can change an object's implementa-

tion without affecting the applications that use it. You may want to change the implementation of an object to improve performance, fix a bug, consolidate code, or support porting. Encapsulation is not unique to OO languages, but the ability to combine data structure and behavior in a single entity makes encapsulation cleaner and more powerful than in prior languages, such as Fortran, Cobol, and C.

1.3.3 Combining Data and Behavior

The caller of an operation need not consider how many implementations exist. Operator polymorphism shifts the burden of deciding what implementation to use from the calling code to the class hierarchy. For example, non-OO code to display the contents of a window must distinguish the type of each figure, such as polygon, circle, or text, and call the appropriate procedure to display it. An OO program would simply invoke the *draw* operation on each figure; each object implicitly decides which procedure to use, based on its class. Maintenance is easier, because the calling code need not be modified when a new class is added. In an OO system, the data structure hierarchy matches the operation inheritance hierarchy (Figure 1.3).

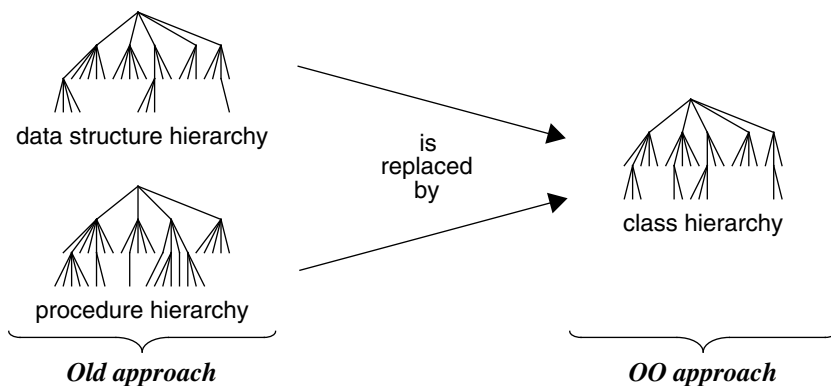


Figure 1.3 OO vs. prior approach. An OO approach has one unified hierarchy for both data and behavior.

1.3.4 Sharing

OO techniques promote sharing at different levels. Inheritance of both data structure and behavior lets subclasses share common code. This sharing via inheritance is one of the main advantages of OO languages. More important than the savings in code is the conceptual clarity from recognizing that different operations are all really the same thing. This reduces the number of distinct cases that you must understand and analyze.

OO development not only lets you share information within an application, but also offers the prospect of reusing designs and code on future projects. OO development provides the tools, such as abstraction, encapsulation, and inheritance, to build libraries of reusable

components. Unfortunately, reuse has been overemphasized as a justification for OO technology. Reuse does not just happen; developers must plan by thinking beyond the immediate application and investing extra effort in a more general design.

1.3.5 Emphasis on the Essence of an Object

OO technology stresses what an object *is*, rather than how it is *used*. The uses of an object depend on the details of the application and often change during development. As requirements evolve, the features supplied by an object are much more stable than the ways it is used, hence software systems built on object structure are more stable in the long run. OO development places a greater emphasis on data structure and a lesser emphasis on procedure structure than functional-decomposition methodologies. In this respect, OO development is similar to information modeling techniques used in database design, although OO development adds the concept of class-dependent behavior.

1.3.6 Synergy

Identity, classification, polymorphism, and inheritance characterize OO languages. Each of these concepts can be used in isolation, but together they complement each other synergistically. The benefits of an OO approach are greater than they might seem at first. The emphasis on the essential properties of an object forces the developer to think more carefully and deeply about what an object is and does. The resulting system tends to be cleaner, more general, and more robust than it would be if the emphasis were only on the use of data and operations.

1.4 Evidence for Usefulness of OO Development

Our work on OO development began with internal applications at the General Electric Research and Development Center. We used OO techniques for developing compilers, graphics, user interfaces, databases, an OO language, CAD systems, simulations, metamodels, control systems, and other applications. We used OO models to document programs that are ill-structured and difficult to understand. Our implementation targets ranged from OO languages to non-OO languages to databases. We successfully taught this approach to others and used it to communicate with application experts.

Since the mid 1990s we have expanded our practice of OO technology beyond General Electric to companies throughout the world. When we wrote the first edition of this book, object orientation and OO modeling were relatively new approaches without much large-scale experience. OO technology can no longer be considered a fad or a speculative approach. It is now part of the computer science and software engineering mainstream.

The annual OOPSLA (Object-Oriented Programming Systems, Languages, and Applications), ECOOP (European Conference on Object-Oriented Programming), and TOOLS (Technology of Object-Oriented Languages and Systems) conferences are important forums for disseminating new OO ideas and application results. The conference proceedings describe many applications that have benefited from an OO approach. Articles on OO systems have also appeared in major publications, such as *IEEE Computer* and *Communications of the ACM*.

1.5 OO Modeling History

Our work at GE R&D led to the development of the Object Modeling Technique (OMT), which the previous edition of this book introduced in 1991. OMT was a success, but so were several other approaches. The popularity of OO modeling led to a new problem—a plethora of alternative notations. The notations expressed similar ideas but had different symbols, confusing developers and making communication difficult.

As a result, the software community began to focus on consolidating the various notations. In 1994 Jim Rumbaugh joined Rational (now part of IBM) and began working with Grady Booch on unifying the OMT and Booch notations. In 1995, Ivar Jacobson also joined Rational and added Objectory to the unification work.

In 1996 the Object Management Group (OMG) issued a request for proposals for a standard OO modeling notation. Several companies responded, and eventually the competing proposals were coalesced into a final proposal. Rational led the final proposal team, with Booch, Rumbaugh, and Jacobson deeply involved. The OMG unanimously accepted the resulting Unified Modeling Language (UML) as a standard in November 1997. The participating companies transferred UML rights to the OMG, which owns the trademark and specification for UML and controls its future development.

The UML was highly successful and replaced the other notations in most publications. Most of the authors of other methods adopted UML notation, willingly or because of market pressure. The UML has ended the OO notation wars and is now clearly *the* accepted OO notation. We have used UML in this book because it is now the standard notation.

In 2001 OMG members started work on a revision to add features missing from the initial specification and to fix problems that were discovered by experience with UML 1. This book is based on the UML 2.0 revision approved in 2004. For access to the official specification documents, see the OMG Web site at www.omg.org.

1.6 Organization of This Book

The remainder of this book is organized into four parts: modeling concepts, analysis/design, implementation, and software engineering. Appendices provide a glossary of terms and answer some of the exercises. The inside covers summarize the notation used in the book.

Part 1 explains OO concepts and presents a graphical notation for expressing them. Chapter 2 introduces modeling and three kinds of models—class, state, and interaction. Chapters 3 and 4 describe the class model, which deals with the structural “data” aspects of a system—these chapters are the heart of Part 1, and mastery of the class model is essential for successful OO development. Chapters 5 and 6 present the state model, which concerns the control aspects of a system. Chapters 7 and 8 describe the interaction model, which captures the interactions among different objects in a system. Chapter 9 summarizes the three models and how they relate to each other. The concepts dealt with in Part 1 permeate the software development cycle, applying equally to analysis, design, and implementation. The entire book uses the notation described in Part 1.

Part 2 shows how to prepare an OO model and use it to analyze and design a system. Chapter 10 summarizes the process, and then Chapter 11 discusses system conception, the invention of an application. Chapters 12 and 13 discuss analysis, the process of describing and understanding the application. Analysis begins with a problem statement from the customer. The analyst incorporates customer information and application knowledge to construct domain and application models. Chapter 14 addresses system design, which is primarily a task of partitioning a system into subsystems and making high-level policy decisions. Chapter 15 presents class design, the augmentation of the analysis model with design decisions. These decisions include the specification of algorithms, assigning functionality to objects, and optimization. Chapter 16 summarizes the process.

Part 3 addresses implementation, with Chapter 17 discussing issues apart from the target language. Chapters 18 and 19 address C++, Java, and databases. Chapter 20 presents guidelines for enhancing readability, reusability, and maintainability using good OO programming style.

Part 4 focuses on software engineering. Although Part 2 presents the stages in a linear order, as a book must, we do not believe that development should proceed in a waterfall fashion. Chapter 21 describes iterative development, in which the process stages are repeated multiple times to build the complete system. Chapter 22 provides advice for managing models. It is easiest to understand and to apply OO development on a new system, but most projects do not have the luxury of working on a clean slate. Chapter 23 describes issues involved in working with existing systems.

Most chapters contain exercises. Selected answers are included in the back of the book. We suggest that you try to work the exercises as you read this book, even if you are not a student. The exercises bring out many subtle points. They provide practice with OO technology and serve as a stepping stone to applications.

abstraction	encapsulation	object-oriented (OO)
analysis	identity	polymorphism
class design	implementation	state model
class model	inheritance	system design
classification	interaction model	

Figure 1.4 Key concepts for Chapter 1

Bibliographic Notes

[Taylor-98] provides a well-written overview of OO technology. [Meyer-97] is also an informative source, even though it is primarily an OO language book. [Love-93] presents examples of industrial projects that have used OO technology.

The purpose of this book is to teach OO concepts and thinking, not serve as a UML reference manual (see [Rumbaugh-05] for that). A textbook should emphasize important con-

cepts, not fine details. We therefore present the most useful aspects of the UML, but we do not try to describe everything. You will learn faster by focusing on core concepts.

We have made a similar condensation of the development process. The process we describe is simple and aimed at small and medium projects. It contains highlights of the Unified Process (see [Jacobson-99]).

The UML contains the concept of a *classifier*, a more general form of a class that abstracts various kinds of modeling entities. For most purposes, there is little difference between a class and a classifier. In this book, we use the word *class* in preference to *classifier*, because modelers will work mostly with classes.

References

- [Brooks-95] Frederick P. Brooks, Jr. *The Mythical Man-Month, Anniversary Edition*. Boston: Addison-Wesley, 1995.
- [Jacobson-99] Ivar Jacobson, Grady Booch, James Rumbaugh. *The Unified Software Development Process*. Boston: Addison-Wesley, 1999.
- [Love-93] Tom Love. *Object Lessons: Lessons Learned in Object-Oriented Development Practices*. New York: SIGS Books, 1993.
- [Meyer-97] Bertrand Meyer. *Object-Oriented Software Construction, Second Edition*. Hertfordshire, England: Prentice Hall International, 1997.
- [Rumbaugh-05] James Rumbaugh, Ivar Jacobson, Grady Booch. *The Unified Modeling Language Reference Manual, Second Edition*. Boston: Addison-Wesley, 2005.
- [Taylor-98] David A. Taylor. *Object Technology: A Manager's Guide, Second Edition*. Boston: Addison-Wesley, 1998.

Exercises

The number in parentheses next to each exercise indicates the difficulty, from 1 (easy) to 10 (very difficult).

- 1.1 (3) What major problems have you encountered during past software projects? Estimate what percentage of your time you spend on analysis, design, coding, and testing/debugging/fixing. How do you go about estimating how much effort a project will require?
- 1.2 (3) Recall a past system that you created. Briefly describe it. What obstacles did you encounter in the design? What software engineering methodology, if any, did you use? What were your reasons for choosing or not choosing a methodology? Are you satisfied with the system as it exists? How difficult is it to add new features to the system? Is it maintainable?
- 1.3 (3) Describe a recent large software system that was behind schedule, over budget, or failed to perform as expected. What factors were blamed? How could the failure have been avoided?
- 1.4 (3) From a user's point of view, criticize a hardware or software system that has a flaw that especially annoys you. For example, some cars require the bumper to be removed to replace a tail light. Describe the system, the flaw, how it was overlooked, and how it could have been avoided with a bit more thought during design.

- 1.5 (5) All objects have identity and are distinguishable. However, for large collections of objects, it may not be a trivial matter to devise a scheme to distinguish them. Furthermore, a scheme may depend on the purpose of the distinction. For each of the following collections of objects, describe how they could be distinguished.
- All persons in the world for the purpose of sending mail
 - All persons in the world for the purpose of criminal investigations
 - All customers with safe deposit boxes in a given bank
 - All telephones in the world for making telephone calls
 - All customers of a telephone company for billing purposes
 - All electronic mail addresses throughout the world
 - All employees of a company to restrict access for security reasons
- 1.6 (4) Prepare a list of classes that you would expect each of the following systems to handle.
- A program for laying out a newspaper
 - A program to compute and store bowling scores
 - A telephone voice mail system with delivery options, message forwarding, and group lists
 - A controller for a video cassette recorder
 - A catalog store order entry system
- 1.7 (6) Classes and operations are listed below. For each class, select the operations that make sense for objects in that class. You may place an operation in multiple classes. Discuss the behavior of each operation.
- Classes:**
- variable-length array — ordered collection of objects, indexed by an integer, whose size can vary at run time
 - symbol table — a table that maps text keywords into descriptors
 - set — unordered collection of objects with no duplicates
- Operations:**
- append — add an object to the end of a collection
 - copy — make a copy of a collection
 - count — return the number of elements in a collection
 - delete — remove an element from a collection
 - index — retrieve an object from a collection at a given position
 - intersect — determine the common elements of two collections
 - insert — place an object into a collection at a given position
 - update — add an element to a collection, writing over whatever is already there
- 1.8 (4) Discuss what the classes in each of the following lists have in common. You may add more classes to each list.
- scanning electron microscope, eyeglasses, telescope, bomb sight, binoculars
 - pipe, check valve, faucet, filter, pressure gauge
 - bicycle, sailboat, car, truck, airplane, glider, motorcycle, horse
 - nail, screw, bolt, rivet
 - tent, cave, shed, garage, barn, house, skyscraper

Part 1

Modeling Concepts

Chapter 2	Modeling as a Design Technique	15
Chapter 3	Class Modeling	21
Chapter 4	Advanced Class Modeling	60
Chapter 5	State Modeling	90
Chapter 6	Advanced State Modeling	110
Chapter 7	Interaction Modeling	131
Chapter 8	Advanced Interaction Modeling	147
Chapter 9	Concepts Summary	161

Part 1 describes the concepts and notations involved in object-oriented modeling. The concepts and notation apply to analysis, design, and implementation.

Chapter 2 discusses modeling in general and then introduces the three kinds of object-oriented models—class, state, and interaction.

Chapter 3 presents the class model which describes the static structure of a system. The class model provides the context for the other two kinds of models. Chapter 4 covers advanced class modeling concepts that you can skip upon a first reading of the book.

Chapter 5 explains the state model which describes the aspects of a system that change over time as well as control behavior. Chapter 6 covers advanced state modeling concepts that you can also skip upon a first reading.

Chapter 7 presents the interaction model and completes the treatment of the three models. The interaction model describes how objects collaborate to achieve overall results. Chapter 8 is an advanced chapter on interaction modeling that you can skip upon an initial reading.

Chapter 9 briefly summarizes the three kinds of models and how they relate to each other.

After reading Part 1, you will understand object-oriented concepts and the UML notation for expressing them. You will be ready to apply the concepts to software development in subsequent parts of the book.

This page intentionally left blank

2

Modeling as a Design Technique

A *model* is an abstraction of something for the purpose of understanding it before building it. Because a model omits nonessential details, it is easier to manipulate than the original entity. Abstraction is a fundamental human capability that permits us to deal with complexity. Engineers, artists, and craftsmen have built models for thousands of years to try out designs before executing them. Development of hardware and software systems is no exception. To build complex systems, the developer must abstract different views of the system, build models using precise notations, verify that the models satisfy the requirements of the system, and gradually add detail to transform the models into an implementation.

2.1 Modeling

Designers build many kinds of models for various purposes before constructing things. Examples include architectural models to show customers, airplane scale models for wind-tunnel tests, pencil sketches for composition of oil paintings, blueprints of machine parts, storyboards of advertisements, and outlines of books. Models serve several purposes.

- **Testing a physical entity before building it.** The medieval masons did not know modern physics, but they built scale models of the Gothic cathedrals to test the forces on the structure. Engineers test scale models of airplanes, cars, and boats in wind tunnels and water tanks to improve their dynamics. Recent advances in computation permit the simulation of many physical structures without the need to build physical models. Not only is simulation cheaper, but it provides information that is too fleeting or inaccessible to be measured from a physical model. Both physical models and computer models are usually cheaper than building a complete system and enable early correction of flaws.
- **Communication with customers.** Architects and product designers build models to show their customers. Mock-ups are demonstration products that imitate some or all of the external behavior of a system.

- **Visualization.** Storyboards of movies, television shows, and advertisements let writers see how their ideas flow. They can modify awkward transitions, dangling ends, and unnecessary segments before detailed writing begins. Artists' sketches let them block out their ideas and make changes before committing them to oil or stone.
- **Reduction of complexity.** Perhaps the main reason for modeling, which incorporates all the previous reasons, is to deal with systems that are too complex to understand directly. The human mind can cope with only a limited amount of information at one time. Models reduce complexity by separating out a small number of important things to deal with at a time.

2.2 Abstraction

Abstraction is the selective examination of certain aspects of a problem. The goal of abstraction is to isolate those aspects that are important for some purpose and suppress those aspects that are unimportant. Abstraction must always be for some purpose, because the purpose determines what is, and is not, important. Many different abstractions of the same thing are possible, depending on the purpose for which they are made.

All abstractions are incomplete and inaccurate. Reality is a seamless web. Anything we say about it, any description of it, is an abridgement. All human words and language are abstractions—incomplete descriptions of the real world. This does not destroy their usefulness. The purpose of an abstraction is to limit the universe so we can understand. In building models, therefore, you must not search for absolute truth but for adequacy for some purpose. There is no single “correct” model of a situation, only adequate and inadequate ones.

A good model captures the crucial aspects of a problem and omits the others. Most computer languages, for example, are poor vehicles for modeling algorithms because they force the specification of implementation details that are irrelevant to the algorithm. A model that contains extraneous detail unnecessarily limits your choice of design decisions and diverts attention from the real issues.

2.3 The Three Models

We find it useful to model a system from three related but different viewpoints, each capturing important aspects of the system, but all required for a complete description. The *class model* represents the static, structural, “data” aspects of a system. The *state model* represents the temporal, behavioral, “control” aspects of a system. The *interaction model* represents the collaboration of individual objects, the “interaction” aspects of a system. A typical software procedure incorporates all three aspects: It uses data structures (class model), it sequences operations in time (state model), and it passes data and control among objects (interaction model). Each model contains references to entities in other models. For example, the class model attaches operations to classes, while the state and interaction models elaborate the operations.

The three kinds of models separate a system into distinct views. The different models are not completely independent—a system is more than a collection of independent parts—but each model can be examined and understood by itself to a large extent. The different models have limited and explicit interconnections. Of course, it is always possible to create bad designs in which the three models are so intertwined that they cannot be separated, but a good design isolates the different aspects of a system and limits the coupling between them.

Each of the three models evolves during development. First analysts construct a model of the application without regard for eventual implementation. Then designers add solution constructs to the model. Implementers code both application and solution constructs. The word *model* has two dimensions—a view of a system (class model, state model, or interaction model) and a stage of development (analysis, design, or implementation). The meaning is generally clear from context.

2.3.1 Class Model

The *class model* describes the structure of objects in a system—their identity, their relationships to other objects, their attributes, and their operations. The class model provides context for the state and interaction models. Changes and interactions are meaningless unless there is something to be changed or with which to interact. Objects are the units into which we divide the world, the molecules of our models.

Our goal in constructing a class model is to capture those concepts from the real world that are important to an application. In modeling an engineering problem, the class model should contain terms familiar to engineers; in modeling a business problem, terms from the business; in modeling a user interface, terms from the application. An analysis model should not contain computer constructs unless the application being modeled is inherently a computer problem, such as a compiler or an operating system. The design model describes how to solve a problem and may contain computer constructs.

Class diagrams express the class model. Generalization lets classes share structure and behavior, and associations relate the classes. Classes define the attribute values carried by each object and the operations that each object performs or undergoes.

2.3.2 State Model

The *state model* describes those aspects of objects concerned with time and the sequencing of operations—events that mark changes, states that define the context for events, and the organization of events and states. The state model captures *control*, the aspect of a system that describes the sequences of operations that occur, without regard for what the operations do, what they operate on, or how they are implemented.

State diagrams express the state model. Each state diagram shows the state and event sequences permitted in a system for one class of objects. State diagrams refer to the other models. Actions and events in a state diagram become operations on objects in the class model. References between state diagrams become interactions in the interaction model.

2.3.3 Interaction Model

The *interaction model* describes interactions between objects—how individual objects collaborate to achieve the behavior of the system as a whole. The state and interaction models describe different aspects of behavior, and you need both to describe behavior fully.

Use cases, sequence diagrams, and activity diagrams document the interaction model. Use cases document major themes for interaction between the system and outside actors. Sequence diagrams show the objects that interact and the time sequence of their interactions. Activity diagrams show the flow of control among the processing steps of a computation.

2.3.4 Relationship Among the Models

Each model describes one aspect of the system but contains references to the other models. The class model describes data structure on which the state and interaction models operate. The operations in the class model correspond to events and actions. The state model describes the control structure of objects. It shows decisions that depend on object values and causes actions that change object values and state. The interaction model focuses on the exchanges between objects and provides a holistic overview of the operation of a system.

There are occasional ambiguities about which model should contain a piece of information. This is natural, because any abstraction is only a rough cut at reality; something will inevitably straddle the boundaries. Some properties of a system may be poorly represented by the models. This is also normal, because no abstraction is perfect; the goal is to simplify the system description without loading down the model with so many constructs that it becomes a burden and not a help. For those things that the model does not adequately capture, natural language or application-specific notation is still perfectly acceptable.

2.4 Chapter Summary

Models are abstractions built to understand a problem before implementing a solution. All abstractions are subsets of reality selected for a particular purpose.

We recommend three kinds of models. The class model describes the static structure of a system in terms of classes and relationships. The state model describes the control structure of a system in terms of events and states. The interaction model describes how individual objects collaborate to achieve the behavior of the system as a whole. Different problems place different emphasis on the three kinds of models.

abstraction	modeling
class model	relationship among models
interaction model	state model

Figure 2.1 Key concepts for Chapter 2

Bibliographic Notes

The first edition of this book also had three models (object, dynamic, and functional), but they were organized differently than those in this second edition.

The object model in the first edition is the same as the class model presented here. We have changed the name to *class model* to stress that the modeling entities are descriptors (classes and relationships) rather than instances (objects and links). Our presentation of the class model in this book also includes constraint modeling, which was missing from the first edition.

Similarly, the dynamic model in the first edition is the same as the state model in this book. We changed the name to *state model* to avoid confusion with other representations of dynamic behavior. The UML contains multiple kinds of models with various degrees of overlap—we cover the most important ones in this book.

We have dropped the functional model from the second edition. Certainly, the eventual software has functionality, but we seldom capture it with data flow diagrams as was shown in the first edition. We included data flow diagrams in the first edition for continuity with the structured analysis / structured design approach of the past. The functional model was not as useful as we envisioned, so we have now dropped it.

In its place, the second edition adds the interaction model. State diagrams do express dynamic behavior fully, but often in an inscrutable manner. Each state diagram focuses on a single class. When many classes have a significant state diagram, it can be difficult to understand an entire system. The interaction model focuses on collaboration and helps a software developer obtain a more comprehensive understanding than with state diagrams alone.

Exercises

- 2.1 (1) Some characteristics of an automotive tire are its size, material, internal construction (bias ply, steel belted, for example), tread design, cost, expected life, and weight. Which factors are important in deciding whether or not to buy a tire for your car? Which ones might be relevant to someone simulating the performance of a computerized anti-skid system for cars? Which ones are important to someone constructing a swing for a child?
- 2.2 (2) Suppose your bathroom sink is clogged and you have decided to try to unclog it by pushing a wire into the drain. You have several types of wire available around the house, some insulated and some not. Which of the following wire characteristics would you need to consider in selecting a wire for the job? Explain your answers.
 - a. Immunity to electrical noise
 - b. Color of the insulation
 - c. Resistance of the insulation to salt water
 - d. Resistance of the insulation to fire
 - e. Cost
 - f. Stiffness
 - g. Ease of stripping the insulation

- h. Weight
 - i. Availability
 - j. Strength
 - k. Resistance to high temperatures
 - l. Resistance to stretching
- 2.3 (3) Wire is used in the following applications. For each application, prepare a list of wire characteristics that are relevant and explain why each is important for the application.
- a. Selecting wire for a transatlantic cable
 - b. Choosing wire that you will use to create colorful artwork
 - c. Designing the electrical system for an airplane
 - d. Hanging a bird feeder from a tree
 - e. Designing a piano
 - f. Designing the filament for a light bulb
- 2.4 (3) If you were designing a protocol for transferring computer files from one computer to another over telephone lines, which of the following details would you select as relevant? Explain how they are relevant.
- a. Electrical noise on the communication lines
 - b. The speed at which serial data is transmitted
 - c. Availability of a database
 - d. Availability of a good full-screen editor
 - e. Buffering and flow control, such as an XON/XOFF protocol to regulate an incoming stream of data
 - f. Number of tracks and sectors on a disk drive
 - g. Character interpretation, such as special handling of control characters
 - h. File organization, linear stream of bytes versus record-oriented, for example
 - i. Math co-processor
- 2.5 (2) There are several models used in the analysis and design of electrical motors. An electrical model involves voltages, currents, electromagnetic fields, inductance, and resistance. A mechanical model considers stiffness, density, motion, forces, and torques. A thermal model handles heat dissipation and heat transfer. A fluid model describes the flow of cooling air. Which model(s) can answer the following questions? Discuss your conclusions.
- a. How much power is required to run a motor? How much of it is wasted as heat?
 - b. How much does a motor weigh?
 - c. How hot does a motor get?
 - d. How much vibration does a motor create?
 - e. How long will it take for the bearings of a motor to wear out?
- 2.6 (3) Decide which model(s) (class, state, interaction) are relevant for the following aspects of a computer chess player. A video display will show the board and pieces. A cursor controlled by a mouse will indicate human moves. Of course, in some cases, more than one model may apply. Explain your answers.
- a. User interface that displays computer moves and accepts human moves
 - b. Representation of a configuration of pieces on the board
 - c. Consideration of a sequence of possible legal moves
 - d. Validation of a move requested by the human player

3

Class Modeling

A class model captures the static structure of a system by characterizing the objects in the system, the relationships between the objects, and the attributes and operations for each class of objects. The class model is the most important of the three models. We emphasize building a system around objects rather than around functionality, because an object-oriented system more closely corresponds to the real world and is consequently more resilient with respect to change. Class models provide an intuitive graphic representation of a system and are valuable for communicating with customers.

Chapter 3 discusses basic class modeling concepts that will be used throughout the book. We define each concept, present the corresponding UML notation, and provide examples. Some important concepts that we consider are object, class, link, association, generalization, and inheritance. You should master the material in this chapter before proceeding in the book.

3.1 Object and Class Concepts

3.1.1 Objects

The purpose of class modeling is to describe objects. For example, *Joe Smith*, *Simplex company*, *process number 7648*, and *the top window* are objects.

An **object** is a concept, abstraction, or thing with identity that has meaning for an application. Objects often appear as proper nouns or specific references in problem descriptions and discussions with users. Some objects have real-world counterparts (Albert Einstein and the General Electric company), while others are conceptual entities (simulation run 1234 and the formula for solving a quadratic equation). Still others (binary tree 634 and the array bound to variable *a*) are introduced for implementation reasons and have no correspondence to physical reality. The choice of objects depends on judgment and the nature of a problem; there can be many correct representations.

All objects have identity and are distinguishable. Two apples with the same color, shape, and texture are still individual apples; a person can eat one and then eat the other. Similarly, identical twins are two distinct persons, even though they may look the same. The term *identity* means that objects are distinguished by their inherent existence and not by descriptive properties that they may have.

3.1.2 Classes

An object is an *instance*—or occurrence—of a class. A *class* describes a group of objects with the same properties (attributes), behavior (operations), kinds of relationships, and semantics. *Person*, *company*, *process*, and *window* are all classes. Each person has name and birthdate and may work at a job. Each process has an owner, priority, and list of required resources. Classes often appear as common nouns and noun phrases in problem descriptions and discussions with users.

Objects in a class have the same attributes and forms of behavior. Most objects derive their individuality from differences in their attribute values and specific relationships to other objects. However, objects with identical attribute values and relationships are possible. The choice of classes depends on the nature and scope of an application and is a matter of judgment.

The objects in a class share a common semantic purpose, above and beyond the requirement of common attributes and behavior. For example, a barn and a horse may both have a cost and an age. If barn and horse were regarded as purely financial assets, they could belong to the same class. If the developer took into consideration that a person paints a barn and feeds a horse, they would be modeled as distinct classes. The interpretation of semantics depends on the purpose of each application and is a matter of judgment.

Each object “knows” its class. Most OO programming languages can determine an object’s class at run time. An object’s class is an implicit property of the object.

If objects are the focus of modeling, why bother with classes? The notion of abstraction is at the heart of the matter. By grouping objects into classes, we abstract a problem. Abstraction gives modeling its power and ability to generalize from a few specific cases to a host of similar cases. Common definitions (such as class name and attribute names) are stored once per class rather than once per instance. You can write operations once for each class, so that all the objects in the class benefit from code reuse. For example, all ellipses share the same procedures to draw them, compute their areas, and test for intersection with a line; polygons would have a separate set of procedures. Even special cases, such as circles and squares, can use the general procedures, though more efficient procedures are possible.

3.1.3 Class Diagrams

We began this chapter by discussing some basic modeling concepts, specifically *object* and *class*. We have described these concepts with examples and prose. This approach is vague and insufficient for dealing with the complexity of applications. We need a means for expressing models that is coherent, precise, and easy to formulate. There are two kinds of models of structure—class diagrams and object diagrams.

Class diagrams provide a graphic notation for modeling classes and their relationships, thereby describing possible objects. Class diagrams are useful both for abstract modeling and for designing actual programs. They are concise, easy to understand, and work well in practice. We will use class diagrams throughout this book to represent the structure of applications.

We will also occasionally use object diagrams. An **object diagram** shows individual objects and their relationships. Object diagrams are helpful for documenting test cases and discussing examples. A class diagram corresponds to an infinite set of object diagrams.

Figure 3.1 shows a class (left) and instances (right) described by it. Objects *JoeSmith*, *MarySharp*, and an anonymous person are instances of class *Person*. The UML symbol for an object is a box with an object name followed by a colon and the class name. The object name and class name are both underlined. Our convention is to list the object name and class name in boldface.

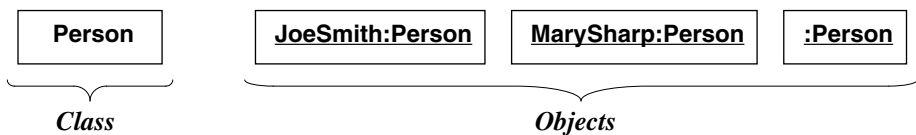


Figure 3.1 A class and objects. Objects and classes are the focus of class modeling.

The UML symbol for a class also is a box. Our convention is to list the class name in boldface, center the name in the box, and capitalize the first letter. We use singular nouns for the names of classes.

Note how we run together multiword names, such as *JoeSmith*, separating the words with intervening capital letters. This is the convention we use for referring to objects, classes, and other constructs. Alternative conventions would be to use intervening spaces (Joe Smith) or underscores (Joe_Smith). The mixed capitalization convention is popular in the OO literature but is not a UML requirement.

3.1.4 Values and Attributes

A **value** is a piece of data. You can find values by examining problem documentation for examples. An **attribute** is a named property of a class that describes a value held by each object of the class. You can find attributes by looking for adjectives or by abstracting typical values. The following analogy holds: Object is to class as value is to attribute. Structural constructs—that is, classes and relationships (to be explained)—dominate class models. Attributes are of lesser importance and serve to elaborate classes and relationships.

Name, *birthdate*, and *weight* are attributes of *Person* objects. *Color*, *modelYear*, and *weight* are attributes of *Car* objects. Each attribute has a value for each object. For example, attribute *birthdate* has value “21 October 1983” for object *JoeSmith*. Paraphrasing, Joe Smith was born on 21 October 1983. Different objects may have the same or different values for a given attribute. Each attribute name is unique within a class (as opposed to being unique across all classes). Thus class *Person* and class *Car* may each have an attribute called *weight*.

Do not confuse values with objects. An attribute should describe values, not objects. Unlike objects, values lack identity. For example, all occurrences of the integer “17” are indistinguishable, as are all occurrences of the string “Canada.” The country Canada is an object, whose *name* attribute has the value “Canada” (the string).

Figure 3.2 shows modeling notation. Class *Person* has attributes *name* and *birthdate*. *Name* is a string and *birthdate* is a date. One object in class *Person* has the value “Joe Smith” for name and the value “21 October 1983” for birthdate. Another object has the value “Mary Sharp” for name and the value “16 March 1950” for birthdate.

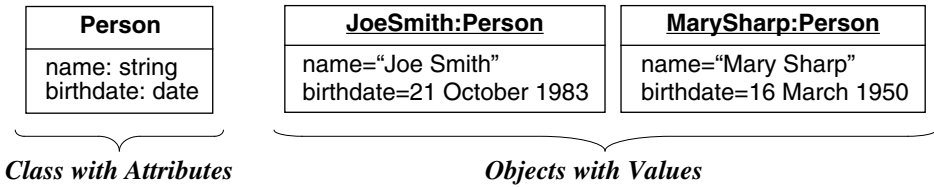


Figure 3.2 Attributes and values. Attributes elaborate classes.

The UML notation lists attributes in the second compartment of the class box. Optional details, such as type and default value, may follow each attribute. A colon precedes the type. An equal sign precedes the default value. Our convention is to show the attribute name in regular face, left align the name in the box, and use a lowercase letter for the first letter.

You may also include attribute values in the second compartment of object boxes. The notation is to list each attribute name followed by an equal sign and the value. We also left align attribute values and use regular type face.

Some implementation media require that an object have a unique identifier. These identifiers are implicit in a class model—you need not and should not list them explicitly. Figure 3.3 emphasizes the point. Most OO languages automatically generate identifiers with which to reference objects. You can also readily define them for databases. Identifiers are a computer artifact and have no intrinsic meaning.

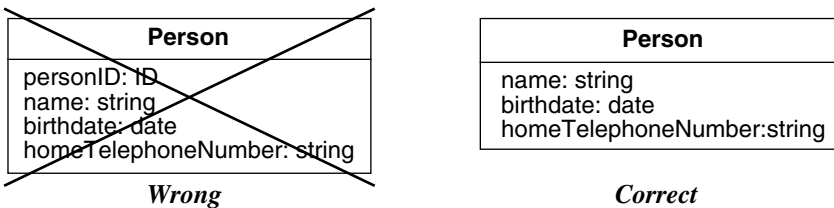


Figure 3.3 Object identifiers. Do not list object identifiers; they are implicit in models.

Do not confuse internal identifiers with real-world attributes. Internal identifiers are purely an implementation convenience and have no application meaning. In contrast, tax payer number, license plate number, and telephone number are not internal identifiers because they have meaning in the real world. Rather they are legitimate attributes.

3.1.5 Operations and Methods

An **operation** is a function or procedure that may be applied to or by objects in a class. *Hire*, *fire*, and *payDividend* are operations on class *Company*. *Open*, *close*, *hide*, and *redisplay* are operations on class *Window*. All objects in a class share the same operations.

Each operation has a target object as an implicit argument. The behavior of the operation depends on the class of its target. An object “knows” its class, and hence the right implementation of the operation.

The same operation may apply to many different classes. Such an operation is **polymorphic**; that is, the same operation takes on different forms in different classes. A **method** is the implementation of an operation for a class. For example, the class *File* may have an operation *print*. You could implement different methods to print ASCII files, print binary files, and print digitized picture files. All these methods logically perform the same task—printing a file; thus you may refer to them by the generic operation *print*. However, a different piece of code may implement each method.

An operation may have arguments in addition to its target object. Such arguments may be placeholders for values, or for other objects. The choice of a method depends entirely on the class of the target object and not on any object arguments that an operation may have. (A few OO languages, notably CLOS, permit the choice of method to depend on any number of arguments, but such generality leads to considerable semantic complexity, which we shall not explore.)

When an operation has methods on several classes, it is important that the methods all have the same **signature**—the number and types of arguments and the type of result value. For example, *print* should not have *fileName* as an argument for one method and *filePointer* for another. The behavior of all methods for an operation should have a consistent intent. It is best to avoid using the same name for two operations that are semantically different, even if they apply to distinct sets of classes. For example, it would be unwise to use the name *invert* to describe both a matrix inversion and turning a geometric figure upside-down. In a large project, some form of name scoping may be necessary to accommodate accidental name clashes, but it is best to avoid any possibility of confusion.

In Figure 3.4, the class *Person* has attributes *name* and *birthdate* and operations *changeJob* and *changeAddress*. *Name*, *birthdate*, *changeJob*, and *changeAddress* are features of *Person*. **Feature** is a generic word for either an attribute or operation. Similarly, *File* has a *print* operation. *GeometricObject* has *move*, *select*, and *rotate* operations. *Move* has argument *delta*, which is a *Vector*; *select* has one argument *p*, which is of type *Point* and returns a *Boolean*; and *rotate* has argument *angle*, which is an input of type float with a default value of 0.0.

The UML notation is to list operations in the third compartment of the class box. Our convention is to list the operation name in regular face, left align the name in the box, and use a lowercase letter for the first letter. Optional details, such as an argument list and result type, may follow each operation name. Parentheses enclose an argument list; commas separate the arguments. A colon precedes the result type. An empty argument list in parentheses shows explicitly that there are no arguments; otherwise you cannot draw conclusions. We do not list operations for objects, because they do not vary among objects of the same class.

Person	File	GeometricObject
name birthdate	fileName sizeInBytes lastUpdate	color position
changeJob changeAddress	print	move (delta : Vector) select (p : Point): Boolean rotate (in angle : float = 0.0)

Figure 3.4 Operations. An operation is a function or procedure that may be applied to or by objects in a class.

3.1.6 Summary of Notation for Classes

Figure 3.5 summarizes the notation for classes. A box represents a class and may have as many as three compartments. The compartments contain, from top to bottom: class name, list of attributes, and list of operations. Optional details such as type and default value may follow each attribute name. Optional details such as argument list and result type may follow each operation name.

ClassName
attributeName1 : dataType1 = defaultValue1 attributeName2 : dataType2 = defaultValue2 • • •
operationName1 (argumentList1) : resultType1 operationName2 (argumentList2) : resultType2 • • •

Figure 3.5 Summary of modeling notation for classes. A box represents a class and may have as many as three compartments.

Figure 3.6 shows that each argument may have a direction, name, type, and default value. The **direction** indicates whether an argument is an input (*in*), output (*out*), or an input argument that can be modified (*inout*). A colon precedes the type. An equal sign precedes the default value. The default value is used if no argument is supplied for the argument.

direction argumentName : type = defaultValue

Figure 3.6 Notation for an argument of an operation. The direction, type, and default value are optional. Direction may be *in*, *out*, or *inout*.

The attribute and operation compartments of class boxes are optional, and you may or may not show them. A missing attribute compartment means that attributes are unspecified. Similarly, a missing operation compartment means that operations are unspecified. In contrast, an empty compartment means that attributes (operations) are specified and that there are none.

3.2 Link and Association Concepts

Links and associations are the means for establishing relationships among objects and classes.

3.2.1 Links and Associations

A *link* is a physical or conceptual connection among objects. For example, Joe Smith *WorksFor* Simplex company. Most links relate two objects, but some links relate three or more objects. This chapter discusses only binary associations; Chapter 4 discusses n-ary associations. Mathematically, we define a link as a tuple—that is, a list of objects. A link is an instance of an association.

An *association* is a description of a group of links with common structure and common semantics. For example, a person *WorksFor* a company. The links of an association connect objects from the same classes. An association describes a set of potential links in the same way that a class describes a set of potential objects. Links and associations often appear as verbs in problem statements.

Figure 3.7 is an excerpt of a model for a financial application. Stock brokerage firms need to perform tasks such as recording ownership of various stocks, tracking dividends, alerting customers to changes in the market, and computing margin requirements. The top portion of the figure shows a class diagram and the bottom shows an object diagram.

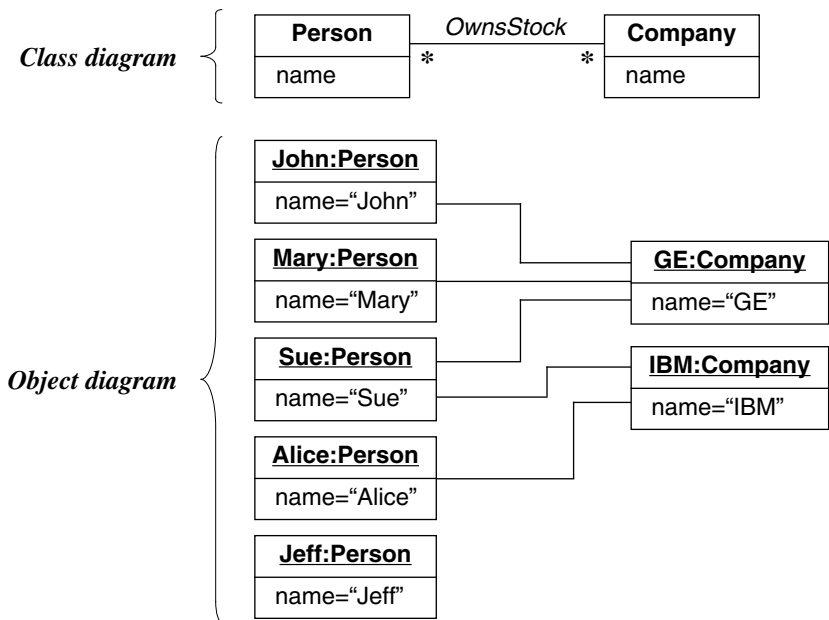


Figure 3.7 Many-to-many association. An association describes a set of potential links in the same way that a class describes a set of potential objects.

In the class diagram, a person may own stock in zero or more companies; a company may have multiple persons owning its stock. The object diagram shows some examples. John, Mary, and Sue own stock in the GE company. Sue and Alice own stock in the IBM company. Jeff does not own stock in any company and thus has no link. The asterisk is a multiplicity symbol. Multiplicity specifies the number of instances of one class that may relate to a single instance of another class and is discussed in the next section.

The UML notation for a link is a line between objects; a line may consist of several line segments. If the link has a name, it is underlined. For example, John owns stock in the GE company. An association connects related classes and is also denoted by a line (with possibly multiple line segments). For example, persons own stock in companies. Our convention is to show link and association names in italics and to confine line segments to a rectilinear grid. It is good to arrange the classes in an association to read from left-to-right, if possible.

The association name is optional, if the model is unambiguous. Ambiguity arises when a model has multiple associations among the same classes (*person works for company* and *person owns stock in company*). When there are multiple associations, you must use association names or association end names (Section 3.2.3) to resolve the ambiguity.

Associations are inherently bidirectional. The name of a binary association usually reads in a particular direction, but the binary association can be traversed in either direction. For example, *WorksFor* connects a person to a company. The inverse of *WorksFor* could be called *Employs*, and it connects a company to a person. In reality, both directions of traversal are equally meaningful and refer to the same underlying association; it is only the names that establish a direction.

Developers often implement associations in programming languages as references from one object to another. A **reference** is an attribute in one object that refers to another object. For example, a data structure for *Person* might contain an attribute *employer* that refers to a *Company* object, and a *Company* object might contain an attribute *employees* that refers to a set of *Person* objects. Implementing associations as references is perfectly acceptable, but you should not model associations this way.

A link is a relationship among objects. Modeling a link as a reference disguises the fact that the link is not part of either object by itself, but depends on both of them together. A company is not part of a person, and a person is not part of a company. Furthermore, using a pair of matched references, such as the reference from *Person* to *Company* and the reference from *Company* to a set of *Persons*, hides the fact that the forward and inverse references depend on each other. Therefore, you should model all connections among classes as associations, even in designs for programs.

The OO literature emphasizes encapsulation, that implementation details should be kept private to a class, and we certainly agree with this. Associations are important, precisely because they break encapsulation. Associations cannot be private to a class, because they transcend classes. Failure to treat associations on an equal footing with classes can lead to programs containing hidden assumptions and dependencies. Such programs are difficult to extend and the classes are difficult to reuse.

Although modeling treats associations as bidirectional, you do not have to implement them in both directions. You can readily implement associations as references if they are only

traversed in a single direction. Chapter 17 discusses some trade-offs to consider when implementing associations.

3.2.2 Multiplicity

Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class. Multiplicity constrains the number of related objects. The literature often describes multiplicity as being “one” or “many,” but more generally it is a (possibly infinite) subset of the nonnegative integers. UML diagrams explicitly list multiplicity at the ends of association lines. The UML specifies multiplicity with an interval, such as “1” (exactly one), “1..*” (one or more), or “3..5” (three to five, inclusive). The special symbol “*” is a shorthand notation that denotes “many” (zero or more).

Figure 3.7 illustrates many-to-many multiplicity. A person may own stock in many companies. A company may have multiple persons holding its stock. In this particular case, John and Mary own stock in the GE company; Alice owns stock in the IBM company; Sue owns stock in both companies; Jeff does not own any stock. GE stock is owned by three persons; IBM stock is owned by two persons.

Figure 3.8 shows a one-to-one association and some corresponding links. Each country has one capital city. A capital city administers one country. (In fact, some countries, such as The Netherlands and Switzerland, have more than one capital city for different purposes. If this fact were important, the model could be modified by changing the multiplicity or by providing a separate association for each kind of capital city.)

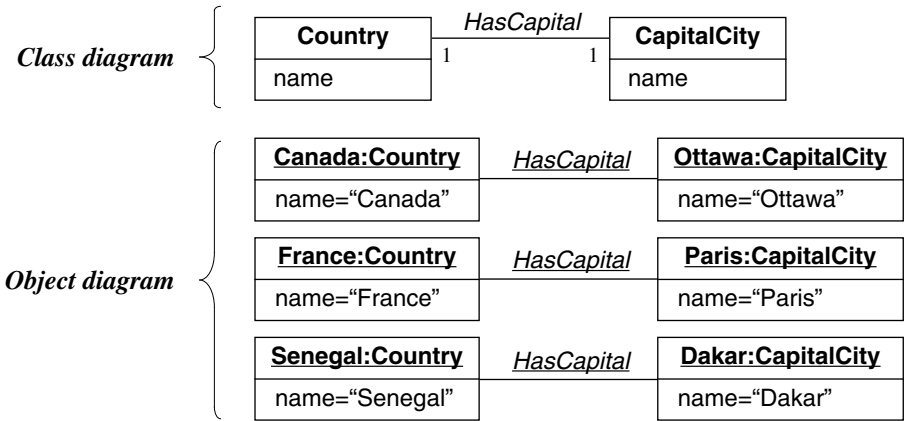


Figure 3.8 One-to-one association. Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class.

Figure 3.9 illustrates zero-or-one multiplicity. A workstation may have one of its windows designated as the console to receive general error messages. It is possible, however, that no console window exists. (The word “console” on the diagram is an association end name, discussed in Section 3.2.3.)



Figure 3.9 Zero-or-one multiplicity. It may be optional whether an object is involved in an association.

Do not confuse “multiplicity” with “cardinality.” Multiplicity is a *constraint* on the size of a collection; cardinality is the *count* of elements that are actually in a collection. Therefore, multiplicity is a constraint on the cardinality.

A multiplicity of “many” specifies that an object may be associated with multiple objects. However, for each association there is at most one link between a given pair of objects (except for bags and sequences, see Section 3.2.5). As Figure 3.10 and Figure 3.11 show, if you want two links between the same objects, you must have two associations.

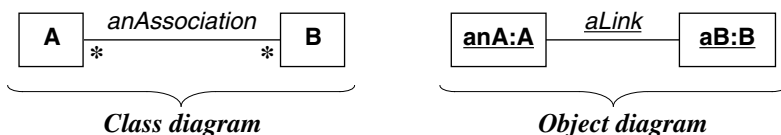


Figure 3.10 Association vs. link. A pair of objects can be instantiated at most once per association (except for bags and sequences).

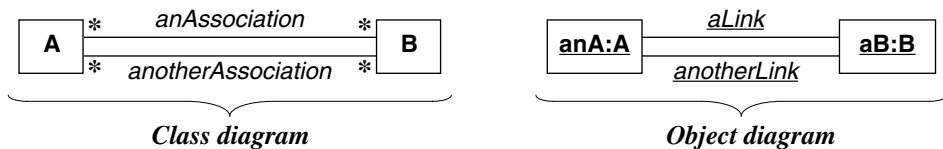


Figure 3.11 Association vs. link. You can use multiple associations to model multiple links between the same objects.

Multiplicity depends on assumptions and how you define the boundaries of a problem. Vague requirements often make multiplicity uncertain. Do not worry excessively about multiplicity early in software development. First determine classes and associations, then decide on multiplicity. If you omit multiplicity notation from a diagram, multiplicity is considered to be unspecified.

Multiplicity often exposes hidden assumptions built into a model. For example, is the *WorksFor* association between *Person* and *Company* one-to-many or many-to-many? It depends on the context. A tax collection application would permit a person to work for multiple companies. On the other hand, the member records for an auto workers’ union may consider second jobs irrelevant. Class diagrams help to elicit these hidden assumptions, making them visible and subject to scrutiny.

The most important multiplicity distinction is between “one” and “many.” Underestimating multiplicity can restrict the flexibility of an application. For example, many programs

cannot accommodate persons with multiple phone numbers. On the other hand, overestimating multiplicity imposes overhead and requires the application to supply additional information to distinguish among the members of a “many” set. In a true hierarchical organization, for example, it is better to represent “boss” with a multiplicity of “zero or one,” rather than allow for nonexistent matrix management.

3.2.3 Association End Names

Our discussion of multiplicity implicitly referred to the ends of associations. For example, a one-to-many association has two ends—an end with a multiplicity of “one” and an end with a multiplicity of “many.” The notion of an *association end* is an important concept in the UML. You can not only assign a multiplicity to an association end, but you can give it a name as well. (Chapter 4 discusses additional properties of association ends.)

Association end names often appear as nouns in problem descriptions. As Figure 3.12 shows, a name appears next to the association end. In the figure *Person* and *Company* participate in association *WorksFor*. A person is an *employee* with respect to a company; a company is an *employer* with respect to a person. Use of association end names is optional, but it is often easier and less confusing to assign association end names instead of, or in addition to, association names.

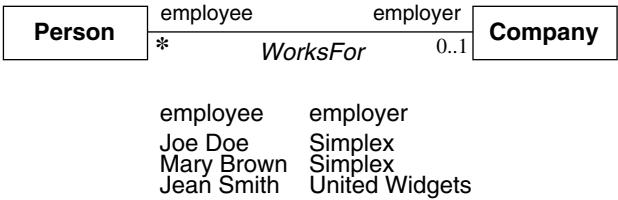


Figure 3.12 Association end names. Each end of an association can have a name.

Association end names are especially convenient for traversing associations, because you can treat each one as a pseudo attribute. Each end of a binary association refers to an object or set of objects associated with a source object. From the point of view of the source object, traversal of the association is an operation that yields related objects. Association end names provide a means of traversing an association, without explicitly mentioning the association. Section 3.5 talks further about traversing class models.

Association end names are necessary for associations between two objects of the same class. For example, in Figure 3.13 *container* and *contents* distinguish the two usages of *Directory* in the self-association. A directory may contain many lesser directories and may optionally be contained itself. Association end names can also distinguish multiple associations between the same pair of classes. In Figure 3.13 each directory has exactly one user who is an owner and many users who are authorized to use the directory. When there is only a single association between a pair of distinct classes, the names of the classes often suffice, and you may omit association end names.

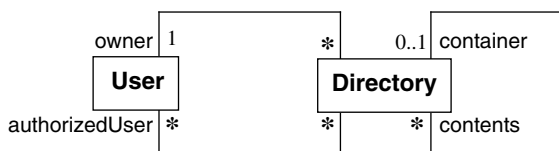


Figure 3.13 Association end names. Association end names are necessary for associations between two objects of the same class. They can also distinguish multiple associations between a pair of classes.

Association end names let you unify multiple references to the same class. When constructing class diagrams you should properly use association end names and not introduce a separate class for each reference, as Figure 3.14 shows. In the wrong model, two instances represent a person with a child, one for the child and one for the parent. In the correct model, one person instance participates in two or more links, twice as a parent and zero or more times as a child. (In the correct model, we must show a child as having an optional parent, so that the recursion eventually terminates.)

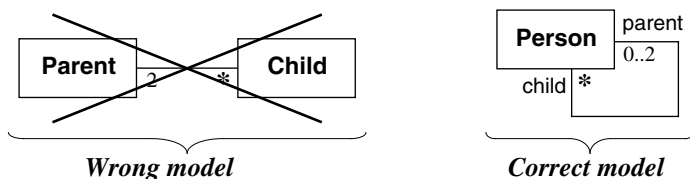


Figure 3.14 Association end names. Use association end names to model multiple references to the same class.

Because association end names distinguish objects, all names on the far end of associations attached to a class must be unique. Although the name appears next to the destination object on an association, it is really a pseudo attribute of the source class and must be unique within it. For the same reason, no association end name should be the same as an attribute name of the source class.

3.2.4 Ordering

Often the objects on a “many” association end have no explicit order, and you can regard them as a set. Sometimes, however, the objects have an explicit order. For example, Figure 3.15 shows a workstation screen containing a number of overlapping windows. Each window on a screen occurs at most once. The windows have an explicit order, so only the top-most window is visible at any point on the screen. The ordering is an inherent part of the association. You can indicate an ordered set of objects by writing “{ordered}” next to the appropriate association end.

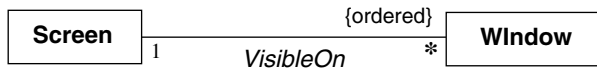


Figure 3.15 Ordering the objects for an association end. Ordering sometimes occurs for “many” multiplicity.

3.2.5 Bags and Sequences

Ordinarily a binary association has at most one link for a pair of objects. However, you can permit multiple links for a pair of objects by annotating an association end with *{bag}* or *{sequence}*. A **bag** is a collection of elements with duplicates allowed. A **sequence** is an ordered collection of elements with duplicates allowed. In Figure 3.16 an itinerary is a sequence of airports and the same airport can be visited more than once. Like the *{ordered}* indication, *{bag}* and *{sequence}* are permitted only for binary associations.

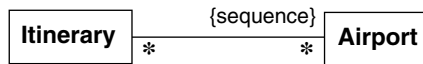


Figure 3.16 An example of a sequence. An itinerary may visit multiple airports, so you should use *{sequence}* and not *{ordered}*.

UML1 did not permit multiple links for a pair of objects. Some modelers misunderstood this restriction with ordered association ends and constructed incorrect models, assuming that there could be multiple links. With UML2 the modeler’s intent is now clear. If you specify *{bag}* or *{sequence}*, then there can be multiple links for a pair of objects. If you omit these annotations, then the association has at most one link for a pair of objects.

Note that the *{ordered}* and the *{sequence}* annotations are the same, except that the first disallows duplicates and the other allows them. A sequence association is an ordered bag, while an ordered association is an ordered set.

3.2.6 Association Classes

Just as you can describe the objects of a class with attributes, so too you can describe the links of an association with attributes. The UML represents such information with an association class. An **association class** is an association that is also a class. Like the links of an association, the instances of an association class derive identity from instances of the constituent classes. Like a class, an association class can have attributes and operations and participate in associations. You can find association classes by looking for adverbs in a problem statement or by abstracting known values.

In Figure 3.17, *accessPermission* is an attribute of *AccessibleBy*. The sample data at the bottom of the figure shows the value for each link. The UML notation for an association class is a box (a class box) attached to the association by a dashed line.

type of arguments to an operation, and operation return type. Tightening the type of an attribute or operation argument to be a subclass of the original type is a form of restriction and must be done with care. It is common to boost performance by overriding a general method with a special method that takes advantage of specific information but does not alter the operation semantics (such as *Circle.rotate* in Figure 3.25).

You should never override a feature so that it is inconsistent with the original inherited feature. A subclass *is* a special case of its superclass and should be compatible with it in every respect. A common, but unfortunate, practice in OO programming is to “borrow” a class that is similar to a desired class and then modify it by changing and ignoring some of its features, even though the new class is not really a special case of the original class. This practice can lead to conceptual confusion and hidden assumptions built into programs.

3.4 A Sample Class Model

Figure 3.26 shows a class model of a workstation window management system. This model is greatly simplified—a real model would require a number of pages—but it illustrates many class modeling constructs and shows how they fit together.

Class *Window* defines common parameters of all kinds of windows, including a rectangular boundary defined by the attributes *x1*, *y1*, *x2*, *y2*, and operations to display and undisplay a window and to raise it to the top (foreground) or lower it to the bottom (background) of the entire set of windows.

A canvas is a region for drawing graphics. It inherits the window boundary from *Window* and adds the dimensions of the underlying canvas region defined by attributes *cx1*, *cy1*, *cx2*, *cy2*. A canvas contains a set of elements, shown by the association to class *Shape*. All shapes have color and line width. Shapes can be lines, ellipses, or polygons, each with their own parameters. A polygon consists of a list of vertices. Ellipses and polygons are both closed shapes, which have a fill color and a fill pattern. Lines are one dimensional and cannot be filled. Canvas windows have operations to add and delete elements.

TextWindow is a kind of a *ScrollingWindow*, which has a two-dimensional scrolling offset within its window, as specified by *xOffset* and *yOffset*, as well as an operation *scroll* to change the scroll value. A text window contains a string and has operations to insert and delete characters. *ScrollingCanvas* is a special kind of canvas that supports scrolling; it is both a *Canvas* and a *ScrollingWindow*. This is an example of *multiple inheritance*, to be explained in Chapter 4.

A *Panel* contains a set of *PanelItem* objects, each identified by a unique *itemName* within a given panel, as shown by the qualified association. Each panel item belongs to a single panel. A panel item is a predefined icon with which a user can interact on the screen. Panel items come in three kinds: buttons, choice items, and text items. A button has a string that appears on the screen; a button can be pushed by the user and has an attribute *depressed*. A choice item allows the user to select one of a set of predefined choices, each of which is a *ChoiceEntry* containing a string to be displayed and a value to be returned if the entry is selected. There are two associations between *ChoiceItem* and *ChoiceEntry*; a one-to-many as-

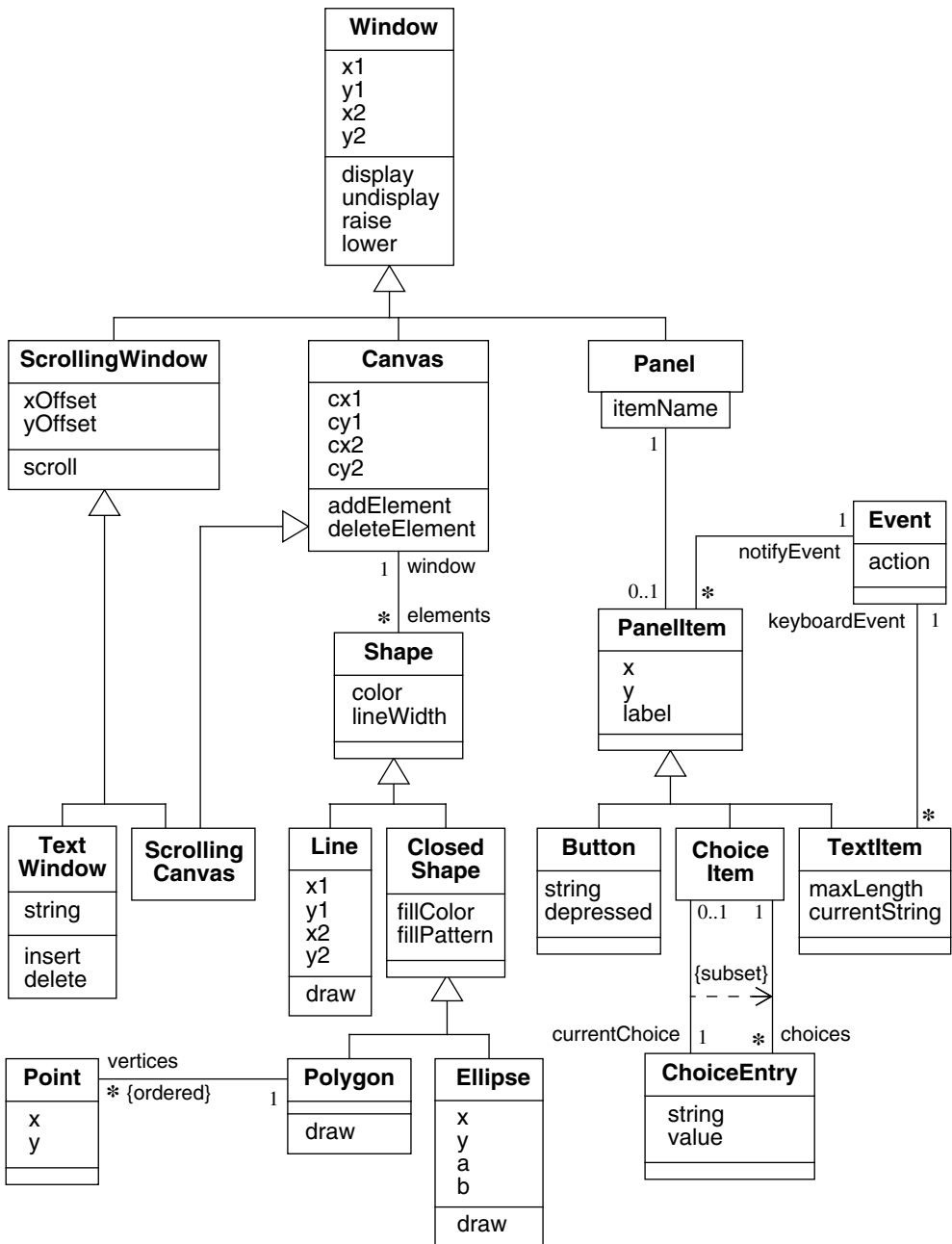


Figure 3.26 Class model of a windowing system

sociation defines the set of allowable choices, while a one-to-one association identifies the current choice. The current choice must be one of the allowable choices, so one association is a subset of the other as shown by the arrow between them labeled “{subset}.” This is an example of a constraint, to be explained in Chapter 4.

When a panel item is selected by the user, it generates an *Event*, which is a signal that something has happened together with an action to be performed. All kinds of panel items have *notifyEvent* associations. Each panel item has a single event, but one event can be shared among many panel items. Text items have a second kind of event, which is generated when a keyboard character is typed while the text item is selected. The association with end name *keyboardEvent* shows these events. Text items also inherit the *notifyEvent* from superclass *PanelItem*; the *notifyEvent* is generated when the entire text item is selected with a mouse.

There are many deficiencies in this model. For example, perhaps we should define a type *Rectangle*, which can then be used for the window and canvas boundaries, rather than having two similar sets of four position attributes. Maybe a line should be a special case of a *polyline* (a connected series of line segments), in which case both *Polyline* and *Polygon* could be subclasses of a new superclass that defines a list of points. Many attributes, operations, and classes are missing from a description of a realistic windowing system. Certainly the windows have associations among themselves, such as overlapping one another. Nevertheless, this simple model gives a flavor of the use of class modeling. We can criticize its details because it says something precise. It would serve as the basis for a fuller model.

3.5 Navigation of Class Models

So far we have shown how class models can express the structure of an application. Now we show how they can also express the behavior of navigating among classes. Navigation is important because it lets you exercise a model and uncover hidden flaws and omissions so that you can repair them. You can perform navigation manually (an informal technique) or write navigation expressions (as we will explain).

Consider the simple model for credit card accounts in Figure 3.27. An institution may issue many credit card accounts, each identified by an account number. Each account has a maximum credit limit, a current balance, and a mailing address. The account serves one or more customers who reside at the mailing address. The institution periodically issues a statement for each account. The statement lists a payment due date, finance charge, and minimum payment. The statement itemizes various transactions that have occurred throughout the billing interval: cash advances, interest charges, purchases, fees, and adjustments to the account. The name of the merchant is printed for each purchase.

We can pose a variety of questions against the model.

- What transactions occurred for a credit card account within a time interval?
- What volume of transactions were handled by an institution in the last year?
- What customers patronized a merchant in the last year by any kind of credit card?

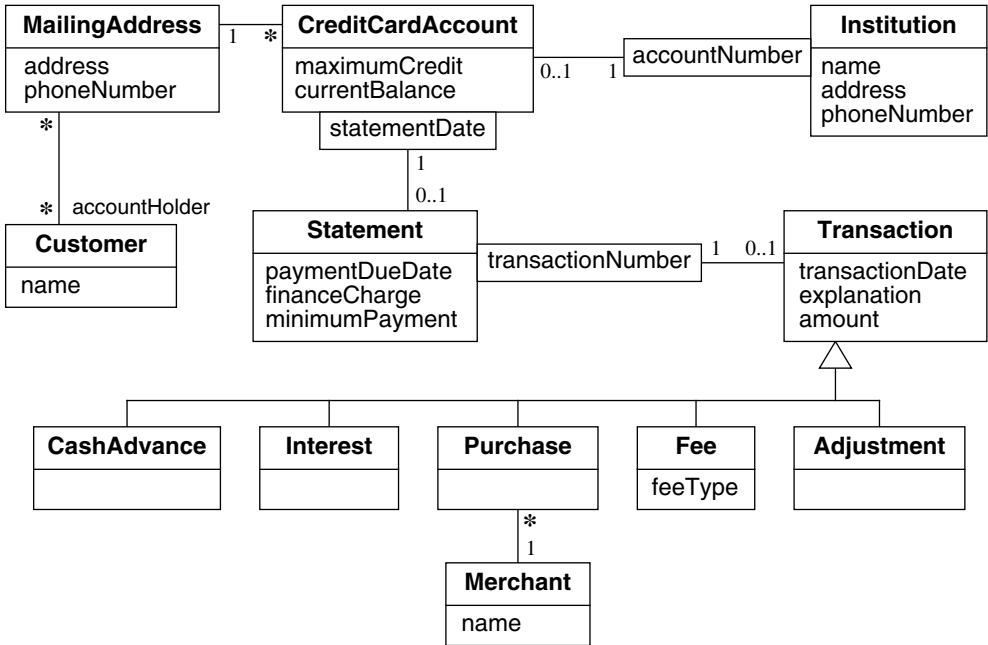


Figure 3.27 Class model for managing credit card accounts

- How many credit card accounts does a customer currently have?
- What is the total maximum credit for a customer, for all accounts?

The UML incorporates a language that can express these kinds of questions—the *Object Constraint Language (OCL)* [Warmer-99]. The next two sections discuss the OCL, and Section 3.5.3 then expresses the credit card questions using the OCL. By no means do we cover the complete OCL; we just cover the portions relevant to traversing class models.

3.5.1 OCL Constructs for Traversing Class Models

The OCL can traverse the constructs in class models.

- **Attributes.** You can traverse from an object to an attribute value. The syntax is the source object, followed by a dot, and then the attribute name. For example, the expression *aCreditCardAccount.maximumCredit* takes a *CreditCardAccount* object and finds the value of *maximumCredit*. (We use the convention of preceding a class name by “a” to refer to an object.) Similarly, you can access an attribute for each object in a collection, returning a collection of attribute values. In addition, you can find an attribute value for a link, or a collection of attribute values for a collection of links.
- **Operations.** You can also invoke an operation for an object or a collection of objects. The syntax is the source object or object collection, followed by a dot, and then the operation. An operation must be followed by parentheses, even if it has no arguments, to

avoid confusion with attributes. You may invoke operations from your class model or predefined operations that are built into the OCL.

The OCL has special operations that operate on entire collections (as opposed to operating on each object in a collection). For example, you can count the objects in a collection or sum a collection of numeric values. The syntax for a collection operation is the source object collection, followed by “->”, and then the operation.

- **Simple associations.** A third use of the *dot* notation is to traverse an association to a target end. The target end may be indicated by an association end name or, where there is no ambiguity, a class name. In the example, *aCustomer.MailingAddress* yields a set of addresses for a customer (the target end has “many” multiplicity). In contrast, *aCreditCardAccount.MailingAddress* yields a single address (the target end has multiplicity of one).
- **Qualified associations.** A qualifier lets you make a more precise traversal. The expression *aCreditCardAccount.Statement[30 November 1999]* finds the statement for a credit card account with the statement date of 30 November 1999. The syntax is to enclose the qualifier value in brackets. Alternatively, you can ignore the qualifier and traverse a qualified association as if it were a simple association. Thus the expression *aCreditCardAccount.Statement* finds the multiple statements for a credit card account. (The multiplicity is “many” when the qualifier is not used.)
- **Association classes.** Given a link of an association class, you can find the constituent objects. Alternatively, given a constituent object, you can find the multiple links of an association class.
- **Generalizations.** Traversal of a generalization hierarchy is implicit for the OCL notation.
- **Filters.** There is often a need to filter the objects in a set. The OCL has several kinds of filters, the most common of which is the *select* operation. The *select* operation applies a predicate to each element in a collection and returns the elements that satisfy the predicate. For example, *aStatement.Transaction->select(amount>\$100)* finds the transactions for a statement in excess of \$100.

3.5.2 Building OCL Expressions

The real power of the OCL comes from combining primitive constructs into expressions. For example, an OCL expression could chain together several association traversals. There could be several qualifiers, filters, and operators as well.

With the OCL, a traversal from an object through a single association yields a singleton or a set (or a bag if the association has the annotation *{bag}* or *{sequence}*). In general, a traversal through multiple associations can yield a bag (depending on the multiplicities), so you must be careful with OCL expressions. A set is a collection of elements without duplicates. A bag is a collection of elements with duplicates allowed.

The example in Figure 3.28 illustrates how an OCL expression can yield a bag. A company might want to send a single mailing to each stockholder address. Starting with the GE company, we traverse the *OwnsStock* association and get a set of three persons. Starting with

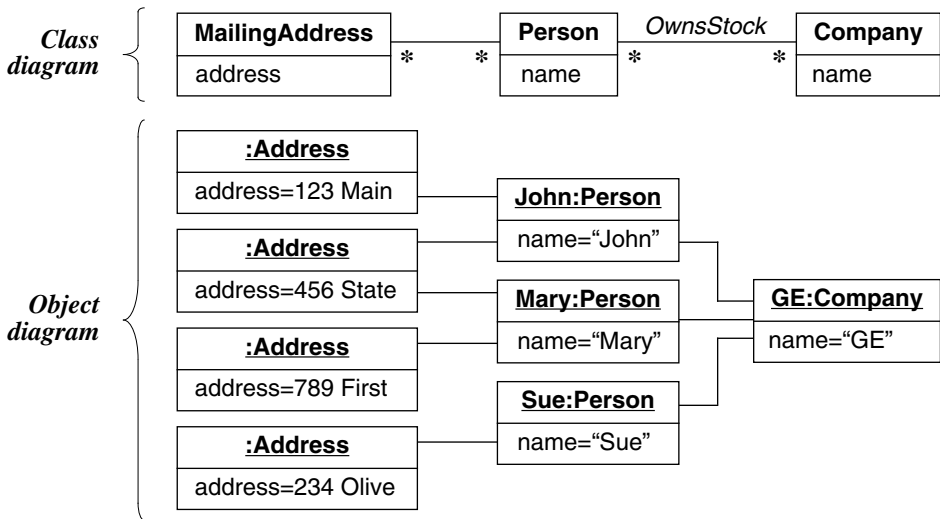


Figure 3.28 A sample model and examples. Traversal of multiple associations can yield a bag.

these three persons and traversing to mailing address, we get a bag obtaining the address *456 State* twice.

[Warmer-99] does not mention null values, since they only discuss the specification of constraints for a correctly implemented system. (*Null* is a special value denoting that an attribute value is unknown or not applicable.) Handling of exceptions and run-time errors is also outside the scope of their book.

In contrast, the purpose in this chapter is not to specify constraints, but rather to discuss navigation of class models. Nulls do not arise for properly phrased and valid constraints. But they certainly do arise with model navigation. For example, a person may lack a mailing address. We extend the meaning of OCL expressions to accommodate nulls—a traversal may yield a null value, and an OCL expression evaluates to null if the source object is null.

3.5.3 Examples of OCL Expressions

We can use the OCL to answer the credit card questions.

- What transactions occurred for a credit card account within a time interval?

```

aCreditCardAccount.Statement.Transaction->
select(aStartDate <= transactionDate and
transactionDate <= anEndDate)
  
```

The expression traverses from a *CreditCardAccount* object to *Statement* and then to *Transaction*, resulting in a set of transactions. (Traversal of the two associations results in a set, rather than a bag, because both associations are one-to-many.) Then we use the OCL *select* operator (a collection operator) to find the transactions within the time interval bounded by *aStartDate* and *anEndDate*.

- What volume of transactions were handled by an institution in the last year?

```
anInstitution.CreditCardAccount.Statement.Transaction->
select(aStartDate <= transactionDate and
transactionDate <= anEndDate).amount->sum()
```

The expression traverses from an *Institution* object to *CreditCardAccount*, then to *Statement*, and then to *Transaction*. (Traversal results in a set, rather than a bag, because all three associations are one-to-many.) The OCL *select* operator finds the transactions within the time interval bounded by *aStartDate* and *anEndDate*. (We choose to make the time interval more general than *last year*.) Then we find the amount for each transaction and compute the total with the OCL *sum* operator (a collection operator).

- What customers patronized a merchant in the last year by any kind of credit card?

```
aMerchant.Purchase->
select(aStartDate <= transactionDate and
transactionDate <= anEndDate).Statement.
CreditCardAccount.MailingAddress.Customer->asSet()
```

The expression traverses from a *Merchant* object to *Purchase*. The OCL *select* operator finds the transactions within the time interval bounded by *aStartDate* and *anEndDate*. (Traversal across a generalization, from *Purchase* to *Transaction*, is implicit in the OCL.) For these transactions, we then traverse to *Statement*, then to *CreditCardAccount*, then to *MailingAddress*, and finally to *Customer*. The association from *MailingAddress* to *Customer* is many-to-many, so traversal to *Customer* yields a bag. The OCL *asSet* operator converts a bag of customers to a set of customers, resulting in our answer.

- How many credit card accounts does a customer currently have?

```
aCustomer.MailingAddress.CreditCardAccount->size()
```

Given a *Customer* object, we find a set of *MailingAddress* objects. Then, given the set of *MailingAddress* objects, we find a set of *CreditCardAccount* objects. (This traversal yields a set, and not a bag, because each *CreditCardAccount* pertains to a single *MailingAddress*.) For the set of *CreditCardAccount* objects we apply the OCL *size* operator, which returns the cardinality of the set.

- What is the total maximum credit for a customer, for all accounts?

```
aCustomer.MailingAddress.CreditCardAccount.
maximumCredit->sum()
```

The expression traverses from a *Customer* object to *MailingAddress*, and then to *CreditCardAccount*, yielding a set of *CreditCardAccount* objects. For each *CreditCardAccount*, we find the value of *maximumCredit* and compute the total with the OCL *sum* operator.

Note that these kinds of questions exercise a model and uncover hidden flaws and omissions that can then be repaired. For example, the query on the number of credit card accounts suggests that we may need to differentiate past accounts from current accounts.

Keep in mind that the OCL was originally intended as a constraint language (see Chapter 4). However, as we explain here, the OCL is also useful for navigating models.

3.6 Practical Tips

We have gleaned the following tips for constructing class models from our application work. Many of these tips have been mentioned throughout the chapter.

- **Scope.** Don't begin class modeling by merely jotting down classes, associations, and inheritance. First, you must understand the problem to be solved. The content of a model is driven by relevance to the solution. You must exercise judgment in deciding which objects to show and which objects to ignore. A model represents only the relevant aspects of a problem. (Section 3.1.1)
- **Simplicity.** Strive to keep your models simple. A simple model is easier to understand and takes less development effort. Try to use a minimal number of classes that are clearly defined and not redundant. Be suspicious of classes that are difficult to define. You may need to reconsider such classes and restructure the model.
- **Diagram layout.** Draw your diagrams in a manner that elicits symmetry. Often there is a superstructure to a problem that lies outside the notation. Try to position important classes so that they are visually prominent on a diagram. Try to avoid crossing lines.
- **Names.** Carefully choose names. Names are important and carry powerful connotations. Names should be descriptive, crisp, and unambiguous. Do not bias names toward one aspect of an object. Choosing good names is one of the most difficult aspects of modeling. You should use singular nouns for the names of classes.
- **References.** Do not bury object references inside objects as attributes. Instead, model these as associations. This is clearer and captures the true intent rather than an implementation approach. (Section 3.2.1)
- **Multiplicity.** Challenge association ends with a multiplicity of one. Often the object on either end is optional and zero-or-one multiplicity may be more appropriate. Other times "many" multiplicity is needed. (Section 3.2.2)
- **Association end names.** Be alert for multiple uses of the same class. Use association end names to unify references to the same class. (Section 3.2.3)
- **Bags and sequences.** An ordinary binary association has at most one link for a pair of objects. However, you can permit multiple links for a pair of objects by annotating an association end with *{bag}* or *{sequence}*. (Section 3.2.5)
- **Attributes of associations.** During analysis, do not collapse attributes of associations into one of the related classes. You should directly describe the objects and links in your models. During design and implementation, you can always combine information for more efficient execution. (Section 3.2.6)
- **Qualified associations.** Challenge association ends with a multiplicity of "many." A qualifier can often improve the precision of an association and highlight important navigation paths. (Section 3.2.7)
- **Generalization levels.** Try to avoid deeply nested generalizations. (Section 3.3.1)

- **Overriding features.** You may override methods and default values of attributes. However, you should never override a feature so that it is inconsistent with the signature or semantics of the original inherited feature. (Section 3.3.3)
- **Reviews.** Try to get others to review your models. Expect that your models will require revision. Class models require revision to clarify names, improve abstraction, repair errors, add information, and more accurately capture structural constraints. Nearly all of our models have required several revisions.
- **Documentation.** Always document your models. The diagram specifies the structure of a model but cannot describe the rationale. The written explanation guides the reader and explains subtle reasons for why the model was constructed a particular way.

3.7 Chapter Summary

Class models describe the static data structure of objects and their relationships to one another. The content of a model is a matter of judgment and is driven by the needs of an application. An object is a concept, abstraction, or thing with identity that has meaning for an application. A class describes a group of objects with the same attributes, behavior, kinds of relationships, and semantics. An attribute is a named property of a class that describes a value held by each object of the class. An operation is a function or procedure that may be applied to or by objects in a class.

A link is a physical or conceptual connection among objects and is an instance of an association. An association is a description of a group of links with common structure and semantics. An association describes a set of potential links in the same way that a class describes a set of potential objects. An association is a logical construct, of which a reference is an implementation alternative. There are other ways of implementing associations besides using references.

You can refer to an end of an association and give it a name and multiplicity. Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class. An association class is an association that is also a class; an association class may have attributes, operations, and participate in associations. A qualified association is an association in which the objects in a “many” association end are partially or fully disambiguated by an attribute called the qualifier. The qualifier selects among the target objects, reducing the effective multiplicity, often from “many” to “one.” Names are often qualifiers.

Generalization is the relationship between a class (the superclass) and one or more variations of the class (the subclasses). Generalization organizes classes by their similarities and differences, structuring the description of objects. A subclass inherits the attributes, operations, and associations of its superclasses. Through inheritance, a subclass can reuse superclass properties or override them; a subclass can add new properties.

Generalization is an important construct for both conceptual modeling and implementation. During conceptual modeling, generalization lets the developer organize classes on the basis of similarities and differences. During implementation, inheritance facilitates polymorphism and code reuse. Inheritance may occur across an arbitrary number of levels, where

each level represents one aspect of an object. An object accumulates attributes, operations, and associations from each level of a generalization hierarchy.

Class models are useful for more than just data structure. In particular, navigation of class models lets you express certain behavior. Furthermore, navigation exercises a class model and uncovers hidden flaws and omissions, which you can then repair. The UML incorporates a language that can be used for navigation, the Object Constraint Language (OCL).

The various class modeling constructs work together to describe a complex system precisely, as shown by our example of a model for a windowing system. Once a model is available, even a simplified one, you can compare it against the requirements of an application, criticize it, and improve it.

ancestor	default value	link	polymorphism
association	descendant	method	qualified association
association class	direction	multiplicity	qualifier
association end	feature	navigation	sequence
attribute	generalization	object	signature
bag	generalization set name	object diagram	specialization
class	identity	operation	subclass
class diagram	inheritance	ordering	superclass
class model	instance	override	value

Figure 3.29 Key concepts for Chapter 3

Bibliographic Notes

The class modeling approach described in this book builds on the OMT notation originally proposed in [Loomis-87], which has now been superseded by the UML [Booch-99] [Rumbaugh-05] [UML]. The UML *class model* corresponds to the *OMT notation* discussed in [Loomis-87]. [Blaha-98] also covers the UML class modeling notation with an emphasis on the constructs that are relevant to database applications.

The class modeling notation is one of a score of approaches descended from the seminal entity-relationship (ER) model of [Chen-76]. All the descendants attempt to improve on the ER approach. Enhancements to the ER model have been pursued for several reasons. The ER technique has been successful for database modeling and as a result, there has been great demand for additional power. Also, ER modeling addresses only database design and not programming. There are too many extensions to ER for us to discuss them here.

A noteworthy aspect of the OMT notation and its successor UML is the emphasis on associations. As with inheritance, associations are important for conceptual modeling and implementation. [Rumbaugh-87] is the original source of the association ideas. The use of the term *relation* in [Rumbaugh-87] is synonymous with our use of *association* in this book.

In the data modeling notations, such as ER and IDEF1X, a binary association has at most one link for a pair of objects. UML1 follows the data modeling convention and also

restricts a binary association to at most one link for a pair of objects. Note that UML2 has an exception to this behavior. In UML2 a binary association with the annotation *{bag}* or *{sequence}* can have multiple links for a pair of objects.

[Khoshafian-86] defines the concept of object identity and its importance to programming languages and database systems.

[Warmer-99] is the reference for the Object Constraint Language (OCL) that is part of the UML. We use the OCL in this chapter for navigating class models.

[Rayside-00] compares OO concepts with philosophy. He emphasizes the importance of crisp names and clear thinking.

[Chonoles-03], [Fowler-00], and [Larman-02] are additional books that you can read to help you learn about the UML. We thank Michael Chonoles for the example (Figure 3.10, Figure 3.11) clarifying that each association has at most one link between a given pair of objects (other than bags and sequences).

References

- [Blaha-98] Michael Blaha and William Premerlani. *Object-Oriented Modeling and Design for Database Applications*. Upper Saddle River, NJ: Prentice Hall, 1998.
- [Booch-99] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. Boston: Addison-Wesley, 1999.
- [Chen-76] P.P.S. Chen. The Entity-Relationship model—toward a unified view of data. *ACM Transactions on Database Systems* 1, 1 (March 1976), 9–36.
- [Chonoles-03] Michael Jesse Chonoles and James A. Schardt. *UML2 for Dummies*. New York: Wiley, 2003.
- [Fowler-00] Martin Fowler. *UML Distilled, Second Edition*. Boston: Addison-Wesley, 2000.
- [Khoshafian-86] S.N. Khoshafian and G.P. Copeland. Object identity. *OOPSLA'86 as ACM SIGPLAN* 21, 11 (November 1986), 406–416.
- [Larman-02] Craig Larman. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process*. Upper Saddle River, NJ: Prentice Hall, 2002.
- [Loomis-87] Mary E.S. Loomis, Ashwin V. Shah, and James E. Rumbaugh. An object modeling technique for conceptual design. *European Conference on Object-Oriented Programming*, Paris, France, June 15–17, 1987, published as *Lecture Notes in Computer Science*, 276, Springer-Verlag, 192–202.
- [Rayside-00] Derek Rayside and Gerard Campbell. An Aristotelian understanding of object-oriented programming. *OOPSLA'00 as ACM SIGPLAN* 35, 10 (October 2000), 337–353.
- [Rumbaugh-87] James E. Rumbaugh. Relations as semantic constructs in an object-oriented language. *OOPSLA'87 as ACM SIGPLAN* 22, 12 (December 1987), 466–481.
- [Rumbaugh-05] James Rumbaugh, Ivar Jacobson, Grady Booch. *The Unified Modeling Language Reference Manual, Second Edition*. Boston: Addison-Wesley, 2005.
- [UML] www.uml.org
- [Warmer-99] Jos Warmer and Anneke Kleppe. *The Object Constraint Language*. Boston: Addison-Wesley, 1999.

Exercises

- 3.1 (3) Prepare a class diagram from the object diagram in Figure E3.1.

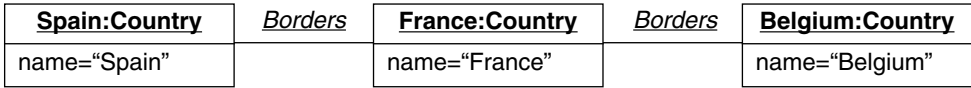


Figure E3.1 Object diagram for a portion of Europe

- 3.2 (5) Prepare a class diagram from the object diagram in Figure E3.2. Explain your multiplicity decisions. Each point has an x coordinate and a y coordinate. What is the smallest number of points required to construct a polygon? Does it make a difference whether or not a point may be shared between polygons? Your answer should address the fact that points are ordered.

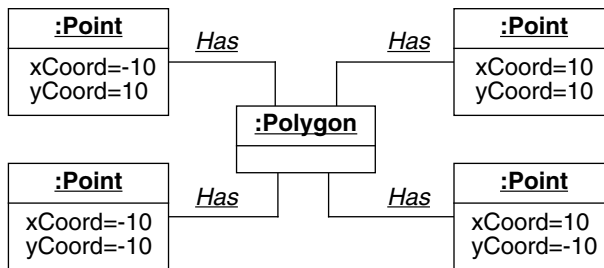


Figure E3.2 Object diagram for a polygon that happens to be a square

- 3.3 (5) Using your class diagram for Exercise 3.2, prepare an object diagram for two triangles with a common side under the following conditions.
- A point belongs to exactly one polygon.
 - A point belongs to one or more polygons.
- 3.4 (5) Prepare a class diagram from the object diagram in Figure E3.3. How does your diagram express the fact that points are ordered? Assume that a point belongs to at most one polygon.
- 3.5 (2) Prepare a written description for the class diagrams from Exercise 3.2 and Exercise 3.4.
- 3.6 (6) Prepare a class diagram from the object diagram in Figure E3.4.
- 3.7 (5) Prepare a class diagram from the object diagram in Figure E3.5. This particular document has 4 pages. The first page has a red point and a yellow square displayed on it. The second page contains a line and an ellipse. An arc, a circle, and a rectangle appear on the last two pages. In preparing your diagram, use one or more generalizations.
- 3.8 (4) Figure E3.6 is a partially completed class diagram of an air transportation system. Multiplicity has been omitted. Add multiplicity to the diagram. Demonstrate how multiplicity decisions depend on your perception of the world.
- 3.9 (3) Add association names to the unlabeled associations in Figure E3.6.

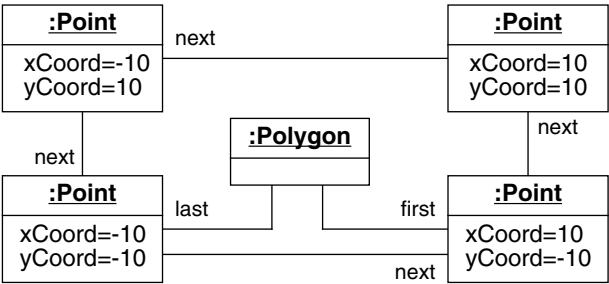


Figure E3.3 Object diagram for a polygon that happens to be a square

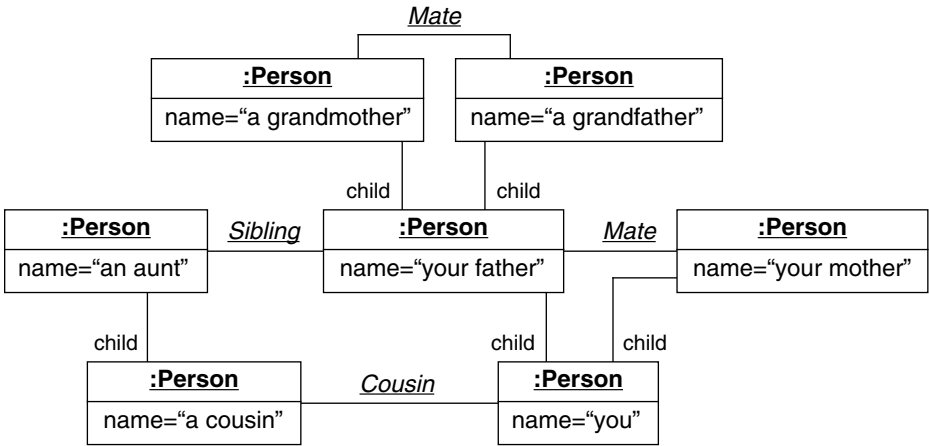


Figure E3.4 Object diagram for part of your family tree

- 3.10 (3) Add association end names to Figure E3.6. Add only meaningful names that are different from the class names. You should add at least six association end names to the diagram.
- 3.11 (2) Add the following operations to the class diagram in Figure E3.6: heat, hire, fire, refuel, reserve, clean, de-ice, take off, land, repair, cancel, delay. It is permissible to add an operation to more than one class.
- 3.12 (6) Prepare an object diagram for an imaginary round trip you took last weekend to London. Include at least one instance of each class. Fortunately, direct flights on a hypersonic plane were available. A friend went with you but decided to stay a while and is still there. Captain Johnson was your pilot on both flights. You had a different seat each way, but you noticed it was on the same plane because of a distinctive dent in the tail section. Students should indicate unknown values with a “?”.
- 3.13 Prepare a class diagram for each group of classes. Add at least 10 relationships (associations and generalizations) to each diagram. Use association names and association end names where

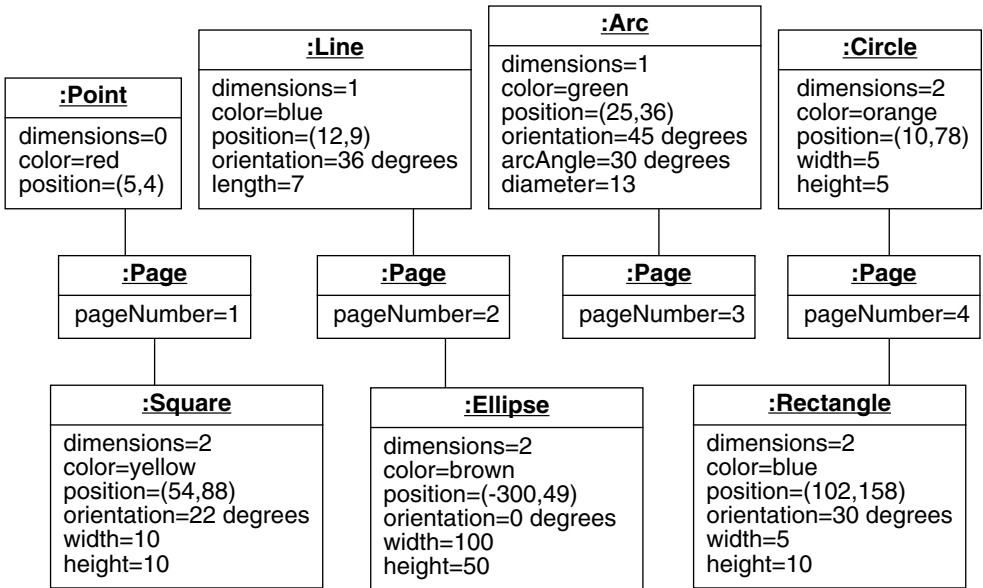


Figure E3.5 Object diagram for a geometrical document

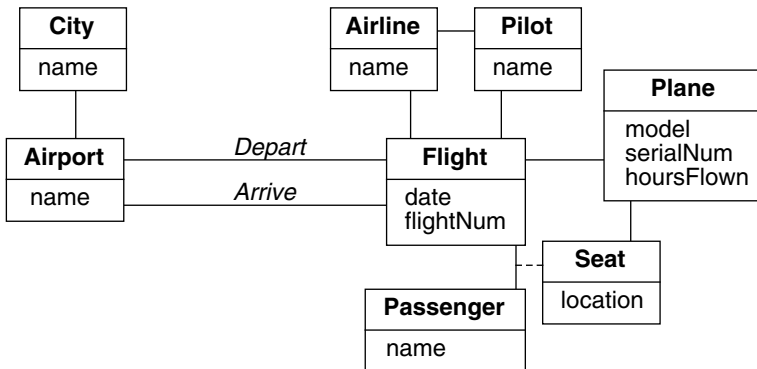


Figure E3.6 Partially completed class model of an air transportation system

needed. Also use qualified associations and show multiplicity. You do not need to show attributes or operations. As you prepare the diagrams, you may add classes. Be sure to explain your diagrams.

- (6) school, playground, principal, school board, classroom, book, student, teacher, cafeteria, restroom, computer, desk, chair, ruler, door, swing
- (4) automobile, engine, wheel, brake, brake light, door, battery, muffler, tail pipe
- (4) castle, moat, drawbridge, tower, ghost, stairs, dungeon, floor, corridor, room, window, stone, lord, lady, cook

- d. (8) expression, constant, variable, function, argument list, relational operator, term, factor, arithmetic operator, statement, computer program
 - e. (6) file system, file, ASCII file, binary file, directory file, disc, drive, track, sector
 - f. (4) gas furnace, blower, blower motor, room thermostat, furnace thermostat, humidifier, humidity sensor, gas control, blower control, hot air vent
 - g. (7) chess piece, rank, file, square, board, move, tree of moves
 - h. (4) sink, freezer, refrigerator, table, light, switch, window, smoke alarm, burglar alarm, cabinet, bread, cheese, ice, door, kitchen
- 3.14 (4) Add at least 10 attributes and at least 5 methods to each of the class diagrams you prepared in the previous exercise.
- 3.15 (6) Figure E3.7 is a portion of a class diagram for a computer program for playing several types of card games. Deck, hand, discard pile, and draw pile are collections of cards. The initial size of a hand depends on the type of game. Each card has a suit and rank. Add the following operations to the diagram: *display*, *shuffle*, *deal*, *initialize*, *sort*, *topOfPile*, *bottomOfPile*, *insert*, *draw*, and *discard*. Some operations may appear in more than one class. For each class in which an operation appears, describe the arguments to the operation and what the operation should do to an instance of that class.

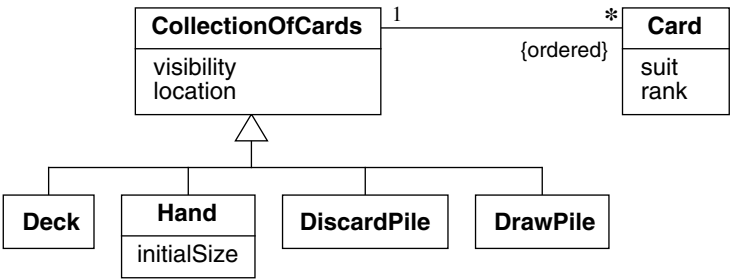


Figure E3.7 Portion of a class diagram for a card-playing system

- 3.16 (5) Figure E3.8 is a portion of a class diagram for a computer system for laying out a newspaper. The system handles newspaper pages which may contain, among other things, columns of text. The user may edit the width and length of a column of text, move it around on a page, or move it from one page to another. As shown, a column is displayed on exactly one page.

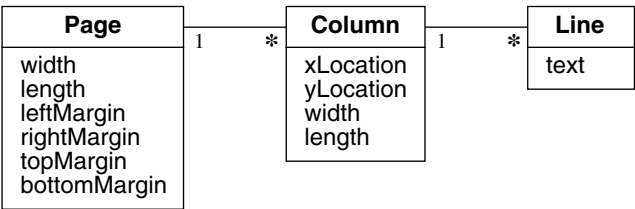


Figure E3.8 Portion of a class diagram for a newspaper publishing system

Modify the class diagram so that portions of the same column may appear on more than one page. If the user edits the text on one page, the changes should appear automatically on other pages. You should change x location and y location into attributes of an association.

- 3.17 (6) Figure E3.9 is a class diagram that might be used in developing a system to simplify the scheduling and scoring of judged athletic competitions such as gymnastics, diving, and figure skating. There are multiple events and competitors. Each competitor may enter several events and each event has many competitors.

Each event has several judges who subjectively rate the performance of competitors in that event. A judge rates every competitor for an event. In some cases, a judge may score more than one event.

Trials are the focus of the competition. Each trial is an attempt by one competitor to perform his or her best in one event. A trial is scored by the panel of judges for that event and a net score determined. Add multiplicity to the diagram.

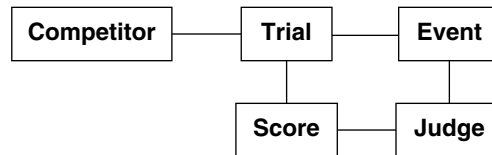


Figure E3.9 Portion of a class diagram for an athletic-event scoring system

- 3.18 (3) Add the following attributes to Figure E3.9: address, age, date, difficulty factor, score, and name. In some cases, you may wish to use the same attribute in more than one class.
- 3.19 (3) Add an association to Figure E3.9 to make it possible to determine a competitor's intended events before trials are held.
- 3.20 (6) Prepare a class model to describe undirected graphs. An undirected graph consists of a set of vertices and a set of edges. Edges connect pairs of vertices. Your model should capture only the structure of graphs (i.e., connectivity) and need not be concerned with layout such as location of vertices or lengths of edges. Figure E3.10 shows a typical undirected graph.

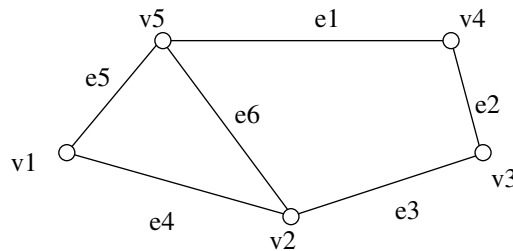


Figure E3.10 Sample undirected graph

- 3.21 (4) Prepare an object diagram for Figure E3.10. [Instructor's note: You may want to give the students our answer to Exercise 3.20.]

- 3.22 (5) Extend the class diagram you prepared in Exercise 3.20 with layout details, including locations of vertices and thickness and color of edges. Also add names of vertices and edges. [Instructor’s note: You may want to give the students our answer to Exercise 3.20.]
- 3.23 (7) Prepare a class model to describe directed graphs. A directed graph is similar to an undirected graph, except the edges are oriented. Figure E3.11 shows a typical directed graph.

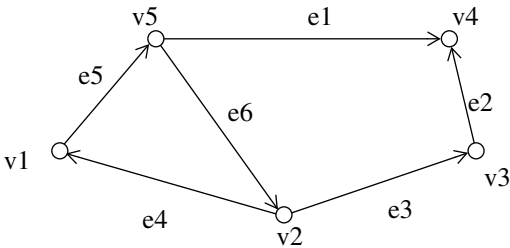


Figure E3.11 Sample directed graph

- 3.24 (4) Prepare an object diagram for Figure E3.11. [Instructor’s note: You may want to give the students our answer to Exercise 3.23.]
- 3.25 (7) Several classes shown in Figure E3.12 have attributes that are really references to other classes and could be replaced with associations. A person may have up to three companies as employers. Each person has an ID. A car is assigned an ID. Cars may be owned by persons, companies, or banks. Owner ID refers to the ID of the person, company, or bank who owns the car. A car loan may be involved in the purchase of a car.

Burying object references as references is the incorrect way to construct a model. Prepare a class diagram without IDs and using association and generalization. Try to assign multiplicities. You may need to add one or more classes of your own.

Person	Car	CarLoan	Company	Bank
name birthdate employer1ID employer2ID employer3ID personID address	ownerID vehicleID ownerType model year	vehicleID customerType customerID accountNumber bankID interestRate currentBalance	name companyID	name bankID

Figure E3.12 Classes with some attributes that are references.

- 3.26 (4) A problem arises when several independent systems need to identify the same object. For example, the department of motor vehicles, an insurance company, a bank, and the police may wish to identify a given motor vehicle. Discuss the relative merits of using the following identification methods:
- a. Identify by its owner
 - b. Identify by attributes such as manufacturer, model, and year

- c. Use the vehicle identification number (VIN) assigned to the car by its manufacturer
- d. Use IDs generated internally by each interested agency

3.27 (7) Prepare a class model that might be used to troubleshoot a 4-cycle lawn mower engine. Use three separate diagrams for the model, with one diagram for each of the following paragraphs.

Power is developed in such an engine by the combustion of a mixture of air and gasoline against a piston. The piston is attached to a crankshaft via a connecting rod, and it moves up and down inside a cylinder as the shaft rotates. As the piston moves down, an intake valve opens, allowing the piston to draw a mixture of fuel and air into the cylinder. At the bottom of the stroke, the intake valve closes. The piston compresses and heats the mixture as it moves upward. Rings in grooves around the piston rub against the cylinder wall, providing a seal necessary for compression and spreading lubricating oil. At the top of the stroke, an electrical spark from a spark plug detonates the mixture. The expanding gases develop power during the downward stroke. At the bottom, an exhaust valve is opened. On the next upward stroke, the exhaust gases are driven out.

Fuel is mixed with air in a carburetor. Dust and dirt in the air, which could cause excessive mechanical wear, are removed by an air filter. The optimum ratio of fuel to air is set by adjusting a tapered mixture screw. A throttle plate controls the amount of mixture pulled into the cylinder. The throttle plate, in turn, is controlled through springs by the operator throttle control and a governor, a mechanical device which stabilizes the engine speed under varying mechanical loads. Intake and exhaust valves are normally held closed by springs and are opened at the right time by a cam shaft, which is gear driven by the crankshaft.

The electrical energy for the spark is provided and timed by a magnet, coil, condenser, and a normally closed switch called the points. The coil has a low-voltage primary circuit connected to the points and a high-voltage secondary connected to the spark plug. The magnet is mounted on a flywheel and as it rotates past the coil, it induces a current in the shorted primary circuit. The points are driven open at the right instant by a cam on the crankshaft. With the aid of the condenser, they interrupt the current in the primary circuit, inducing a high-voltage pulse in the secondary.

3.28 (6) Prepare a class diagram for the dining philosopher problem. There are 5 philosophers and 5 forks around a circular table. Each philosopher has access to 2 forks, one on either side. Each fork is shared by 2 philosophers. Each fork may be either on the table or in use by one philosopher. A philosopher must have 2 forks to eat.

3.29 (7) The tower of Hanoi is a problem frequently used to teach recursive programming techniques. The goal is to move a stack of disks from one of three long pegs to another, using the third peg for maneuvering. Each disk is a different size. Disks may be moved from the top of a stack on a peg to the top of the stack on any other peg, one at a time, provided a disk is never placed on another disk that is smaller than itself. The details of the algorithm for listing the series of required moves depend on the structure of the class diagram used. Prepare class diagrams for each of the following descriptions. Show classes and associations. Do not show attributes or operations:

- a. A tower consists of 3 pegs. Each peg has several disks on it, in a certain order.
- b. A tower consists of 3 pegs. Disks on the pegs are organized into subsets called stacks. A stack is an ordered set of disks. Every disk is in exactly one stack. A peg may have several stacks on it, in order.
- c. A tower consists of 3 pegs. Disks on the pegs are organized into subsets called stacks, as in (b), with several stacks on a peg. However, the structure of a stack is recursive. A stack con-

sists of one disk (the disk that is physically on the bottom of the stack) and zero or one stack, depending on the height of the stack.

- d. Similar to (c), except only one stack is associated with a peg. Other stacks on the peg are associated in a linked list.
- 3.30 (8) The recursive algorithm for producing the series of moves described in the previous exercise focuses on a stack of disks. To move a stack of height N , where $N > 1$, first move the stack of height $N - 1$ to the free peg using a recursive call. Then move the bottom disk to the desired peg. Finally, move the stack on the free peg to the desired peg. The recursion terminates, because moving a stack of height 1 is trivial. Which one of the several class diagrams that you prepared in the previous exercise is best suited for this algorithm? Discuss why. Also, add attributes and operations to the diagram. What are the arguments for each operation? Describe what each operation is supposed to do to each class for which it is defined.
- 3.31 (6) Consider Figure E3.6. Write an OCL expression to compute the set of names of airlines that a person flew in a given year. Assume you have a function *getYear(date)* that extracts the year given a date. (Instructor’s note: You should give the students our answer to Exercise 3.10 as the basis for this exercise.)
- 3.32 (6) Consider Figure E3.6. Write an OCL expression to find the nonstop flights from *aCity1* to *aCity2*. (Instructor’s note: You should give the students our answer to Exercise 3.10 as the basis for this exercise.)
- 3.33 (6) Consider Figure E3.9. Write an OCL expression to find the total score a competitor received from a judge. (Instructor’s note: You should give the students our answer to Exercise 3.18 as the basis for this exercise.)
- 3.34 (6) Compare the class models in Figure E3.13. The left model represents *Subscription* as an association class; the right model treats *Subscription* as an ordinary class.
- A person may have multiple magazine subscriptions. A magazine has multiple subscribers. For each subscription, it is important to track the date and amount of each payment as well as the current expiration date.

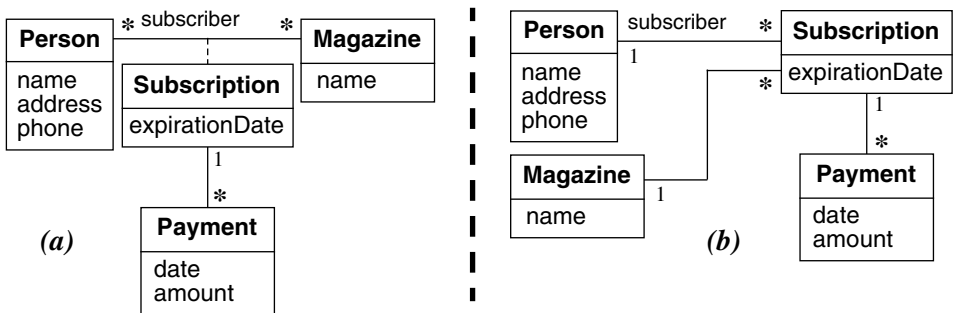


Figure E3.13 Class diagram for magazine subscriptions

4

Advanced Class Modeling

Chapter 4 continues our discussion of class modeling concepts with a treatment of advanced topics. This chapter provides subtleties for improved modeling that you can skip upon a first reading of this book.

4.1 Advanced Object and Class Concepts

4.1.1 Enumerations

A data type is a description of values. Data types include numbers, strings, and enumerations. An *enumeration* is a data type that has a finite set of values. For example, the attribute *accessPermission* in Figure 3.17 is an enumeration with possible values that include *read* and *read-write*. Figure 3.25 also has some enumerations that Figure 4.1 illustrates. *Figure.penType* is an enumeration that includes *solid*, *dashed*, and *dotted*. *TwoDimensional.fillType* is an enumeration that includes *solid*, *grey*, *none*, *horizontal lines*, and *vertical lines*.

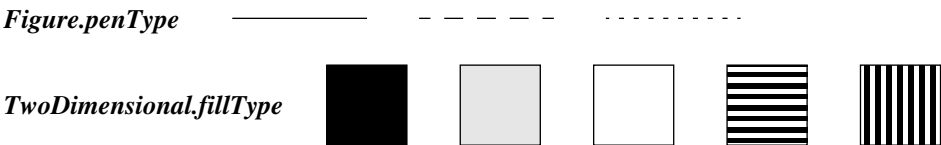


Figure 4.1 Examples of enumerations. Enumerations often occur and are important to users. Implementations must enforce the finite set of values.

When constructing a model, you should carefully note enumerations, because they often occur and are important to users. Enumerations are also significant for an implementation;

you may display the possible values with a pick list and you must restrict data to the legitimate values.

Do not use a generalization to capture the values of an enumerated attribute. An enumeration is merely a list of values; generalization is a means for structuring the description of objects. You should introduce generalization only when at least one subclass has significant attributes, operations, or associations that do not apply to the superclass. As Figure 4.2 shows, you should not introduce a generalization for *Card*, because most games do not differentiate the behavior of spades, clubs, hearts, and diamonds.

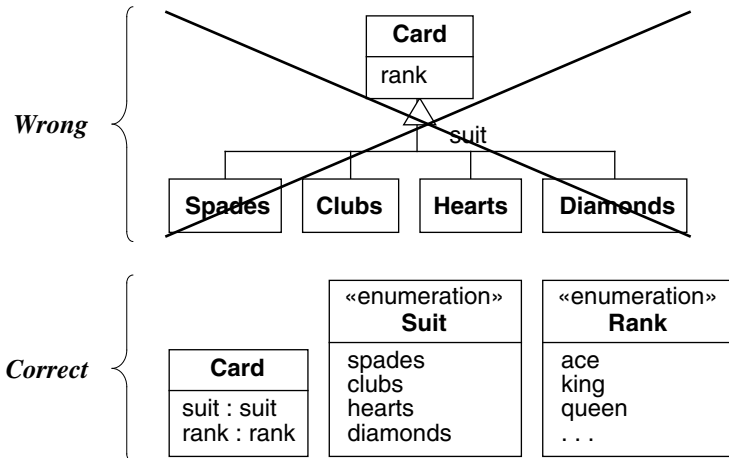


Figure 4.2 Modeling enumerations. Do not use a generalization to capture the values of an enumerated attribute.

In the UML an enumeration is a data type. You can declare an enumeration by listing the keyword *enumeration* in guillemets («») above the enumeration name in the top section of a box. The second section lists the enumeration values.

4.1.2 Multiplicity

Multiplicity is a constraint on the cardinality of a set. Chapter 3 explained multiplicity for associations. Multiplicity also applies to attributes.

It is often helpful to specify multiplicity for an attribute, especially for database applications. **Multiplicity for an attribute** specifies the number of possible values for each instantiation of an attribute. The most common specifications are a mandatory single value [1], an optional single value [0..1], and many [*]. Multiplicity specifies whether an attribute is mandatory or optional (in database terminology whether an attribute can be null). Multiplicity also indicates if an attribute is single valued or can be a collection. If not specified, an attribute is assumed to be a mandatory single value ([1]). In Figure 4.3 a person has one name, one or more addresses, zero or more phone numbers, and one birthdate.

Person
name : string [1] address : string [1..*] phoneNumber : string [*] birthDate : date [1]

Figure 4.3 Multiplicity for attributes. You can specify whether an attribute is single or multivalued, mandatory or optional.

4.1.3 Scope

Chapter 3 presented features for individual objects. This is the default usage, but there can also be features for an entire class. The *scope* indicates if a feature applies to an object or a class. An underline distinguishes features with class scope (static) from those with object scope. Our convention is to list attributes and operations with class scope at the top of the attribute and operation boxes, respectively.

It is acceptable to use an attribute with class scope to hold the *extent* of a class (the set of objects for a class)—this is common with OO databases. Otherwise, you should avoid attributes with class scope because they can lead to an inferior model. It is better to model groups explicitly and assign attributes to them. For example, the upper model in Figure 4.4 shows a simple model of phone mail. Each message has an owner mailbox, date recorded, time recorded, priority, message contents, and a flag indicating if it has been received. A message may have a mailbox as the source or it may be from an external call. Each mailbox has a phone number, password, and recorded greeting. For the *PhoneMessage* class we can store the maximum duration for a message and the maximum days a message will be retained. For the *PhoneMailbox* class we can store the maximum number of messages that can be stored.

The upper model is inferior, however, because the maximum duration, maximum days retained, and maximum message count have a single value for the entire phone mail system. In the lower model these limits can vary for different kinds of users, yielding a more flexible and extensible phone mail system.

In contrast to attributes, it is acceptable to define operations of class scope. The most common use of class-scoped operations is to create new instances of a class. Sometimes it is convenient to define class-scoped operations to provide summary data. You should be careful with the use of class-scoped operations for distributed applications.

4.1.4 Visibility

Visibility refers to the ability of a method to reference a feature from another class and has the possible values of *public*, *protected*, *private*, and *package*. The precise meaning depends on the programming language. (See Chapter 18 for details.) Any method can freely access *public* features. Only methods of the containing class and its descendants via inheritance can access *protected* features. (Protected features also have package accessibility in Java.) Only methods of the containing class can access *private* features. Methods of classes defined in the same package as the target class can access *package* features.

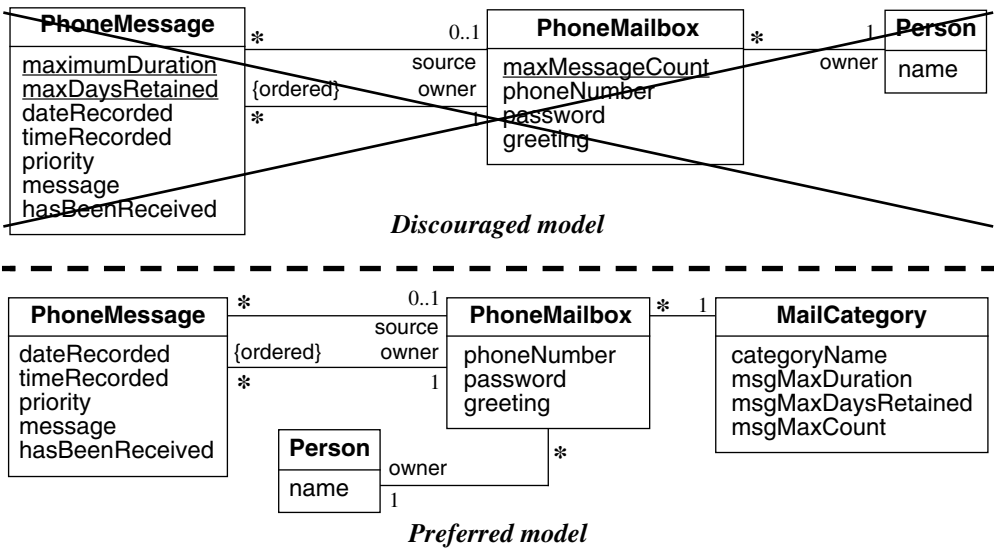


Figure 4.4 Attribute scope. Instead of assigning attributes to classes, model groups explicitly.

The UML denotes visibility with a prefix. The character “+” precedes public features. The character “#” precedes protected features. The character “-” precedes private features. And the character “~” precedes package features. The lack of a prefix reveals no information about visibility.

There are several issues to consider when choosing visibility.

- **Comprehension.** You must understand all public features to understand the capabilities of a class. In contrast, you can ignore private, protected, and package features—they are merely an implementation convenience.
- **Extensibility.** Many classes can depend on public methods, so it can be highly disruptive to change their signature (number of arguments, types of arguments, type of return value). Since fewer classes depend on private, protected, and package methods, there is more latitude to change them.
- **Context.** Private, protected, and package methods may rely on preconditions or state information created by other methods in the class. Applied out of context, a private method may calculate incorrect results or cause the object to fail.

4.2 Association Ends

As the name implies, an *association end* is an end of an association. A binary association has two ends, a ternary association (Section 4.3) has three ends, and so forth. Chapter 3 discussed the following properties.

- **Association end name.** An association end may have a name. The names disambiguate multiple references to a class and facilitate navigation. Meaningful names often arise, and it is useful to place the names within the proper context.
- **Multiplicity.** You can specify multiplicity for each association end. The most common multiplicities are “1” (exactly one), “0..1” (at most one), and “*” (“many”—zero or more).
- **Ordering.** The objects for a “many” association end are usually just a set. However, sometimes the objects have an explicit order.
- **Bags and sequences.** The objects for a “many” association end can also be a bag or sequence.
- **Qualification.** One or more qualifier attributes can disambiguate the objects for a “many” association end.

Association ends have some additional properties.

- **Aggregation.** The association end may be an aggregate or constituent part (Section 4.4). Only a binary association can be an aggregation; one association end must be an aggregate and the other must be a constituent.
- **Changeability.** This property specifies the update status of an association end. The possibilities are *changeable* (can be updated) and *readonly* (can only be initialized).
- **Navigability.** Conceptually, an association may be traversed in either direction. However, an implementation may support only one direction. The UML shows navigability with an arrowhead on the association end attached to the target class. Arrowheads may be attached to zero, one, or both ends of an association.
- **Visibility.** Similar to attributes and operations (Section 4.1.4), association ends may be *public*, *protected*, *private*, or *package*.

4.3 N-ary Associations

Chapter 3 presented binary associations (associations between two classes). However, you may occasionally encounter *n-ary associations* (associations among three or more classes.) You should try to avoid n-ary associations—most of them can be decomposed into binary associations, with possible qualifiers and attributes. Figure 4.5 shows an association that at first glance might seem to be an n-ary but can readily be restated as binary associations.

A nonatomic n-ary association—a person makes the purchase of stock in a company...

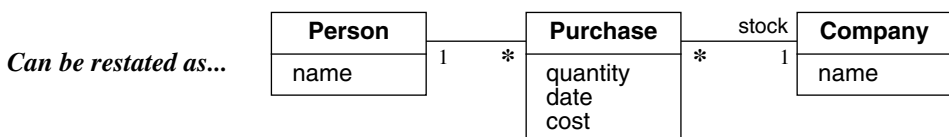


Figure 4.5 Restating an n-ary association. You can decompose most n-ary associations into binary associations.

exist apart from an assembly) and a more restrictive form called composition (the constituent part can belong to at most one assembly and has a coincident lifetime).

A state diagram for an assembly is a collection of state diagrams, one for each part. The aggregate state corresponds to the combined states of all the parts. The aggregate state is one state from the first diagram, *and* a state from the second diagram, *and* a state from each other diagram. In the more interesting cases, the part states interact.

- **State aggregation.** Some states can be partitioned into lesser states, each operating independently and each having its own subdiagram. The state of the object comprises one state from each subdiagram.

This completes our treatment of concepts and notation for object-oriented modeling.

Part 2

Analysis and Design

Chapter 10	Process Overview	167
Chapter 11	System Conception	173
Chapter 12	Domain Analysis	181
Chapter 13	Application Analysis	216
Chapter 14	System Design	240
Chapter 15	Class Design	270
Chapter 16	Process Summary	298

Part 1 covers *concepts*, specifically the concepts and notation for the class, state, and interaction models. We now shift our focus in Parts 2 and 3 and present a *process* for devising the models. Part 1 discusses *what* constitutes a model; Parts 2 and 3 explain *how to* formulate a model. Our treatment of process is language independent and applies equally well to OO languages, non-OO languages, and databases.

Chapter 10 provides an overview of the process for building models and emphasizes that development is normally iterative and seldom a rigid sequence of steps.

Chapter 11 presents the first stage of development—system conception—during which a visionary conceives an application and sells the idea to an organization.

Once you have a concept for an application, you elaborate and refine the concept by building models as Chapters 12 and 13 explain. First build a domain model that focuses on the real-world things that carry the semantics of the application. Then build an application model that addresses the computer aspects of the application that are visible to users.

The analysis models give you a thorough understanding of an application. The next stage is to address the practicalities of realizing the models. Chapter 14 covers system design, in which you devise a high-level strategy for building a solution. Chapter 15 covers class design, in which you flesh out the details for classes, associations, and operations.

Chapter 16 concludes Part 2 by summarizing the analysis and design portion of the development process.

After reading Part 2, you will understand the basics of how to prepare OO models. You will not be an expert, but you will have a good start on learning a valuable software development skill. You will be ready to study implementation and software engineering in the final two parts.

This page intentionally left blank

10

Process Overview

A *software development process* provides a basis for the organized production of software, using a collection of predefined techniques and notations. The process in this book starts with formulation of the problem, then continues through analysis, design, and implementation. The presentation of the stages is linear, but the actual process is seldom linear.

10.1 Development Stages

Software development has a sequence of well-defined stages, each with a distinct purpose, input, and output.

- **System conception.** Conceive an application and formulate tentative requirements.
- **Analysis.** Deeply understand the requirements by constructing models. The goal of analysis is to specify *what* needs to be done, not *how* it is done. You must understand a problem before attempting a solution.
- **System design.** Devise a high-level strategy—the architecture—for solving the application problem. Establish policies to guide the subsequent class design.
- **Class design.** Augment and adjust the real-world models from analysis so that they are amenable to computer implementation. Determine algorithms for realizing the operations.
- **Implementation.** Translate the design into programming code and database structures.
- **Testing.** Ensure that the application is suitable for actual use and that it truly satisfies the requirements.
- **Training.** Help users master the new application.
- **Deployment.** Place the application in the field and gracefully cut over from legacy applications.
- **Maintenance.** Preserve the long-term viability of the application.

The entire process is seamless. You continually elaborate and optimize models as your focus shifts from analysis to design to implementation. Throughout development the same concepts and notation apply; the only difference is the shift in perspective from the initial emphasis on business needs to the later emphasis on computer resources.

An OO approach moves much of the software development effort up to analysis and design. It is sometimes disconcerting to spend more time during analysis and design, but this extra effort is more than compensated by faster and simpler implementation. Because the resulting design is cleaner and more adaptable, future changes are much easier.

Part 2 covers the first four topics and Part 3 covers implementation. In this book we emphasize development and only briefly consider testing, training, deployment, and maintenance. These last four topics are important, but are not the focus of this book.

10.1.1 System Conception

System conception deals with the genesis of an application. Initially somebody thinks of an idea for an application, prepares a business case, and sells the idea to the organization. The innovator must understand both business needs and technological capabilities.

10.1.2 Analysis

Analysis focuses on creation of models. Analysts capture and scrutinize requirements by constructing models. They specify *what* must be done, not *how* it should be done. Analysis is a difficult task in its own right, and developers must fully understand the problem before addressing the additional complexities of design. Sound models are a prerequisite for an extensible, efficient, reliable, and correct application. No amount of implementation patches can repair an incoherent application and compensate for a lack of forethought.

During analysis, developers consider the available sources of information (documents, business interviews, related applications) and resolve ambiguities. Often business experts are not sure of the precise requirements and must refine them in tandem with software development. Modeling quickens the convergence between developers and business experts, because it is much faster to work with multiple iterations of models than with multiple implementations of code. Models highlight omissions and inconsistencies so that they can be resolved. As developers elaborate and refine a model, it gradually becomes coherent.

There are two substages of analysis: domain analysis and application analysis. **Domain analysis** focuses on real-world things whose semantics the application captures. For example, an airplane flight is a real-world object that a flight reservation system must represent. Domain objects exist independently of any application and are meaningful to business experts. You find them during domain analysis or by prior knowledge. Domain objects carry information about real-world objects and are generally passive—domain analysis emphasizes concepts and relationships, with much of the functionality being implicit in the class model. The job of constructing a domain model is mainly to decide which information to capture and how to represent it.

Domain analysis is then followed by **application analysis** that addresses the computer aspects of the application that are visible to users. For example, a flight reservation screen is

part of a flight reservation system. Application objects do not exist in the problem domain and are meaningful only in the context of an application. Application objects, however, are not merely internal design decisions, because the users see them and must agree with them. The application model does not prescribe the implementation of the application. It describes how the application appears from the outside—the black-box view of it. You cannot find application classes with domain analysis, but you can often reuse them from previous applications. Otherwise, you must devise application objects during analysis as you think about interfaces with other systems and how your application interacts with users.

10.1.3 System Design

During *system design*, the developer makes strategic decisions with broad consequences. You must formulate an architecture and choose global strategies and policies to guide the subsequent, more detailed portion of design. The *architecture* is the high-level plan or strategy for solving the application problem. The choice of architecture is based on the requirements as well as past experience. If possible, the architecture should include an executable skeleton that can be tested. The system designer must understand how a new system interacts with other systems. The architecture must also support future modification of the application.

For straightforward problems, preparation of the architecture follows analysis. However, for large and complex problems their preparation must be interleaved. The architecture helps to establish a model's scope. In turn, modeling reveals important issues of strategy to resolve. For large and complex problems, there is much interplay between the construction of a model and the model's architecture, and they must be built together.

10.1.4 Class Design

During *class design*, the developer expands and optimizes analysis models; there is a shift in emphasis from application concepts toward computer concepts. Developers choose algorithms to implement major system functions, but they should continue to defer the idiosyncrasies of particular programming languages.

10.1.5 Implementation

Implementation is the stage for writing the actual code. Developers map design elements to programming language and database code. Often, tools can generate some of the code from the design model.

10.1.6 Testing

After implementation, the system is complete, but it must be carefully tested before being commissioned for actual use. The ideas that inspired the original project should have been nurtured through the previous stages by the use of models. Testers once again revisit the original business requirements and verify that the system delivers the proper functionality. Testing can also uncover accidental errors (bugs) that have been introduced. If an application runs on multiple hardware and operating system platforms, it should be tested on all of them.

Developers should check a program at several levels. Unit tests exercise small portions of code, such as methods or possibly entire classes. Unit tests discover local problems and often require that extra instrumentation be built into the code. System tests exercise a major subsystem or the entire application. In contrast to unit tests, system tests can discover broad failures to meet specifications. Both unit and system tests are necessary. Testing should not wait until the entire application is coded. It must be planned from the beginning, and many tests can be performed during implementation.

10.1.7 Training

An organization must train users so that they can fully benefit from an application. Training accelerates users on the software learning curve. A separate team should prepare user documentation in parallel to the development effort. Quality control can then check the software against the user documentation to ensure that the software meets its original goals.

10.1.8 Deployment

The eventual system must work in the field, on various platforms and in various configurations. Unexpected interactions can occur when a system is deployed in a customer environment. Developers must tune the system under various loads and write scripts and install procedures. Some customers will require software customizations. Staff must also localize the product to different spoken languages and locales. The result is a usable product release.

10.1.9 Maintenance

Once development is complete and a system has been deployed, it must be maintained for continued success. There are several kinds of maintenance. Bugs that remain in the original system will gradually appear during use and must be fixed. A successful application will also lead to enhancement requests and a long-lived application will occasionally have to be restructured.

Models ease maintenance and transitions across staff changes. A model expresses the business intent for an application that has been driven into the programming code, user interface, and database structure.

10.2 Development Life Cycle

An OO approach to software development supports multiple life-cycle styles. You can use a waterfall approach performing the phases of analysis, design, and implementation in strict sequence for the entire system. However, we typically recommend an iterative development strategy. We summarize the distinction here and elaborate in Chapter 21.

10.2.1 Waterfall Development

The waterfall approach dictates that developers perform the software development stages in a rigid linear sequence with no backtracking. Developers first capture requirements, then construct an analysis model, then perform a system design, then prepare a class design, fol-

lowed by implementation, testing, and deployment. Each stage is completed in its entirety before the next stage is begun.

The waterfall approach is suitable for well-understood applications with predictable outputs from analysis and design, but such applications seldom occur. Too many organizations attempt to follow a waterfall when requirements are fluid. This leads to the familiar situation where developers complain about changing requirements, and the business complains about inflexible software development. A waterfall approach also does not deliver a useful system until completion. This makes it difficult to assess progress and correct a project that has gone awry.

10.2.2 Iterative Development

Iterative development is more flexible. First you develop the nucleus of a system—analyzing, designing, implementing, and delivering working code. Then you grow the scope of the system, adding properties and behavior to existing objects, as well as adding new kinds of objects. There are multiple iterations as the system evolves to the final deliverable.

Each iteration includes a full complement of stages: analysis, design, implementation, and testing. Unlike the strict sequence of the waterfall method, iterative development can interleave the different stages and need not construct the entire system in lock step. Some parts may be completed early, while other, less crucial parts are completed later. Each iteration ultimately yields an executable system that can be integrated and tested. You can accurately gauge progress and make adjustments to your plans based on feedback from the early iterations. If there is a problem, you can move backward to an earlier stage for rework.

Iterative development is the best choice for most applications because it gracefully responds to change and minimizes risk of failure. Management and business users get early feedback about progress.

10.3 Chapter Summary

A software engineering process provides a basis for the organized production of software. There is a sequence of well-defined stages that you can apply to each of the pieces of a system. For example, parallel development teams might develop a database design, key algorithms, and a user interface. An iterative development of software is flexible and responsive to evolving requirements. First you prepare a nucleus of a system, and then you successively grow its scope until you realize the final desired software.

analysis	domain analysis	system conception
application analysis	implementation	system design
architecture	iterative development	testing
class design	life cycle	training
deployment	maintenance	waterfall development

Figure 10.1 Key concepts for Chapter 10

Bibliographic Notes

The *class design* stage is renamed from *object design* in the first edition of this book.

Exercises

- 10.1 (2) It seems there is never enough time to do a job right the first time, but there is always time to do it over. Discuss how the approach presented in this chapter overcomes this tendency of human behavior. What kinds of errors do you make if you rush into the implementation phase of a software project? Compare the effort required to prevent errors with that needed to detect and correct them.
- 10.2 (4) This book explains how to use OO techniques to implement programs and databases. Discuss how OO techniques could be applied in other areas, such as language design, knowledge representation, and hardware design.

11

System Conception

System conception deals with the genesis of an application. Initially some person, who understands both business needs and technology, thinks of an idea for an application. Developers must then explore the idea to understand the needs and devise possible solutions. The purpose of system conception is to defer details and understand the big picture—what need does the proposed system meet, can it be developed at a reasonable cost, and will the demand for the result justify the cost of building it?

This chapter introduces the automated teller machine (ATM) case study that threads throughout the remainder of the book.

11.1 Devising a System Concept

Most ideas for new systems are extensions of existing ideas. For example, a human relations department may have a database of employee benefit choices and require that a clerk enter changes. An obvious extension is to allow employees to view and enter their own changes. There are many issues to resolve (security, reliability, privacy, and so on), but the new idea is a straightforward extension of an existing concept.

Occasionally a new system is a radical departure from the past. For example, an online auction automates the ancient idea of buyers bidding against each other for products, but the first online auction systems were brand new software. The concept became feasible when several enabling technologies came into place: the Internet, widespread personal computer access, and reliable servers. The large customer base and low unit cost due to automation changed the nature of auctions—an online auction can sell inexpensive items and still make a profit. In addition, online systems have made the auction process concurrent and distributed.

Here are some ways to find new system concepts.

- **New functionality.** Add functionality to an existing system.

- **Streamlining.** Remove restrictions or generalize the way a system works.
- **Simplification.** Let ordinary persons perform tasks previously assigned to specialists.
- **Automation.** Automate manual processes.
- **Integration.** Combine functionality from different systems.
- **Analogies.** Look for analogies in other problem domains and see if they have useful ideas.
- **Globalization.** Travel to other countries and observe their cultural and business practices.

11.2 Elaborating a Concept

Most systems start as vague ideas that need more substance. A good system concept must answer the following questions.

- **Who is the application for?** You should clearly understand which persons and organizations are stakeholders of the new system. Two of the most important kinds of stakeholders are the financial sponsors and the end users.

The financial sponsor are important because they are paying for the new system. They expect the project to be on schedule and within budget. You should get the financial sponsors to agree to some measure of success. You need to know when the system is complete and meets their expectations.

The users are also stakeholders, but in another sense. The users will ultimately determine the success of the new system by an increase (or decrease) in their productivity or effectiveness. Users can help you if they are receptive and provide critical comments. They can improve your system by telling you what is missing and what could be improved. In general, users will not consider new software unless they have a compelling interest—either personal or business. You should try to help them find a vested interest in your project so that you can obtain their buy-in. If you cannot get their buy-in, you should question the need for the project and reconsider doing it.

- **What problems will it solve?** You must clearly bound the size of the effort and establish its scope. You should determine which features will be in the new system and which will not. You must reach various kinds of users in different organizations with their own viewpoints and political motivations. You must not only decide which features are appropriate, but you must also obtain the agreement of influential persons.
- **Where will it be used?** At this early stage, it is helpful to get a general idea of where the new system might be used. You should determine if the new system is mission-critical software for the organization, experimental software, or a new capability that you can deploy without disrupting the workflow. You should have a rough idea about how the new system will complement the existing systems. It is important to know if the software will be used locally or will be distributed via a network. For a commercial product, you should characterize the customer base.

- **When is it needed?** Two aspects of time are important. The first is the *feasible* time, the time in which the system can be developed within the constraints of cost and available resources. The other is the *required* time, when the system is needed to meet business goals. You must make sure that the timing expectations driven by technical feasibility are consistent with the timing the business requires. If there is a disconnect, you must initiate a dialogue between technologists and business experts to reach a solution.
- **Why is it needed?** You may need to prepare a business case for the new system if someone has not already done so. The business case contains the financial justification for the new system, including the cost, tangible benefits, intangible benefits, risk, and alternatives. You must be sure that you clearly understand the motivation for the new system. The business case will give you insight into what stakeholders expect, roughly indicate the scope, and may even provide information for seeding your models. For a commercial product, you should estimate the number of units that can be sold and determine a reasonable selling price; the revenue must cover costs and a profit.
- **How will it work?** You should brainstorm about the feasibility of the problem. For large systems you should consider the merits of different architectures. The purpose of this speculation is not to choose a solution, but to increase confidence that the problem can be solved reasonably. You might need some prototyping and experimentation.

11.2.1 The ATM Case Study

Figure 11.1 lists our original system concept for an Automated Teller Machine (ATM). We ask high-level questions to elaborate the initial concept.

Develop software so that customers can access a bank's computers and carry out their own financial transactions without the mediation of a bank employee.

Figure 11.1 System concept for an automated teller machine

- **Who is the application for?** A number of companies provide ATM products. Consequently, only a vendor or a large financial company could possibly justify the cost and effort of building ATM software.

A vendor would be competing for customers in an established market. A large vendor could certainly enter such a market, but might find it advantageous to partner with or acquire an existing supplier. A small vendor would need some special feature to differentiate itself from the crowd and attract attention.

It is unlikely that a financial company could justify developing ATM software just for its own use, because it would probably be more expensive than purchasing a product. If a financial company wanted special features, it could partner with a vendor. Or it might decide to create a separate organization that would build the software, sell it to the sponsoring company, and then market it to others.

For the ATM case study, we will assume that we are a vendor building the software. We will assume that we are developing an ordinary product, since deep complexities of the ATM problem domain are beyond the scope of this book.

- **What problems will it solve?** The ATM software is intended to serve both the bank and the customer. For the bank, ATM software increases automation and reduces manual handling of routine paperwork. For the customer, the ATM is ubiquitous and always available, handling routine transactions whenever and wherever the customer desires. ATM software must be easy to use and convenient so that customers will use it in preference to bank tellers. It must be reliable and secure since it will be handling money.
- **Where will it be used?** ATM software has become essential to financial institutions. Customers take it for granted that a bank will have an ATM machine. ATM machines are available at many stores, sporting events, and other locations throughout the world.
- **When is it needed?** Any software development effort is a financial proposition. The investment in development ultimately leads to a revenue stream. From an economic perspective, it is desirable to minimize the investment, maximize the revenue, and realize revenue as soon as possible. Thoughtful modeling and OO techniques are conducive to this goal.
- **Why is it needed?** There are many reasons why a vendor might decide to build a software product. If other companies are making money with similar products, there is an economic incentive to participate. A novel product could outflank competitors and lead to premium pricing. Businesses commission internal efforts for technology that is difficult to buy and critical to them. We have no real motivation to develop ATM software, other than to demonstrate the techniques in this book.
- **How will it work?** We will adopt a three-tier architecture to separate the user interface from programming logic, and programming logic from the database. In reality, the architecture is n -tier, because there can be any number of intermediate programming levels communicating with each other. We will discuss architecture further in the *System Design* chapter.

11.3 Preparing a Problem Statement

Once you have fleshed out the raw idea by answering the high-level questions, you are ready to write a requirements statement that outlines the goals and general approach of the desired system.

Throughout development, you should distinguish among requirements, design, and implementation. Requirements describe how a system behaves from the user's point of view. The system is considered as a black box—all we care about is its external behavior. For example, some requirements for a car are that when you press on the accelerator pedal, the car goes faster, and when you step on the brake, the car slows down. Design decisions are engineering choices that provide the behavior specified by the requirements. For example, some design decisions are how the internal linkages are routed, how the engine is controlled, and

what kinds of brake pads are on the wheels. Implementation deals with the ultimate realization in programming code.

Frequently customers mix true requirements with design decisions. Usually this is a bad idea. If you separate requirements from design decisions, you preserve the freedom to change a design. Typically there are many possible ways to design a system, and you should defer a solution until you fully understand a problem.

A system concept document may include an example implementation. The purpose of the example is to show how the system could be implemented using current technology at a reasonable cost. It is a “proof of existence” statement. However, make it clear that the sample implementation could be done differently in the final system. The sample implementation is merely proposed as a possibility.

For example, when the Apollo program to put a man on the moon in the 1960s was first proposed, the plan was to place a rocket in earth orbit, then launch a landing vehicle directly to the moon’s surface. In the final successful program, the rocket was launched directly into a lunar orbit, from which the lander was launched to the moon’s surface. It was not a bad thing to make the first proposal, however, as this gave confidence that there was a feasible approach.

As Figure 11.2 shows, the problem statement should state what is to be done and not how it is to be implemented. It should be a statement of needs, not a proposal for a system architecture. The requestor should avoid describing system internals, as this restricts development flexibility. Performance specifications and protocols for interaction with external systems are legitimate requirements. Software engineering standards, such as modular construction, design for testability, and provision for future extensions, are also proper.

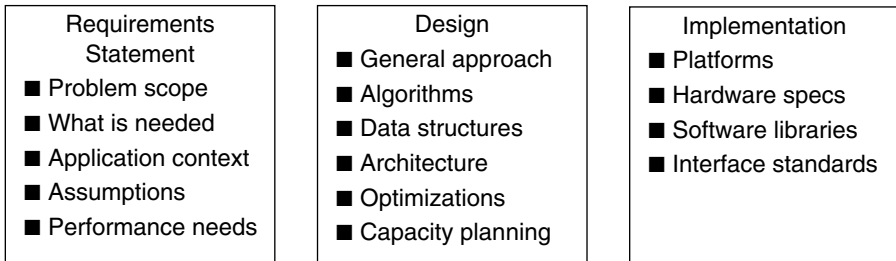


Figure 11.2 Kinds of requirements. Do not make early design and implementation decisions or you will compromise development.

A problem statement may have more or less detail. A requirement for a conventional product, such as a payroll program or a billing system, may have considerable detail. A requirement for a research effort in a new area may lack details, but presumably the research has some objective that should be clearly stated.

Most problem statements are ambiguous, incomplete, or even inconsistent. Some requirements are just plain wrong. Some requirements, although precisely stated, have unpleasant consequences on the system behavior or impose unreasonable implementation costs. Some requirements do not work out as well as the requestor thought. The problem

statement is just a starting point for understanding the problem, not an immutable document. The purpose of the subsequent analysis (next chapter) is to fully understand the problem and its implications. There is no reason to expect that a problem statement prepared without a full analysis will be correct.

11.3.1 The ATM Case Study

Figure 11.3 shows a problem statement for an automated teller machine (ATM) network.

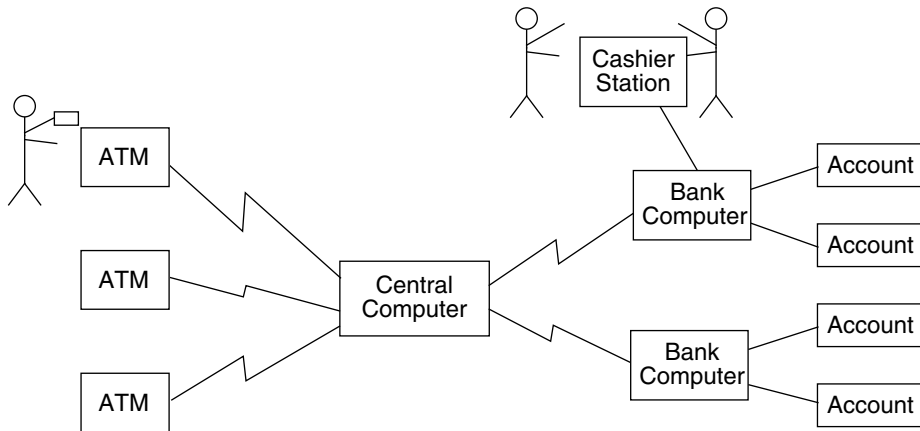


Figure 11.3 ATM network. The ATM case study threads throughout the remainder of this book.

Design the software to support a computerized banking network including both human cashiers and automatic teller machines (ATMs) to be shared by a consortium of banks. Each bank provides its own computer to maintain its own accounts and process transactions against them. Cashier stations are owned by individual banks and communicate directly with their own bank's computers. Human cashiers enter account and transaction data.

Automatic teller machines communicate with a central computer that clears transactions with the appropriate banks. An automatic teller machine accepts a cash card, interacts with the user, communicates with the central system to carry out the transaction, dispenses cash, and prints receipts. The system requires appropriate recordkeeping and security provisions. The system must handle concurrent accesses to the same account correctly.

The banks will provide their own software for their own computers; you are to design the software for the ATMs and the network. The cost of the shared system will be apportioned to the banks according to the number of customers with cash cards.

11.4 Chapter Summary

The first stage of a project is to devise a new idea. The idea can involve a new system or an improvement to an existing system. Before investing time and money into development, it is

necessary to evaluate the feasibility of the system, the difficulty and risk of developing it, the demand for the system, and the cost-benefit ratio. This process should consider the view-points of all the stakeholders of the system and should make the trade-offs necessary to provide a good chance of success, not just technical success, but also business success. This process usually results in some adjustments to the original idea. When the system conception stage is complete, write a problem statement that serves as the starting point for analysis. The problem statement need not be complete, and it will change during development, but the writing of the statement helps to focus the attention of the project.

business case	problem statement
cost-benefit trade-off	requirement
design decision	stakeholder
implementation constraint	system conception

Figure 11.4 Key concepts for Chapter 11

Exercises

- 11.1 (3) Consider a new antilock braking system for crash avoidance in an automobile. Elaborate the following high-level questions and explain your answers.
 - a. Who is the application for? Who are the stakeholders? Estimate how many persons in your country are potential customers.
 - b. Identify three features that should be included and three features that should be omitted.
 - c. Identify three systems with which it must work.
 - d. What are two of the largest risks?
- 11.2 (3) Repeat Exercise 11.1 for software that supports Internet selling of books.
- 11.3 (3) Repeat Exercise 11.1 for software that supports the remodeling of kitchens.
- 11.4 (3) Repeat Exercise 11.1 for an online auction system.
- 11.5 (4) Prepare a problem statement, similar to that for the ATM system, for each of the following systems. You may limit the scope of the system, but be precise and avoid making implementation decisions. Use 75–150 words per specification.
 - a. bridge player
 - b. change-making machine
 - c. car cruise control
 - d. electronic typewriter
 - e. spelling checker
 - f. telephone answering machine
- 11.6 (3) Rephrase the following requirements to make them more precise. Remove any design decisions posing as requirements:
 - a. A system to transfer data from one computer to another over a telecommunication line. The system should transmit data reliably over noisy channels. Data must not be lost if the receiv-

ing end cannot keep up or if the line drops out. Data should be transmitted in packets, using a master-slave protocol in which the receiving end acknowledges or negatively acknowledges all exchanges.

- b. A system for automating the production of complex machined parts. The parts will be designed using a three-dimensional drafting editor that is part of the system. The system will produce tapes that can be used by numerical control (N/C) machines to actually produce the parts.
- c. A desktop publishing system, based on a what-you-see-is-what-you-get philosophy. The system will support text and graphics. Graphics include lines, squares, rectangles, polygons, circles, and ellipses. Internally, a circle is represented as a special case of an ellipse and a square as a special case of a rectangle. The system should support interactive, graphical editing of documents.
- d. A system for generating nonsense. The input is a sample document. The output is random text that mimics the input text by imitating the frequencies of combinations of letters of the input. The user specifies the order of the imitation and the length of the desired output. For order N , every output sequence of N characters is found in the input and at approximately the same frequency. As the order increases, the style of the output more closely matches the input.

The system should generate its output with the following method: Select a position at random in the document being imitated. Scan forward in the input text until a sequence of characters is found that exactly matches the last $N - 1$ characters of the output. If you reach the end of the input, continue scanning from the beginning. When a match is found, copy the letter that follows the matched sequence from the input to the output. Repeat until the desired amount of text is generated.

- e. A system for distributing electronic mail over a network. Each user of the system should be able to send mail from any computer account and receive mail on one designated account. There should be provisions for answering or forwarding mail, as well as saving messages in files or printing them. Also, users should be able to send messages to several other users at once through distribution lists. Each computer on the net should hold any messages destined for computers that are down.

Domain Analysis

Domain analysis, the next stage of development, is concerned with devising a precise, concise, understandable, and correct model of the real world. Before building anything complex, the builder must understand the requirements. Requirements can be stated in words, but these are often imprecise and ambiguous. During analysis, we build models and begin to understand the requirements deeply.

To build a domain model, you must interview business experts, examine requirements statements, and scrutinize related artifacts. You must analyze the implications of the requirements and restate them rigorously. It is important to abstract important features first and defer small details until later. The successful analysis model states what must be done, without restricting how it is done, and avoids implementation decisions.

In this chapter you will learn how to take OO concepts and apply them to construct a domain model. The model serves several purposes: It clarifies the requirements, it provides a basis for agreement between the stakeholders and the developers, and it becomes the starting point for design and implementation.

12.1 Overview of Analysis

As Figure 12.1 shows, analysis begins with a problem statement generated during system conception. The statement may be incomplete or informal; analysis makes it more precise and exposes ambiguities and inconsistencies. The problem statement should not be taken as immutable but should serve as a basis for refining the real requirements.

Next, you must understand the real-world system described by the problem statement, and abstract its essential features into a model. Statements in natural language are often ambiguous, incomplete, and inconsistent. The analysis model is a precise, concise representation of the problem that permits answering questions and building a solution. Subsequent design steps refer to the analysis model, rather than the original vague problem statement.

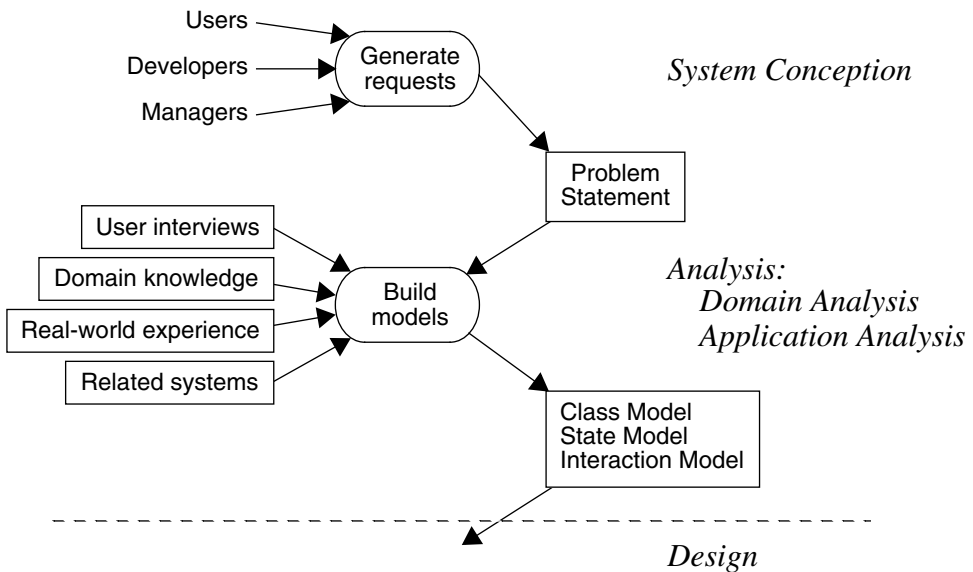


Figure 12.1 Overview of analysis. The problem statement should not be taken as immutable, but rather as a basis for refining the requirements.

Perhaps even more important, the process of constructing a rigorous model of the problem domain forces the developer to confront misunderstandings early in the development process while they are still easy to correct.

The analysis model addresses the three aspects of objects: static structure of objects (class model), interactions among objects (interaction model), and life-cycle histories of objects (state model). All three submodels are not equally important in every problem. Almost all problems have useful class models derived from real-world entities. Problems concerning reactive control and timing, such as user interfaces and process control, have important state models. Problems containing significant computation as well as systems that interact with other systems and different kinds of users have important interaction models.

Analysis is not a mechanical process. The exact representations involve judgment and in many regards are a matter of art. Most problem statements lack essential information, which must be obtained from the requestor or from the analyst's knowledge of the real-world problem domain. Also there is a choice in the level of abstraction for the model. The analyst must communicate with the requestor to clarify ambiguities and misconceptions. The analysis models enable precise communication.

We have divided analysis into two substages. The first, *domain analysis*, is covered in this chapter and focuses on understanding the real-world essence of a problem. The second, *application analysis*, is covered in the next chapter and builds on the domain model—incorporating major application artifacts that are seen by users and must be approved by them.

12.2 Domain Class Model

The first step in analyzing the requirements is to construct a domain model. The domain model shows the static structure of the real-world system and organizes it into workable pieces. The domain model describes real-world classes and their relationships to each other. During analysis, the class model precedes the state and interaction models because static structure tends to be better defined, less dependent on application details, and more stable as the solution evolves. Information for the domain model comes from the problem statement, artifacts from related systems, expert knowledge of the application domain, and general knowledge of the real world. Make sure you consider all information that is available and do not rely on a single source.

Find classes and associations first, as they provide the overall structure and approach to the problem. Next add attributes to describe the basic network of classes and associations. Then combine and organize classes using inheritance. Attempts to specify inheritance directly without first understanding classes and their attributes can distort the class structure to match preconceived notions. Operations are usually unimportant in a domain model. The main purpose of a domain model is to capture the information content of a domain.

It is best to get ideas down on paper before trying to organize them too much, even though they may be redundant and inconsistent, so as not to lose important details. An initial analysis model is likely to contain flaws that must be corrected by later iterations. The entire model need not be constructed uniformly. Some aspects of the problem can be analyzed in depth through several iterations while other aspects are still sketchy.

You must perform the following steps to construct a domain class model.

- Find classes. [12.2.1–12.2.2]
- Prepare a data dictionary. [12.2.3]
- Find associations. [12.2.4–12.2.5]
- Find attributes of objects and links. [12.2.6–12.2.7]
- Organize and simplify classes using inheritance. [12.2.8]
- Verify that access paths exist for likely queries. [12.2.9]
- Iterate and refine the model. [12.2.10]
- Reconsider the level of abstraction. [12.2.11]
- Group classes into packages. [12.2.12]

12.2.1 Finding Classes

The first step in constructing a class model is to find relevant classes for objects from the application domain. Objects include physical entities, such as houses, persons, and machines, as well as concepts, such as trajectories, seating assignments, and payment schedules. All classes must make sense in the application domain; avoid computer implementation constructs, such as linked lists and subroutines. Not all classes are explicit in the problem statement; some are implicit in the application domain or general knowledge.

As Figure 12.2 shows, begin by listing candidate classes found in the written description of the problem. Don't be too selective; write down every class that comes to mind. Classes often correspond to nouns. For example, in the statement "a reservation system to sell tickets to performances at various theaters" tentative classes would be *Reservation*, *System*, *Ticket*, *Performance*, and *Theater*. Don't operate blindly, however. The idea is to capture concepts; not all nouns are concepts, and concepts are also expressed in other parts of speech.

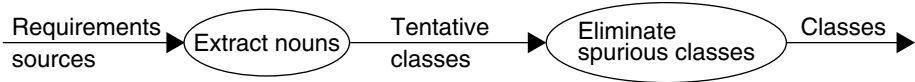


Figure 12.2 Finding classes. You can find many classes by considering nouns.

Don't worry much about inheritance or high-level classes; first get specific classes right so that you don't subconsciously suppress detail in an attempt to fit a preconceived structure. For example, if you are building a cataloging and checkout system for a library, identify different kinds of materials, such as books, magazines, newspapers, records, videos, and so on. You can organize them into broad categories later, by looking for similarities and differences.

ATM example. Examination of the concepts in the ATM problem statement from Chapter 11 yields the tentative classes shown in Figure 12.3. Figure 12.4 shows additional classes that do not appear directly in the statement but can be identified from our knowledge of the problem domain.

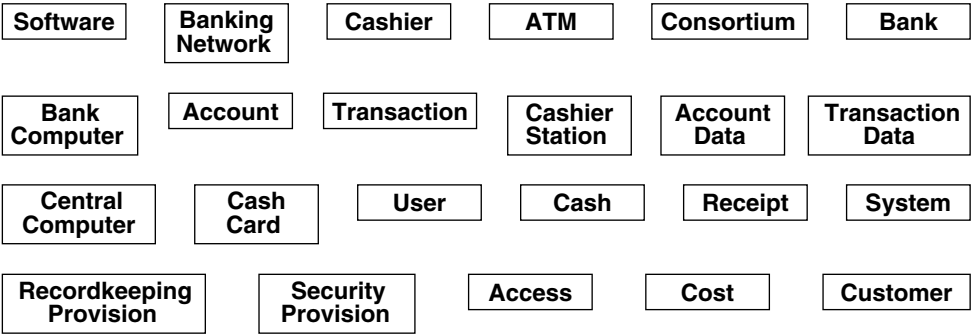


Figure 12.3 ATM classes extracted from problem statement nouns



Figure 12.4 ATM classes identified from knowledge of problem domain

12.2.2 Keeping the Right Classes

Now discard unnecessary and incorrect classes according to the following criteria. Figure 12.5 shows the classes eliminated from the ATM example.

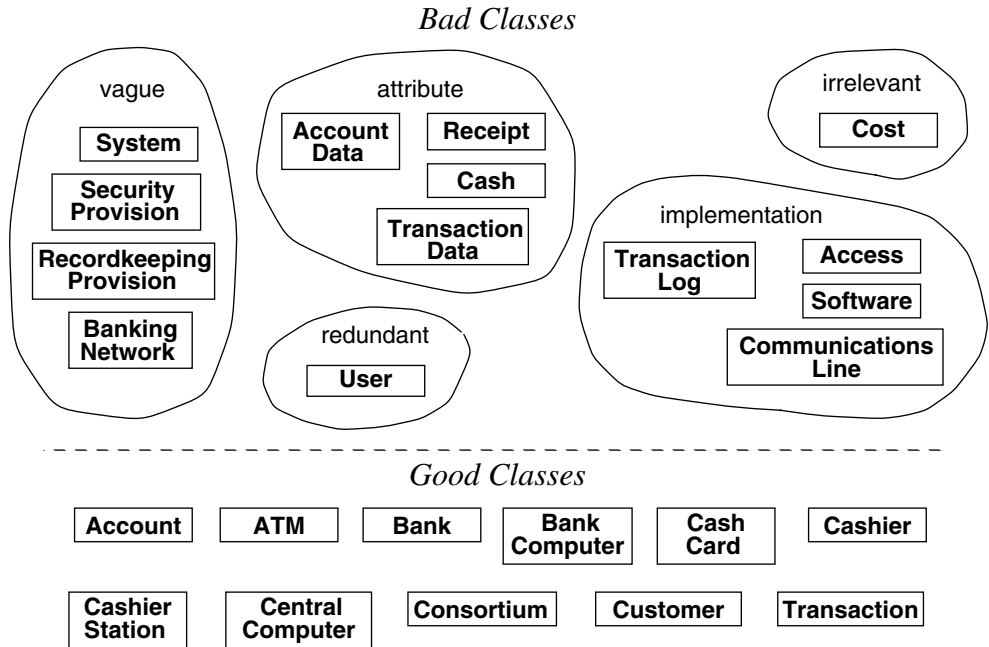


Figure 12.5 Eliminating unnecessary classes from ATM problem

- **Redundant classes.** If two classes express the same concept, you should keep the most descriptive name. For example, although *Customer* might describe a person taking an airline flight, *Passenger* is more descriptive. On the other hand, if the problem concerns contracts for a charter airline, *Customer* is also an appropriate word, since a contract might involve several passengers.

ATM example. *Customer* and *User* are redundant; we retain *Customer* because it is more descriptive.

- **Irrelevant classes.** If a class has little or nothing to do with the problem, eliminate it. This involves judgment, because in another context the class could be important. For example, in a theater ticket reservation system, the occupations of the ticket holders are irrelevant, but the occupations of the theater personnel may be important.

ATM example. Apportioning *Cost* is outside the scope of the ATM software.

- **Vague classes.** A class should be specific. Some tentative classes may have ill-defined boundaries or be too broad in scope.

ATM example. *RecordkeepingProvision* is vague and is handled by *Transaction*. In other applications, this might be included in other classes, such as *StockSales*, *TelephoneCalls*, or *MachineFailures*.

- **Attributes.** Names that primarily describe individual objects should be restated as attributes. For example, *name*, *birthdate*, and *weight* are usually attributes. If the independent existence of a property is important, then make it a class and not an attribute. For example, an employee's office would be a class in an application to reassign offices after a reorganization.

ATM example. *AccountData* is underspecified but in any case probably describes an account. An ATM dispenses cash and receipts, but beyond that cash and receipts are peripheral to the problem, so they should be treated as attributes.

- **Operations.** If a name describes an operation that is applied to objects and not manipulated in its own right, then it is not a class. For example, a telephone call is a sequence of actions involving a caller and the telephone network. If we are simply building telephones, then *Call* is part of the state model and not a class.

An operation that has features of its own should be modeled as a class, however. For example, in a billing system for telephone calls a *Call* would be an important class with attributes such as *date*, *time*, *origin*, and *destination*.

- **Roles.** The name of a class should reflect its intrinsic nature and not a role that it plays in an association. For example, *Owner* would be a poor name for a class in a car manufacturer's database. What if a list of drivers is added later? What about persons who lease cars? The proper class is *Person* (or possibly *Customer*), which assumes various different roles, such as *owner*, *driver*, and *lessee*.

One physical entity sometimes corresponds to several classes. For example, *Person* and *Employee* may be distinct classes in some circumstances and redundant in others. From the viewpoint of a company database of employees, the two may be identical. In a government tax database, a person may hold more than one job, so it is important to distinguish *Person* from *Employee*; each person can correspond to zero or more instances of employee information.

- **Implementation constructs.** Eliminate constructs from the analysis model that are extraneous to the real world. You may need them later during design, but not now. For example, CPU, subroutine, process, algorithm, and interrupt are implementation constructs for most applications, although they are legitimate classes for an operating system. Data structures, such as linked lists, trees, arrays, and tables, are almost always implementation constructs.

ATM example. Some tentative classes are really implementation constructs. *TransactionLog* is simply the set of transactions; its exact representation is a design issue. Communication links can be shown as associations; *CommunicationsLine* is simply the physical implementation of such a link.

- **Derived classes.** As a general rule, omit classes that can be derived from other classes. If a derived class is especially important, you can include it, but do so only sparingly. Mark all derived classes with a preceding slash ('/') in the class name.

12.2.3 Preparing a Data Dictionary

Isolated words have too many interpretations, so prepare a data dictionary for all modeling elements. Write a paragraph precisely describing each class. Describe the scope of the class within the current problem, including any assumptions or restrictions on its use. The data dictionary also describes associations, attributes, operations, and enumeration values. Figure 12.6 shows a data dictionary for the classes in the ATM problem.

12.2.4 Finding Associations

Next, find associations between classes. A structural relationship between two or more classes is an association. A reference from one class to another is an association. As we discussed in Chapter 3, attributes should not refer to classes; use an association instead. For example, class *Person* should not have an attribute *employer*; relate class *Person* and class *Company* with association *WorksFor*. Associations show relationships between classes at the same level of abstraction as the classes themselves, while object-valued attributes hide dependencies and obscure their two-way nature. Associations can be implemented in various ways, but such implementation decisions should be kept out of the analysis model to preserve design freedom.

Associations often correspond to stative verbs or verb phrases. These include physical location (*NextTo*, *PartOf*, *ContainedIn*), directed actions (*Drives*), communication (*TalksTo*), ownership (*Has*, *PartOf*), or satisfaction of some condition (*WorksFor*, *MarriedTo*, *Manages*). Extract all the candidates from the problem statement and get them down on paper first; don't try to refine things too early. Again, don't treat grammatical forms blindly; the idea is to capture relationships, however they are expressed in natural language.

ATM example. Figure 12.7 shows associations. The majority are taken directly from verb phrases in the problem statement. For some associations the verb phrase is implicit in the statement. Finally, some associations depend on real-world knowledge or assumptions. These must be verified with the requestor, as they are not in the problem statement.

12.2.5 Keeping the Right Associations

Now discard unnecessary and incorrect associations, using the following criteria.

- **Associations between eliminated classes.** If you have eliminated one of the classes in the association, you must eliminate the association or restate it in terms of other classes.

ATM example. We can eliminate *Banking network includes cashier stations and ATMs*, *ATM dispenses cash*, *ATM prints receipts*, *Banks provide software*, *Cost apportioned to banks*, *System provides recordkeeping*, and *System provides security*.

- **Irrelevant or implementation associations.** Eliminate any associations that are outside the problem domain or deal with implementation constructs.

ATM example. For example, *System handles concurrent access* is an implementation concept. Real-world objects are inherently concurrent; it is the implementation of the access algorithm that must be concurrent.

12.5.1 Refining the Analysis Model

The overall analysis model may show inconsistencies and imbalances within and across models. Iterate the different portions to produce a cleaner, more coherent model. Try to refine classes to increase sharing and improve structure. Add details that you glossed over during the first pass.

Some constructs will feel awkward and won't seem to fit in right. Reexamine them carefully; you may have the wrong concepts. Sometimes major restructuring in the model is needed as your understanding increases. It is easier to do now than it will ever be, so don't avoid changes just because you already have a model in place. When there are many constructs that appear similar but don't quite fit together, you have probably missed or miscast a more general concept. Watch out for generalizations factored on the wrong aspects.

A common difficulty is a physical object that has two logically distinct aspects. Each aspect should be modeled with a distinct object. An indication of this problem is a class that doesn't fit in cleanly and seems to have two sets of unrelated attributes, associations, and operations.

Other indications to watch for include exceptions, many special cases, and lack of expected symmetry. Consider restructuring your model to capture constraints better within its structure.

Be wary of codifying arbitrary business practices in your model. Software should facilitate operation of the business and not inhibit reasonable changes. Often you can introduce abstractions that increase business flexibility without substantially complicating a model.

Remove classes or associations that seemed useful at first but now appear extraneous. Often two classes in the analysis can be combined, because the distinction between them doesn't affect the rest of the model in any meaningful way. There is a tendency for models to grow as analysis proceeds. This is a concern, since the amount of development work escalates as a model becomes larger in size. Take a close look at your model for minor concepts to cut or abstractions that can simplify the model.

A good model feels right and does not appear to have extraneous detail. Don't worry if it doesn't seem perfect; even a good model will often have a few small areas where the design is adequate but never feels quite right.

12.5.2 Restating the Requirements

When the analysis is complete, the model serves as the basis for the requirements and defines the scope of future discourse. Most of the real requirements will be part of the model. In addition you may have some performance constraints; these should be stated clearly, together with optimization criteria. Other requirements specify the method of solution and should be separated and challenged, if possible.

You should verify the final model with the requestor. During analysis some requirements may appear to be incorrect or impractical; confirm corrections to the requirements. Also business experts should verify the analysis model to make sure that it correctly models the real world. We have found analysis models to be an effective means of communication with business experts who are not computer experts.

The final verified analysis model serves as the basis for system architecture, design, and implementation. You should revise the original problem statement to incorporate corrections and understanding discovered during analysis.

12.5.3 Analysis and Design

The goal of analysis is to specify the problem fully without introducing a bias to any particular implementation, but it is impossible in practice to avoid all taints of implementation. There is no absolute line between the various development stages, nor is there any such thing as a perfect analysis. Don't treat the rules we have given too rigidly. The purpose of the rules is to preserve flexibility and permit changes later, but remember that the goal of modeling is to accomplish the total job, and flexibility is just a means to an end.

ATM example. We have no further changes to the ATM model at this time. A true application is more likely to incur revision than a textbook example, because you have reviewers who are passionate about the application and have a vested interest in it.

12.6 Chapter Summary

The domain model captures general knowledge about an application—concepts and relationships known to experts in the domain. The domain model has class models and sometimes state models, but seldom has an interaction model. The purpose of analysis is to understand the problem and the application so that a correct design can be constructed. A good analysis captures the essential features of the problem without introducing implementation artifacts that prematurely restrict design decisions.

The domain class model shows the static structure of the real world. First find classes. Then find associations between classes. Note attributes, though you can defer minor ones. You can use generalization to organize and simplify the class structure. Group tightly coupled classes and associations into packages. Supplement the class models with a data dictionary—brief textual descriptions, including the purpose and scope of each element.

If a domain class has several qualitatively different states during its life cycle, make a state diagram for it, but most domain classes will not require state diagrams.

Methodologies are never as linear as they appear in books. This one is no exception. Any complex analysis is constructed by iteration on multiple levels. You need not prepare all parts of the model at the same pace. The result of analysis replaces the original problem statement and serves as the basis for design.

Bibliographic Notes

Abbott explains how to use nouns and verbs in the problem statement to seed thinking about an application [Abbott-83]. [Coad-95] is a good book with some examples of analysis patterns.

building the domain class model	finding classes
building the domain state model	finding events
data dictionary	finding states
domain analysis	refining a model with inheritance
finding associations	shifting the level of abstraction
finding attributes	testing the model

Figure 12.15 Key concepts for Chapter 12

References

[Abbott-83] Russell J. Abbott. Program Design by Informal English Descriptions. *Communications of the ACM* 26, 11 (November 1983), 882–894.

[Coad-95] Peter Coad, David North, and Mark Mayfield. *Object Models: Strategies, Patterns, and Applications*. Upper Saddle River, NJ: Yourdon Press, 1995.

Exercises

- 12.1 (3) For each of the following systems, identify the relative importance of the three aspects of modeling: 1) class modeling, 2) state modeling, 3) interaction modeling. Explain your answers. For example, for a compiler, the answer might be 3, 1, and 2. Interaction modeling is most important for a compiler because it is dominated by data transformation concerns.
- a. bridge player
 - b. change-making machine
 - c. car cruise control
 - d. electronic typewriter
 - e. spelling checker
 - f. telephone answering machine
- 12.2 (7) Create a class diagram for each system from Exercise 11.6. Note that the requirements are incomplete, so your class models will also be incomplete.

Exercises 12.3–12.8 are related. Do the exercises in sequence. The following are tentative specifications for a simple diagram editor that could be used as the core of a variety of applications.

The editor will be used interactively to create and modify drawings. A drawing contains several sheets. Drawings are saved to and loaded from named ASCII files. Sheets contain boxes and links. Each box may optionally contain a single line of text. Text is allowed only in boxes. The editor must automatically adjust the size of a box to fit any enclosed text. The font size of the text is not adjustable. Any pair of boxes on the same sheet may be linked by a series of alternating horizontal and vertical lines. Figure E12.1 shows a simple, one sheet drawing.

The editor will be menu driven, with pop-up menus. A three-button mouse will be used for menu, object, and link selections. The following are some operations the editor should provide: create sheet, delete sheet, next sheet, previous sheet, create box, link boxes, enter text, group selection, cut selections, move selections, copy selections, paste, edit text, save drawing, and load drawing. Copy, cut,

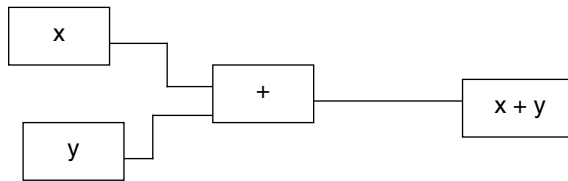


Figure E12.1 A sample drawing

and paste will work through a buffer. Copy will create a copy of selections from a sheet to the buffer. Cut will remove selections to the buffer. Paste will copy the contents of the buffer to the sheet. Each copy and cut operation overwrites the previous contents of the buffer. Pan and zoom will not be allowed; sheets will have fixed size. When boxes are moved, enclosed text should move with them and links should be stretched.

- 12.3 (3) The following is a list of candidate classes. Prepare a list of classes that should be eliminated for any of the reasons given in this chapter. Give a reason for each elimination. If there is more than one reason, give the main one.

character, line, x coordinate, y coordinate, link, position, length, width, collection, selection, menu, mouse, button, computer, drawing, drawing file, sheet, pop-up, point, menu item, selected object, selected line, selected box, selected text, file name, box, buffer, line segment coordinate, connection, text, name, origin, scale factor, corner point, end point, graphics object.

- 12.4 (3) Prepare a data dictionary for proper classes from the previous exercise.

- 12.5 (3) The following is a list of candidate associations and generalizations for the diagram editor. Prepare a list of associations and generalizations that should be eliminated or renamed for any of the reasons given in this chapter. Give a reason for each elimination or renaming. If there is more than one reason, give the main one.

a box has text, a box has a position, a link logically associates two boxes, a box is moved, a link has points, a link is defined by a sequence of points, a selection or a buffer or a sheet is a collection, a character string has a location, a box has a character string, a character string has characters, a line has length, a collection is composed of links and boxes, a link is deleted, a line is moved, a line is a graphical object, a point is a graphical object, a line has two points, a point has an x coordinate, a point has a y coordinate

- 12.6 Figure E12.2 is a partially completed class diagram for the diagram editor. Show how could it be used for each of the following queries. Use a combination of the OCL (see Chapter 3) and pseudocode to express your queries.

- (2) Find all selected boxes and links.
- (4) Given a box, determine all other boxes that are directly linked to it.
- (8) Given a box, find all other boxes that are directly or indirectly linked to it.
- (2) Given a box and a link, determine if the link involves the box.
- (3) Given a box and a link, find the other box logically connected to the given box through the other end of the link.
- (4) Given two boxes, determine all links between them.
- (6) Given a selection, determine which links are “bridging” links. If a selection does not include all boxes on a sheet, “bridging” links may result. A “bridging” link is a link that con-

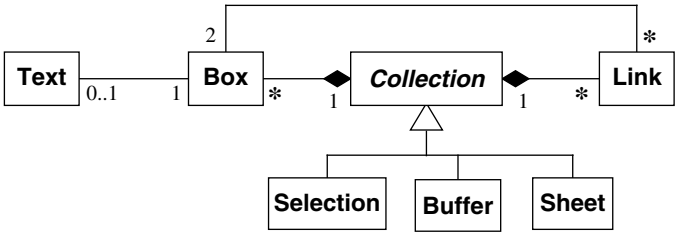


Figure E12.2 Partially completed class diagram for a diagram editor

nects a box that has been selected to a box that has not. A link that connects two boxes that are selected or two boxes that are not selected is not a “bridging” link. “Bridging” links require special handling during a *cut* or a *move* operation on a selection.

12.7 (6) Figure E12.3 is a variation of the class diagram in which the class *Connection* explicitly represents the connection of a link to a box. Redo the queries from the previous exercise using this representation.

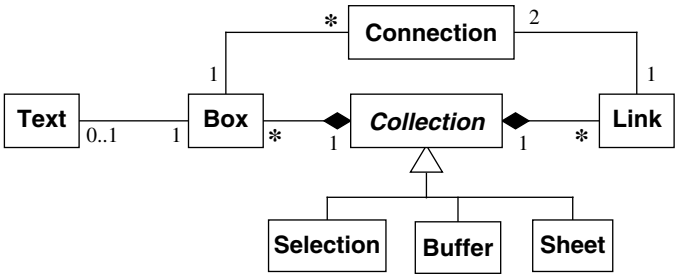


Figure E12.3 Alternative partially completed class diagram for a diagram editor

12.8 (5) What classes require state diagrams? Describe some relevant states and events.

Exercises 12.9–12.13 are related. Do the exercises in sequence. These exercises concern a computerized scoring system that you have volunteered to create for the benefit of a local children’s synchronized swimming league. Teams get together for competitions called meets during which the children perform in two types of events: figures and routines. Figure events, which are performed individually, are particular water ballet maneuvers such as swimming on your back with one leg raised straight up. Routines, which are performed by the entire team, are water ballets. Both figures and routines are scored, but your system need only address figures.

Each child must provide his or her name, age, address, and team name to register prior to the meet. To simplify scoring, each contestant is assigned a number.

During a meet, figure events are held simultaneously at several stations that are set up around a swimming pool, usually one at each corner. There are volunteer judges and scorekeepers. Scorekeepers tend to tire, so there is often turnover in their ranks. Several judges and scorekeepers are assigned to each station during a meet. Over the course of a season each judge and scorekeeper may serve sev-

eral stations. For scoring consistency, each figure is held at exactly one station with the same judges. A station may process several figure events in the course of a meet.

Contestants are organized into groups, with each group starting at a different station. When a child is finished at one station, he or she proceeds to another station for another event. When everyone has been processed at a station for a given event, the station switches to the next event assigned to it.

Each competitor gets one try at each event, called a trial. Just before a trial, the child's number is announced to the child and to the scorekeepers. Sometimes the children get out of order or the scorekeepers become confused and the station stops while the problem is fixed. Each judge indicates a raw score for each observed trial by holding up numbered cards. The raw scores are read to the scorekeepers, who record them and compute a net score for the trial. The highest and lowest raw scores are discarded, and the average of the remaining scores is multiplied by a difficulty factor for the figure.

Individual and team prizes are awarded at the conclusion of a meet based on top individual and team scores. There are several age categories, with separate prizes for each category. Individual prizes are based on figures only. Team prizes are based on figures and routines.

Your system will be used to store all information needed for scheduling, registering, and scoring. At the beginning of a season, all swimmers will be entered into the system and a season schedule will be prepared, including deciding which figures will be judged at which meets. Prior to a meet, the system will be used to process registrations. During a meet, it will record scores and determine winners.

12.9 (3) The following is a list of candidate classes for the scoring system. Prepare a list of classes that should be eliminated for any of the reasons given in this chapter. Give a reason for each elimination. If there is more than one reason, give the main one.

address, age, age category, average score, back, card, child, child's name, competitor, compute average, conclusion, contestant, corner, date, difficulty factor, event, figure, file of team member data, group, individual, individual prize, judge, league, leg, list of scheduled meets, meet, net score, number, person, pool, prize, register, registrant, raw score, routine, score, scorekeeper, season, station, team, team prize, team name, trial, try, water ballet.

12.10 (3) Prepare a data dictionary for proper classes from the previous exercise.

12.11 (4) The following is a list of candidate associations and generalizations for the scoring system. Prepare a list of associations and generalizations that should be eliminated or renamed for any of the reasons given in this chapter. Give a reason for each elimination or renaming. If there is more than one reason, give the main one.

a season consists of several meets, a competitor registers, a competitor is assigned a number, a number is announced, competitors are split into groups, a meet consists of several events, several stations are set up at a meet, several events are processed at a station, several judges are assigned to a station, routines and figures are events, raw scores are read, highest score is discarded, lowest score is discarded, figures are processed, a league consists of several teams, a team consists of several competitors, a trial of a figure is made by a competitor, a trial receives several scores from the judges, prizes are based on scores.

12.12 Figure E12.4 is a partially completed class diagram for the scoring system. The association between meet and event is not derived, because an event may be determined for a meet before a station is assigned to it. Show how it could be used for each of the following queries. Use a combination of the OCL (see Chapter 3) and pseudocode to express your queries.

- a. (2) Find all the members of a given team.
- b. (6) Find which figures were held more than once in a given season.

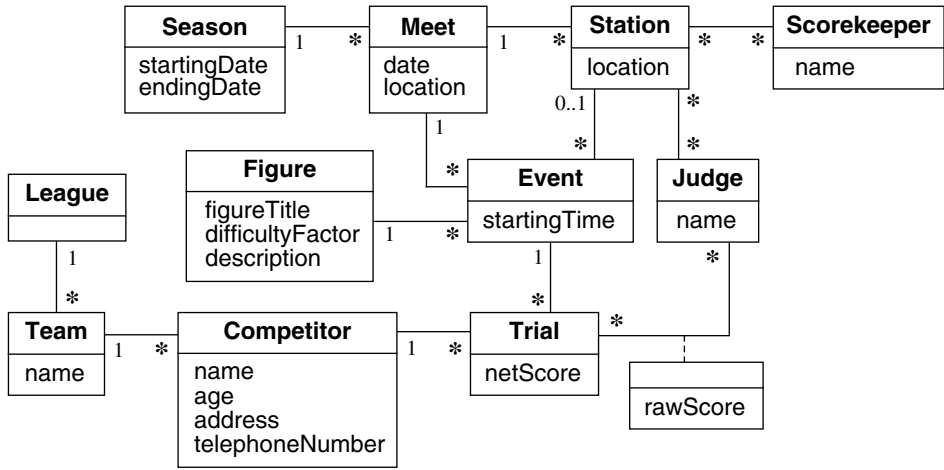


Figure E12.4 Partially completed class diagram for a scoring system

- c. (6) Find the net score of a competitor for a given figure at a given meet.
 - d. (6) Find the team average over all figures in a given season.
 - e. (6) Find the average score of a competitor over all figures in a given meet.
 - f. (6) Find the team average in a given figure at a given meet.
 - g. (4) Find the set of all individuals who competed in any events in a given season.
 - h. (7) Find the set of all individuals who competed in all of the events held in a given season.
 - i. (6) Find all the judges who judged a given figure in a given season.
 - j. (6) Find the judge who awarded the lowest score during a given event.
 - k. (6) Find the judge who awarded the lowest score for a given figure.
 - l. (7) Modify the diagram so that the competitors registered for an event can be determined.
- 12.13 (5) What classes require state diagrams? Describe some relevant states and events.
- 12.14 (7) Revise the diagrams in Figure E12.5, Figure E12.6, Figure E12.7, and Figure E12.8 to eliminate ternary associations. In some cases you will have to promote the association to a class.
- Figure E12.5 is a relationship between *Doctor*, *Patient*, and *DateTime* that might be encountered in a system used by a clinic with several doctors on the staff. The combination of *DateTime* + *Patient* is unique as well as *DateTime* + *Doctor*.
- Figure E12.6 is a relationship between *Student*, *Professor*, and *University* that might be used to express the contacts between students attending and professors teaching at several universities. There is one link in the relationship for a student that takes one or more classes from a professor at a university. The combination of *Student* + *Professor* + *University* is unique.
- Figure E12.7 shows the relationship expressing the seating arrangement at a concert. *Concert* + *Seat* is unique.
- Figure E12.8 expresses the connectivity of a directed graph. Each edge of a directed graph is connected in a specific order to exactly two vertices. More than one edge can be connected between a given pair of vertices. The attribute *Edge* is unique for the relationship.
- In each case, try to come as close as possible to the original intent and compare the merits of the original and the revised models.

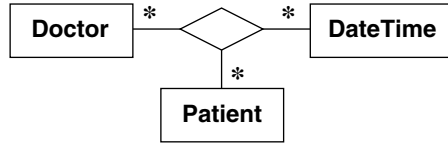


Figure E12.5 Ternary association for *Doctor*, *Patient*, and *DateTime*

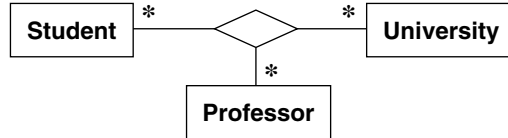


Figure E12.6 Ternary association for *Student*, *Professor*, and *University*

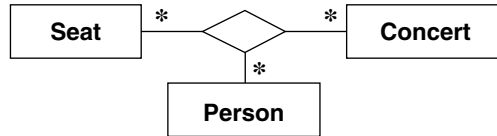


Figure E12.7 Ternary association for *Seat*, *Person*, and *Concert*

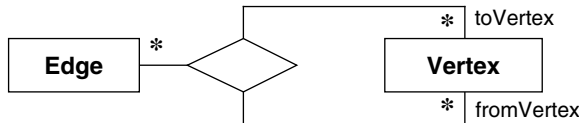


Figure E12.8 Ternary association for directed graphs

12.15 (9) Figure E12.9 lists requirements for a document manager. We then prepared the initial model in Figure E12.10. Note some flaws in the model.

Develop software for managing professional records of papers, books, journals, notes, and computer files. The system must be able to record authors of published works in the appropriate order, name of work, date of publication, publisher, publisher city, an abstract, as well as a comment. The software must be able to group published works into various categories that are defined by the user to facilitate searching. The user must be able to assign a quality indicator of the perceived value of each work.

Only some of the papers in each issue of a journal may be of interest. It would also be helpful to be able to attach comments to sections or even individual pages of a work.

Figure E12.9 Requirements for a document manager

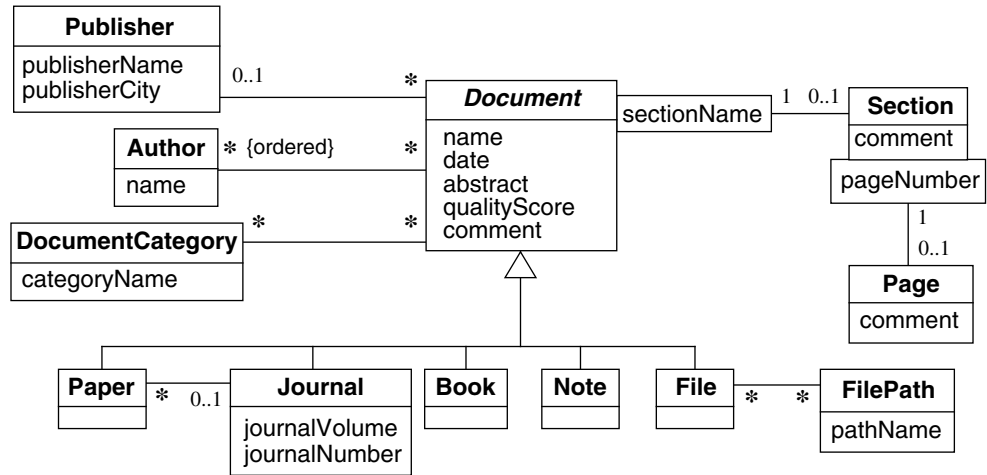


Figure E12.10 Initial model for a document manager

- There is little difference between subclasses. Is an outline of a paper a “paper” or a “note”? How should we handle a paper that is in both an electronic file and a binder? How should we represent information about slides for talks?
- We would like to handle both standard comments (applicable to many documents and chosen by point and click in a user interface) and custom comments (applicable to one document and specifically typed by the user).
- We should be able to comment on a numbered page without having sections.

Improve the model by making it more abstract. (Hint: You should have generic classes for location, document properties, and comments. It is adequate to represent document composition with a hierarchy.)

Exercises 12.16–12.19 are related. Do the exercises in sequence. The following are tentative specifications for scheduling software.

The scheduling software must support the following functions: arranges meetings, schedules appointments, plans tasks, and tracks holidays (including vacations).

The scheduler runs on a network that many users share. Each user may have a schedule. A schedule contains multiple entries. Most entries belong to a single schedule; however, a meeting entry may appear in many schedules.

There are four kinds of entries: meetings, appointments, tasks, and holidays. Meetings and appointments both occur within a single day and have a start time and end time. In contrast, tasks and holidays may extend over several days and just have a start date and end date. Any entry may be repeated. Repeat information includes how often the entry should be repeated, when it starts, and when it ends.

12.16 (3) The following is a list of candidate classes. Prepare a list of classes that should be eliminated for any of the reasons given in this chapter. Give a reason for each elimination. If there is more than one reason, give the main one.

scheduling software, function, meeting, appointment, task, holiday, vacation, scheduler, network, user, schedule, entry, meeting entry, day, start time, end time, start date, end date, repeat information.

12.17 (3) Prepare a data dictionary for proper classes from the previous exercise.

12.18 (4) The following is a list of candidate associations and generalizations for the scoring system. Prepare a list of associations and generalizations that should be eliminated or renamed for any of the reasons given in this chapter. Give a reason for each elimination or renaming. If there is more than one reason, give the main one.

- scheduling software that supports the following functions
- the scheduler runs on a network that many users share
- user may have a schedule
- a schedule contains multiple entries
- entries pertain to a single schedule
- a meeting entry may appear in many schedules
- meetings and appointments both occur within a single day and have a start time and end time
- tasks and holidays may extend over several days and just have a start date and end date.

12.19 (5) Construct a class model for the scheduling software.

Exercises 12.20–12.23 are related. Do the exercises in sequence. The following provides requirements for meetings and extends the scheduling software from Exercises 12.16–12.19.

The scheduling software facilitates meetings. When a user (the chairperson) arranges a meeting, the software places a meeting entry in the schedule of each attendee. The chairperson uses the scheduler to reserve a room for the meeting, to identify the attendees, and to find time on their schedules when everyone is available. The chairperson can indicate whether the attendance for each attendee is required or optional. The system tracks the acceptance status for each attendee—whether an attendee has accepted or declined.

The scheduler manages meeting notices. When a meeting is set up, the scheduler sends invitations to all attendees, who are able to view meeting information. Each invitee can accept or refuse as well as possibly cancel later on. The system also manages notices in case the meeting is rescheduled or cancelled.

12.20 (3) The following is a list of candidate classes. Prepare a list of classes that should be eliminated for any of the reasons given in this chapter. Give a reason for each elimination. If there is more than one reason, give the main one.

scheduling software, meeting, user, chairperson, software, meeting entry, schedule, attendee, scheduler, room, time, everyone, attendance, acceptance status, meeting notice, invitation, meeting information, invitee, notice.

12.21 (3) Prepare a data dictionary for proper classes from the previous exercise.

12.22 (4) The following is a list of candidate associations and generalizations. Prepare a list of associations and generalizations that should be eliminated or renamed for any of the reasons given in this chapter. Give a reason for each elimination or renaming. If there is more than one reason, give the main one.

- scheduling software facilitates meetings

- user (the chairperson) arranges a meeting
- software places a meeting entry in the schedule of each attendee
- chairperson uses the scheduler to reserve a room for the meeting, to identify the attendees, and to find time on their schedules when everyone is available
- chairperson can indicate whether the attendance for each attendee is required or optional
- system tracks the acceptance status for each attendee—whether an attendee has accepted or declined
- scheduler manages meeting notices
- scheduler sends invitations to all attendees, who are able to view meeting information
- system also manages notices in case the meeting is rescheduled or cancelled.

12.23 (7) Construct a class model for the extension to the scheduling software. Your answer should resolve a problem from Exercise 12.19. In the class model for our answer to Exercise 12.19, we cannot tell which user owns an entry. (Hint: You should reconcile the chairperson and attendee associations from the extended requirements with the association between *Schedule* and *Entry* from the Exercise 12.16–12.19 requirements.)

13

Application Analysis

This chapter completes our treatment of analysis by adding major application artifacts to the domain model from the prior chapter. We include these application artifacts in analysis, because they are important, visible to users, and must be approved by them. In general, you cannot find the application classes in the domain itself, but must find them in use cases.

13.1 Application Interaction Model

Most domain models are static and operations are unimportant, because a domain as a whole usually doesn't *do* anything. The focus of domain modeling is on building a model of intrinsic concepts. After completing the domain model we then shift our attention to the details of an application and consider interaction.

Begin interaction modeling by determining the overall boundary of the system. Then identify use cases and flesh them out with scenarios and sequence diagrams. You should also prepare activity diagrams for use cases that are complex or have subtleties. Once you fully understand the use cases, you can organize them with relationships. And finally check against the domain class model to ensure that there are no inconsistencies.

You can construct an application interaction model with the following steps.

- Determine the system boundary. [13.1.1]
- Find actors. [13.1.2]
- Find use cases. [13.1.3]
- Find initial and final events. [13.1.4]
- Prepare normal scenarios. [13.1.5]
- Add variation and exception scenarios. [13.1.6]
- Find external events. [13.1.7]
- Prepare activity diagrams for complex use cases. [13.1.8]

- Organize actors and use cases. [13.1.9]
- Check against the domain class model. [13.1.10]

13.1.1 Determining the System Boundary

You must know the precise scope of an application—the boundary of the system—in order to specify functionality. This means that you must decide what the system includes and, more importantly, what it omits. If the system boundary is drawn correctly, you can treat the system as a black box in its interactions with the outside world—you can regard the system as a single object, whose internal details are hidden and changeable. During analysis, you determine the purpose of the system and the view that it presents to its actors. During design, you can change the internal implementation of the system as long as you maintain the external behavior.

Usually, you should not consider humans as part of a system, unless you are modeling a human organization, such as a business or a government department. Humans are actors that must interact with the system, but their actions are not under the control of the system. However, you must allow for human error in your system.

ATM example. The original problem statement from Chapter 11 says to “design the software to support a computerized banking network including both human cashiers and automatic teller machines...” Now it is important that cashier transactions and ATM transactions be seamless—from the customer’s perspective either method of conducting business should yield the same effect on a bank account. However, in commercial practice an ATM application would be separate from a cashier application—an ATM application spans banks while a cashier application is internal to a bank. Both applications would share the same underlying domain model, but each would have its own distinct application model. For this chapter we focus on ATM behavior and ignore cashier details.

13.1.2 Finding Actors

Once you determine the system boundary, you must identify the external objects that interact directly with the system. These are its *actors*. Actors include humans, external devices, and other software systems. The important thing about actors is that they are not under control of the application, and you must consider them to be somewhat unpredictable. That is, even though there may be an expected sequence of behavior by the actors, an application’s design should be robust so that it does not crash if an actor fails to behave as expected.

In finding actors, we are not searching for individuals but for archetypical behavior. Each actor represents an idealized user that exercises some subset of the system functionality. Examine each external object to see if it has several distinct faces. An actor is a coherent face presented to the system, and an external object may have more than one actor. It is also possible for different kinds of external objects to play the part of the same actor.

ATM example. A particular person may be both a bank teller and a customer of the same bank. This is an interesting but usually unimportant coincidence—a person approaches the bank in one or the other role at a time. For the ATM application, the actors are *Customer*, *Bank*, and *Consortium*.

may use memory data structures and files. An accounting system may use a database to connect subsystems.

Files are cheap, simple, and permanent. However, file operations are low level, and applications must include additional code to provide a suitable level of abstraction. File implementations vary for different computer systems, so portable applications must carefully isolate file-system dependencies. Implementations for sequential files are mostly standard, but commands and storage formats for random-access files and indexed files vary. Figure 14.3 characterizes the kind of data that belongs in files.

- Data with high volume and low information density (such as archival files or historical records).
- Modest quantities of data with simple structure.
- Data that are accessed sequentially.
- Data that can be fully read into memory.

Figure 14.3 Data suitable for files. Files provide a low-tech solution to data management and should not be overlooked.

Databases, managed by database management systems (DBMSs), are another kind of data store. Various types of DBMSs are available from vendors, including relational and OO. DBMSs cache frequently accessed data in memory in order to achieve the best combination of cost and performance from memory and disk storage. Databases make applications easier to port to different hardware and operating system platforms, since the vendor ports the DBMS code. One disadvantage of DBMSs is their complex interface—many database languages integrate awkwardly with programming languages. Figure 14.4 characterizes the kinds of data that belong in a database.

- Data that require updates at fine levels of detail by multiple users.
- Data that must be accessed by multiple application programs.
- Data that require coordinated updates via transactions.
- Large quantities of data that must be handled efficiently.
- Data that are long-lived and highly valuable to an organization.
- Data that must be secured against unauthorized and malicious access.

Figure 14.4 Data suitable for databases. Databases provide heavyweight data management and are used for most important business applications.

OO-DBMSs have not become popular in the mass market. Consequently you should consider them only for specialty applications that have a wide variety of data types or that

must access low-level data management primitives. These applications include engineering applications, multimedia applications, knowledge bases, and electronic devices with embedded software. For most applications that need a database, you should use a relational DBMS (RDBMS). RDBMSs dominate the marketplace, and their features are sufficient for most applications. RDBMSs can also provide a very good implementation of an OO model, *if* they are used properly—Chapter 19 presents the details.

ATM example. The typical bank computer would use a relational DBMS—they are fast, readily available, and cost-effective for these kinds of financial applications.

The ATM might also use a database, but the paradigm for that is less obvious. Relational and OO-DBMSs would both be possibilities. Many OO-DBMSs permit access to low-level primitives, and a stripped-down database might enable mass production of ATM software at a low cost. A stripped-down database might also simplify ATM operation. Alternatively, RDBMSs are mature products with many features that might reduce development effort.

14.8 Handling Global Resources

The system designer must identify global resources and determine mechanisms for controlling access to them. There are several kinds of global resources.

- **Physical units.** Examples include processors, tape drives, and communication satellites.
- **Space.** Examples include disk space, a workstation screen, and the buttons on a mouse.
- **Logical names.** Examples include object IDs, filenames, and class names.
- **Access to shared data.** Databases are an example.

If the resource is a physical object, then it can control itself by establishing a protocol for obtaining access. If the resource is a logical entity, such as an object ID or a database, then there is danger of conflicting access in a shared environment. Independent tasks could simultaneously use the same object ID, for example.

You can avoid conflict by having a “guardian object” own each global resource and control access to it. One guardian object can control several resources. All access to the resource must pass through the guardian object. Allocating each shared global resource to a single object is a recognition that the resource has identity.

You can also partition a resource logically, assigning subsets to different guardian objects for independent control. For example, one strategy for object ID generation in a parallel distributed environment is to preallocate a range of possible IDs to each processor in a network; each processor allocates the IDs within its preallocated range without the need for global synchronization.

In a time-critical application, the cost of passing all access to a resource through a guardian object is sometimes too high, and clients must access the resource directly. In this case, locks can be placed on subsets of the resource. A **lock** is a logical object associated with some defined subset of a resource that gives the lock holder the right to access the resource directly. A guardian object must still exist to allocate the locks, but after one interaction with the guardian to obtain a lock the user of the resource can access the resource directly. This approach is more dangerous, because each resource user must be trusted to behave itself in its

access to the resource. Do not use direct access to shared resources unless it is absolutely necessary.

ATM example. Bank codes and account numbers are global resources. Bank codes must be unique within the context of a consortium. Account codes must be unique within the context of a bank.

14.9 Choosing a Software Control Strategy

The analysis model shows interactions as events between objects. Hardware control closely matches the analysis model, but there are several ways for implementing control in software. Although all subsystems need not use the same implementation, it is best to choose a single control style for the whole system. There are two kinds of control flows in a software system: external control and internal control.

External control concerns the flow of externally visible events among the objects in the system. There are three kinds of control for external events: procedure-driven sequential, event-driven sequential, and concurrent. The appropriate control style depends on the available resources (language, operating system) and on the kind of interactions in the application.

Internal control refers to the flow of control within a process. It exists only in the implementation and therefore is neither inherently concurrent nor sequential. The designer may choose to decompose a process into several tasks for logical clarity or for performance (if multiple processors are available). Unlike external events, internal transfers of control, such as procedure calls or intertask calls, are under the direction of the program and can be structured for convenience. Three kinds of control flow are common: procedure calls, quasi-concurrent intertask calls, and concurrent intertask calls. Quasi-concurrent intertask calls, such as coroutines or lightweight processes, are programming conveniences in which multiple address spaces or call stacks exist but only a single thread of control can be active at once.

14.9.1 Procedure-driven Control

In a procedure-driven sequential system, control resides within the program code. Procedures request external input and then wait for it; when input arrives, control resumes within the procedure that made the call. The location of the program counter and the stack of procedure calls and local variables define the system state.

The major advantage of procedure-driven control is that it is easy to implement with conventional languages; the disadvantage is that it requires the concurrency inherent in objects to be mapped into a sequential flow of control. The designer must convert events into operations between objects. A typical operation corresponds to a pair of events: an output event that performs output and requests input and an input event that delivers the new values. This paradigm cannot easily accommodate asynchronous input, because the program must explicitly request input. The procedure-driven paradigm is suitable only if the state model shows a regular alternation of input and output events. Flexible user interfaces and control systems are hard to build with this style.

Note that all major OO languages, such as C++ and Java, are procedural languages. Do not be fooled by the OO phrase *message passing*. A message is a procedure call with a built-

in case statement that depends on the class of the target object. A major drawback of conventional OO languages is that they fail to support the concurrency inherent in objects. Some concurrent OO languages have been designed, but they are not yet widely used.

14.9.2 Event-driven Control

In an event-driven sequential system, control resides within a dispatcher or monitor that the language, subsystem, or operating system provides. Developers attach application procedures to events, and the dispatcher calls the procedures when the corresponding events occur (“callback”). Procedure calls to the dispatcher send output or enable input but do not wait for it in-line. All procedures return control to the dispatcher, rather than retaining control until input arrives. Consequently, the program counter and stack cannot preserve state. Procedures must use global variables to maintain state, or the dispatcher must maintain local state for them. Event-driven control is more difficult to implement with standard languages than procedure-driven control but is often worth the extra effort.

Event-driven systems permit more flexible control than procedure-driven systems. Event-driven systems simulate cooperating processes within a single multithreaded task; an errant procedure can block the entire application, so you must be careful. Event-driven user interface subsystems are particularly useful.

Use an event-driven system for external control in preference to a procedure-driven system whenever possible, because the mapping from events to program constructs is simpler and more powerful. Event-driven systems are also more modular and can handle error conditions better than procedure-driven systems.

14.9.3 Concurrent Control

In a concurrent system, control resides concurrently in several independent objects, each a separate task. Such a system implements events directly as one-way messages (*not* OO language “messages”) between objects. A task can wait for input, but other tasks continue execution. The operating system resolves scheduling conflicts among tasks and usually supplies a queuing mechanism, so that events are not lost if a task is executing when they arrive. If there are multiple CPUs, then different tasks can actually execute concurrently.

14.9.4 Internal Control

During design, the developer expands operations on objects into lower-level operations on the same or other objects. Internal object interactions are similar to external object interactions, because you can use the same implementation mechanisms. However, there is an important difference—external interactions inherently involve waiting for events, because objects are independent and cannot force other objects to respond; objects generate internal operations as part of the implementation algorithm, so their form of response is predictable. Consequently, you can think of most internal operations as procedure calls, in which the caller issues a request and waits for the response. There are algorithms for parallel processing, but many computations are well represented sequentially and can easily be folded onto a single thread of control.

14.9.5 Other Paradigms

We assume that the reader is primarily interested in procedural programming, but other paradigms are possible, such as rule-based systems, logic programming systems, and other forms of nonprocedural programs. These constitute another control style in which explicit control is replaced by declarative specification with implicit evaluation rules, possibly non-deterministic or highly convoluted. Developers currently use such languages in limited areas, such as artificial intelligence and knowledge-based programming, but we expect their use to grow in the future. Because these languages are totally different from procedural languages (including OO languages), the remainder of this book has little to say about them.

ATM example. Event-driven control is the appropriate paradigm for the ATM station. The ATM services a single user, so there is little need for concurrent control. The ATM must be responsive in its user interactions, and event-driven control is much better at that than procedure-driven control.

14.10 Handling Boundary Conditions

Although most of system design concerns steady-state behavior, you must consider boundary conditions as well and address the following kinds of issues.

- **Initialization.** The system must proceed from a quiescent initial state to a sustainable steady state. The system must initialize constant data, parameters, global variables, tasks, guardian objects, and possibly the class hierarchy itself. During initialization only a subset of the functionality of the system is usually available. Initializing a system containing concurrent tasks is most difficult, because independent objects must not get either too far ahead or too far behind other independent objects during initialization.
- **Termination.** Termination is usually simpler than initialization, because many internal objects can simply be abandoned. The task must release any external resources that it had reserved. In a concurrent system, one task must notify other tasks of its termination.
- **Failure.** Failure is the unplanned termination of a system. Failure can arise from user errors, from the exhaustion of system resources, or from an external breakdown. The good system designer plans for orderly failure. Failure can also arise from bugs in the system and is often detected as an “impossible” inconsistency. In a perfect design, such errors would never happen, but the good designer plans for a graceful exit on fatal bugs by leaving the remaining environment as clean as possible and recording or printing as much information about the failure as possible before terminating.

14.11 Setting Trade-off Priorities

The system designer must set priorities that will be used to guide trade-offs for the rest of design. These priorities reconcile desirable but incompatible goals. For example, a system can often be made faster by using extra memory, but that increases power consumption and costs more. Design trade-offs involve not only the software itself but also the process of developing it. Sometimes it is necessary to sacrifice complete functionality to get a piece of

software into use (or into the marketplace) earlier. Sometimes the problem statement specifies priority, but often the burden falls on the designer to reconcile the incompatible desires of the client and decide how to make trade-offs.

The system designer must determine the relative importance of the various criteria as a guide to making design trade-offs. The system designer does not *make* all the trade-offs, but establishes the priorities for making them. For example, the first video games ran on processors with limited memory. Conserving memory was the highest priority, followed by fast execution. Designers had to use every programming trick in the book, at the expense of maintainability, portability, and understandability. As another example, mathematical subroutine packages run on a wide range of machines. Well-conditioned numerical behavior is crucial to such packages, as well as portability and understandability. These cannot be sacrificed for fast development.

Design trade-offs affect the entire character of a system. The success or failure of the final product may depend on how well its goals are chosen. Even worse, if no system-wide priorities are established, then the various parts of the system may optimize opposing goals (“suboptimization”), resulting in a system that wastes resources. Even on small projects, programmers often forget the real goals and become obsessed with “efficiency” when it is really unimportant.

Setting trade-off priorities is at best vague. You cannot expect numerical accuracy (“speed 53%, memory 31%, portability 15%, cost 1%”). Priorities are rarely absolute; for example, trading memory for speed does not mean that any increase in speed, no matter how small, is worth any increase in memory, no matter how large. We cannot even give a full list of design criteria that might be subject to trade-offs. Instead, the priorities are a statement of design philosophy. Subsequent design will still require judgment and interpretation when trade-offs are actually made.

ATM example. The ATM station is a mass-market product. Consequently, the manufacturing cost is a concern, and the resulting product must have a polished user interface. The software must be robust and resilient in the face of failure. Development cost is a lesser concern, since the cost can be amortized across numerous copies.

14.12 Common Architectural Styles

Several prototypical architectural styles are common in existing systems. Each of these is well suited to a certain kind of system. If you have an application with similar characteristics, you can save effort by using the corresponding architecture, or at least using it as a starting point for your design. Some kinds of systems are listed below.

- **Batch transformation**—a data transformation executed once on an entire input set. [14.12.1]
- **Continuous transformation**—a data transformation performed continuously as inputs change. [14.12.2]
- **Interactive interface**—a system dominated by external interactions. [14.12.3]

- **Dynamic simulation**—a system that simulates evolving real-world objects. [14.12.4]
- **Real-time system**—a system dominated by strict timing constraints. [14.12.5]
- **Transaction manager**—a system concerned with storing and updating data, often including concurrent access from different physical locations. [14.12.6]

This is not meant to be a complete list of known systems and architectures but a list of common forms. Some problems require a new kind of architecture, but most can use an existing style or at least a variation on it. Many problems combine aspects of these architectures.

14.12.1 Batch Transformation

A *batch transformation* performs sequential computations. The application receives the inputs, and the goal is to compute an answer; there is no ongoing interaction with the outside world. Examples include standard computational problems such as compilers, payroll processing, VLSI automatic layout, stress analysis of a bridge, and many others. The state model is trivial or nonexistent for batch transformation problems. The class model is important—there are class models for the input, output, and the intervening stages. The interaction model documents the computation and couples the class models. The most important aspect of a batch transformation is to define a clean series of steps.

In the past, when we worked at GE R&D, one of our colleagues (Bill Premierlani) built a compiler that received an ASCII file of graphical pictures as input and generated relational database definition code as output. This work preceded the availability of commercial OO modeling tools. Figure 14.5 shows the sequence of steps. The compiler had five class models—one for the input, one for the output, and three for intermediate representations.

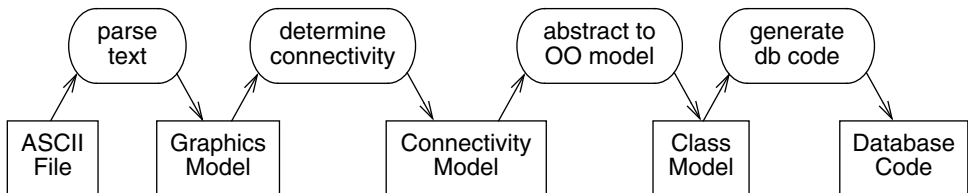


Figure 14.5 Sequence of steps for a compiler. A batch transformation is a sequential input-to-output transformation that does not interact with the outside world.

The steps in designing a batch transformation are as follows.

- Break the overall transformation into stages, with each stage performing one part of the transformation.
- Prepare class models for the input, output, and between each pair of successive stages. Each stage knows only about the models on either side of it.
- Expand each stage in turn until the operations are straightforward to implement.
- Restructure the final pipeline for optimization.

14.12.2 Continuous Transformation

A *continuous transformation* is a system in which the outputs actively depend on changing inputs. Unlike a batch transformation that computes the outputs only once, a continuous transformation updates outputs frequently (in theory continuously, although in practice they are computed discretely at a fine time scale). Because of severe time constraints, the system cannot recompute the entire set of outputs each time an input changes (otherwise the application would be a batch transformation). Instead, the system must compute outputs incrementally. Typical applications include signal processing, windowing systems, incremental compilers, and process monitoring systems. The class, state, and interaction models have similar purposes as with the batch transformation.

One way to implement a continuous transformation is with a pipeline of functions. The pipeline propagates the effect of each input change. Developers can define intermediate and redundant objects to improve the performance of the pipeline. Some high-performance systems, such as signal processing, need to synchronize values within the pipeline. Such systems perform operations at well-defined times and carefully balance the flow path of operations so that values arrive at the right place at the right time without bottlenecks.

Figure 14.6 shows the example of a graphics application. The application first maps geometric figures in user-defined coordinates to window coordinates. Then it clips the figures to fit the window bounds. Finally it offsets each figure by its window position to yield its screen position.

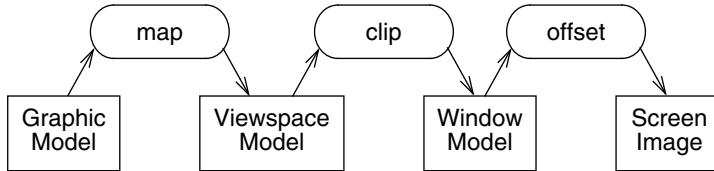


Figure 14.6 Sequence of steps for a graphics application. A continuous transformation repeatedly propagates input changes to the output.

The steps in designing a pipeline for a continuous transformation are as follows.

- Break the overall transformation into stages, with each stage performing one part of the transformation.
- Define input, output, and intermediate models between each pair of successive stages, as for the batch transformation.
- Differentiate each operation to obtain incremental changes to each stage. That is, propagate the incremental effects of each change to an input through the pipeline as a series of incremental updates.
- Add additional intermediate objects for optimization.

14.12.3 Interactive Interface

An *interactive interface* is a system that is dominated by interactions between the system and external agents, such as humans or devices. The external agents are independent of the system, so the system cannot control the agents, although it may solicit responses from them. An interactive interface usually includes only part of an entire application, one that can often be handled independently from computations. Examples of interactive systems include a forms-based query interface, a workstation windowing system, and the control panel for a simulation.

The major concerns of an interactive interface are the communications protocol between the system and the external agents, the syntax of possible interactions, the presentation of output (the appearance on the screen, for instance), the flow of control within the system, performance, and error handling. Interactive interfaces are dominated by the state model. The class model represents interaction elements, such as input and output tokens and presentation formats. The interaction model shows how the state diagrams interact.

The steps in designing an interactive interface are as follows.

- Isolate interface classes from the application classes.
- Use predefined classes to interact with external agents, if possible. For example, windowing systems have extensive collections of predefined windows, menus, buttons, forms, and other kinds of classes ready to be adapted to applications.
- Use the state model as the structure of the program. Interactive interfaces are best implemented using concurrent control (multitasking) or event-driven control (interrupts or call-backs). Procedure-driven control (writing output and then waiting for input in-line) is awkward for anything but rigid control sequences.
- Isolate physical events from logical events. Often a logical event corresponds to multiple physical events. For example, a graphical interface can take input from a form, from a pop-up menu, from a function button on the keyboard, from a typed-in command sequence, or from an indirect command file.
- Fully specify the application functions that are invoked by the interface. Make sure that the information to implement them is present.

14.12.4 Dynamic Simulation

A *dynamic simulation* models or tracks real-world objects. Examples include molecular motion modeling, spacecraft trajectory computation, economic models, and video games. Simulations are perhaps the simplest system to design using an OO approach. The objects and operations come directly from the application. There are two ways for implementing control: an explicit controller external to the application objects can simulate a state machine, or objects can exchange messages among themselves, similar to the real-world situation.

Unlike an interactive system, the internal objects in a dynamic simulation do correspond to real-world objects, so the class model is usually important and often complex. Like an interactive system, the state and interaction models are also important.

The steps in designing a dynamic simulation are as follows.

- Identify active real-world objects from the class model. These objects have attributes that are periodically updated.
- Identify discrete events. Discrete events correspond to discrete interactions with the object, such as turning power on or applying the brakes. Discrete events can be implemented as operations on the object.
- Identify continuous dependencies. Real-world attributes may be dependent on other real-world attributes or vary continuously with time, altitude, velocity, or steering wheel position, for example. These attributes must be updated at periodic intervals, using numerical approximation techniques to minimize quantization error.
- Generally a simulation is driven by a timing loop at a fine time scale. Discrete events between objects can often be exchanged as part of the timing loop.

Usually, the hardest problem with simulations is providing adequate performance. In an ideal world, an arbitrary number of parallel processors would execute the simulation in an exact analogy to the real-world situation. In practice, the system designer must estimate the computational cost of each update cycle and provide adequate resources. Discrete steps must approximate continuous processes.

14.12.5 Real-time System

A *real-time system* is an interactive system with tight time constraints on actions. Hard real-time software involves critical applications that require a guaranteed response within the time constraints. In contrast, soft real-time software must also be highly reliable, but can occasionally violate time constraints. Typical real-time applications include process control, data acquisition, communications devices, device control, and overload relays.

Real-time design is complex and involves issues such as interrupt handling, prioritization of tasks, and coordinating multiple CPUs. Unfortunately, real-time systems are frequently designed to operate close to their resource limits, so that severe, nonlogical restructuring of the design is often needed to achieve the necessary performance. Such contortions come at the cost of portability and maintainability. Real-time design is a specialized topic that we do not cover in this book.

14.12.6 Transaction Manager

A *transaction manager* is a system whose main function is to store and retrieve data. Most transaction managers deal with multiple users who read and write data at the same time. They also must secure their data to protect it from unauthorized access as well as accidental loss. Transaction managers are often built on top of a database management system (DBMS)—this is a form of reuse. A DBMS has generic functionality for managing data that you can reuse and need not implement. Examples of transaction managers include airline reservations, inventory control, and order fulfillment.

The class model is dominant. The state model is occasionally important, especially for specifying the evolution of an object as well as constraints and methods that apply at different points in time. The interaction model is seldom significant.

The steps in designing an information system are as follows.

- Map the class model to database structures. See Chapter 19 for advice.
- Determine the units of concurrency—that is, the resources that inherently or by specification cannot be shared. Introduce new classes as needed.
- Determine the unit of transaction—that is, the set of resources that must be accessed together during a transaction. A transaction succeeds or fails in its entirety.
- Design concurrency control for transactions. Most database management systems provide this. The system may need to retry failed transactions several times before giving up.

14.13 Architecture of the ATM System

The ATM system is a hybrid of an interactive interface and a transaction management system. The entry stations are interactive interfaces—their purpose is to interact with a human to gather information needed to formulate a transaction. Specifying the entry stations consists of constructing a class model and a state model. The consortium and banks are primarily a distributed transaction management system. Their purpose is to maintain data and allow it to be updated over a distributed network under controlled conditions. Specifying the transaction management part of the system consists primarily of constructing a class model. Figure 14.2 shows the architecture of the ATM system.

The only permanent data stores are in the bank computers. A database ensures that data is consistent and available for concurrent access. The ATM system processes each transaction as a single batch operation, locking an account until the transaction is complete.

Concurrency arises because there are many ATM stations, each of which can be active at any time. There can be only one transaction per ATM station, but each transaction requires the assistance of the consortium computer and a bank computer. As Figure 14.2 shows, a transaction cuts across physical units; the diagram shows each transaction as three connected pieces. During design, each piece will become a separate implementation class. Although there is only one transaction per ATM station, there may be many concurrent transactions per consortium computer or bank computer. This does not pose any special problem, because the database synchronizes access to any one account.

The consortium computer and bank computers will be event driven. Each of them queues input events but processes them one at a time in the order received. The consortium computer has minimal functionality. It simply forwards a message from an ATM station to a bank computer and from a bank computer to an ATM station. The consortium computer must be large enough to handle the transaction load. It may be acceptable to block an occasional transaction, provided the user receives an appropriate message.

The bank computer is the only unit with any nontrivial procedures, but even those are mostly just database updates. The only complexity might come from failure handling. The bank computers must have capacity to handle the expected worst-case load, and they must have enough disk storage to record all transactions.

The system must contain operations for adding and deleting ATM stations and bank computers. Each physical unit must protect itself against the failure or disconnection from the rest of the network. A database protects against loss of data. However, special attention must be paid to failure during a transaction so that neither the user nor the bank loses money—this may require a complicated acknowledgment protocol before committing the transaction. The ATM station should display an appropriate message if the connection is down. The ATM must handle other kinds of failure as well, such as exhaustion of cash or paper for receipts.

On a financial system such as this, fail-safe transactions are the highest priority. If there is any doubt about the integrity of a transaction, then the ATM must abort the transaction with an appropriate message to the user.

14.14 Chapter Summary

After analyzing an application and before beginning the class design, the system designer must decide on the basic approach to the solution. The form of the high-level strategy for building the system is called the system architecture.

Early in the planning for a new system you should estimate the performance. The intention is to have a rough idea of what to expect. You want to make sure that it is reasonable and that there are no big surprises as development proceeds.

Next, prepare a reuse plan. Reuse is often cited as a benefit of OO technology, but it does not happen automatically. There are two different aspects of reuse. Most developers should focus on reusing existing models, libraries, frameworks, and patterns that are relevant to their applications. In addition, elite developers can create artifacts for reuse by others.

A system can be divided into horizontal layers and vertical partitions. Each layer defines a different abstract world that may differ completely from other layers. Each layer is a client of services of the layer or layers below it and a server of services for the layer or layers above it. Systems can also have partitions, each performing a general kind of service. Simple system topologies, such as pipelines or stars, reduce complexity. Most systems are a mixture of layers and partitions.

Inherently concurrent objects execute in parallel, and a single thread of control cannot combine them; they require separate hardware devices or separate tasks in a processor. You can combine nonconcurrent objects onto a single thread of control and implement them as a single task.

A system must have enough processors and special-purpose hardware units to meet performance goals. You should assign objects to hardware so that hardware use is balanced and meets concurrency constraints. You can do this by estimating computational throughput and allowing for queuing effects in configuring the hardware. You may want to use special-purpose hardware for compute-intensive computations. One goal in partitioning a hardware network is to minimize communications traffic between physically distinct modules.

Data stores can cleanly separate subsystems within an architecture and give application data some degree of permanence. In general, memory data structures, files, and databases can implement data stores. Files are simple, cheap, and permanent but may provide too low a level of abstraction for an application and necessitate much additional programming. Da-

tabases provide a higher level of abstraction than files, but they too involve compromises in terms of overhead costs and complexity.

The system designer must identify global resources and determine mechanisms for controlling access to them. Some common mechanisms are: establishing a “guardian” object that serializes all access, partitioning global resources into disjoint subsets which are managed at a lower level, and locking.

Hardware control is inherently concurrent, but software control can be procedure driven, event driven, and concurrent. Control for a procedure-driven system resides within the program code; the location of the program counter and the stack of procedure calls and local variables define the system state. In an event-driven system control resides within a dispatcher or monitor; application procedures are attached to events and are called by the dispatcher when the corresponding events occur. In a concurrent system, control resides concurrently in multiple independent objects. Event-driven and concurrent implementations are much more flexible than procedure-driven control.

Most of system design is concerned with steady-state behavior, but boundary conditions (initialization, termination, and failure) are also important.

An essential aspect of system architecture is making trade-offs between time and space, hardware and software, simplicity and generality, and efficiency and maintainability. These trade-offs depend on the goals of the application. The system designer must state the priorities, so that trade-off decisions during subsequent design will be consistent.

Several kinds of systems are frequently encountered for which standard architectural styles exist. These include two kinds of functional transformations: batch computation and continuous transformation; three kinds of time-dependent systems: interactive interface, dynamic simulation, and real-time; and a database system: transaction manager. Most application systems are usually a hybrid of several forms, possibly one for each major subsystem. Other kinds of architecture are possible.

architecture	hardware requirements	service
client-server	inherent concurrency	subsystem
concurrency	layer	system design
data management	partition	system topology
event-driven system	peer-to-peer	thread of control
framework	reuse plan	trade-off priorities

Figure 14.7 Key concepts for Chapter 14

Bibliographic Notes

Simple software applications do not require much systems engineering, but complex systems must be decomposed and the parts assigned to the appropriate specialists. [Clements-02] presents a process for evaluating software architectures. Essentially a group of stakeholders meet and prioritize criteria that the architecture should satisfy; they quantify the criteria with

specific scenarios. Then they analyze the architecture to determine its compliance with the high-priority scenarios.

Patterns are a popular topic in the literature and the subject of a number of books. There are patterns for analysis [Coad-95], architecture [Buschmann-96] [Shaw-96], design [Gamma-95], and implementation [Coplien-92]. There have also been a number of conferences over the years that have focused on patterns, many of which have been sponsored by the Pattern Languages of Programming [PLoP].

References

- [Berlin-90] Lucy Berlin. When objects collide: Experiences with reusing multiple class hierarchies. *ECOOP/OOPSLA 1990 Proceedings*, October 21–25, 1990, Ottawa, Ontario, Canada, 181–193.
- [Buschmann-96] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. *Pattern-Oriented Software Architecture: A System of Patterns*. Chichester, UK: Wiley, 1996.
- [Clements-02] Paul Clements, Rick Kazman, and Mark Klein. *Evaluating Software Architectures*. Boston: Addison-Wesley, 2002.
- [Coad-95] Peter Coad, David North, and Mark Mayfield. *Object Models: Strategies, Patterns, and Applications*. Upper Saddle River, NJ: Yourdon Press, 1995.
- [Coplien-92] James O. Coplien. *Advanced C++ Programming Styles and Idioms*. Boston: Addison-Wesley, 1992.
- [Gamma-95] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1995.
- [Johnson-88] Ralph E. Johnson and Brian Foote. Designing reusable classes. *Journal of Object-Oriented Programming* 1, 3 (June/July 1988), 22–35.
- [Korson-92] Tim Korson and John D. McGregor. Technical criteria for the specification and evaluation of object-oriented libraries. *Software Engineering Journal* (March 1992), 85–94.
- [PLoP] jerry.cs.uiuc.edu/~plop
- [Shaw-96] Mary Shaw and David Garlan. *Software Architecture*. Upper Saddle River, NJ: Prentice Hall, 1996.

Exercises

- 14.1 (4) For each of the following systems, list the applicable style(s) of system architecture: batch transformation, continuous transformation, interactive interface, dynamic simulation, real-time system, and transaction manager. Explain your selection(s). For systems that fit more than one style, group features of the system by style.
 - a. **An electronic chess companion.** The system consists of a chess board with a built-in computer, lights, and membrane switches. The human player registers moves by pressing chess pieces on the board, activating membrane switches mounted under each square. The computer indicates moves through lights also mounted under each square. The human moves the chess pieces for the computer. The computer should make only legal moves, should reject attempted illegal human moves, and should try to win.
 - b. **An airplane flight simulator for a video game system.** The video game system has already been implemented and consists of a computer with joystick and pushbutton inputs and an output interface for a color television. Your job is to develop the software for the computer

to display the view from the cockpit of an airplane. The joystick and pushbutton control the airplane. The display should be based on a terrain description stored in memory. When your program is complete, it will be sold on cartridges that plug into the video game system.

- c. **A floppy disk controller chip.** The chip is going to use a microprogram for internal control. You are concerned with the microprogram. The chip bridges the gap between a computer and a floppy disk drive. Your portion of the control will be responsible for positioning the read/write head and reading the data. Information on the diskette is organized into tracks and sectors. Tracks are equally spaced circles of data on the diskette. Data within a track is organized into sectors. Your architecture will need to support the following operations: Find track 0, find a given track, read a track, read a sector, write a track, and write a sector.
- d. **A sonar system.** You are concerned with the portion of the system that detects undersea objects and computes how far away they are (range). This is done by transmitting an acoustic pulse and analyzing any resulting echo. A technique called correlation is used to perform the analysis, in which a time-delayed copy of the transmitted pulse is multiplied by the returned echo and integrated for many values of time delay. If the result is large for a particular value of time delay, it is an indication that there is an object with a range that corresponds to that delay.

14.2 (3) Discuss how you would implement control for the applications described in the previous exercise.

14.3 (7) As the system architect for a new signal-processing product, you must decide how to store data in real time. The product uses analog to digital convertors to sample an analog input signal at the rate of 16,000 bytes/second (128,000 bits/second) for 10 seconds. Unfortunately, the needed calculations are too time consuming to do as the samples are received, so you are going to have to store the samples temporarily. The decision has already been made to limit the amount of memory used for buffers to 64,000 bytes. The system has a floppy disk drive that uses diskettes organized into 77 tracks for a total of 243,000 bytes of storage per diskette. It takes 10 milliseconds to move the disk drive read/write head from one track to another and 83 milliseconds, on average, to find the beginning of a track once the head is positioned. The disk drive will be positioned at the correct track prior to the start of data acquisition.

Two solutions to the problem are being considered: (1) Simply write the data samples on the diskette as they become available. Why doesn't this work? (2) Use memory as a buffer. Data samples are placed in memory as they are acquired and written to the floppy disk as fast as possible on sequential tracks. Will this method work? Describe the method in more detail. How much memory is needed for the buffer? How many tracks will be used on the diskette? Prepare a few scenarios. Describe how the control might work.

14.4 (6) Consider a system for automating the drilling of holes in rectangular metal plates. The size and location of holes are described interactively, using a graphical editor on a personal computer. When the user is satisfied with a particular drawing, a peripheral device on the personal computer punches a numerical control (N/C) tape. The tape can then be used on a variety of commercially available N/C drilling machines that have moving drill heads and can change drill sizes.

You are concerned only with the editing of the drawings and the punching of the N/C tapes. The tapes contain sequences of instructions to move the drill head, change drills, and drill. Since it takes some time to move the drill between holes, and even longer to change drills, the system should determine a reasonably efficient drilling sequence. It is not necessary to achieve the absolute minimum time, but the system should not be grossly inefficient either. The drill head is controlled independently in the x and y directions, so the time it takes to move between holes is

proportional to the larger of the required displacements in the x and the y direction. Prepare a system architecture. How would you characterize the style of the system?

- 14.5 (5) Consider a system for interactive symbolic manipulation of polynomials. The basic idea is to allow a mathematician to be more accurate and productive in developing formulas. The user enters mathematical expressions and commands a line at a time. Expressions are ratios of polynomials, which are constructed from constants and variables. Intermediate expressions may be assigned to variables for later recall. Operations include addition, subtraction, multiplication, division, and differentiation with respect to a variable.

Develop an architecture for the system. How would you characterize the style of the system? How would you save work in progress to resume at a future time?

- 14.6 (4) An architecture for the system described in the previous exercise could involve the following subsystems. Organize them into partitions and layers.
- line syntax—scan a line of user input for tokens
 - line semantics—determine the meaning of a line of input
 - command processing—execute user input, error checking
 - construct expression—build an internal representation of an input expression
 - apply operation—carry out an operation on one or more expressions
 - save work—save the current context
 - load work—read in previously saved context
 - substitute—substitute one expression for a variable in another expression
 - rationalize—convert an expression to canonical form
 - evaluate—replace a variable in an expression with a constant and simplify the expression

- 14.7 (6) Consider a system for editing, saving, and printing class diagrams and generating relational database schema. The system supports only a limited subset of the class modeling notation—classes with attributes and binary associations with multiplicity. The system also includes editing functions such as create class, create association, cut, copy, and paste. The editor must understand the semantics of class diagrams. For example, when a class rectangle is moved, the lines representing any attached associations are stretched. If a class is deleted, attached associations are also deleted. When the user is satisfied with the diagram, the system will generate the corresponding relational database schema. Discuss the relative advantages of a single program that performs all functions versus two programs, one that edits class diagrams and the other that generates database schema from class diagrams.

- 14.8 (6) In the previous exercise, both physical and logical aspects of class diagrams must be considered. Physical aspects include location and sizes of lines, boxes, and text. Logical aspects include connectivity, classes, attributes, and associations. Discuss basing your architecture on the following strategies. Consider the issues involved in editing and saving class diagrams as well as generating database schema.

- Model only the geometrical aspects of class diagrams. Treat logical aspects as derived.
- Model both the geometrical and logical aspects of class diagrams.

- 14.9 (5) Another approach to the system described in Exercise 14.7 is to use a commercially available desktop publishing system for class diagram preparation instead of implementing your own class diagram editor. The desktop editor can dump its output in an ASCII markup language. The vendor supplies the grammar for the markup language.

Compare the two approaches. One approach is to build your own editor that understands the semantics of class diagrams. The other is to use a commercially available desktop publishing

system to edit class diagrams. What happens if new versions of the desktop publishing system become available? Can you assume that the user prepares a diagram using a notation that your database generator will understand? Is it worth the effort to implement functions such as cut, copy, and paste that commercial systems already do so well? Who is going to help the users if they run into problems? How is your system going to be supported and maintained? How soon can you get the system completed?

- 14.10 (6) A common issue in many systems is how to store data so it is preserved in the event of power loss or hardware failure. The ideal solution should be reliable, low-cost, small, fast, maintenance free, and simple to incorporate into a system. Also, it should be immune to heat, dirt, and humidity. Compromises in the available technology often influence the functional requirements. Compare each of the following solutions in terms of the ideal. Note that this is not an exhaustive list of solutions.
- Do not worry about it at all. Reset all data every time the system is turned on.
 - Never turn the power off if it can be helped. Use a special power supply, including backup generators, if necessary.
 - Keep critical information on a magnetic disk drive. Periodically make full and/or incremental copies on magnetic tape.
 - Use a battery to maintain power to the system memory when the rest of the system is off. It might even be possible to continue to provide limited functionality.
 - Use a special memory component, such as a magnetic bubble memory or an electronically erasable programmable read-only memory.
 - Critical parameters are entered by the user through switches. Several types of switches are commercially available for this use, including several toggle switches in a package that connects the same way as an integrated circuit.
- 14.11 (7) For each of the following systems, select one or more of the strategies for data storage from the previous exercise. In each case explain your reasoning and give an estimate (order of magnitude) of how much memory, in bytes, is required:
- Four-function pocket calculator.** Main source of power is light. Performs basic arithmetic.
 - Electronic typewriter.** Main source of power is either rechargeable batteries or alternating current. Operates in two modes. In one mode, documents are typed a line at a time. Editing may be performed on a line before it is typed. A liquid crystal display will display up to 16 characters for editing purposes. In the other mode, an entire document can be entered and edited before printing. The typewriter should be able to save the working document for up to a year with the main power off.
 - System clock for a personal computer.** Main power is direct current supplied by the personal computer when it is on. Provides time and date information to the computer. Must maintain the correct date and time for at least five years with the main power off.
 - Airline reservation system.** Main power is alternating current. Used to reserve seats on airline flights. The system must be kept running at all times, at all costs. If, for some reason, the system must be shut off, no data should be lost.
 - Digital control and thermal protection unit for a motor.** The device provides thermal protection for motors over a wide range of horsepower ratings by calculating motor temperature based on measured current and a simulation of motor heat dissipation. If the calculated motor temperature exceeds safe limits, the motor is shut off and not allowed to be restarted until it cools down. The main source of power is alternating current, which may be interrupted. The system must provide protection as soon as it is turned on. Parameters needed for thermal

simulation are initially set at the factory, but provision must be made to change them, if necessary, after the system is installed. Because the motor temperature is not measured directly, it is necessary to continue to simulate the motor temperature for at least an hour after loss of main power, in case power is restored before the motor cools.

- 14.12 (9) The design of file formats is a common task for system design. A BNF diagram is a convenient way to express file formats. Figure E14.1 is a portion of a BNF diagram of a language for describing classes and binary associations. Nonterminal symbols are shown in rectangles, and terminal symbols are shown in circles or rectangles with rounded corners. With the exception of *character*, the diagram defines all nonterminals. A diagram consists of classes and associations. A class has a unique name and many attributes. An association has an optional name and two ends. An association end contains the name one of the classes being associated and multiplicity information. Textual information is described by quoted strings. A character is any ASCII character except quote.

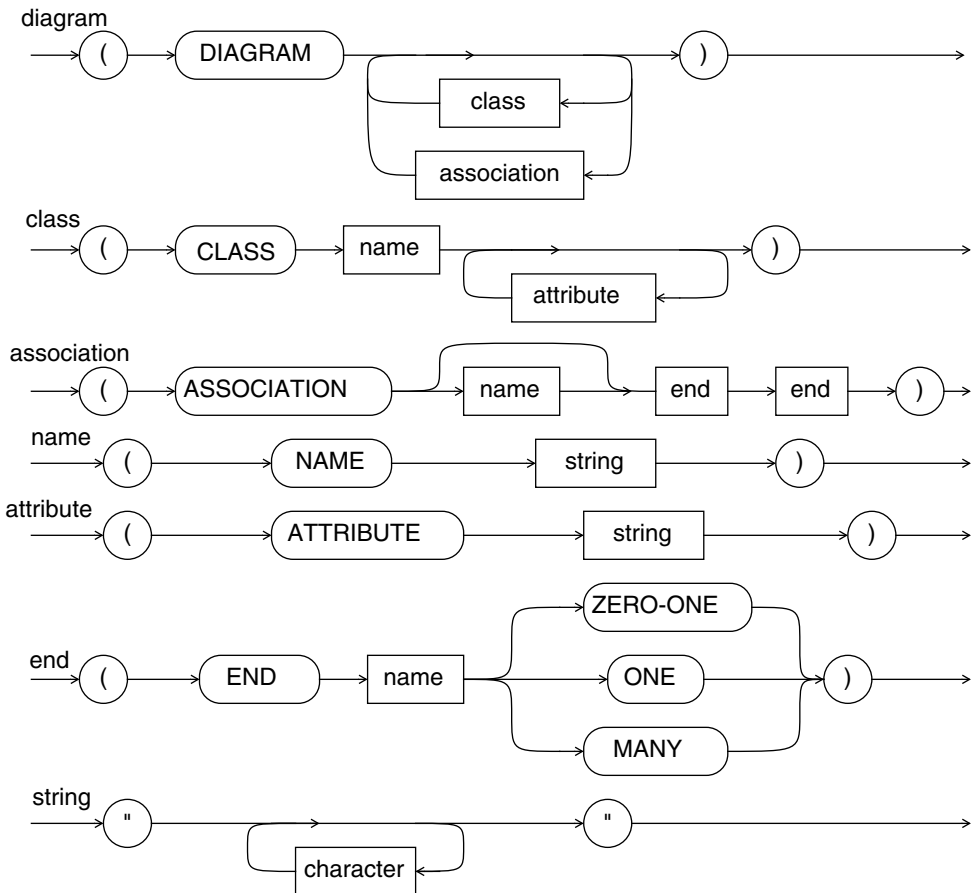


Figure E14.1 BNF diagram for a language that describes classes and associations

- a. Use the language in Figure E14.1 to describe the class diagram in Figure E14.2.
- b. Discuss similarities and differences between data in storage and data in motion. For example, the description you prepared in the previous part could be used to store a class diagram in a file or to transmit a diagram from one location to another.
- c. The language in this problem is used to describe the structure of class diagrams. Invent a language to describe two-dimensional polygons. Use BNF to describe your language. Describe a square and a triangle in your language.

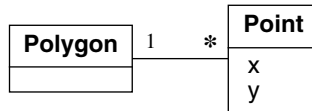


Figure E14.2 Class diagram of polygons

- 14.13 (6) A common problem encountered in digital systems is data corruption due to noise or hardware failure. One solution is to use a cyclic redundancy code (CRC). When data is stored or transmitted, a code is computed from the data and appended to it. When data is retrieved or received, the code is recomputed and compared with the value that was appended to the data. A match is necessary but not sufficient to indicate that the data is correct. The probability that errors will be detected depends on the sophistication of the function used to compute the CRC. Some functions can be used for error correction as well as detection. Parity is an example of a simple function that detects single-bit errors.

The function to compute a CRC can be implemented in hardware or software. The choice for a given problem is a compromise involving speed, cost, flexibility, and complexity. The hardware solution is fast, but may add unnecessary complexity and cost to the system hardware. The software solution is cheaper and more flexible, but may not be fast enough and may make the system software more complex.

For each of the following subsystems, decide whether or not a CRC is needed. If so, decide whether to implement the CRC in hardware or software. Explain your choices.

- a. floppy disk controller
- b. system to transmit data files from one computer to another over telephone lines
- c. memory board on a computer board in the space shuttle
- d. magnetic tape drive
- e. validation of an account number (a CRC can be used to distinguish between valid accounts and those generated at random)

- 14.14 (6) Consider the scheduler software in Exercises 12.16–12.19 and 12.20–12.23.

With scheduling software it is also important to manage security—that is, the schedules that each user is permitted to read and write.

An obvious way to maintain security is to maintain a list of access permissions for each combination of user and schedule. However, this can become tedious to monitor and maintain.

Another solution is to allow permissions to be entered also for a group. A user can belong to multiple groups; each group may have multiple users and lesser groups. The users may access schedules for which they have permission or for which their groups have permission.

Extend the class models from Exercises 12.19 and 12.23 for this model of security. (Instructor's note: You should give the students our answers to Exercises 12.19 and 12.23.)

15

Class Design

The analysis phase determines what the implementation must do, and the system design phase determines the plan of attack. The purpose of class design is to complete the definitions of the classes and associations and choose algorithms for operations.

This chapter shows how to take the analysis model and flesh it out to provide a basis for implementation. The system design strategy guides your decisions, but during class design, you must now resolve the details. There is no need to change from one model to another, as the OO paradigm spans analysis, design, and implementation. The OO paradigm applies equally well in describing the real-world specification and computer-based implementation.

15.1 Overview of Class Design

The analysis model describes the information that the system must contain and the high-level operations that it must perform. You *could* prepare the design model in a completely different manner, with entirely new classes. Most of the time, however, the simplest and best approach is to carry the analysis classes directly into design. Class design then becomes a process of adding detail and making fine decisions. Moreover, if you incorporate the analysis model during design, it is easier to keep the analysis and design models consistent as they evolve.

During design, you choose among different ways to realize the analysis classes with an eye toward minimizing execution time, memory, and other cost measures. In particular, you must flesh out operations, choosing algorithms and breaking complex operations into simpler operations. This decomposition is an iterative process that is repeated at successively lower levels of abstraction. You may decide to introduce new classes to store intermediate results during program execution and avoid recomputation. However, it is important to avoid overoptimization, as ease of implementation, maintainability, and extensibility are also important concerns.

OO design is an iterative process. When you think that the class design is complete at one level of abstraction, you should consider the next lower level of abstraction. For each

level, you may need to add new operations, attributes, and classes. You may even need to revise the relationships between objects (including changes to the inheritance hierarchy). Do not be surprised if you find yourself iterating several times.

Class design involves the following steps.

- Bridge the gap from high-level requirements to low-level services. [15.2]
- Realize use cases with operations. [15.3]
- Formulate an algorithm for each operation. [15.4]
- Recurse downward to design operations that support higher-level operations. [15.5]
- Refactor the model for a cleaner design. [15.6]
- Optimize access paths to data. [15.7]
- Reify behavior that must be manipulated. [15.8]
- Adjust class structure to increase inheritance. [15.9]
- Organize classes and associations. [15.10]

15.2 Bridging the Gap

Figure 15.1 summarizes the essence of design. There is a set of features that you want your system to achieve. You have a set of available resources. Think of the distance between them as a gap. Your job is to build a bridge across the gap. There are several sources of high-level needs: use cases, application commands, and system operations and services. Resources include the operating system infrastructure, class libraries, and previous applications.

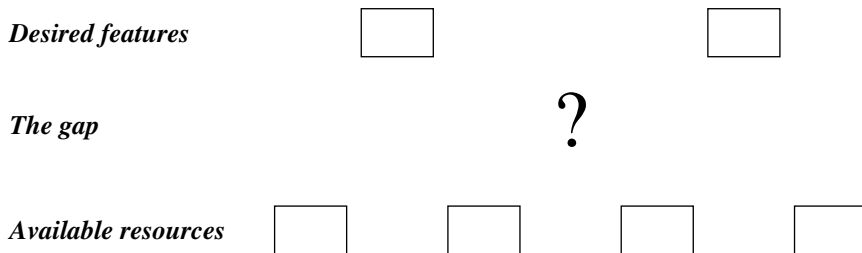


Figure 15.1 The design gap. There is often a disconnect between the desired features and the available resources.

If you can directly construct each feature from the resources, you are done. For example, a salesman can use a spreadsheet to construct a formula for his commission based on various assumptions. The resources are a good match for the task.

But usually it's not so easy. Suppose you want to build a Web-based ordering system. Now you cannot readily build the system from a spreadsheet or a programming language, because there is too big a gap between the features and the resources. You must invent some intermediate elements, so that each element can be expressed in terms of a few elements at

the next lower level (Figure 15.2). Furthermore, if the gap is large you will need to organize the intermediate elements into multiple levels. The intermediate elements may be operations, classes, or other UML constructs. Inventing good intermediate elements is the essence of successful design.

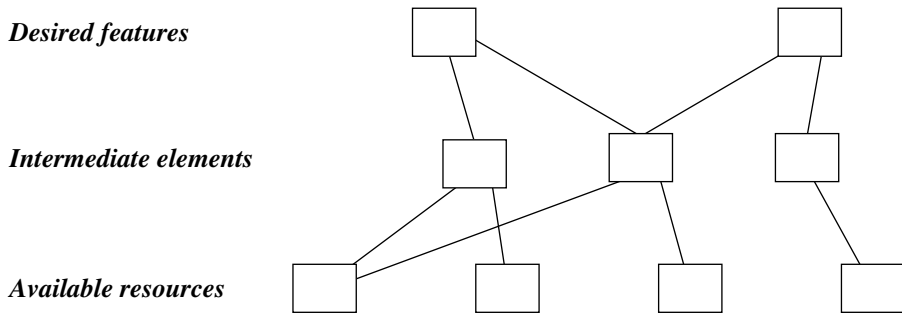


Figure 15.2 Bridging the gap. You must invent intermediate elements to bridge the gap between the desired features and the available resources.

Often the intermediate elements are not obvious. There can be many ways to decompose a high-level operation. You must guess at a likely set of intermediate operations, then try to build them. Be alert to intermediate operations that are similar but not identical. You can reduce the size of your code and increase its clarity by folding these similar operations into a smaller number of common operations. These reworked operations may be less than ideal for some of the higher-level operations. You may have to compromise, because a good design optimizes an entire system and not each separate decision.

If the intermediate elements have already been built, you can just use them, but the principle of bridging the gap is the same. You still have to find the elements in a framework or a class library, select them, and fit them together. The problem isn't making up individual elements—anybody can do that well. The problem is to fit the entire system together cleanly.

Design is difficult because it is not a pure analytic task. You cannot merely study system requirements and derive the ideal system. There are far too many choices of intermediate elements to try them all, so you must apply heuristics. Design requires synthesis: You have to invent new intermediate elements and try to fit them together. It is a creative task, like solving puzzles, proving theorems, playing chess, building bridges, or writing symphonies. You can't expect to push a button or follow a recipe and automatically get a design. A development process provides guidance, just as chess books and engineering handbooks and music theory courses help, but eventually it takes an act of creativity to produce a design.

15.3 Realizing Use Cases

In Chapter 13 we added major operations to the class model. Now during class design we elaborate the complex operations, most of which come from use cases.

Use cases define the required behavior, but they do not define its realization. That is the purpose of design—to choose among the options and prepare for implementation. Each choice has advantages and disadvantages. It is not sufficient merely to deliver the behavior, although that is a primary goal. You must also consider the consequences of each choice on performance, reliability, ease of future enhancement, and many other “ilities”. Design is the process of realizing functionality while balancing conflicting needs.

Use cases define system-level behavior. During design you must invent new operations and new objects that provide this behavior. Then, in turn, you must define each of these new operations in terms of lower-level operations involving more objects. Eventually you can implement operations directly in terms of existing operations. Inventing the right intermediate operations is what we have called “bridging the gap.”

To start, list the responsibilities of a use case or operation. A **responsibility** is something that an object knows or something it must do [Wirfs-Brock-90]. A responsibility is not a precise concept; it is meant to get the thought process going. For example, in an online theater ticket system, making a reservation has the responsibility of finding unoccupied seats to the desired show, marking the seats as occupied, obtaining payment from the customer, arranging delivery of the tickets, and crediting payment to the proper account. The theater system itself must track which seats are occupied, know the prices of various seats, and so on.

Each operation will have various responsibilities. Some of these may be shared by other operations, and others may be reused in the future. Group the responsibilities into clusters and try to make each cluster coherent. That is, each cluster should consist of related responsibilities that can be serviced by a single lower-level operation. Sometimes, if the responsibilities are broad and independent, each responsibility is in its own cluster.

Now define an operation for each responsibility cluster. Define the operation so that is not restricted to special circumstances, but don’t make it so general that it is unfocused. The goal is to anticipate future uses of the new operation. If the operation can be used in several different places in the current design, you probably don’t have to make it more general, except to cover the existing uses.

Finally, assign the new lower-level operations to classes. If there is no good class to hold an operation, you may need to invent a new lower-level class.

ATM example. One of the use cases from Chapter 13 is *process transaction*. Recall that a *Transaction* is a set of *Updates* and that the logic varies according to withdrawal, deposit, and transfer.

- **Withdrawal.** A withdrawal involves a number of responsibilities: get amount from customer, verify that amount is covered by the account balance, verify that amount is within the bank’s policies, verify that ATM has sufficient cash, disburse funds, debit bank account, and post entry on the customer’s receipt. Note that some of these responsibilities must be performed within the context of a database transaction. A database transaction ensures all-or-nothing behavior—all operations within the scope of a transaction happen or none of the operations happen. For example, the disbursement of funds and debiting of the bank account must both happen together.
- **Deposit.** A deposit involves several responsibilities: get amount from customer, accept funds envelope from customer, time-stamp envelope, credit bank account, and post en-

try on the customer's receipt. Some of these responsibilities must also be performed within the context of a database transaction.

- **Transfer.** Responsibilities include: get source account, get target account, get amount, verify that source account covers amount, verify that the amount is within the bank's policies, debit the source account, credit the target account, and post an entry on the customer's receipt. Once again some of the responsibilities must happen within a database transaction.

You can see that there is some overlap between the operations. For example, *withdrawal*, *deposit*, and *transfer* all request the amount from the customer. *Transfer* and *withdrawal* both verify that the source account has sufficient funds. A reasonable design would coalesce this behavior and build it once.

15.4 Designing Algorithms

Now formulate an *algorithm* for each operation. The analysis specification tells *what* the operation does for its clients, but the algorithm shows *how* it is done. Perform the following steps to design algorithms.

- Choose algorithms that minimize the cost of implementing operations.
- Select data structures appropriate to the algorithms.
- Define new internal classes and operations as necessary.
- Assign operations to appropriate classes.

15.4.1 Choosing Algorithms

Many operations are straightforward because they simply traverse the class model to retrieve or change attributes or links. The OCL (see Chapter 3) provides a convenient notation for expressing such traversals.

However, a class-model traversal cannot fully express some operations. We often use pseudocode to handle these situations. Pseudocode helps us think about the algorithm while deferring programming details. For example, many applications involve graphs and the use of transitive closure. (The transitive closure is the set of nodes that can be reached, directly or indirectly, from some starting node.) Figure 15.3 shows a simple model for an undirected graph and pseudocode for computing the transitive closure.

When efficiency is not an issue, you should use simple algorithms. In practice, only a few operations tend to be application bottlenecks. Typically, 20% of the operations consume 80% of execution time. For the remaining operations, it is better to have a design that is simple, understandable, and easy to program than to wring out minor improvements. You can focus your creativity on the algorithms for the operations that are a bottleneck. For example, scanning a set of size n for a value requires an average of $n/2$ operations, whereas a binary search takes $\log n$ operations and a hash search takes less than 2 operations on average. Here are some considerations for choosing among alternative algorithms.

15.8 Reification of Behavior

Behavior written in code is rigid. You can execute it, but cannot manipulate it at run time. Most of the time this is fine, because all you want to do is execute it. But if you need to store, pass, or modify the behavior at run time, you should reify it.

Reification is the promotion of something that is not an object into an object. Behavior usually meets this description. It isn't usually something that you would normally manipulate. If you reify behavior, you can store it, pass it to other operations, and transform it. Reification adds complexity but can dramatically expand the flexibility of a system.

You reify behavior by encoding it into an object and decoding it when it is run. The resulting run-time cost may or may not be significant. If the encoding invokes high-level procedures, the cost is only a few indirections. If the entire behavior is encoded in a different language, however, it must be interpreted, which can be an order of magnitude slower than direct execution of code. If the encoded behavior constitutes a small part of the run-time execution time, the performance overhead may not matter.

Exercise 4.16 in Chapter 4 illustrates reification. In one sense you can regard the tasks of a recipe as operations; in another sense they could be data in a class model.

[Gamma-95] lists a number of behavioral patterns that reify behavior. These include encoding a state machine as a table with a run-time interpreter (*State*), encoding a sequence of requests as parameterized command objects (*Command*), and parameterizing a procedure in terms of an operation that it uses (*Strategy*). These techniques have been around for a long time, but the language of patterns is convenient for describing them and weighing their benefits and costs. For example, the *Strategy* pattern was used in Fortran days for purposes such as passing a function to a mathematical integration routine. However, in Fortran there was no way to ensure correct matching of passed functions, and errors were easy to make. By encoding the passed function as an instance of a function class, you get extensibility via polymorphism, yet can enforce the signatures of an entire family of functions. In this case, OO technology permits a cleaner solution than previous techniques.

ATM example. We have already used reification in the case study. In one sense a transaction is an action—withdrawing, depositing, and transferring funds. We promoted transaction to a class so that we could describe it. The functionality that we need is routine and can readily be obtained by traversing the class model.

15.9 Adjustment of Inheritance

As class design progresses, you can often adjust the definitions of classes and operations to increase inheritance by performing the following steps.

- Rearrange classes and operations to increase inheritance.
- Abstract common behavior out of groups of classes.
- Use delegation to share behavior when inheritance is semantically invalid.

15.9.1 Rearranging Classes and Operations

Sometimes several classes define the same operation and can easily inherit it from a common ancestor, but more often operations in different classes are similar but not identical. By adjusting the definitions of the operations, you may be able to cover them with a single inherited operation.

Before using inheritance, the operations must match. All operations must have the same signature—that is, the same number and types of arguments and results. In addition, the operations must have the same semantics. You can use the following kinds of adjustments to increase the chance of inheritance.

- **Operations with optional arguments.** You may be able to align signatures by adding optional arguments that can be ignored. For example, a draw operation on a monochromatic display does not need a color parameter, but it can accept the parameter and ignore it for consistency with color displays.
- **Operations that are special cases.** Some operations may have fewer arguments because they are special cases of more general operations. Implement the special operations by calling the general operation with appropriate parameter values. For example, appending an element to a list is a special case of inserting an element into list; the insert point simply follows the last element.
- **Inconsistent names.** Similar attributes in different classes may have different names. Give the attributes the same name and move them to a common ancestor class. Then operations that access the attributes will match better. Also watch for similar operations with different names. A consistent naming strategy is important.
- **Irrelevant operations.** Several classes in a group may define an operation, but some others may not. Define the operation on the common ancestor class and declare it as a no-op on the classes that don't care about it. For example, rotation is meaningful for geometric figures, but is unimportant for circles.

ATM example. An ATM can post remote transactions on a receipt. It would seem that we should also be able to issue a receipt for cashier transactions. However, a receipt for a *RemoteTransaction* involves a *CashCard*, while a receipt for a *CashierTransaction* directly involves a *Customer*. Furthermore, the cashier software is apart from the ATM software. We will have two different kinds of receipts, a *RemoteReceipt* and a *CashierReceipt*.

15.9.2 Abstracting Out Common Behavior

You will not recognize all opportunities for inheritance during analysis, so it is worthwhile to reexamine the class model looking for commonality. In addition, you will be adding new classes and operations during design. If two classes seem to repeat several operations and attributes, it is possible that the two classes are really specializations of the same thing, when viewed at a higher level of abstraction.

When there is common behavior, you can create a common superclass for the shared features, leaving only the specialized features in the subclasses. This transformation of the class model is called **abstracting out** a common superclass or common behavior. We advise

you to make only abstract superclasses, meaning that there are no direct instances of it, but the behavior it defines belongs to all instances of its subclasses. (You can always do this by adding an *Other* subclass.) For example, a *draw()* operation of a geometric figure on a display screen requires setup and rendering of the geometry. The rendering varies among different figures, such as circles, lines, and splines, but the figures can inherit the setup, such as setting the color, line thickness, and other parameters, from the abstract class *Figure*.

Sometimes it is worthwhile to abstract out a superclass even when your application has only one subclass that inherits from it. Although this does not yield any immediate sharing of behavior, the abstract superclass may be reusable in future projects. It may even be a worthwhile addition to your class library. When you complete a project, you should consider the potentially reusable classes for future applications.

Abstract superclasses have benefits beyond sharing and reuse. The splitting of a class into two classes that separate the specific aspects from the more general aspects is a form of modularity. Each class is a separately maintained component with a well-documented interface.

The creation of abstract superclasses also improves the extensibility of a software product. Imagine that you are developing a temperature-sensing module for a large computerized control system. You must use a specific type of sensor (Model J55) with a particular way of reading the temperature, and a formula for converting the raw numeric reading into degrees Celsius. You could place all this behavior in a single class, with one instance for each sensor in the system. But realizing that the J55 sensor is not the only type available, you create an abstract *Sensor* superclass that defines the general behavior common to all sensors. A particular subclass called *SensorJ55* provides reading and conversion that is particular to J55.

Now, when your control system converts to a new sensor model, you need only prepare a subclass that has the specialized behavior for the new model. The superclass already has the common behavior. Perhaps best of all, you will not have to change a single line of code in the large control system that uses these sensors, because the interface is the same, as defined by the *Sensor* superclass.

There is a subtle but important way that abstract superclasses improve configuration management for software maintenance and distribution. Suppose that you must distribute your control system software to many plants throughout the country, each having a different configuration that involves (among other things) a different mix of temperature sensors. Some plants still use the old J55 model, while others have converted to the newer K99 model, and some plants may have a mixture of both types. Generating customized versions of your software to match each different configuration could be tedious.

Instead, you distribute one version of software that contains a subclass for each known sensor model. When the software starts up, it reads the customer's configuration file that tells it which sensor model is used in which location and creates instances of the particular subclasses for the relevant sensors. All the rest of the code treats the sensors as if they were all the same as defined by the *Sensor* superclass. It is even possible to change from one type of sensor to another on-the-fly (while the system is running) if the software creates a new object for the new type of sensor.

ATM example. We did pay attention to inheritance during analysis when we constructed the class model. We do not see any additional inheritance at this time. In a full-fledged

application there would be much more design detail and increased opportunities for inheritance.

15.9.3 Using Delegation to Share Behavior

Inheritance is a mechanism for implementing generalization, in which the behavior of a superclass is shared by all its subclasses. Sharing of behavior is justifiable only when a true generalization relationship occurs—that is, only when it can be said that the subclass *is* a form of the superclass. Operations of the subclass that override the corresponding operation of the superclass must provide the same services as the superclass. When class *B* inherits the specification of class *A*, you can assume that every instance of class *B* *is* an instance of class *A* because it behaves the same.

Sometimes programmers use inheritance as an implementation technique with no intention of guaranteeing the same behavior. It often happens that an existing class already implements some of the behavior that you want to provide in a newly defined class, although in other respects the two classes differ. The programmer is then tempted to inherit from the existing class to achieve part of the implementation of the new class. This can lead to problems if other inherited operations provide unwanted behavior. We discourage this *inheritance of implementation* because it can lead to incorrect behavior.

As an example of implementation inheritance, suppose that you need a *Stack* class and a *List* class is available. You may be tempted to make *Stack* inherit from *List*. You can push an element onto a stack by adding an element to the end of the list. Similarly, you can pop an element from a stack by removing an element from the end of the list. But you will also inherit unwanted operations that add or remove elements from arbitrary positions in the list. If these are ever used (by mistake or as a “short-cut”), then the *Stack* class will not behave as expected.

Often when you are tempted to use inheritance as an implementation technique, you could achieve the same goal in a safer way by making one class an associate of the other class. Then, one object can selectively invoke the desired operations of another class, using delegation rather than inheritance. *Delegation* consists of catching an operation on one object and sending it to a related object. You delegate only meaningful operations, so there is no danger of inheriting meaningless operations by accident.

A safer design of *Stack* would delegate to *List*, as Figure 15.7 shows. Every *Stack* instance contains a private *List* instance. (You could optimize the actual implementation of the aggregation by using an embedded object or a pointer attribute.) The *Stack.push()* operation delegates to *List* by calling its *last()* and *add()* operations to add an element at the end. The *pop()* operation has a similar implementation using the *last()* and *remove()* operations. The ability to corrupt the stack by adding or removing arbitrary elements is hidden from the client of the *Stack* class.

Some languages, such as C++ and Java, permit a subclass to inherit the form of a superclass but to selectively inherit operations from ancestors and selectively export operations to clients. This is tantamount to the use of delegation, because the subclass *is not* a form of the superclass in all respects and is not confused with it.

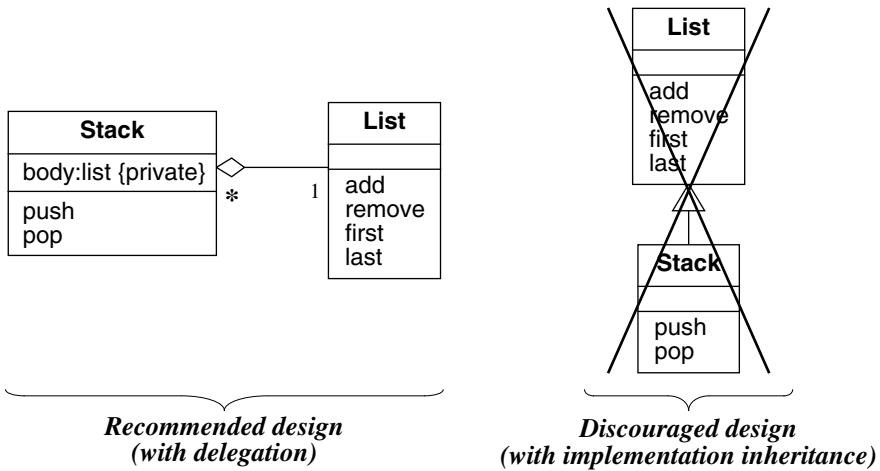


Figure 15.7 Alternative designs. Do not use implementation inheritance.

ATM example. Our use of inheritance is deep and structural. That is the only way we ever use inheritance.

15.10 Organizing a Class Design

Programs consist of discrete physical units that can be edited, compiled, imported, or otherwise manipulated. In some languages, such as C and Fortran, the units are source files. In Ada and Java, the package is an explicit language construct. [Coplien-92] shows how to use a C++ class to group static member functions, lesser classes, enumerations, and constants. You can improve the organization of a class design with the following steps.

- Hide internal information from outside view.
- Maintain coherence of entities.
- Fine-tune definition of packages.

15.10.1 Information Hiding

During analysis we did not consider information visibility—rather our focus was on understanding the application. The goals of design are different. During design we adjust the analysis model so that it is practical to implement and maintain. One way to improve the viability of a design is by carefully separating external specification from internal implementation. This is called **information hiding**. Then you can change the implementation of a class without requiring that clients of the class modify code. In addition, the resulting “firewalls” around classes limit the effects of changes so that you can better understand them.

There are several ways to hide information.

- **Limit the scope of class-model traversals.** Taken to an extreme, a method could traverse all associations of the class model to locate and access an object. Such unconstrained visibility is appropriate during analysis, when you are trying to understand a problem, but methods that know too much about a model are fragile and easily invalidated by changes. During design you should try to limit the scope of any one method [Lieberherr-88]. An object should access only objects that are directly related (directly connected by an association). An object can access indirectly related objects via the methods of intervening objects.
- **Do not directly access foreign attributes.** Generally speaking, it is acceptable for subclasses to access the attributes of their superclasses. However, classes should not access the attributes of an associated class. Instead, call an operation to access the attributes of an associated class.
- **Define interfaces at a high a level of abstraction.** It is desirable to minimize class couplings. One way to do this is by raising the level of abstraction of interfaces. It is fine to call a method on another class for a meaningful task, but you should avoid doing so for minutia.
- **Hide external objects.** Use boundary objects to isolate the interior of a system from its external environment. A *boundary object* is an object whose purpose is to mediate requests and responses between the inside and the outside. It accepts external requests in a client-friendly form and transforms them into a form convenient for the internal implementation.
- **Avoid cascading method calls.** Avoid applying a method to the result of another method, unless the result class is already a supplier of methods to the caller. Instead consider writing a method to combine the two operations.

15.10.2 Coherence of Entities

Coherence is another important design principle. An entity, such as a class, an operation, or a package, is coherent if it is organized on a consistent plan and all its parts fit together toward a common goal. An entity should have a single major theme; it should not be a collection of unrelated parts.

A method should do one thing well. A single method should not contain both policy and implementation. *Policy* is the making of context-dependent decisions. *Implementation* is the execution of fully-specified algorithms. Policy involves making decisions, gathering global information, interacting with the outside world, and interpreting special cases. A policy method contains I/O statements, conditionals, and accesses data stores. A policy method does not contain complicated algorithms but instead calls the appropriate implementation methods. An implementation method encodes exactly one algorithm, without making any decisions, assumptions, defaults, or deviations. All its information is supplied as arguments, so the argument list may be long.

Separating policy and implementation greatly increases the possibility of reuse. The implementation methods do not contain any context dependencies, so they are likely to be re-

usable. Usually you must rewrite policy methods in a new application, but they are often simple and consist mostly of high-level decisions and low-level calls.

For example, consider an operation to credit interest on a checking account. Interest is compounded daily based on the balance, but all interest for a month is lost if the account is closed. The interest logic consists of two parts: an implementation method that computes the interest due between a pair of days, without regard to any forfeitures or other provisions; and a policy method that decides whether and for what interval the implementation method is called. The implementation method is complex, but likely to be reused. Policy methods are less likely to be reusable, but simpler to write.

A class should not serve too many purposes at once. If it is too complicated, you can break it up using either generalization or aggregation. Smaller pieces are more likely to be reusable than large complicated pieces. Exact numbers are somewhat risky, but as a rule of thumb consider breaking up a class if it contains more than about 10 attributes, 10 associations, or 20 operations. Always break a class if the attributes, associations, or operations sharply divide into two or more unrelated groups.

15.10.3 Fine-Tuning Packages

During analysis you partitioned the class model into packages. This initial organization may not be suitable or optimal for implementation. You should define packages so that their interfaces are minimal and well defined. The interface between two packages consists of the associations that relate classes in one package to classes in the other and operations that access classes across package boundaries.

You can use the connectivity of the class model as a guide for forming packages. As a rough rule of thumb, classes that are closely connected by associations should be in the same package, while classes that are unconnected, or loosely connected, may be in separate packages. Of course there are other aspects to consider. Packages should have some theme, functional cohesiveness, or unity of purpose.

The number of different operations that traverse a given association is a good measure of its coupling strength. We are referring to the number of different ways that the association is used, not the frequency of traversal. Try to keep strong coupling within a single package.

15.11 ATM Example

Figure 15.8 shows our final ATM domain class model after class design. (Keep in mind that the full class model also includes the application class model from Figure 13.8.)

15.12 Chapter Summary

Class design does not begin from scratch but rather elaborates the previous stages of analysis and system design. Class design adds details, such as designing algorithms, refactoring operations, optimizing classes, adjusting inheritance, and refining packages.

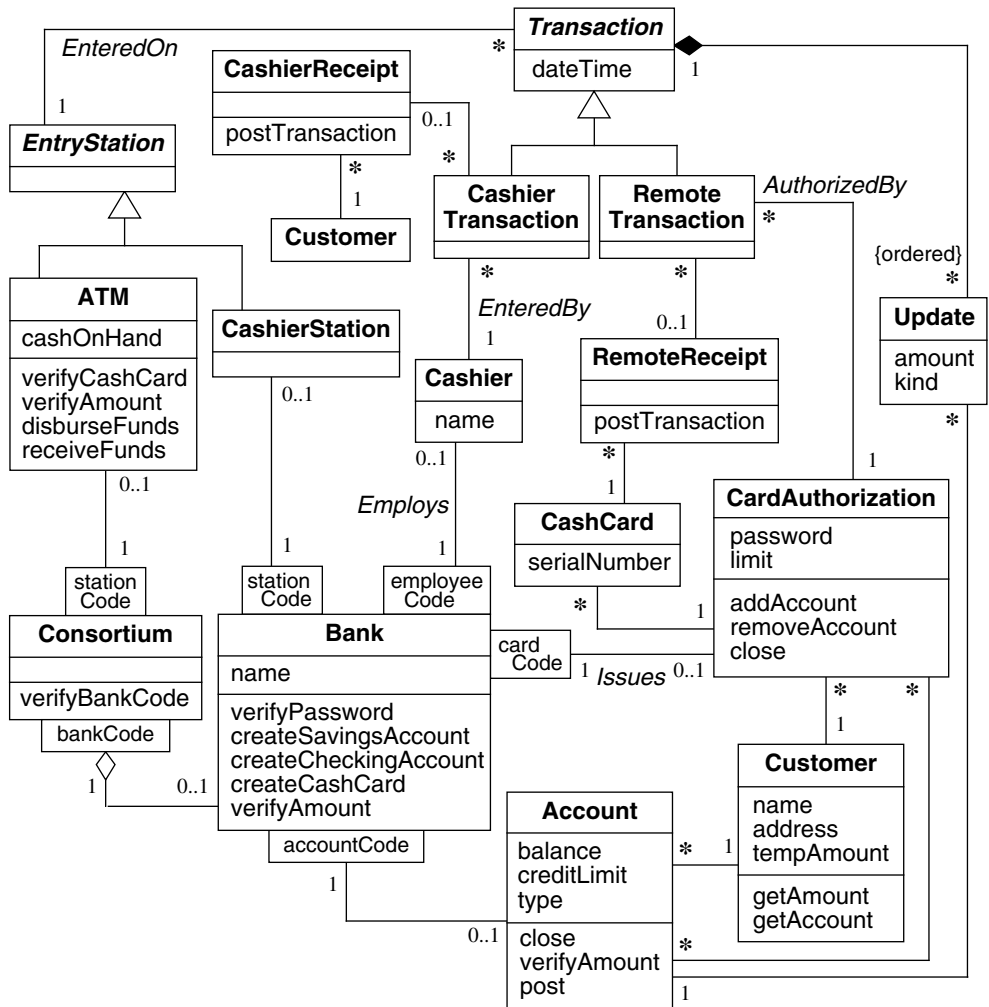


Figure 15.8 Final ATM domain class model after class design

The first step of class design is to add operations according to the use cases. Use cases define the required behavior but not its realization. The class designer invents operations that will deliver the behavior specified by the use cases.

Next, you must devise an algorithm for each operation. Class design focuses on computational complexity, but you should sacrifice small amounts of performance for greater clarity. You will need to recurse to define low-level operations to realize high-level operations. Recursion stops when you have operations that are already available or that are straightforward to implement.

The initial design of a set of operations will contain inconsistencies, redundancies, and inefficiencies. This is natural, because it is impossible to get a large design correct in one pass. Furthermore, as a design evolves, it also degrades. It is inevitable that an operation or class conceived for one purpose will not fully fit additional purposes. As you proceed with a design, you should occasionally refactor operations to improve their clarity and resilience.

During design, you may need to rework the analysis model for efficiency. Optimization does not discard the original information but adds new redundant information to speed access paths and preserve intermediate results. It can be helpful to rearrange algorithms and reduce the number of operations that need to be executed.

You should consider reification—the promotion of something that is not an object into an object. For example, a deposit, withdrawal, or transfer of funds would normally be an operation—it is something that someone *does*. However, for the ATM case study we promoted transaction to a class so that we could describe it.

As class design progresses, you can often adjust the definitions of classes and operations to increase inheritance. These adjustments include modifying the argument list of a method, moving attributes and operations from a class into a superclass, defining an abstract superclass to cover the shared behavior of several classes, and splitting an operation into an inherited part and a specific part. You should use delegation rather than inheritance when a class is similar to another class but not truly a subclass.

You must organize programs into physical units for editors and compilers as well as for the convenience of programming teams. Information hiding is a primary goal to ensure that future changes affect limited amounts of code. Packages should be coherent and organized about a common theme.

abstracting out a superclass	derived attributes	policy vs. implementation
adjusting inheritance	implementation inheritance	recurring
algorithm	index	refactoring
data structure	information hiding	reification
delegation	optimization	responsibility
derived associations	package	use case

Figure 15.9 Key concepts for Chapter 15

Bibliographic Notes

Algorithms and data structures are part of the basic computer science curriculum. [Aho-83] and [Sedgewick-95] are well-written books about algorithms.

Adding indexes and rearranging access order to improve performance is a mature technique in database optimization. See [Ullman-02] for examples.

[Lieberherr-88] is an early attempt to provide visibility guidelines (the law of Demeter) that preserve OO modularity. [Meyer-97] suggests style rules for using classes and operations.

The *class design* stage is renamed from *object design* in the first edition of this book.

References

- [Aho-83] Alfred Aho, John Hopcroft, and Jeffrey Ullman. *Data Structures and Algorithms*. Boston: Addison-Wesley, 1983.
- [Coplien-92] James O. Coplien. *Advanced C++: Programming Styles and Idiom*. Boston: Addison-Wesley, 1992.
- [Fowler-99] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Boston: Addison-Wesley, 1999.
- [Gamma-95] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1995.
- [Lieberherr-88] K. Lieberherr, I. Holland, A. Riel. Object-oriented programming: an objective sense of style. *OOPSLA '88 as ACM SIGPLAN 23*, 11 (November 1988), 323–334.
- [Meyer-97] Bertrand Meyer. *Object-Oriented Software Construction, Second Edition*. Upper Saddle River, NJ: Prentice Hall, 1997.
- [Sedgewick-95] Robert Sedgewick, Philippe Flajolet, and Peter Gordon. *An Introduction to the Analysis of Algorithms*. Boston: Addison-Wesley, 1995.
- [Ullman-02] Jeffrey Ullman and Jennifer Widom. *A First Course in Database Systems*. Upper Saddle River, NJ: Prentice Hall, 2002.
- [Wirfs-Brock-90] Rebecca Wirfs-Brock, Brian Wilkerson, and Lauren Wiener. *Designing Object-Oriented Software*. Upper Saddle River, NJ: Prentice Hall, 1990.

Exercises

- 15.1 (6) Take the use cases from Exercise 13.9 and list at least four responsibilities for each one. [Instructor's note: You may want to give the students our answer to Exercise 13.9.]
- 15.2 (6) Take the first three use cases from Exercise 13.16 and list at least four responsibilities for each one. [Instructor's note: You may want to give the students our answer to Exercise 13.16.]
- 15.3 (4) Write algorithms to draw the following figures on a graphics terminal. The figures are not filled. Assume pixel-based graphics. State any assumptions that you make.
 - a. circle
 - b. ellipse
 - c. square
 - d. rectangle
- 15.4 (3) Discuss whether or not the algorithm that you wrote in the previous exercise to draw an ellipse is suitable for drawing circles and whether or not the rectangle algorithm is suitable for squares.
- 15.5 (3) By careful ordering of multiplications and additions, the total number of arithmetic steps needed to evaluate a polynomial can be minimized. For example, one way to evaluate the polynomial $a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$ is to compute each term separately, adding each term to the total as it is computed, which requires 10 multiplications and 4 additions. Another way is to rearrange the order of the arithmetic to $x \cdot (x \cdot (x \cdot (x \cdot a_4 + a_3) + a_2) + a_1) + a_0$, which requires only 4 multiplications and 4 additions. How many multiplications and additions are required by each method for an n th-order polynomial? Discuss the relative merits of each approach.

- 15.6 (4) Improve the class diagram in Figure E15.1 by generalizing the classes *Ellipse* and *Rectangle* to the class *GraphicsPrimitive*, transforming the class diagram so that there is only a single one-to-one association to the object class *BoundingBox*. In effect, you will be changing the 0..1 multiplicity to exactly-one multiplicity. As it stands, the class *BoundingBox* is shared between *Ellipse* and *Rectangle*. A *BoundingBox* is the smallest rectangular region that will contain the associated *Ellipse* or *Rectangle*.

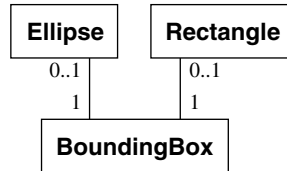


Figure E15.1 Portion of a class diagram with a shared class

- 15.7 (5) Which class(es) for the previous exercise must supply a delete operation visible to the outside world? To delete means to destroy an object and remove it from the application. Explain your answer.
- 15.8 (4) Modify the class diagram in Figure E15.2 so that a separate class provides margins. Different categories of pages may have a default margin, and specific pages may override the default.

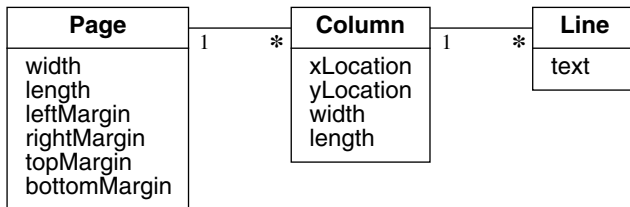


Figure E15.2 Portion of a class diagram of a newspaper

- 15.9 (3) Modify Figure E15.2 to make it possible to be able to determine what *Page* a *Line* is on without first determining the *Column*.
- 15.10 (7) Write pseudocode for each method in Figure E15.3. *Initialize* causes a deck to start with 52 cards and anything else to become empty. *Insert* and *delete* take a card as a single argument and insert or delete the card into a collection, forcing the collection to redisplay itself. *Delete* is allowed only on the top card of a pile. *Sort* is used to sort a hand by suit and rank.
- Pile* is an abstract class. *TopOfPile* and *bottomOfPile* are queries. *Draw* deletes the top card from a pile and inserts the card into a hand, which is passed as an argument.
- Shuffle* mixes a deck. *Deal* selects cards from the top of the deck one at a time, deleting them from the deck and inserting them into hands that are created and returned as an array of hands. *Display* displays a card. *Compare* determines which of two cards has the largest value. *Discard* deletes a card from the collection that contains it and places it on top of the draw pile that is passed as an argument.

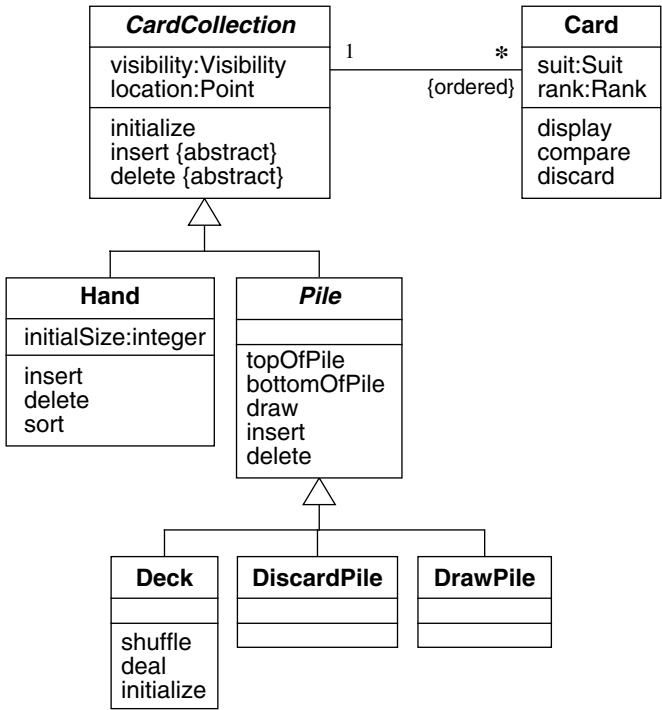


Figure E15.3 Portion of a class diagram of a card-playing program

- 15.11 (5) Write pseudocode for computing the net score for a trial in Figure E12.4.
- Each attempt of a competitor in an event (a *Trial*) is observed by several judges. Each judge rates the attempt and holds up a score. A reader assigned to the group of judges announces the scores one at a time to a panel of scorekeepers. Three scorekeepers write the scores down, cross off the highest and the lowest scores, and total the rest. They check each other's total to detect recording and arithmetic errors. In some cases, they may ask the reader to repeat the scores. When they are satisfied, they hand their figures to three other scorekeepers, who multiply the total score by a difficulty factor for the event and take the average to determine a net score. The net scores are compared to detect and correct scoring errors.
- 15.12 Prepare pseudocode for the following operations to classes in Figure E12.4. You will need to add a many-to-many association *RegisteredFor* between a set of *Events* and an ordered list of *Competitors* to track who is registered for which *Events*. Use the registration order in scheduling trials.
- a. (3) Find the event for a figure and meet.
 - b. (3) Register a competitor for an event.
 - c. (3) Register a competitor for all events at a meet.
 - d. (5) Select and schedule events for a meet.
 - e. (3) Schedule meets in a season.
 - f. (4) Assign judges and scorekeepers to stations.

- 15.13 (8) Figure E15.4 is a portion of a class diagram metamodel that might be used in a compiler of an OO language. Write pseudocode for the *traceInheritancePath* method that traces an inheritance hierarchy as follows. Input to the method is a pair of classes. The method returns an ordered list of classes on the path from the more general class to the more specific class. Tracing is only through generalizations; aggregations and associations are ignored. If tracing fails, an empty list is returned. You may assume that multiple inheritance is not allowed.

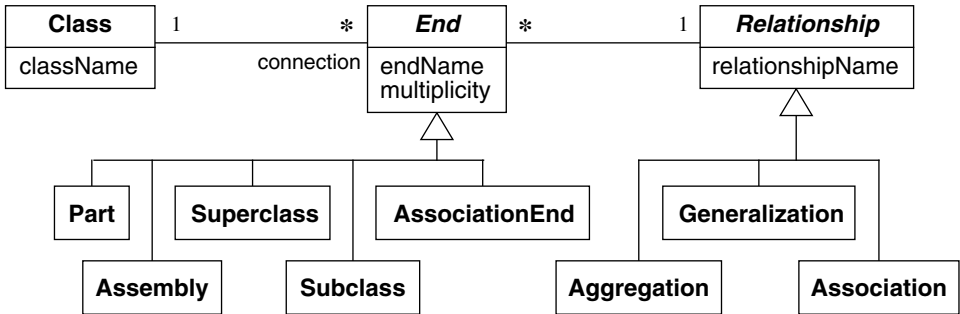


Figure E15.4 Portion of a class diagram metamodel

- 15.14 (8) Refine Figure E15.4 by eliminating the associations to the classes *End* and *Relationship*, replacing them with associations to the subclasses of *End* and *Relationship*. This is an example of a transformation on a class diagram. Write pseudocode for the *traceInheritancePath* method for the new diagram.
- 15.15 (7) Referring to Figure E15.4, prepare an algorithm for an operation that will generate a name for an association that does not already have one. Input to the operation is an instance of *Association*. The operation must return a globally unique *relationshipName*. If the association already has a name, the operation should return it. Otherwise the operation should generate a name using a strategy that you must devise.
- The precise strategy is not critical, but the generated names must be unique, and anyone reading the names should be able to determine which association the name refers to. Assume all associations are binary. You may assume that a similar operation on the class *End* already has been designed that will return an *endName* unique within the context of a relationship. If the name that would be formed collides with an existing name, modify the name in some way to make it unique. If you feel you need to modify the diagram or use additional data structures, go ahead, but be sure to describe them.
- 15.16 (7) Improve the class diagram in Figure E15.5 by transforming it, adding the class *PoliticalParty*. Associate *Voter* with a party. Discuss why the transformation is an improvement.
- 15.17 (7) Sometimes an airline will substitute a smaller aircraft for a larger one for a flight with few passengers. Write an algorithm for reassigning seats so that passengers with low row numbers do not have to be reassigned. Assume both aircraft have the same number of seats per row.
- 15.18 (8) The need for implementation efficiency may force you to create classes that are not in the original problem statement. For example, a two-dimensional CAD system may use specialized data structures to determine which points fall within a rectangular window specified by the user.

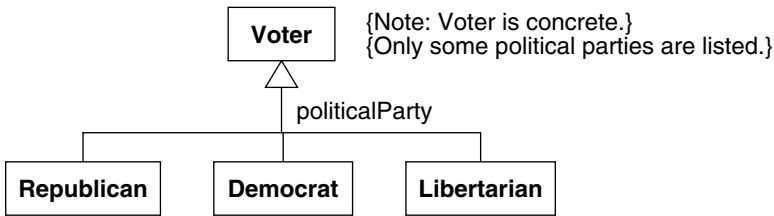


Figure E15.5 Class diagram representing voter membership in a political party

One technique is to maintain a collection of points sorted on x and then y . Points that fall within a rectangular window can usually be found without having to check all points.

Prepare a class diagram that describes collections of points sorted on x and y . Write pseudocode for the operations *search*, *add*, and *delete*. The input to *search* is a description of a rectangular region and a collection of points. The output of *search* is a set of points from the input collection that fall within the region. Inputs to both *add* and *delete* are a point and a collection of points. The input point is added or deleted from the collection.

- 15.19 (8) Determine how the time required by the search operation in the previous exercise depends on the number of points in a collection. Explicitly state any assumptions you make.
- 15.20 (3) In selecting an algorithm, it may be important to evaluate its resource requirements. How does the time required to execute the following algorithms depend on the following parameters?
 - a. The algorithm in Exercise 15.13 on the depth of the inheritance hierarchy.
 - b. The algorithm in Exercise 15.17 on the number of passengers.

16

Process Summary

As Figure 16.1 shows, a development process provides a basis for the organized production of software. We advocate that a process be based on OO concepts and the UML notation.

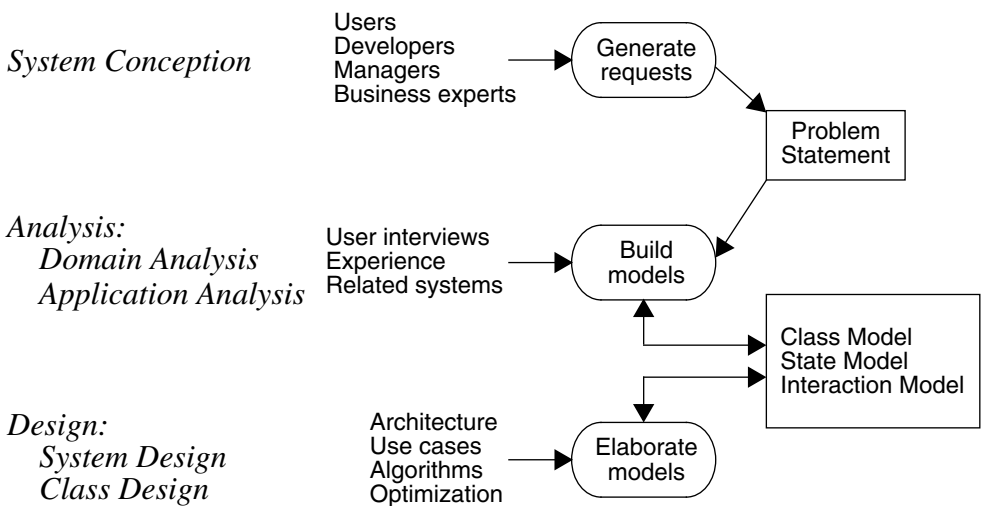


Figure 16.1 Summary of development process. A development process provides a basis for the organized production of software.

Note that there is no need to change from one model to another, as the OO paradigm spans analysis, design, and implementation. The OO paradigm applies equally well in describing the real-world specification and computer-based implementation.

This book's presentation of the stages is linear (an artifact of presentation), but in practice OO development is an iterative process, as Figure 16.1 emphasizes. When you think that

a model is complete at one level of abstraction, you should consider the next lower level of abstraction. For each level, you may need to add new operations, attributes, and classes. You may even need to revise the relationships between objects (including changes to the inheritance hierarchy). Do not be surprised if you find yourself iterating several times. Chapter 21 explains more about iterative development.

16.1 System Conception

System conception deals with the genesis of an application. Initially some person, who understands both technology and business needs, thinks of an idea for an application. The purpose of system conception is to understand the big picture—what need does the proposed system meet, can it be developed at a reasonable cost, and will the demand for the result justify the cost of building it? The input to system conception is the raw idea for a new application. The output is a problem statement that is the starting point for careful analysis.

16.2 Analysis

Analysis focuses on preparing models to get a deep understanding of the requirements. The goal of analysis is to specify what needs to be done, not how it is done. You must understand a problem before attempting a solution. It is important to consider all available inputs, including requirements statements, user interviews, real-world experience, and artifacts from related systems. The output from analysis is a set of models that specify a system in a rigorous and complete manner. There are two substages to analysis: domain analysis and application analysis.

16.2.1 Domain Analysis

Domain analysis captures general knowledge about an application—concepts and relationships known to experts in the domain. The concern is with devising a precise, concise, understandable, and correct model of the real world. Before building anything complex, the builder must understand the requirements. Domain analysis leads to class models and sometimes state models, but seldom interaction models. The job of constructing a domain model is mainly to decide which information to capture (determine the application's scope) and how to represent it (the level of abstraction).

16.2.2 Application Analysis

Application analysis follows and addresses the computer aspects of the application (application objects) that are visible to users. Application objects are not merely internal design decisions, because the users see them and must agree with them. Application classes include controllers, devices, and boundary objects. The interaction model dominates application analysis, but the class and state models are also important.

16.3 Design

Analysis addresses the *what* of an application; design addresses the *how*. Once you have a thorough understanding of an application from analysis, you are ready to deal with the details of building a practical and maintainable solution. You could prepare the design model in a completely different manner than analysis, but most of the time, the simplest and best approach is to carry the analysis classes forward into design. Design then becomes a process of adding detail and making fine decisions. There are two substages to design: system design and class design.

16.3.1 System Design

The purpose of system design is to devise a high-level strategy—the architecture—for solving the application problem. The choice of architecture is an important decision with broad consequences and is based on the requirements as well as past experience. The system designer must also establish policies to guide the subsequent class design.

16.3.2 Class Design

Class design augments and adjusts the real-world models from analysis so that they are ready for implementation. Class designers complete the definitions of the classes and associations and choose algorithms for operations.

Part 3

Implementation

Chapter 17	Implementation Modeling	303
Chapter 18	OO Languages	314
Chapter 19	Databases	348
Chapter 20	Programming Style	380

Parts 1 and 2 have presented OO concepts and an analysis and design process for applying the concepts. Part 3 covers the remainder of software development and discusses the specific details for implementing a system with C++, Java, and databases. OO models are also helpful with non-OO languages, but we do not cover non-OO languages in this book, because most developers now days are using OO languages.

Chapter 17 discusses implementation issues that transcend the choice of language. The focus is on techniques for realizing associations, since few languages have intrinsic support.

Chapter 18 explains the principles of how to implement an OO design with C++ and Java. We cover C++ and Java because they are the most popular OO programming languages.

Chapter 19 shows how to implement an OO design with a database. Our focus is on relational databases, because they dominate the marketplace. As you would expect, OO designs can also be implemented with OO databases, but OO databases have but a small market share and are only used in specialty situations.

Chapter 20 concludes with style recommendations for programming in any language or database. The programming code is the ultimate embodiment of the solution to the problem, so the way in which it is written is important for maintainability and extensibility.

Part 3 completes our explanation of how to take OO concepts and use them to develop applications. Part 4 deals with software engineering issues that are especially important for large and complex applications.

This page intentionally left blank

17

Implementation Modeling

Implementation is the final development stage that addresses the specifics of programming languages. Implementation should be straightforward and almost mechanical, because you should have made all the difficult decisions during design. To a large extent, your programming code should simply translate design decisions. You must add details while writing code, but each one should affect only a small part of the program.

17.1 Overview of Implementation

It is now, during implementation, that you finally capitalize on your careful preparation from analysis and design. First you should address implementation issues that transcend languages. This is what we call *implementation modeling* and involves the following steps.

- Fine-tune classes. [17.2]
- Fine-tune generalizations. [17.3]
- Realize associations. [17.4]
- Prepare for testing [17.5]

The first two steps are motivated by the theory of transformations. A *transformation* is a mapping from the domain of models to the range of models. When modeling, it is important not only to focus on customer requirements, but to also take an abstract mathematical perspective.

17.2 Fine-tuning Classes

Sometimes it is helpful to fine-tune classes before writing code in order to simplify development or to improve performance. Keep in mind that the purpose of implementation is to realize the models from analysis and design. Do not alter the design model unless there is a compelling reason. If there is, consider the following possibilities.

- **Partition a class.** In Figure 17.1, we can represent home and office information for a person with a single class or we can split the information into two classes. Both approaches are correct. If we have much home and office data, it would be better to separate them. If we have a modest amount of data, it may be easier to combine them.

The partitioning of a class can be complicated by generalization and association. For example, if *Person* was a superclass and we split it into home and office classes, it would be less convenient for the subclasses to obtain both kinds of information. The subclasses would have to multiply inherit, or we would have to introduce an association between the home and office classes. Furthermore, if there were associations to *Person*, you would need to decide how to associate to the partitioned classes.

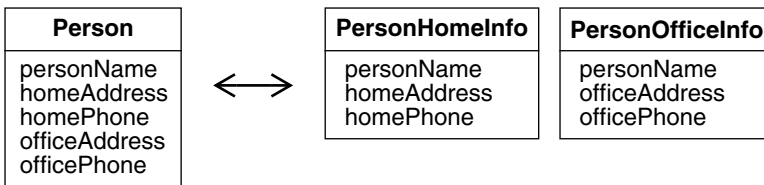


Figure 17.1 Partitioning a class. Sometimes it is helpful to fine-tune a model by partitioning or merging classes.

- **Merge classes.** The converse to partitioning a class is to merge classes. If we had started with *PersonHomeInfo* and *PersonOfficeInfo* in Figure 17.1, we could combine them. Figure 17.2 shows another example with intervening associations. Neither representation is inherently superior, because both are mathematically correct. Once again, you must consider the effects of generalization and association in your decisions.

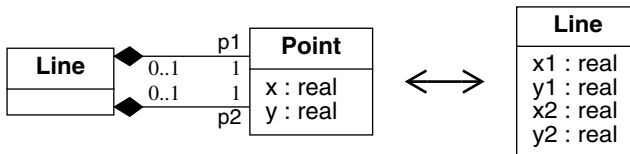


Figure 17.2 Merging classes. It is acceptable to rework your definitions of classes, but only do so for compelling development or performance reasons.

- **Partition / merge attributes.** You can also adjust attributes by partitioning and merging, as Figure 17.3 illustrates.
- **Promote an attribute / demote a class.** As Figure 17.4 shows, we can represent address as an attribute, as one class, or as several related classes. The bottom model would be helpful if we were preloading address data for an application.

ATM example. We may want to split *Customer address* into several classes if we are prepopulating address data. For example, we may preload *city*, *stateProvince*, and *postalCode*

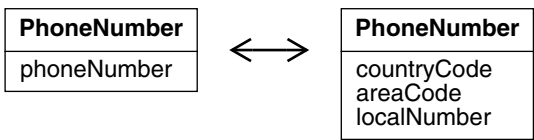


Figure 17.3 Partitioning / merging attributes

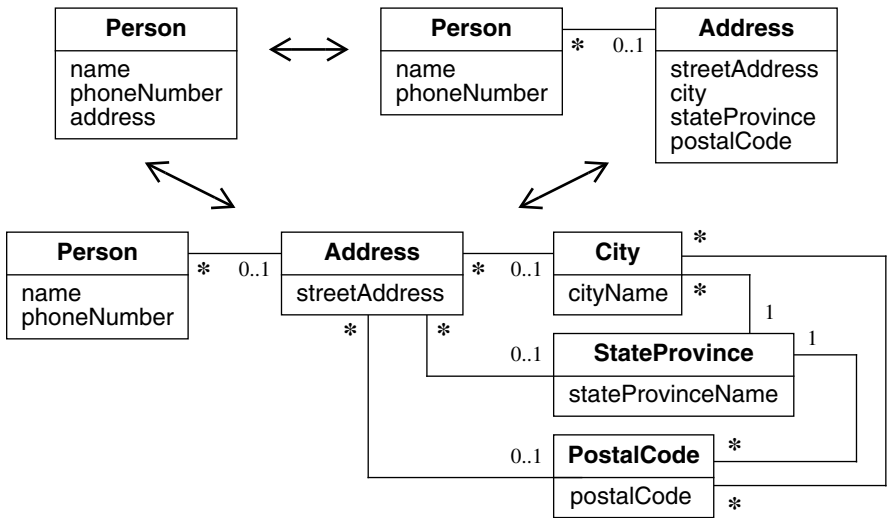


Figure 17.4 Promoting an attribute / demoting a class

data for the convenience of customer service representatives when creating a new *Customer* record.

We may also want to place *Account type* in its own class. Then it would be easier to program special behavior. For example, the screens may look different for checking accounts than for savings accounts.

All in all, the ATM model is small in size and carefully prepared, so we are not inclined to make many changes.

17.3 Fine-tuning Generalizations

As you can reconsider classes, so too you can reconsider generalizations. Sometimes it is helpful to remove a generalization or to add one prior to coding.

Figure 17.5 shows a translation model from one of our recent applications. A language translation service converts a *TranslationConcept* into a *Phrase* in the desired language. A *MajorLanguage* is a language such as English, French, or Japanese. A *MinorLanguage* is a

dialect such as American English, British English, or Australian English. All entries in the application database that must be translated store a *translationConceptID*. The translator first tries to find the phrase for a concept in the specified *MinorLanguage* and then, if that is not found, looks for the concept in the corresponding *MajorLanguage*.

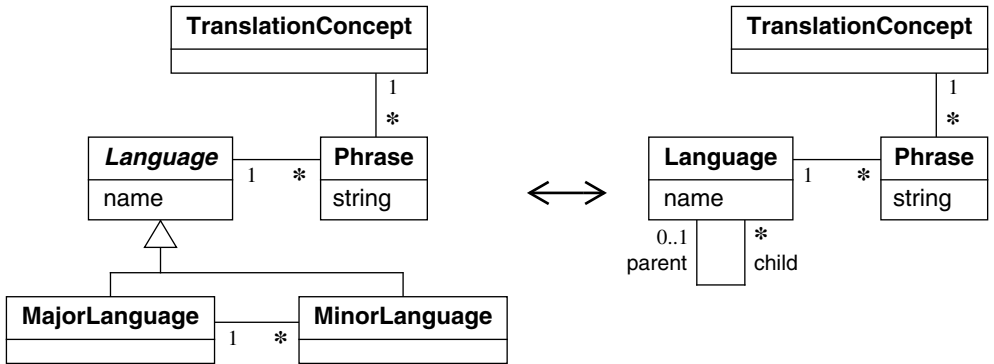


Figure 17.5 Removing / adding generalization. Sometimes it can simplify implementation to remove or add a generalization.

For implementation simplicity, we removed the generalization and used the right model. Since the translation service is separate from the application model, there were no additional generalizations or associations to consider, and it was easy to make the simplification.

ATM example. Back in Section 13.1.1 we mentioned that the ATM domain class model encompassed two applications—ATM and cashier. We did not concern ourselves with this during analysis—the purpose of analysis is to understand business requirements, and the eventual customer does not care how services are structured. Furthermore, we wanted to make sure that both applications had similar behavior. However, now that we are implementing, we must separate the applications and limit the scope to what we will actually build. Figure 17.6 deletes cashier information from the domain class model, leading to a removal of both generalizations.

Figure 17.6 is the full ATM class model. The top half (*Account* and above) presents the domain class model; the bottom half (*UserInterface*, *ConsortiumInterface*, and below) presents the application class model. The operations are representative, but only some are listed.

17.4 Realizing Associations

Associations are the “glue” of the class model, providing access paths between objects. Now we must formulate a strategy for implementing them. Either we can choose a global strategy for implementing all associations uniformly, or we can select a particular technique for each association, taking into account the way the application will use it. We will start by analyzing how associations are traversed.

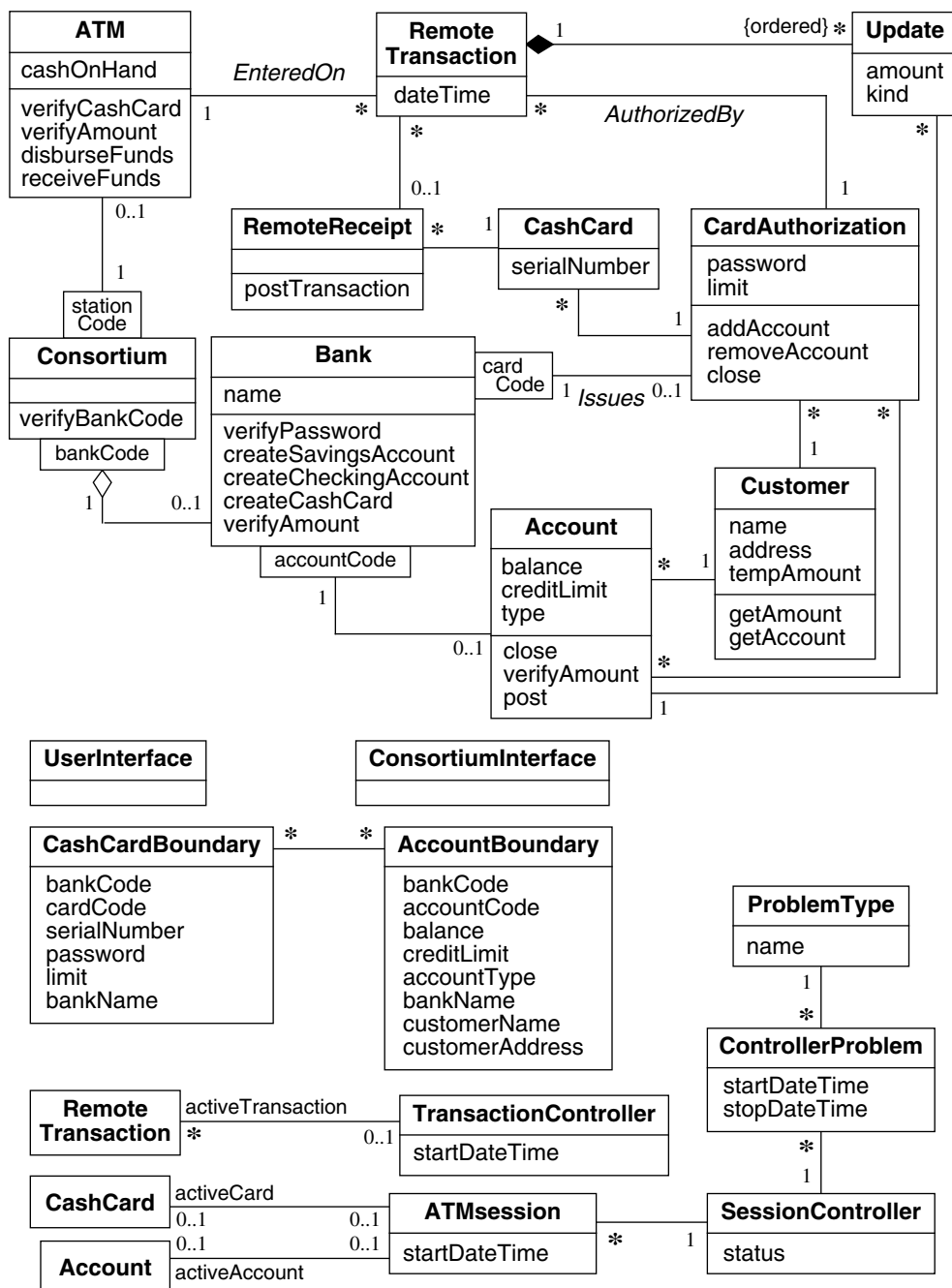


Figure 17.6 Full ATM implementation class model

17.4.1 Analyzing Association Traversal

We have assumed until now that associations are inherently bidirectional, which is certainly true in an abstract sense. But if your application has some associations that are traversed in only one direction, their implementation can be simplified. Be aware, however, that future requirements may change, and you may need to add a new operation later that traverses the association in the reverse direction.

For prototype work we always use bidirectional associations, so that we can add new behavior and modify the application quickly. For production work we optimize some associations. In any case, you should hide the implementation, using access methods to traverse and update the association. Then you can change your decision more easily.

17.4.2 One-way Associations

If an association is traversed only in one direction, you can implement it as a *pointer*—an attribute that contains an object reference. (Note that this chapter uses the word *pointer* in the logical sense. The actual implementation could be a programming-language pointer, a programming-language reference, or even a database foreign key.) If the multiplicity is “one,” as Figure 17.7 shows, then it is a simple pointer; if the multiplicity is “many,” then it is a set of pointers.

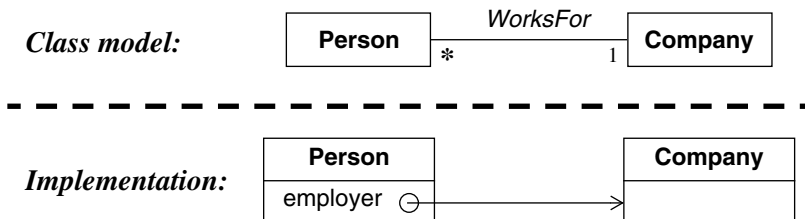


Figure 17.7 Implementing a one-way association with pointers. If an association is traversed only in one direction, you can implement it as a pointer.

17.4.3 Two-way Associations

Many associations are traversed in both directions, although not usually with equal frequency. There are three approaches to their implementation.

- **Implement one-way.** Implement as a pointer in one direction only and perform a search when backward traversal is required. This approach is useful only if there is a great disparity in traversal frequency in the two directions and minimizing both the storage and update costs is important. The rare backward traversal will be expensive.
- **Implement two-way.** Implement with pointers in both directions as Figure 17.8 shows. This approach permits fast access, but if either direction is updated, then the other must also be updated to keep the link consistent. This approach is useful if accesses outnumber updates.

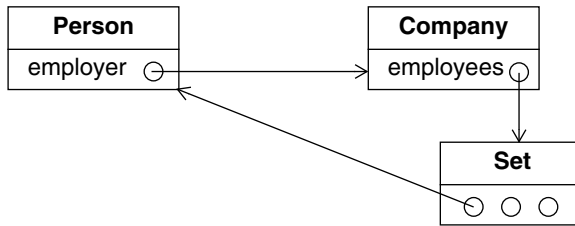


Figure 17.8 Implementing a two-way association with pointers. Dual pointers enable fast traversal of an association in either direction, but introduce redundancy, complicating maintenance.

- **Implement with an association object.** Implement with a distinct association object, independent of either class, as Figure 17.9 shows [Rumbaugh-87]. An association object is a set of pairs of associated objects (triples for qualified associations) stored in a single variable-size object. For efficiency, you can implement an association object using two dictionary objects, one for the forward direction and one for the backward direction. Access is slightly slower than with pointers, but if hashing is used, then access is still constant time. This approach is useful for extending predefined classes from a library that cannot be modified, because the association object does not add any attributes to the original classes. Distinct association objects are also useful for sparse associations, in which most objects of the classes do not participate, because space is used only for actual links.

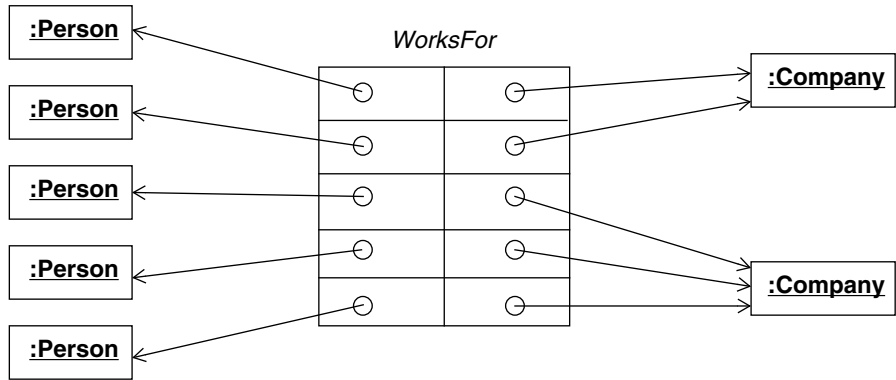


Figure 17.9 Implementing an association as an object. This is the most general approach for implementing associations but requires the most programming skill.

17.4.4 Advanced Associations

The appropriate techniques for implementing advanced associations vary.

- **Association classes.** The usual approach is to promote the association to a class. This handles any attributes of the association as well as associations of the association class. Note that promotion changes the meaning of the model; the promoted association has identity of its own, and methods must compensate to enforce the dependency of the association class on the constituent classes.

Alternatively, if the association is one-to-one and has no further associations, you can implement the association with pointers and store any attributes for the association as attributes of either class. Similarly, if the association is one-to-many and has no further associations, you can implement the association with pointers and store attributes for the association as attributes of the “many” class, since each “many” class appears only once for the association.
- **Ordered associations.** Use an ordered set of pointers similar to Figure 17.8 or a dictionary with an ordered set of pairs similar to Figure 17.9.
- **Sequences.** Same as ordered association, but use a list of pointers.
- **Bags.** Same as ordered association, but use an array of pointers.
- **Qualified associations.** Implement a qualified association with multiplicity “one” as a dictionary object. using the techniques of Figure 17.9. Qualified associations with multiplicity “many” are rare, but you can implement them as a dictionary of object sets.
- **N-ary associations.** Promote the association to a class. Note that there is a change in identity and that you must compensate with additional programming, similar to that for association classes.
- **Aggregation.** Treat aggregation like an ordinary association.
- **Composition.** You can treat composition like an ordinary association. You will need to do some additional programming to enforce the dependency of the part on the assembly.

17.4.5 ATM Example

Exercise 17.1 addresses the implementation of associations from the ATM model.

17.5 Testing

If you have carefully modeled your application, as we advise, you will reduce errors in your software and need less testing. Nevertheless, testing is still important. Testing is a quality-assurance mechanism for catching residual errors. Furthermore, testing provides an independent measure of the quality of your software. The number of bugs found for a given testing effort is an indicator of software quality, and you should find fewer bugs as you become proficient at modeling. You should keep careful records of the bugs that you find, as well as customer complaints.

If your software is sound, the primary difficulty for developers is in finding the occasional, odd error. Fixing the errors is a much easier problem. (In contrast, if your software is haphazard, it can also be difficult to fix the errors.)

You need to test at every stage of development, not just during implementation. The nature of the testing changes, however, as you proceed. During analysis, you test the model against user expectations by asking questions and seeing if the model answers them. During design, you test the architecture and can simulate its performance. During implementation, you test the actual code—the model serves as a guide for paths to traverse.

Testing should progress from small pieces to ultimately the entire application. Developers should begin by testing their own code, their classes and methods—this is called **unit testing**. The next step is **integration testing**—that is, how the classes and methods fit together. You do integration testing in successive waves, putting code together in increasing chunks of scope and behavior. It is important to do integration testing early and often to ensure that the pieces of code cleanly fit together (see Chapter 21). The final step is **system testing**, where you check the entire application.

17.5.1 Unit Testing

Developers normally check their own code and do the unit and integration testing, because they understand the detailed logic and likely sources of error. Unit testing follows the same principles as in pre-OO days: developers should try to cover all paths and cases, use special values of arguments, and try extreme and “off-by-one” values for arguments. If your methods and classes are simple and focused, it will be easier to prepare unit tests.

It is a good idea to instrument objects and methods. You can place assertions (preconditions, postconditions, invariants) in your code to trap errors. You should try to detect problems near the source (where they are easier to understand) rather than downstream (where they can be confusing).

We agree with the use of paired programmers and aggressive code inspection that is part of the agile programming movement. Along the same lines, we also recommend formal software reviews (see Chapter 22), where developers present their work to others and receive comments.

17.5.2 System Testing

Ideally, a separate team apart from the developers should carry out system testing—this is a natural role for a quality assurance (QA) organization. QA should derive their testing from the analysis model of the original requirements and prepare their test suite in parallel to other development activities. Then the system testers are not distracted by the details of development and can provide an independent assessment of an application, reducing the chance of oversights. Once alpha testing is complete, customers perform beta tests, and then if the software looks good, it is ready for general release.

The scenarios of the interaction model define system-level test cases. You can generate additional scenarios from the use cases or state machines. Pick some typical test cases, but also consider atypical situations: zero iterations, the maximum number of iterations, coincidence of events if permitted by the model, and so on. Strange paths though the state machine make good test cases, because they check default assumptions. Also pay attention to performance and stress the software with multiuser and distributed access, if that is appropriate.

As much as possible, use a test suite. The test suite is helpful for rechecking code after bug fixes and detecting errors that creep into future software releases. It can be difficult to automate testing when you have an application with an interactive user interface, but even then you can still document your test scripts for later use.

ATM example. We have carefully and methodically prepared the ATM model. Consequently we would be in a good position for testing, if we were to build a production application.

17.6 Chapter Summary

Implementation is the final development stage that addresses the specifics of programming languages. First you should address implementation issues that transcend languages—we call this implementation modeling. Sometimes it is helpful to fine-tune classes and generalizations before writing code in order to simplify development or to improve performance. Do this only if you have a compelling reason.

Associations are a key concept in UML class modeling, but are poorly supported by most programming languages. Nevertheless, you should keep your thinking clear by using associations as you study requirements and then necessarily degrade them once you reach implementation. There are two primary ways of implementing associations with programming languages—with pointers (for one or both directions) or with association objects. An association object is a pair of dictionary objects, one for the forward direction and one for the backward direction.

Even though careful modeling reduces errors, it does not eliminate the need for testing. You will need unit, integration, and system tests. For unit testing, developers check the classes and methods of their own code. Integration testing combines multiple classes and methods and subjects them to additional tests. System testing exercises the overall application and ensures that it actually delivers the requirements originally uncovered during analysis.

association object	implementation modeling	transformation
association traversal	integration testing	two-way association
dictionary object	one-way association	unit testing
fine-tuning classes	pointer	
fine-tuning generalizations	system testing	

Figure 17.10 Key concepts for Chapter 17

Bibliographic Notes

Transformations provide the motivation for Section 17.2 and Section 17.3. [Batini-92] presents a comprehensive list of transformations. [Blaha-96] and [Blaha-98] present additional transformations.

References

- [Batini-92] Carlo Batini, Stefano Ceri, and Shamkant B. Navathe. *Conceptual Database Design: An Entity-Relationship Approach*. Redwood City, CA: Benjamin Cummings, 1992.
- [Blaha-96] Michael Blaha and William Premerlani. A catalog of object model transformations. *Third Working Conference on Reverse Engineering*, November 1996, Monterey, CA, 87–96.
- [Blaha-98] Michael Blaha and William Premerlani. *Object-Oriented Modeling and Design for Database Applications*. Upper Saddle River, NJ: Prentice Hall, 1998.
- [Rumbaugh-87] James E. Rumbaugh. Relations as semantic constructs in an object-oriented language. *OOPSLA'87 as ACM SIGPLAN 22*, 12 (December 1987), 466–481.

Exercises

- 17.1 (7) Implement each association in Figure 17.6. Use one-way or two-way pointers as the semantics of the problem dictates. Explain your answers.
- 17.2 (5) Implement each association in Figure E12.3. Use one-way pointers wherever possible. Should any of the association ends be ordered? Explain your answers.
- 17.3 (4) Implement each association in Figure E15.2. Use one-way or two-way pointers as the semantics of the problem dictates. Should any of the association ends be ordered? Explain your answers.
- 17.4 (3) Implement the association in Figure E15.3. Use one-way or two-way pointers as the semantics of the problem dictates. Explain your answer.
- 17.5 (7) Implement each association in Figure E12.4. Use one-way or two-way pointers as the semantics of the problem dictates. Should any of the association ends be ordered? Explain your answers.

OO Languages

This chapter discusses how to take a generic design and make the final implementation decisions that are required to realize the design in C++ or Java. These are the two dominant languages used in OO implementation. The goal of this chapter is to produce code for a program. We do not intend to give a C++ or Java tutorial, but to highlight language features and explain their use with models.

Chris Kelsey was the primary author of this chapter and we thank her for her help.

18.1 Introduction

It is relatively easy to implement an OO design with an OO language, since language constructs are similar to design constructs. In this book we will focus on C++ and Java, since they are the dominant OO languages. Even if you are using another language, many of the principles will be the same and the discussion here will be relevant.

C++ and Java have much in common. Java is younger than C++ and borrows heavily from C++ syntax. Both are strongly typed languages, where variables and values must be known to belong to a particular native or user-defined type. Strong typing can improve reliability by detecting mismatched method arguments and assignments, and it increases opportunities for optimization.

18.1.1 Introduction to C++

C++ was developed by Bjarne Stroustrup at AT&T Bell Laboratories in the 1980s, with the intent of extending the widely used, procedural C language to include OO capabilities. It retains the “close to the machine” characteristics that are the hallmark of its C heritage, while adding the “close to the problem” capabilities of more direct expression of OO concepts.

Bell Labs originally implemented C++ as a preprocessor that translated C++ into standard C. As C++ came into mainstream use in the 1990s, direct compilers with symbolic de-

buggers and other development tools appeared. ISO/ANSI standardized the language and its libraries in 1998. C++ compilers for various operating systems are widely available from several major vendors, as well as from the Free Software Foundation.

C++ language syntax is a superset of C. Thus, C++ is a hybrid language, in which some entities are object types and some are traditional primitive types. Because of its origins, it retains features that are inconsistent with “pure” OO programming, such as free-standing functions that are not methods of a class. However, syntax and semantics remain consistent across native and object data types.

C++ supports generalization with inheritance and run-time method resolution (*polymorphism*). Classes may include a mix of polymorphic and nonpolymorphic methods. To enable run-time resolution for a method, a superclass must explicitly declare that method as *virtual*. The implementation is efficient, typically achieved by having each object contain a pointer to a table of methods for its class. There is no shared basic *Object* type as characterizes other OO languages. C++ permits multiple inheritance.

In addition to overriding methods via inheritance, C++ allows *overloading*—methods and functions may share the same name but have parameters that vary in number or type. Upon invocation, the language chooses the method or function with the appropriate parameters. Similarly, C++ operators may be overloaded, allowing a method to be expressed intuitively [such as $a + b$ instead of $a.add(b)$, where a and b are of type *ComplexNumber*].

Access specifiers promote encapsulation by restricting the availability of class members (methods or data) to methods of the class itself (*private*), the class and its subclasses (*protected*), or any class/method/function (*public*). A class may grant selective access to otherwise private members with a *friend* declaration. C++ performs all access restriction on a class, not object, basis—which means there is no access restriction among same-class objects. C++ *namespaces* provide a semantic scope for symbols but do not affect accessibility of visible entities. Namespaces were introduced in part to alleviate name conflicts among external libraries.

C++ exposes its memory management to the programmer, who may customize memory allocation strategies. The compiler statically allocates storage for objects (primitive or class types) declared at compile time. An application may also dynamically obtain storage from the heap at run time by using the *new* operation. A dynamically created object persists in memory until a *delete* operation explicitly destroys it.

Memory addresses remain fixed for the lifetime of an object and identify it. The programmer can get the address of a statically allocated object (using operator $\&$), as well as obtain an address as a handle for a dynamically allocated one. To access the target of a pointer, the pointer is *dereferenced* with the operator $*$, such that for any object O at address A , it is true that $\&O == A$ and $*A == O$. As with C, the programmer is generally not protected from memory-based errors at compile time or run time. C++ offers no run-time error-detection facilities; errors generally result in undefined program behavior.

In addition to the C-style pointers used as operands in memory operations, C++ includes *references* that syntactically appear as object aliases. References must be bound to an existing object at their creation. They are constant in their binding and so cannot be null. Otherwise, references behave much as permanently dereferenced pointers.

Classes have *constructors* and *destructors*, methods that C++ automatically invokes upon creation and destruction of objects. These are typically used for initialization and any operations that must occur upon termination (often deallocation), respectively.

In summary, C++ is a flexible language characterized by a concern for run-time efficiency, the ability to form broad type hierarchies, and uniform semantics. It provides a platform for fine granularity of expression and control at the expense of some simplicity.

18.1.2 Introduction to Java

Java came into being as a by-product of an early 1990s Sun Microsystems, Inc. exploration into programming consumer devices, a project that required a portable and device-independent language. It made its public debut in 1995 as a download on the then-infant World Wide Web, and the ability to run restricted Java programs was soon thereafter incorporated into Web browsers to allow dynamic and interactive content on Web pages. Within a few years, with the explosion of the Internet and distributed application architectures, Java became a staple of commercial development. Commercial, shareware, and free Java tools are widely available, as the language continues to evolve and its libraries grow. Sun maintains control over language releases and makes basic tools available at no charge for most computing platforms.

Java's popularity comes not so much from the core language itself as from its portability and its huge library of associated classes. Java source code compiles to an intermediate byte-code, which in turn runs on a platform-specific Java Virtual Machine. JVMs are available for nearly all operating systems and are readily substitutable with JVMs from other vendors. Native Java compilers are also available.

Just as C++ leveraged the experience of large numbers of C programmers, so Java leveraged the large base of C/C++ speakers. Java syntax is quite similar to C++, although its object and memory models differ significantly.

Java is a strongly typed language, with distinct primitive and object types. The usage of the two differs significantly: Java statically allocates primitives and treats them as value-based variables ($i = j$ assigns j 's value to variable i), but dynamically allocates object types at run time and manipulates them only through reference variables ($i = j$ assigns the physical object referred to by j to an alternative reference i). Java has no reference syntax for primitives but does provide tools for conversion between primitives and corresponding object types, such as *int* and *Integer*. Java version 1.5 adds *autoboxing*, the automatic conversion between primitives and their corresponding wrappers in common circumstances such as parameter passing and use in collections.

All Java object types share a single *Object* ancestor and so are, at the most abstract level, type compatible. Run-time type checking throws exceptions if invalid downcasts (treating an object as a subtype) are used. Polymorphism is automatic, and a programmer can prevent it only by explicitly prohibiting the definition of overriding methods. Java supports only single inheritance, although *interfaces* (uninstantiable class specifications containing only constants and method declarations) can emulate multiple superclasses.

Packages organize classes and provide a scope for identifiers. Each source-code compilation unit (file) in Java can contain at most one public class and must bear the name of that class, although the file may define other supporting classes. A declaration at the top of the

file identifies the package. Files using public symbols from a different package use an *import* directive to access the public elements of that package (such as *import java.io.** to use classes in Java's i/o package/library).

Similar to C++'s access specifiers, Java's access modifiers restrict the availability of attributes and methods. Without explicit specification, an attribute or method has by default **package** accessibility and may be used by all other methods defined within the package. Explicit options are **private** (intra-class access), **public** (universal access) and **protected** (extends access to subclasses defined outside their parent class's package). Thus the Java meaning of *protected* is a bit different than C++.

Memory management is the province of the JVM. All Java code—data and methods—exists within the context of a class. The system loads and unloads classes as needed and relocates code within memory during run time. The programmer need not, and cannot, know the location of objects—they must be addressed through their corresponding references that syntactically appear as object variables.

Object deallocation is done through garbage collection—when all references to an object have been retired, the system will return object memory to its pool. Although the programmer can suggest object destruction, the system cannot be forced to collect garbage on demand. Java also offers run-time memory error detection (such as array out of bounds and attempted use of null references) and throws appropriate exceptions on detection.

In summary, Java emphasizes run-time portability and code safety at the cost of some efficiency and flexibility. Java provides abstract interfaces to emulate multiple inheritance, distinguishes between object and primitive type semantics, and offers a rich library of objects for both general implementation and for system-level integration on distributed platforms.

18.1.3 Comparison of C++ and Java

Table 18.1 compares C++ and Java.

18.2 Abbreviated ATM Model

Figure 18.1 shows a portion of the ATM model that we will use as an example. We have added *CheckingAccount* and *SavingsAccount* so that we can discuss generalization.

18.3 Implementing Structure

The first step in implementing an OO design is to implement the structure specified by the class model. You should perform the following tasks.

- Implement data types. [18.3.1]
- Implement classes. [18.3.2]
- Implement access control. [18.3.3]
- Implement generalizations. [18.3.4]
- Implement associations [18.3.5]

	C++	Java
Memory management	Accessible to programmer. Objects at fixed address.	System controlled. Objects relocatable in memory.
Inheritance model	Single and multiple inheritance. Polymorphism explicit per method. No universal base class. Encourages mix-in hierarchies.	Single inheritance with abstract interfaces. Polymorphism automatic. Universal <i>Object</i> ancestor.
Access control and object protection	Thorough and flexible model with <i>const</i> protection available.	Cumbersome model encourages weak encapsulation.
Type semantics	Consistent between primitive and object types	Differs for primitive and object types.
Program organization	Functions and data may exist external to any class. Global (file) and namespace scopes available.	All functions and data exist within classes. Package scope available.
Libraries	Predominantly low-level functionality. Rich generic (template) container (data structures) and algorithm library.	Massive. Classes for high-level services and system integration included.
Run-time error detection	Programmer responsibility. Results in undefined behavior at run time.	System responsibility. Results in compile-time or run-time termination.
Portability	Source must be recompiled for platform. Native code runs on CPU.	Bytecode classes portable to platform-specific JVMs. JVM must be available.
Efficiency	Excellent.	Good. Can vary with JVM implementation.

Table 18.1 C++ vs. Java. Both are powerful languages with different trade-offs.

18.3.1 Data Types

If you have not already assigned data types to attributes, you must do so now. Certain data types merit special consideration.

Primitives

Floating-point numbers, integer types, characters, and booleans can express simple values. Where possible, use numeric values instead of strings. Numeric types typically allow better storage and processing efficiency, as well as easier maintenance of attribute integrity. (See subsequent section on enumerations.)

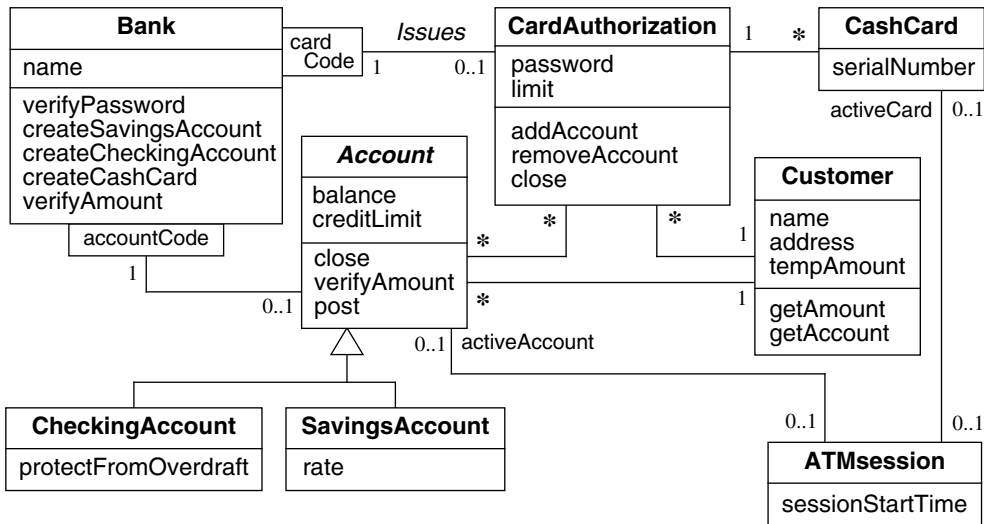


Figure 18.1 Abbreviated ATM implementation class model used in this chapter

Object Types

You can use objects to collect and organize attribute values into richer types. C++ supports the physical nesting of objects, where an instance of one object is physically created within the memory space of another during object construction. Java supports only the referencing of objects as members, so a Java object within another object is analogous to a C++ pointer or reference member that must be explicitly bound to another object.

C++ structs are technically no different than classes except that all members are public by default. They are conventionally used as “POD”—that is, “Plain Old Data”—to bind values together in a logical grouping with no methods. A constructor may be added to initialize data members. Although all members are public, the host object encapsulates access to the struct members.

```

struct address {
    string street;
    string city;
    string state;
    address() : street(""), city(""), state("") {}
};

class Customer {
    string name;
    address addr; // object attribute
    float tempAmount;
public :
    // ...

```

```
        string City() { return addr.city; } // use
    };
```

You can follow a similar strategy with Java, although the host object's initialization or constructor must explicitly create the object-attribute instance. Default package access yields access to object-attribute members within the using class.

```
class Address {
    String street = "";
    String city = "";
    String state = "";
}

public class Customer {
    private String name;
    // note explicit construction of attribute
    private address addr = new Address(); // object attribute
    private float tempAmount;
    // ...
    String city() { return addr.city; } // use
}
```

Reference Types

You can use Java object references, and C++ pointers and references, to implement associations. In Java, all class-type variables represent references to objects, while in C++ a variable may directly represent an object. C++ requires extra care that objects are not mistakenly used where a reference type is intended.

Object Identifiers

OO languages have built-in mechanisms for identifying objects, and ways to test object identity. There is typically no need to create explicit object identifier types. If you need a unique object identifier, you can obtain it at run time from the system.

In C++ an object's actual memory address serves as a unique identifier and can be obtained by applying the `&` (address of) operator to an object or object reference. Object identity can be tested by pointer (address) comparison. Java's `==` operator provides run-time identity comparison. If a run-time identifier is needed, the universal *Object* superclass contains *hashCode()* and *toString()* methods that, if not indiscriminately overridden, yield unique integer and string identifiers. Although memory addresses are not available in Java, most systems implement these methods based on internal object location.

Do not confuse a unique domain identifier, such as a bank account code or taxpayer number, with an object identifier. A domain identifier describes a domain-dependent property, while an object identifier describes a system-based attribute.

Enumerations

Enumerations provide two advantages—a value domain constraint and symbolic representation of values. For example, CLUB, DIAMOND, HEART, SPADE can express the range of playing-card suits.

C++ can directly implement enumerations. Each enumeration is a discrete type, for which you can define methods and operators. Members that are not explicitly assigned a value take on sequential integral values, starting at 0 if not otherwise specified.

```
enum Card = { CLUB, DIAMOND, HEART, SPADE };
```

There is an implicit conversion from an enumerated type value to integral type, but not vice versa. C++ guarantees that the sizes of objects of enumerated type are large enough to hold the domain's largest value. For example, a variable of the enum {FALSE,TRUE} could be sized as a single bit. Enumerations may appear at global scope or may be class members.

In practice, it can be cumbersome to use C++ enumerations, owing to conversion issues and the burden of redefining operations that are natively available for *ints*. Enums are best used to clarify code by providing symbolic constants. Encapsulated attribute values may be internally stored and manipulated as *ints*, while the public interface uses the constants to restrict parameter values, and method implementations may use the constants as bounds.

```
class Car {
public: // make enum public so clients can use it
    enum direction {N,E,S,W};
private:
    int mph;        // car speed
    int nesw;       // car direction
public:
    // require enum type parameter
    Car(int speed,direction dir) : mph(speed), nesw(dir) {}
    // int allows ++, -- without overloading for enum type
    Car& TurnRight() {if (++nesw >W) nesw = N; return *this;}
    Car& TurnLeft()  {if (--nesw <N) nesw = W; return *this;}
    // ...
};

int main()
{
    Car (15,Car::E): // client uses public enum value
    // ...
}
```

Java versions prior to 1.5 do not include the ability to create enumerated types. Java enums are similar to those in C++. In the absence of enumerations, you can use interfaces (see Section 18.3.4) to group and share constants. The explicit field modifiers *public*, *static*, and *final* are optional here, as they apply by default to all *int* fields specified within an interface. (Note that the “Enumeration” found in the Java class library is a deprecated interface providing rudimentary iteration operations and is unrelated to enumerated types.)

```
public interface Card {
    public static final int CLUB = 0;
    public static final int DIAMOND = 1;
    public static final int HEART = 2;
    public static final int SPADE = 3;
```

```

    . . .
}

```

18.3.2 Classes

OO programming languages provide direct support for implementing objects. You must declare each attribute and method in a class model as part of its corresponding C++ or Java class. You will also need to add attributes and methods for implementing associations (see Section 18.3.5). It is good practice to carry forward the names from the design model.

You can regard objects as entities that provide service to client objects that make requests. Services appear as public methods of a class. Methods provide the protocol for obtaining services. In general, method parameters represent information that an object needs to perform a service, and the return value of the method represents the object's response to the client requesting the service.

Because the public interface describes the class's services, it is best to work “outside in” when defining classes. Start with the public interface—methods intended to provide services to others—and add internal methods and attributes as needed to support the public interface methods.

Because the public methods document class behavior, they typically appear at the beginning of a class declaration for easy visibility, although there is no requirement that they do so. Both C++ and Java resolve symbols only after reading an entire class definition, so members can refer to each other in any order.

18.3.3 Access Control

A class should define public methods for services that clients can request or invoke. A class may also specify data that is immutable as public. All other members of the class—attributes and methods used internally to implement public functionality—should be rendered invisible and inaccessible to other objects and functions.

Both Java and C++ rely on *access specifiers* (called *access modifiers* in Java) to control clients' access to methods and data. The most basic specifiers, applied to attributes and methods, are the same for both languages. Only methods of the class can access *private* attributes or methods. Any client can access *public* members of the class.

Access Control in Java

Strong encapsulation in Java requires detailed attention to packages as well as access specifiers. A Java package provides a scope that extends access privileges to other entities within that scope. Packages are indicated by a package declaration at the top of a source file; multiple files can thus be grouped into the same package. Unless the *private* access specifier explicitly qualifies an attribute or method, other methods of classes defined in the same package can freely access it. If no package is specified, a class is considered to be in a global default package where all but its explicitly private members are available to all other clients residing in any other source file without package specification.

We recommend that you avoid the default package and instead name packages to manage access control. Furthermore, you should declare all attributes and nonpublic methods as

private—package access alone provides only weak encapsulation. You should guard access to all attributes by wrapping them with appropriate methods.

In addition to attributes and methods, Java classes have explicit access control. A Java class itself must be declared *public* to allow its public methods to be accessed by clients outside its package. See *Visibility and access to classes* below.

Access Control in C++

C++ access specifiers apply to sections of the class declaration. All members are assumed private until the compiler encounters an access specifier. Specifiers can appear anywhere in the declaration and apply until another specifier is encountered. Again, you should guard all attributes and methods other than the known public interface as private and relax access only with sound justification. Access specification within C++ structs is the same, with the exception that members are by default *public* until a more restrictive specifier is encountered (see Section 18.3.1).

C++ allows selective access to private members through a *friend* declaration. The class containing the declaration grants access to a named function, method, or class and in doing so allows the named entity full access to its private members. Friendship is best used sparingly, as it provides additional paths to encapsulated members.

Visibility and Access to Classes

For a class type to be recognized and its public members available, it must be visible to its potential clients. For both C++ and Java, the basic compilation unit is the file. Classes in a single file are visible to each other, but in practice it is best to place no more than a single class, along with incidental support classes, in a single file. Thus, class types from outside the file must explicitly be made visible to the compilation unit.

In Java, the package structure governs visibility and access. Classes in files declared to be in the same package are always visible to one another. Packages are dependent on disk file structure—all files declared to be in a package must reside in a directory named with the package name in order to be located by the Java system. For both compilation and execution, Java uses the environmental variable CLASSPATH, which gives a relative starting point for the system to search for packages and classes.

Although a package can contain many classes, each Java source file can have at most one *public* class, and the file must have the same name as the sole public class within. Classes outside the package can use only public methods of public classes. For a public class to be visible to a class in a different package, the client class's source file must include an import directive: *import packageName.className* or *import packageName.**. The system locates classes by prepending the classpath to the package and optional file name.

For the ATM case study, we would place each of the classes in a separate file. For example, the source file Bank.java would contain:

```
package bankInfo
public class Bank { ... }
```

and the file Customer.java would contain:

```
package bankInfo
public class Customer { ... }
```

The files `Bank.java`, `Customer.java`, and other classes in the *bankInfo* package must reside in a disk directory named *bankInfo*. For *Bank* and *Customer* to be visible to the *ATMsession* class, which is not part of the *bankInfo* package, the `ATMsession.java` source file must first declare:

```
import bankInfo.*
public class ATMsession { ... }
```

The import statement lets the *ATMsession* implementation see and use the public methods of the public classes found in the *bankInfo* package.

The rules and naming requirements of packages are somewhat onerous, but do not be tempted to simplify by using a global package. Without controlling the visibility and access of the classes themselves, you abandon much of access control. Although private attributes are still respected as private, without packages, methods designed for internal access to those private attributes effectively become public, and nonpublic classes intended for restricted use also become publicly available.

C++ has no language-specified dependence on the location of source code or compiled code. C++ programs divide code into header files, which contain declarations (including classes), and implementation files, which contain the actual code for all but the most simple methods (very small methods are typically implemented within class declarations). To introduce symbols from one file that are needed in another, the `#include<filename>` (when the source resides on a system-specified search path) or `#include "filename"` (when the full path is specified within the quotes) directive is placed at the top of the client file. The directive incorporates the header file containing the declarations of the desired class into the client's compilation unit by substitution, so that the symbols contained in the header are effectively placed into the file at the directive's location.

C++ has a *namespace* feature that serves as a scoping mechanism for program names but does not affect access control. A class is placed into a namespace if it is declared within a namespace declaration in a file and has the fully qualified name of `namespace::classname`. A namespace can span source files, and a source file can contain multiple namespaces. To avoid cumbersome syntax, the symbols of one namespace can be introduced into another with a *using* directive. C++ standard library names exist within the namespace *std*.

When no namespace is specified, symbols are in the global namespace. Because namespaces impact visibility but not access control, they are used primarily to prevent or disambiguate coincident symbol names as might be found when utilizing multiple libraries.

Access Control in Inheritance Hierarchies

Both languages include the *protected* access specifier, but it works differently in C++ and Java. In C++, a protected member is accessible only by methods of the class and its subclasses.

In Java, protected attributes and methods have package access, which is further extended to methods of subclasses that are defined outside the package where the superclass is defined. Java does not have the equivalent of C++'s more restrictive protected access.

Applying Access Control

A UML model may have annotations—{public}, {private}, {protected}, {package} or +, −, #, ~—indicating access specifiers for class members (called *visibility* in the UML). The lack

of an explicit indication implies that a method is part of the public interface. A thorough implementation should include access specifiers and make attributes private.

18.3.4 Generalization

OO languages provide robust support for generalization through *inheritance*. A class can serve as the parent *superclass* to one or more child *subclasses*. A subclass inherits all the members of its parent and may add attributes and methods of its own. The subclass may also *override* superclass methods to let the child express a behavior of the same name and signature (parameters and return) differently. Subclasses in turn may act as superclasses to successive generations of subclasses, allowing the creation of hierarchies of classes, with each level expressing extended and/or more specific behaviors.

Beyond the convenience of making subclasses easy to specify, the inheritance mechanism lets objects be utilized more abstractly. Inheritance enables *polymorphism* (from the Greek “many faces”), where a child type can be addressed as if it were any of its ancestral types. At run time, the system resolves which kind of child is in use, and invokes that subclass’s variation of a behavior specified more generally by the ancestor. In a sound hierarchy, each subclass should be able to sensibly perform all superclass behaviors, such that anywhere the parent type might appear, the child type can fulfill the parent’s behavioral expectations. [Liskov-88]

You need not fully specify all classes in an inheritance hierarchy. You can simply declare higher-level abstractions, deferring implementation detail to subclasses. These are *abstract classes*—classes that are described but remain partially or wholly unimplemented. Abstract classes describe types at a level where there is insufficient knowledge to implement a concrete behavior, but the general specification is known. For example, any *Shape* may be drawn, but until the type of *Shape* is specified, it is impossible to know *how* it is drawn.

Because they are incompletely implemented, you cannot create objects of abstract types. Instead, you create *concrete* objects of their subclass types—ones that fully implement the parent’s specified behavior—and assign a superclass-type reference. The subclass objects are referred to as if they were of the more generic type. For example, a *Square* or *Circle*—concrete subclasses of an abstract *Shape*—might be assigned to a *Shape*-type reference variable. Although both would appear as *Shapes*, when their *draw()* method is invoked, each will polymorphically manifest its proper behavior. In Java,

```
Shape s1 = new Circle();  
Shape s2 = new Square();  
s1.draw(); s2.draw();
```

will result in drawings of different shapes.

C++ and Java implement somewhat different models of inheritance.

Inheritance in Java

All Java objects share a common ancestor, the *Object* class. *Object* contains minimal methods and fields to support programming concerns such as object identity, equivalence, and concurrency. The common basis also allows the manipulation of objects at an entirely generic level, such as forming collections of otherwise unrelated types. Using objects at an “un-

```
    }
};
```

18.3.5 Associations

OO languages lack direct support for associations. However, you can readily implement links with object references or distinct association objects. (See Section 17.4.) You should promote the association to a class if there are attributes describing the association itself. We also recommend promotion for n-ary and qualified associations.

One-Way Associations

One-way associations reduce interdependencies among classes. When one class references another, the referenced class must be visible and accessible to the hosting class. The referencing class must enforce and maintain the association, and typically utilizes the interface of the referenced class. To the extent these dependencies can be reduced to one side of an association, maintenance is reduced and reusability may be enhanced.

For example, in Figure 17.7, if we do not need to retrieve the collection of employees for a company, a pointer from *Person* to *Company* will suffice. In Java, the *Person* class can simply contain a *Company* field.

```
public class Company { ... }
public class Person {
    private Company employer;
    ...
}
```

C++ offers two options for implementing referencing attributes. Most commonly, a pointer may be used, which allows a link to be changed. Assuming a *Person* may change employers leads to the following C++ code. (This C++ code also permits a *null* employer, which is inconsistent with the multiplicity in Figure 17.7. Update methods would have to prevent a *null* employer to enforce the multiplicity.)

```
class Company { ... }
class Person {
    Company* employer;
    ...
}
```

A C++ reference member implies a permanent link in which the containing object has a dependency on the attribute object. References must be bound at initialization. They cannot be null, nor can they be assigned (changed), so the language enforces the preexistence of the link target. Assuming that an *Account* is issued for a particular *Bank* and is not transferable, and that no *Account* may be issued without a *Bank*,

```
class Bank { ... }
class Account {
    Bank& bank;
    ...
}
```

This reference-binding requirement of C++ necessitates the existence of a *Bank* prior to the construction of an *Account* (see Section 18.4.1). You can achieve a partial constraint in Java by declaring an object-type attribute as *final*, which ensures that a reference variable cannot be reassigned to a different object, although it does not entail the preexistence requirement.

When implementing associations through referencing attributes, take care not to subject the objects involved to inadvertent changes. An object that hosts a referencing attribute can potentially open a back door by inappropriately exposing the referenced object itself or its attributes. It is particularly important that attributes representing links should be well encapsulated and be modified or reported only through intentional and safe methods. C++ offers an added level of security with the ability to apply the *const* qualifier to referenced objects in situations where they may be intentionally exposed, but are not intended to be modified in the context of the association. Java does not distinguish between constant and mutable objects.

Two-Way Associations

Two-way associations entail link maintenance for both association ends. You can implement a one-to-one association with either a single reference on each end or an association object. Similarly, a one-to-many association requires a single reference on one end and a collection of pointers on the other end or an association object. For example, a customer may have several accounts and we want to be able to navigate this association in both directions. Both Java and C++ have collection object types available from their libraries that can represent the “many” side of the link. In Java,

```
public class Account {
    private Customer customer; // the 1 side of one-to-many
    ...
}

import java.util.* // to access HashSet class
public class Customer {
    // list of account ref's
    private HashSet accounts = new HashSet();
    ...
}
```

Association Classes

Association classes can increase independence of the objects involved by removing direct references to related classes, but they do so at some loss of efficiency and increase in complexity of implementing operations. Independent association classes decouple linked objects but require the overhead of navigating through the link.

An explicit association object is conceptually a set of tuples, each tuple containing one value from each associated class. A binary association object can be implemented as two dictionary objects, each dictionary mapping in one direction across the association. Both Java and C++ have library support for map objects. To encapsulate links and make link maintenance more intuitive, the maps may be wrapped into an application class that serves as a manager for construction or use of the objects involved.

Choosing an Implementation

If the model has not already prescribed implementation of an association, your choice may depend more on the scale, architecture, or enterprise environment of an application than on the nature of the association itself. If you are constrained from modifying existing classes, an association class may be preferable. Association classes often best implement one-to-many associations where the “many” side can be quite large, or is sparse, and can provide independent and extensible management operations where required.

For simple associations, referencing attributes provide navigation across links. Where possible, one-way associations provide more efficient and safer implementations. Care should be taken to encapsulate links and to prevent inadvertent exposure of a referent’s information or interface through a host object.

18.4 Implementing Functionality

Once you have the structure in place, you can start implementing methods. For each class, specify methods by signature (method name, parameters, return type). The class model implies many methods. Obviously, you can create and destroy objects and links as well as access attribute values. More subtly, you can traverse a class model leading to additional methods. Methods also arise from derived attributes, the state model, and the interaction model.

- Object creation. [18.4.1]
- Object lifetime. [18.4.2]
- Object destruction. [18.4.3]
- Link creation. [18.4.4]
- Link destruction. [18.4.5]
- Derived attributes. [18.4.6]

Objects have state, behavior, and identity. This is reflected in the object life cycle. Objects are created by the system and should be initialized in a valid state. C++ has facilities for destruction of objects on demand, while Java can only suggest destruction to its garbage collector. Both have special methods that run at object creation, and both can specify behavior at the termination of an object’s lifetime.

Object can request services or information from one another. They do so by invoking other objects’ methods. In both Java and C++, object behaviors are invoked using a *membership operator* ‘.’ to indicate that the right-hand operand—a method of the object’s class—should be invoked on the named target object. The same membership syntax accesses attributes—*X.y* names the *y* attribute of the object *X*, though normally attributes are encapsulated and so not reachable from another object.

Within the context of a class, objects have an implicit reference to self, called *this* in both Java and C++. You need not explicitly qualify member names within class definitions: In Java, *fn()* means *this.fn()* and *y=10* means *this.y=10*. In C++, *this* is a pointer, and so the pointer membership operator “->” would be used, *this->fn()*.

For an object to call upon another object for services or information, it must have a name by which it can access its target. This handle may be provided in the form of a link, where the requesting object has a pointer or reference to another, or through a parameter, where one object receives a handle to another at method call. Any invocation of the target objects' methods (or, rarely, the use of target objects' attributes) takes place through the target handle and is governed by the target's members' access specifications.

Both C++ and Java allow *static* class members that are shared by all objects of a class type. Although they are subject to access specification, as any other data or method of a class might be, these static elements have entirely different life cycles than conventional objects and can be accessed through the class itself, apart from any particular instances that may exist. (See Section 18.4.2.)

18.4.1 Object Creation

In OO languages, objects are typically created dynamically (during run time), through a request to the system to create an object of a particular type. In Java, this is a requirement for nonprimitive types, while primitive types are statically (at compile time) allocated simply by declaration. In C++, both object and primitive types can be either statically or dynamically allocated, and the results are considered as objects in either case. Both Java and C++ use the *new* operator to create objects. In C++,

```
// static allocation -- creates a single account
Account acct1;

// statically allocate an array of 10 accounts;
Account accounts1[10];

// dynamic allocation: define a pointer,
// initialize with result of new operation
Account* acct2 = new Account;

// dynamically create array of 10 accounts
Account* accounts2 = new Account[10];
```

In Java,

```
// create a reference variable,
// initialize with result of new operation
Account acct = new Account();

// create array of 10 references -- no accounts are made!
Account accounts = new Account[10];

// now create the accounts for the array:
for (int i = 0; i < 10; i++) accounts[i] = new Account();
```

When a new object is created, the system allocates storage for its attribute values and performs other chores involved with the start of the object life cycle. OO languages free the programmer from having to understand technical details of object implementation such as

object layout and internal identifiers. Java and C++ also let the programmer specify operations to occur at the time of object creation, so that you can ensure that objects come into existence in a valid logical state. Once the system has completed its creation chores, a special method, called a **constructor**, is automatically invoked. The constructor takes the form of a method, with no return value, that shares the class name. It may have any number of parameters and may be overloaded. For example, in Java, the following code specifies that *Account* has two constructors, one requiring an opening balance and one that takes no arguments.

```
public class Account {
    ...
    public Account(float OpeningBalance) {
        balance = OpeningBalance;
    }
    public Account() { balance = 0; }
    ...
}
```

Constructors may be used to assign values to members, to create member objects, or perform other start-of-life processing on an object. Constructors run after the object is fully formed, so they may call other methods and in general have the same capabilities, privileges, and constraints of other methods. If no constructor is defined for an object, a constructor of the form *X(){}* is presumed to exist. Though it performs nothing, it allows creation expressions matching its imaginary declaration. Once you create a parameterized constructor, the system will no longer provide the no-argument form for you. If you need one, you will have to overload *X(...)* with *X()*, as done above for *Account*.

Understanding constructors in subclasses is not difficult, if you consider object construction. Subclass-type objects inherit their attributes and methods because they contain their parent type, plus any members they add at the subclass level. You can think of it as if their parent type is created, and then their own piece is added on, and so while constructors are not inherited, they run sequentially from the most general through the most specific level. In the case of C++, where objects can be statically allocated, member objects get constructed recursively—including the invocation of their respective constructors—before the host object can be fully constructed. Only after the host object is complete does its own constructor run. You can see this in action (here in C++).

```
class X {
    public:
        X() { cout << "X!"; }
};

class Y : public X {
    public:
        Y() { cout << "Y!"; }
};

class Z : public Y {
    public:
```

```

        Z() { cout << "Z!"; }
};

int main() {
    Z z; // simply make a Z...
    return 0;
}

```

Running the code produces the result: X!Y!Z!

It is very poor practice to define no constructors for a class. By default, C++ performs no default initialization of object members, while Java initializes members to 0 or null according to type. Neither provides a valid-state object for most classes. The purpose of the constructor is to have a way to perform initialization and operations on an object so that, at the time of its entry into the program domain, it is a fully formed, logically intact, safe-to-operate-on instance of its type.

If there are no start-of-life operations to be performed by a newly constructed object, Java allows nonzero default initializations to be performed within the class definition. In C++, initialization take place within a *member initialization list* that specifies initial values for members before entry into the constructor code block. Both languages allow assignment to members within the body of the constructor, although initialization should be preferred over assignment wherever possible.

In Java,

```

public class Account {
    private float balance = 0; // initialization
    ...
    public Account() {} // constructor need not assign
}

```

In C++,

```

class Account {
    float balance; // values not allowed
    ...
public:
    Account() : balance(0) {} // initialization via list
}

```

In C++, the system also supplies a copy constructor, that specifies the semantics of copying by assignment. Like the default (no-argument) constructor, if one is not written for the class, a constructor of the form *X(const X& x)* is provided, with the default value-based meaning of shallow (memberwise) copying. Java discourages object copying by requiring the programmer to specify that a class implements the *Cloneable* interface and override *Object's clone()* method.

18.4.2 Object Lifetime

Statically allocated (created at compile time) objects, which may be any type in C++ and primitive types and reference variables in Java, exist within the scope of a code block, indicated by { }. They are automatically destroyed when program control passes out of their

scope. Dynamically allocated (*new*'d) objects persist in memory from the time of their creation until they are explicitly destroyed in C++, or no longer in use by the program in Java (see object destruction below).

Both C++ and Java allow class members to be qualified as *static*. Static members belong not to an instance of a class, but to the class itself, and (if public) can be accessed in the absence of any instance of the class: (C++) *X::StaticMethod()* or (Java) *X.StaticMethod()*. As a convenience, both static data and methods can be addressed through a particular object, but neither are instance members of that object.

Static data members come into existence before any particular object of the class type is created—at program entry in C++ or at class loading in Java—and remain in existence throughout the program. In C++, static members exist as independent objects (memory allocations), while in Java each class itself has an independent instantiation (the *class instance*), of which statics are considered members. Although statics may also be accessed within the scope of their class during method definition or through particular object instances, there is only one instance of a static data member per class, and its life cycle is entirely independent of any object through which it is accessed. Because static methods are not members of a particular object, the notion of *this* (and *super* in Java) has no meaning within the context of a static method, and regular instance data cannot be referenced without qualification with a specific object reference.

In C++, variables defined at global scope (not within a function, including *main*, method, or as a member of another object) have a life cycle much like class-based statics. They are created before the program starts, and are destroyed at program termination. While global data is discouraged as poor practice, global functions are commonly used in C++. Although Java does not allow free-standing functions to be defined at global scope, global functions are regularly emulated in utility library classes that contain groups of static methods. Several of Java's library classes, such as *Integer* or *Collections*, are predominantly or wholly composed of static methods.

Because static members and class instances are created apart from object instances, the initialization of statics occurs independently. Statics in C++ are declared within, but defined and initialized outside, the class, while Java static fields may be initialized conventionally or within a static initialization block. Both C++ and Java statics are guaranteed to be initialized only at some point before use in a program, but neither language specifies a relative instantiation or initialization order.

18.4.3 Object Destruction

When an object is no longer needed, it may be destroyed and its storage returned to the memory pool. With statically allocated (compile-time) objects, this happens as they fall out of scope. If, for some reason, you retain a handle to an object that has fallen out of scope, using it would be an error, somewhat analogous to calling the telephone number for someone who has moved and is no longer available at the number you called.

For dynamically allocated objects, with their potentially unconstrained lifetimes, the problem becomes one of eliminating objects that are no longer of interest, or whose existence is no longer logically valid, and recycling their space in memory. For this, C++ and Java

each pursue one of two basic strategies: Either the programmer takes responsibility for explicitly removing objects that are no longer needed or sensible (C++), or the system keeps an eye out for whether an object is actually in use or not (Java). The latter approach, called **garbage collection**, is typically implemented in a way that the system tracks references to the object. When there are no remaining references (no valid handles to the object exist any more), the system tags the object as eligible for recycling. The system periodically runs a *garbage collector* that reclaims tagged objects' space for the memory pool.

Manual disposal of objects, as C++ practices, places a burden on the programmer to be aware of allocations and responsible for object cleanup. However, it also allows tight control of the object life cycle and the ability to fully model object events, including exit or termination of an object from its domain. Garbage collection frees the programmer from memory management chores, but restricts the ability to exercise end-of-life-cycle control.

C++ classes can have **destructors**, analogous to constructors, that run automatically at the destruction of an object. Destructors take the form of a no-argument constructor, with the ~ before the class name: `X::~~X() { ... }`. They never take arguments and have no return value.

Because C++ object destruction can be explicitly invoked and predictably executed, it is common for the destructor to undo things a constructor has done. Most common is the allocation and deallocation of dynamically created object members, but this pairing of functionality may include increment/decrement of counts, acquisition and release of resources, and so forth. Destructors can also be used to predictably invoke any logically necessary (often cleanup) behaviors that are required at end of object life.

```
Window :: ~Window ()
{
    // erase the window and repaint the underlying region
}
```

The process of destruction is the inverse of construction: First, the destructor code runs, then any members that themselves require destruction are in turn destructed (starting with their destructors), and finally the whole of the memory is returned to the heap.

To explicitly destroy a dynamically allocated object, the **delete** operator is used. Empty array brackets `[]` are used to differentiate the deletion of individual objects from the deletion of a dynamically allocated array.

```
class X { ... }
...
X x1 = new X;
X* *x2 = new X[20];
...
delete x1;
delete [] x2;
```

After deleting a C++ object, it is advisable to immediately assign 0 to any pointers that were pointing to it. *Delete* is implemented to have no effect when (accidentally) called on a null pointer, whereas “double deleting” an object results in undefined—and often disastrous—behavior.

Java objects are not explicitly destroyed by the programmer. Java discerns when there are no more references to an object in use, at which time the object becomes eligible to be destroyed at the garbage collector's convenience. The programmer can attempt to induce destruction, by explicitly setting reference variables to *null* to detach them from objects, and may even suggest garbage collection, but cannot guarantee timely destruction of an object.

Java does allow classes to override *Object's finalize()* method, which acts in a manner similar to the C++ destructor. However, because the exact time of object destruction cannot be triggered, you cannot rely on any operations within *finalize()* to occur at a predictable time.

18.4.4 Link Creation

Links are forged and destroyed (valued or set to null) as objects interact with one another. Referential links should be created in the course of whatever behavior (method) initiates an association between objects, and destroyed in the course of whatever behavior terminates the association. In general, you should avoid direct *set* methods in favor of operations that encapsulate data and forge links only under validated conditions.

To create a link in a one-way association, an object retains a handle to a parameter object. For bidirectional links, handles can be exchanged, although it is usually preferable to initiate the exchange from one side and encapsulate the other to ensure consistent logic and full link updates. Which class governs the exchange is a matter of application logic.

In the following simplified Java example, *Students* maintain a list of current *Courses*, and *Courses* maintain class lists of current *Students*. Because the decision to take a course is generated by a student and conditioned on the student's status, the creation of links is governed by the *Student.addClass(Course)* method. The *addClass* method in turn calls the *Course's enroll(Student)* method, which is accessible to *Student* due to the class's shared package. However, package access prevents public access to clients (in this case, the class *EnrollmentApplication*), thereby restricting link creation to the orderly process prescribed by *Student's addClass* method.

```
// file Course.java
package school;
public class Course {
    private String title;
    private HashSet students = new HashSet();

    public Course (String nm) { title = nm; }
    public String courseName() { return title; }

    boolean enroll(Student stu) {
        // check that course isn't already full, etc.
        // if ok, return result of HashSet.add method
        return students.add(stu);
    }
}
```

```

    public void printClassList() {
        System.out.println("Class List for " + courseName() +
            ":");
        Iterator it = students.iterator();
        while (it.hasNext())
            System.out.println(
                ((Student)it.next()).studentName());
    }
}

//file Student.java
package school;
public class Student {
    private String name;
    private HashSet classes = new HashSet();

    public Student(String nm) { name = nm; }
    public String studentName() { return name; }
    public boolean addClass(Course crs) {
        // validate student's ability to take this course;
        // if ok, request course to enroll student;
        // if course returns true, add course to list
        return ( crs.enroll(this)) ?
            classes.add(crs) : false ;
    }

    public void printCourses() {
        System.out.println("Courses for " + studentName() +
            ":");

        Iterator it = classes.iterator();
        while (it.hasNext())
            System.out.println(
                ((Course)it.next()).courseName());
    }
}

// file EnrollmentApplication.java
import local.school.*
public class EnrollmentApplication {
    public static void main(String [] args) {

        Student mike = new Student("mike");
        Student bill = new Student("bill");
        ...
        Course tt = new Course("Type Theory");
    }
}

```

```

        mike.AddClass(tt);
        bill.AddClass(tt);

        tt.PrintClassList();
        mike.PrintCourses()
    }
}

```

Objects can be constructed with knowledge of their collaborators, and so establish links at the time of object creation. Often, it is desirable to condition object creation on establishment of links. In such cases, it is necessary to delegate construction to “factory” techniques that examine criteria before invoking construction, as neither Java nor C++ constructors can return an error, nor can they abort the construction process. Factories may be classes, or simply static methods (see Section 18.4.2).

Here (in C++), a *Transaction* will not be constructed until an account number is validated and a handle to the associated account obtained.

```

class Transaction {
    ...
protected:
    // private or protected constructor to prevent
    // public construction
    Transaction(Account& acct) { ... }

public:
    ...
    static Transaction* MakeTransaction
        ( const char* acctNumber) {
        Account* acct;
        // null pointer return indicates no transaction
        if ( /* account is not ok */ ) return 0;
        // return a transaction for valid account
        return new Transaction(*acct);
        ...
    }
};

```

18.4.5 Link Destruction

Link destruction is generally the inverse of link creation. The activity that dictates the dissociation of objects is the method where a referential link is typically broken. In most cases, this simply means the handle attribute is set to a null value, or an entry is removed from a collection of links. A *Student* might drop a *Course*,

```

// Student.dropClass(Course):
public boolean dropClass(Course crs) {
    if (!classes.contains(crs)) return false;
    if (!crs.drop(this)) return false;
    classes.remove(classes.indexOf(crs));
}

```

```

        return true;
    }

    // Course.drop(Student):
    boolean drop(Student stu) {
        if (!students.contains(stu)) return false;
        students.remove(students.indexOf(stu));
        return true;
    }

```

When links involving independently existing objects have been created, typically each object is intended to continue existence beyond the life of the link. Care must be taken that the referenced object is accessible through other references, or that a handle to a referenced object is captured from a link-destroying method. Failure to do so can result in a memory leak (C++) or an inaccessible object that may be garbage-collected in Java.

In C++, a common idiom is to pair constructor-destructor activity to implement the creation and destruction of links. Those links often reflect the acquisition and release of resources.

Links involving objects that are existentially dependent on one another may require destruction of one of the linked objects. This, in turn, may imply the updating or destruction of other objects that may be linked to the dependent object, or suggest that the link sustaining the dependent object not be destroyed. This is analogous to database operations, where record deletion may be propagated (cascaded) to related records or prohibited in order to preserve the integrity of existing records.

18.4.6 *Derived Attributes*

For accuracy and currency, it is always desirable to calculate fresh values from independent data at the time of use. Usually the time required to compute fresh values is negligible when weighed against the costs of managing updates and redundancy for stored derived data.

It is a common mistake to include redundant state data in objects. States can often be determined by examining other attributes (in Java).

```

public class Account {
    // wrong: redundant state data for overdrawn
    private boolean overdrawn;
    private float balance;
    ...
    // compute state upon request instead
    public boolean isOverdrawn() { return balance < 0; }
    ...
}

```

States can also be determined via links (here C++):

```

class Customer {
    ...
    List<Account> accts;
public:

```

```

...
bool hasOverdrawnAccount() {
    // iterate through all accounts;
    // return true if any are overdrawn
}
}

```

or may be implied by the presence of links (again, in C++):

```

class Telephone {
    Telephone* connectedTo;
    ...
public:
    // are we connected to another phone?
    // if our link isn't null, we are...
    bool isBusy() { return connectedTo != 0; }
    ...
}

```

18.5 Practical Tips

Here are tips for using C++ and Java to implement an OO design.

- **Enumerations.** Use enumerated types for clarity and enforcement of domain values. (Section 18.3.1)
- **Java packages.** Avoid the default package and instead use named packages to manage access control. (Section 18.3.3)
- **Access control.** Declare all attributes and nonpublic methods as *private*. Relax encapsulation only if it is essential to do so. (Section 18.3.3)
- **C++ friend.** Use the friend declaration sparingly, because it can compromise encapsulation. (Section 18.3.3)
- **Java interfaces.** Consider using Java interfaces as a workaround for multiple inheritance in a model. An interface declares methods and constant fields and also provides a type for accessing objects. (Section 18.3.4)
- **C++ private inheritance.** Avoid private inheritance of classes in C++. Delegation is a better strategy. (Section 18.3.4)
- **One-way associations.** Use a one-way association when two-way association traversal is not needed. One-way associations are easier to maintain and reduce object interdependencies. (Section 18.3.5)
- **C++ reference and Java final.** Note association ends that must be bound at initialization and cannot be changed. C++ references can fully enforce these semantics, and the Java *final* property can partially enforce them. (Section 18.3.5)
- **Constructors.** It is a very poor practice to define no constructors for a class. A constructor should always initialize an object to a valid initial state. (Section 18.4.1)

- **C++ deletion.** When deleting a C++ object, it is advisable to immediately set its pointer(s) to 0. Delete has no effect for a null pointer, but accidental deletion of an already deleted object can be disastrous. (Section 18.4.3)
- **Link destruction.** When a link is destroyed, make sure the associated objects are accessible through other handles or intentionally destroyed. Otherwise you have a memory leak (C++) or an object that is inadvertently garbage collected (Java). (Section 18.4.5)

18.6 Chapter Summary

It is relatively easy to implement an OO design with an OO language, since language constructs are similar to design constructs. C++ and Java are the two dominant OO languages, and hence the subject of this chapter.

The first step in implementing an OO design is to implement the structure specified by the class model. Begin by assigning data types to attributes. Where possible, use numeric values instead of strings. Numeric types typically allow better storage and processing efficiency, as well as easier maintenance of attribute integrity.

Next define classes. It is best to work “outside in” by starting with the public interface. Then add internal methods, attributes, and lesser classes as needed to support the interface methods.

You should pay careful attention to access control—it provides the means to encapsulate attributes and methods and limit access to them. The most basic specifiers are the same for both C++ and Java. Only methods of the class can access private members. Any client can access public members. The *protected* specifier in C++ limits member access to the class and its subclasses. Java is more permissive, letting attributes and methods in the same package also access protected attributes and methods. Java packages are important both for organizing code and controlling access. The C++ *friend* declaration allows selective access to private members.

OO languages provide robust support for generalization through inheritance. C++ supports multiple inheritance, but Java is limited to single inheritance. However, Java does provide some of the benefits of multiple inheritance through the use of interfaces. A Java interface is a class specification with no implementation.

In Java polymorphism is automatic, though you can prevent further subclassing by declaring a class as *final*. Similarly, a *final* qualifier applied to a method prevents the method from being overridden. In contrast, polymorphism is not automatic in C++. To activate polymorphism, a method must be declared as *virtual*.

OO languages lack direct support for associations. However, you can readily implement links with pointers/references or distinct association objects. One-way associations reduce interdependencies among classes, so you should use them when traversal is needed in only one direction. C++ offers two options for implementing association ends. A pointer lets a link be changed or set to null. In contrast, a reference must be bound at initialization and can-

not be changed. Java lacks the full equivalent to the C++ reference—you can define an object-type attribute as `final`, which ensures that a reference variable cannot be changed but does not require binding at initialization.

When you need to traverse an association in both directions, you should use a two-way implementation. You can bury a pointer/reference (or set of pointers/references) in each related class or use an association object. With two-way pointers/references, you have redundancy and must be careful to keep both directions mutually consistent.

Once you have the structure in place, you can start implementing methods. You should carefully define constructors for new objects and be sure to initialize them to a valid state. Similarly, you should pay attention to destructors, being sure to release any system or program resources that are no longer needed upon the end of the object.

You should seldom include redundant data in objects; for accuracy and reliability it is usually better to calculate fresh values from independent data at the time of use.

abstract	derived data	interface	private
access modifier	destructor	namespace	protected
access specifier	enumeration	new	public
association	final	overloading	reference
concrete	friend	package	static
constructor	garbage collection	pointer	virtual
data type	generalization	polymorphism	

Figure 18.2 Key concepts for Chapter 18

Bibliographic Notes

[Stroustrup-94] provides an interesting discussion of language-design issues.

References

[Arnold-00] Ken Arnold, James Gosling, and David Holmes. *The Java Programming Language, Third Edition*. Boston: Addison-Wesley, 2000.

[Gosling-00] James Gosling, Bill Joy, Guy Steele, and Gilad Bracha. *Java Language Specification, Third Edition*. Boston: Addison-Wesley, 2000. (Available online: <http://java.sun.com/docs/books/jls/>)

[Liskov-88] Barbara Liskov. Data abstraction and hierarchy. *SIGPLAN Notices*, 23, 5 (May 1988).

[Stroustrup-94] Bjarne Stroustrup. *The Design and Evolution of C++*. Boston: Addison-Wesley, 1994.

[Stroustrup-97] Bjarne Stroustrup. *The C++ Programming Language, Third Edition*. Boston: Addison-Wesley, 1997.

Exercises

18.1 (1) Assign a data type to each attribute in Figure E18.1.

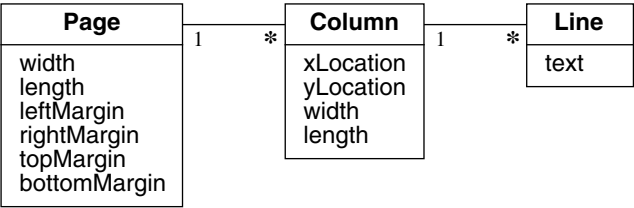


Figure E18.1 Portion of a class diagram of a newspaper

18.2 (4) Consider the model in Figure E18.2. For now, ignore the attributes, methods, and association. See Exercise 15.10 for an explanation of the model.

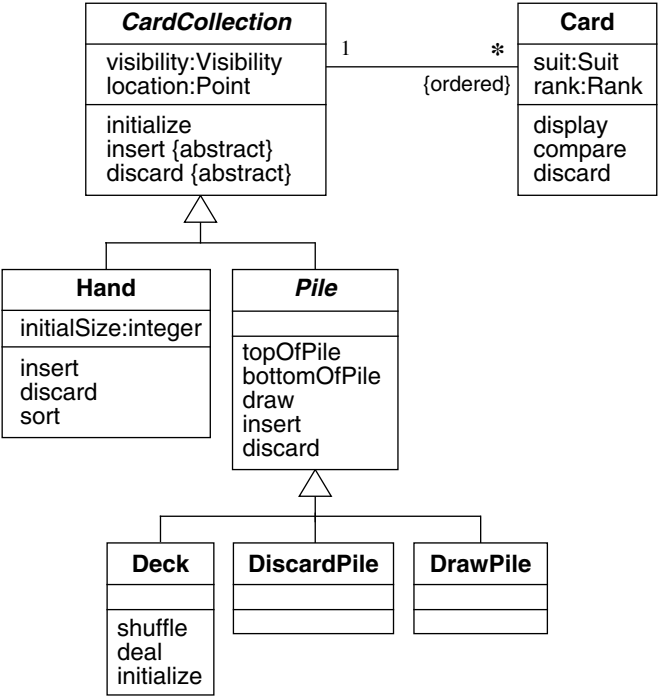


Figure E18.2 Portion of a class diagram of a card playing program

- a. Prepare C++ declarations for the classes and the inheritance. Pay careful attention to access control.

- b. Prepare Java declarations for the classes and the inheritance. Pay careful attention to access control and packages.
- 18.3 (6) In Figure E18.2 *visibility* controls whether the front or the back of a card is displayed. Add to your answer for Exercise 18.2 but, for now, continue to ignore the methods and association. [Instructor's note: You may want to give the students our answer to Exercise 18.2.]
- a. In C++, add declarations for all attributes, except location. Also define the enumerations.
 - b. In Java, add declarations for all attributes, except location. Also define the enumerations.
- 18.4 (7) Add the *CardCollection*—*Card* association to your C++ and Java answers from Exercise 18.3. [Hint: You can use a template which is not explained in this book. You can answer the exercise by considering the implementation of associations and reading about the standard library and templates on the Web or in a C++ language book.]
- 18.5 (6) Declare methods for your answer for Figure E18.2 for both C++ and Java. See Exercise 15.10 for an explanation of the methods.
- 18.6 Write C++ or Java code, including class declarations and methods, to implement the following using pointers.
- a. (9) One-to-one association that is traversed in both directions.
 - b. (6) One-to-many association that is traversed in the direction from one to many. The association is unordered.
 - c. (6) One-to-many association that is traversed in the direction from one to many. The association is ordered.
 - d. (8) Many-to-many association that is traversed in both directions. The association is ordered in one direction, and unordered in the other direction.
- 18.7 (7) Describe strategies for managing memory, assuming that automatic garbage collection is not available. Your answer should provide guidelines that a programmer could use during coding.
- a. **A system for text manipulation.** The system often creates one large string in contiguous memory from several smaller strings. You cannot waste memory and cannot set an upper bound on string length or the number of strings to combine. Write pseudocode for a method that combines strings, recovering memory that is no longer used.
 - b. **A multipass compiler.** Objects are created dynamically. Each pass examines the objects created on the previous pass and produces objects to be used on the next pass. The computer system on which the compiler will run has a practically unlimited virtual address space and an operating system with a good swapping algorithm. The methods in the run-time library for allocating and deallocating memory dynamically are inefficient. Discuss the relative merits of two alternatives: (1) Forget about garbage collection and let the operating system allocate a large amount of virtual memory. (2) Recover deallocated objects with garbage collection.
 - c. **Software that runs for a long time, such as banking software or an air traffic control system.** You have the same computer system and run-time library as described in Exercise 18.7(b). Discuss the relative merits of the two approaches.
 - d. **A method which may create and return an object that uses a large block of memory.** Discuss the relative merits of the following two approaches: (1) Each time the method is called, it destroys the object created the last time it was called, if any. (2) Each time the method is called, it may create a new object. It is up to the calling method to destroy the object when it is no longer needed. Comment on these two approaches.

- 18.8 (6) How might you organize Figure E12.4 into Java packages?
- 18.9 (7) Write C++ and Java declarations for the model in Figure E18.3. Implement the entire model. Be sure to consider access control and Java packages. Pay attention to constructors and destructors. Implement all associations as bidirectional. See the exercises in Chapter 12 for an explanation of the model.

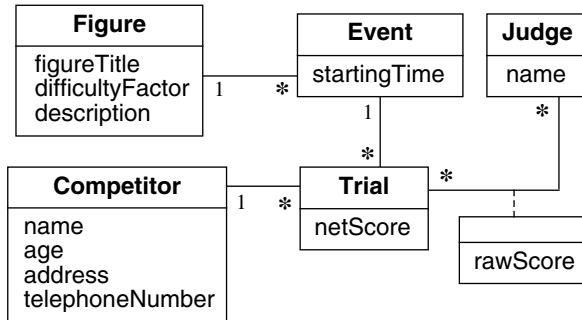


Figure E18.3 Partially completed class diagram for a scoring system

- 18.10 (6) Write C++ or Java code that corresponds to the pseudocode for Exercise 15.11. Make sure that your code respects encapsulation. [Instructor's note: You may want to give the students our answer to Exercise 15.11.]
- 18.11 (8) Write C++ or Java code for each of the OCL expressions in Section 3.5.3. Assume that there is a bidirectional implementation of associations. A specification, such as an OCL expression, need not respect encapsulation, but the code you write should.
- 18.12 (6) Using C++ or Java, implement all associations involving the classes *Box*, *Link*, *LineSegment*, or *Point* in Figure E18.4. Note that the editor allows links only between pairs of boxes.
- 18.13 (7) Implement the *cut* operation on the class *Box* in Figure E18.4 using C++ or Java. For simplicity, *cut* simply deletes the things that are cut and does not store them in a buffer. Propagate the operation from boxes to attached link objects. Update any associations that are involved. Be sure to recover any memory that is released by the operation (for C++ only). You may assume another method will update the display.
- 18.14 (7) Write a method using C++ or Java that will create a link between two boxes. Inputs to the method are two boxes and a list of points. The method should update associations and create object instances as needed. You may assume another method will update the display. Also write a method to destroy a link.
- 18.15 (8) Using C++ or Java, implement the following queries on Figure E18.4:
- Given a box, determine all other boxes that are directly linked to it.
 - Given a box, find all other boxes that are directly or indirectly linked to it.
 - Given a box and a link, determine if the link involves the box.
 - Given a box and a link, find the other box logically connected to the given box through the other end of the link.

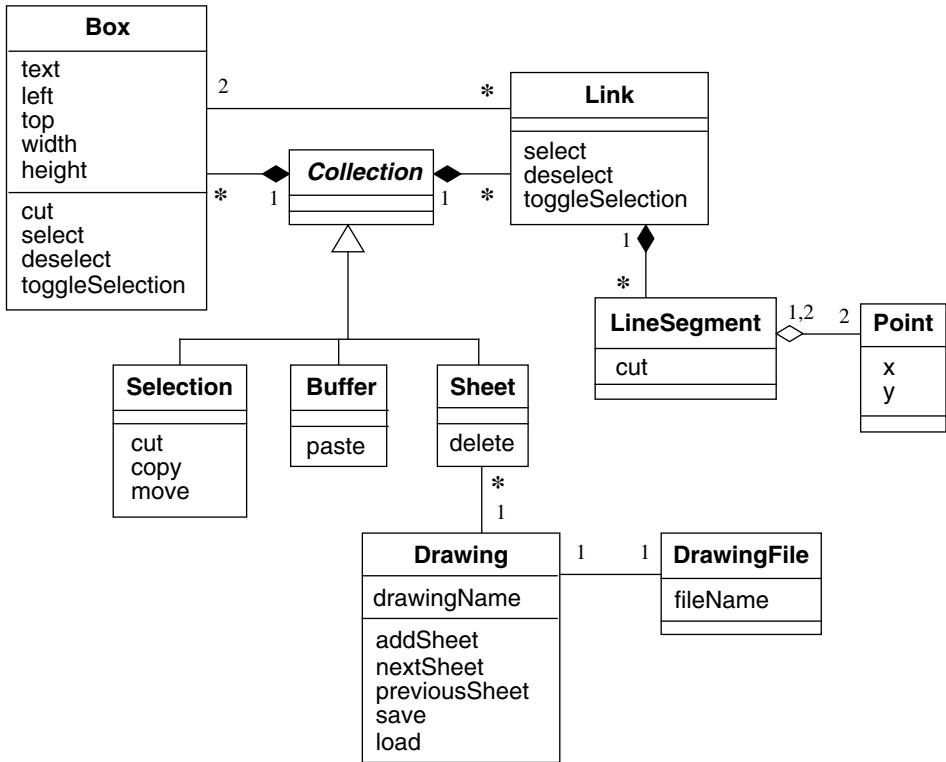


Figure E18.4 Class diagram for a diagram editor

- e. Given two boxes, determine all links between them.
- f. Given a selection and a sheet, determine which links connect a selected box to a deselected box.
- g. Given two boxes and a link, produce an ordered set of points. The first point is where the link connects to the first box, the last point is where the link connects to the second box, and intermediate points trace the link.

19

Databases

The OO paradigm is versatile and applies to databases as well as programming code. It might surprise you, but you can implement UML models not only with OO databases but also with relational databases. The resulting databases are efficient, coherent, and extensible.

You prepare a database by first performing the analysis steps described in Chapter 12 and constructing a domain model. The remaining methodology chapters in Part 2 apply mostly to design of programming code and to a lesser extent to databases. This chapter resumes where Chapter 12 ends. How can we map a model to database structures and tune the result for fast performance? How can we couple the resulting database to programming code? This chapter also includes a brief introduction to databases for new readers.

The chapter emphasizes relational databases because they dominate the marketplace. OO databases are practical only for niche applications and are discussed at the end of the chapter.

19.1 Introduction

19.1.1 Database Concepts

A *database* is a permanent, self-descriptive store of data that is contained in one or more files. Self-description is what sets a database apart from ordinary files. A database contains the data structure or *schema*—description of data— as well as the data.

A *database management system (DBMS)* is the software for managing access to a database. One major objective of OO technology is to promote software reuse; for data-intensive applications DBMSs can replace much application code. You are achieving reuse when you use generic DBMS code, rather than custom-written application code. There are additional reasons for using a DBMS.

- **Data protection.** DBMSs protect data from accidental loss due to hardware crashes, disk media failures, and application errors.
- **Efficiency.** DBMSs have efficient algorithms for managing large quantities of data.
- **Sharing between users.** Multiple users can access the database at the same time.
- **Sharing between applications.** Multiple application programs (presumably related) can read and write data to the same database. A database is a neutral medium that promotes communication among programs.
- **Data quality.** You can specify rules that data must satisfy. A DBMS can control the quality of its data over and above facilities that application programs may provide.
- **Data distribution.** You can partition data across various sites, organizations, and hardware platforms. The DBMS keeps the fragmented data consistent.
- **Security.** A DBMS can restrict reading and writing of data to authorized users.

19.1.2 Relational Database Concepts

A *relational database* has data that is perceived as tables. A *relational DBMS (RDBMS)* manages tables of data and associated structures that increase the functionality and performance of tables. RDBMSs have benefitted from a clear definition by an authoritative figure (EF Codd, the inventor of relational databases) and a standard access language (SQL [Melton-93]). All RDBMSs support a common core of SQL for defining tables, manipulating data in tables, and controlling access to tables. Variations exist for data types, performance tuning, programming access, and system data, though the standard is gradually subsuming these areas. An RDBMS has three major aspects: data structure, operators, and constraints.

- **Data structure.** A relational database appears as a collection of tables. Tables have a specific number of columns and an arbitrary number of rows with a value stored at each row-column intersection. Figure 19.1 shows two sample tables. The *Person* table has five columns and four rows; the *Company* table has three columns and three rows. The column names in boldface are primary keys (to be explained). Note that Jane Brown lacks an employer. *Null* means that an attribute value is unknown or not applicable for a given row.

RDBMSs use special techniques—such as indexing, hashing, and sorting—to speed access, because literal tables are much too slow for practical needs. These tuning techniques are transparent and not visible in the commands for reading and writing to tables. The RDBMS decides when tuning structures are helpful in processing a query, and if so, automatically uses them. The RDBMS automatically updates tuning structures whenever the corresponding tables are modified.

- **Operators.** SQL provides operators for manipulating tables. The SQL *select* statement reads the data in tables. The syntax looks something like (keywords are capitalized):

```
SELECT columnList
FROM tableList
WHERE predicateIsTrue
```

Person table

personID	lastName	firstName	address	employer
1	Smith	Jim	314 Olive St.	1001
5	Brown	Moe	722 Short St.	1002
999	Smith	Jim	1561 Main Dr.	1001
14	Brown	Jane	722 Short St.	NULL

Company table

companyID	companyName	address
1001	Ajax Widgets	33 Industrial Dr.
1002	AAA liquors	724 Short St.
1003	Win-more Sports	1877 Broadway

Figure 19.1 Sample tables. A relational DBMS presents data as tables.

Logically (the actual implementation is more efficient) the RDBMS combines the various tables into one temporary table. The column list specifies the columns to retain. The predicate specifies the rows to retain. The RDBMS returns the resulting data as the answer to the query. SQL has additional commands for inserting, deleting, and updating data in tables.

Interactive SQL commands are set-oriented; they operate on entire tables rather than individual rows or values. SQL provides a similar language for use with application programs that has a row-at-a-time interface.

- **Constraints.** An RDBMS can enforce many constraints (such as candidate, primary, and foreign keys) that are defined as part of the database structure. An RDBMS refuses to store data that violates constraints, returning an error to the user or requesting program.

A **candidate key** is a combination of columns that uniquely identifies each row in a table. The combination must be minimal and include only those columns that are needed for unique identification. No column in a candidate key can be null.

A **primary key** is a candidate key that is preferentially used to access the records in a table. A table can have at most one primary key; normally each table should have a primary key. The boldface in Figure 19.1 indicates the primary key of each table.

A **foreign key** is a reference to a candidate key (normally a reference to a primary key) and is the glue that binds tables. In Figure 19.1 *employer* is a foreign key in the *Person* table that refers to *companyID* in the *Company* table. It would not be permissible to change Moe Brown's *employer* to 1004, since the *Company* table does not define 1004. If the row for Ajax Widgets were deleted in the *Company* table, then both Jim Smith rows would have to be deleted or have their *employer* set to null. The foreign-to-primary-key binding forms a frequent navigation path between tables.

19.1.3 Normal Forms

A *normal form* is a guideline for relational database tables that increases data consistency. As tables satisfy higher levels of normal forms, they are less likely to store redundant or contradictory data. Developers can violate normal forms for good cause, such as to increase performance for a database that is read and seldom updated. Such a relaxation is called *denormalization*. The important issue with normal forms is to violate them deliberately and only when necessary.

Normal forms were first used in the 1970s and 1980s. At that time, developers built databases by creating a list of desired fields, which they then had to organize into meaningful groups before storing them in a database. That was the purpose of normal forms. Normal forms organize fields into groups according to dependencies between fields. Unfortunately, it is easy to overlook dependencies. If any are missed, the resulting database structure may be flawed.

UML models provide a better way to prepare databases. Instead of focusing on the fine granularity of fields, developers think in terms of groups of fields—that is, classes. UML models do not diminish the validity of normal forms—normal forms apply regardless of the development approach.

However, UML modeling does eliminate the need to check normal forms. If developers build a sound model, it will intrinsically satisfy normal forms. The converse also holds—a poor model is unlikely to satisfy normal forms. Furthermore, if developers cannot build a sound model, they will probably be unable to find all the dependencies that are required for checking normal forms. It is less difficult to build models than to find all the dependencies.

The bottom line is that developers can still check normal forms if they want to after modeling, but such a check is unnecessary.

19.1.4 Choosing a DBMS Product

In order to build an application, you must choose a specific DBMS product. Because the core features have been set by the SQL standard, you should choose an RDBMS vendor according to pragmatic concerns.

- **Market share.** Oracle, IBM, and Microsoft are the major market players. The staying power of other vendors is less clear. You may also want to consider an open-source DBMS such as MySQL or PostgreSQL.
- **Vendor and third-party support.** DBMSs are a big commitment for an organization and require ongoing help.
- **Other applications.** You reduce administrative and license costs if you use the same vendor or a small number of vendors for your applications.

With each new product release the major vendors tend to jump ahead of the competition's features and performance benchmarks, only to be surpassed themselves when a competitor has its own new release. Over the long term there tends to be little difference in features and performance, so you should not dwell on them when choosing a product.

19.2 Abbreviated ATM Model

Figure 19.2 shows the portion of the ATM model that this chapter will use as an example. We added *CheckingAccount* and *SavingsAccount* so that we can discuss generalization. We also added the *Address* class to assist our explanation.

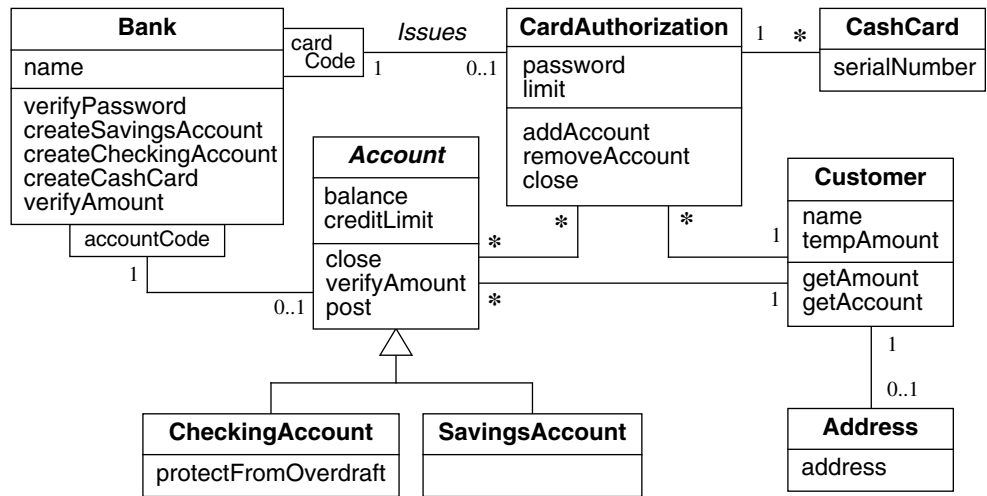


Figure 19.2 Abbreviated ATM implementation class model used in this chapter

19.3 Implementing Structure—Basic

You can readily translate the class model into SQL code. Many tools will do this, but you should still know the rules. Then you understand what the tools are doing and can spot check their results. RDBMSs provide good support for classes and associations but lack support for inheritance, so you must use a workaround. You should perform the following initial tasks.

- Implement classes. [19.3.1]
- Implement associations. [19.3.2]
- Implement generalizations. [19.3.3]
- Implement identity [19.3.4]

19.3.1 Classes

Normally you should map each class to a table and each attribute to a column (Figure 19.3). You can add columns for an object identifier and associations (to be explained). The boldface indicates the primary key. Keywords are in uppercase. Note that operations do not affect table structure. We have chosen data types and *not-null* constraints that seem appropriate for the

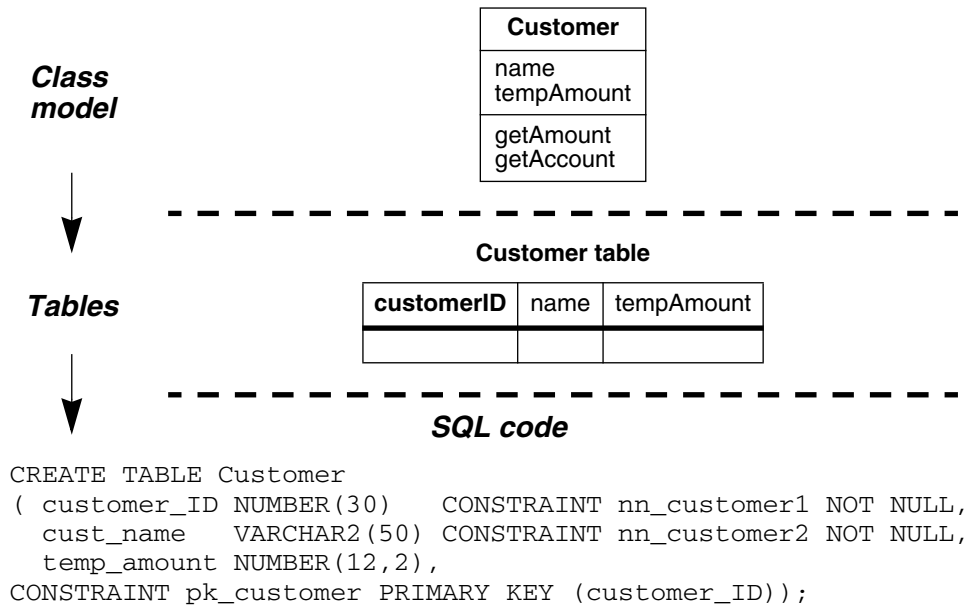


Figure 19.3 Implementing classes. Make each class a table.

problem. All our examples use Oracle syntax. The *nn_customer1* and *nn_customer2* are names of the not-null constraints. The *pk_customer* is the name of the primary key constraint.

19.3.2 Associations

The implementation rules for associations depend on the multiplicity.

- **Many-to-many associations.** Implement the association with a table and make the association's primary key the combination of the classes' primary keys (Figure 19.4). If the association has attributes, they become additional columns.
- **One-to-many associations.** Each one becomes a foreign key buried in the table for the "many" class (Figure 19.5). If there had been a name on the "one" end of the association, we would have used it as the foreign key name. We presume that *serialNumber* is unique for *CashCard*.
- **One-to-one associations.** These seldom occur. You can handle them by burying a foreign key in either class table (Figure 19.6).
- **N-ary associations.** They also seldom occur. You can treat them like many-to-many associations and create a table for the association. Typically, the primary key of the n-ary association combines the primary keys of the related tables.
- **Association classes.** An *association class* is an association that is also a class. It is easier to establish the proper dependencies if you make each association class into a table, regardless of the multiplicity.

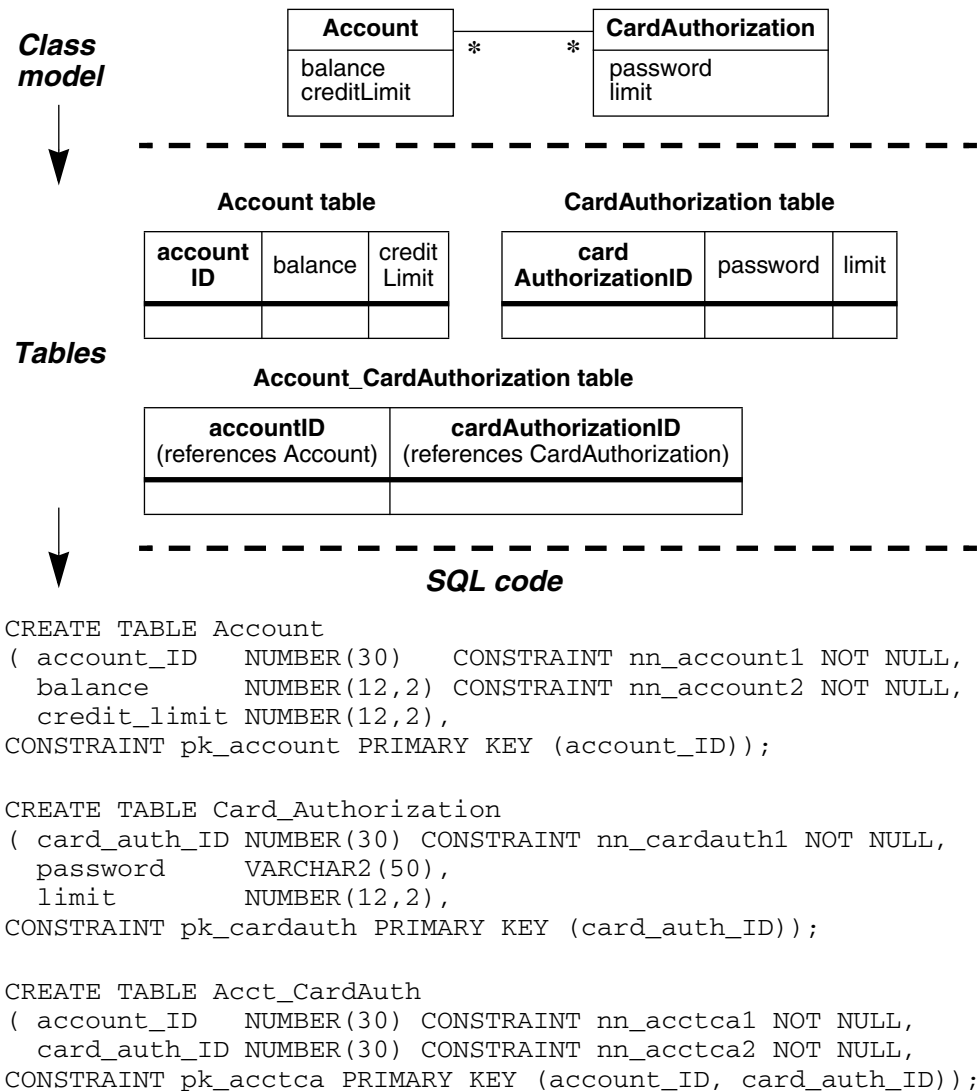
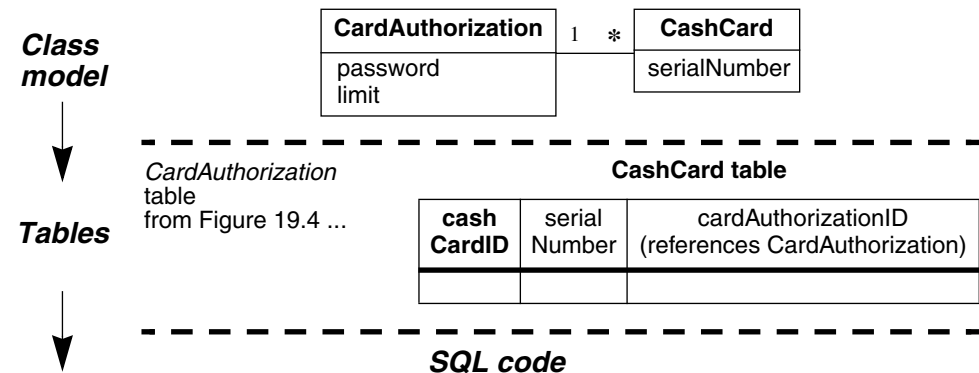


Figure 19.4 Implementing many-to-many associations. Make each one a table.

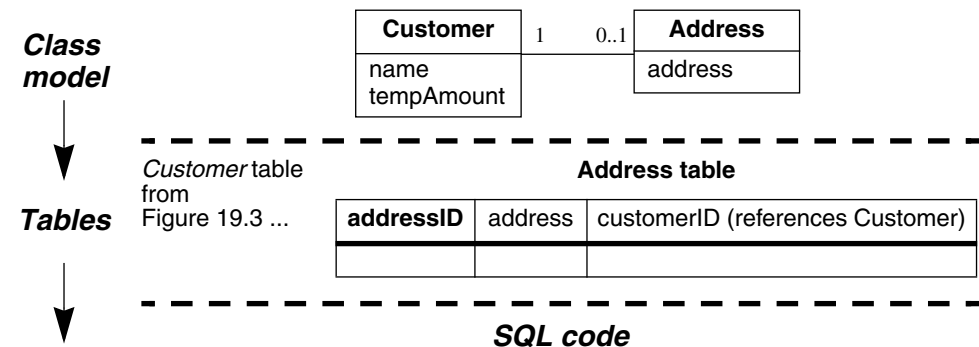
- **Qualified associations.** Qualified associations follow the same rules as the underlying association without the qualifier. Thus we treat Figure 19.7 like a one-to-many association (a bank has many accounts). The notation *ckn* (*n* is a number) denotes a candidate key. Many qualified associations have a candidate key involving the qualifier.
- **Aggregation, composition.** Aggregation and composition follow the same implementation rules as association.



Card_Authorization code from Figure 19.4 ...

```
CREATE TABLE Cash_Card
( cash_card_ID NUMBER(30) CONSTRAINT nn_cashcard1 NOT NULL,
  serial_num VARCHAR2(50) CONSTRAINT nn_cashcard2 NOT NULL,
  card_auth_ID NUMBER(30) CONSTRAINT nn_cashcard3 NOT NULL,
  CONSTRAINT pk_cashcard PRIMARY KEY (cash_card_ID),
  CONSTRAINT uq_cashcard1 UNIQUE (serial_num));
```

Figure 19.5 Implementing one-to-many associations. Bury each one as a foreign key in the “many” class table.



Customer code from Figure 19.3 ...

```
CREATE TABLE Address
( address_ID NUMBER(30) CONSTRAINT nn_address1 NOT NULL,
  address VARCHAR2(200) CONSTRAINT nn_address2 NOT NULL,
  customer_ID NUMBER(30) CONSTRAINT nn_address3 NOT NULL,
  CONSTRAINT pk_address PRIMARY KEY (address_ID));
```

Figure 19.6 Implementing one-to-one associations. Bury a foreign key in either class table.

should make continued project funding contingent on holding a review. Keep in mind that these are *technical* reviews, not *management* reviews—the purpose of the reviews is to deepen technology, not to inform management. [Boehm-01] notes that peer reviews catch 60% of software defects, so technical reviews are clearly important. We advise that the size of technical reviews be kept small (less than ten persons) and confined to developers who are actively interested in the project.

22.4 Modeling Sessions

Chapter 12 explains domain analysis—the building of real-world models to clarify application requirements. Once you become experienced at modeling, you can consider different ways of engaging users and obtaining their input. We will characterize three alternatives—back-room, round-robin, and live modeling—and discuss their trade-offs.

22.4.1 Back-Room Modeling

The most popular way to build a model is to talk to business experts, record their comments (such as with a requirements statement or use cases), and then go off-line and model. Many analysts prefer this *back-room modeling* approach, because they can focus on what the user is saying and wrestle with the model later when they are alone. Over a series of meetings, users answer questions and volunteer information that they think may be helpful. After each meeting, the analyst incorporates the users' comments, and the model gradually improves. Typically, the model stays in the background and users do not see it.

It is better to meet with several users at once rather than have one-on-one meetings. A multiuser meeting has a better chemistry, because users stimulate each other's memory. Also there is a risk of intimidation in a one-on-one session, which is less likely with multiple users. Most analysts prefer to meet with a group of users who share an interest. For example, an analyst might meet separately with salespersons and engineers.

Back-room modeling has the following trade-offs.

- **Advantages.** It requires the least skill and is appropriate for analysts who are tentative with modeling.
- **Disadvantages.** The painstaking cycle of interaction with users is cumbersome for skilled modelers. The slow interaction can also be troublesome for users, because multiple interviews are required. Analysts must carefully transcribe information, or it will be forgotten.

22.4.2 Round-Robin Modeling

Round-robin modeling is more complex than back-room modeling, but more efficient at gathering requirements. The analyst still meets with small groups of users, segmented by interest or functional area, but in round-robin modeling, the users see the model. As users express requirements, the analyst traverses the model and tries to resolve them. An analyst can resolve simple issues during a meeting and complex issues afterward.

We call this approach **round-robin modeling**, because an analyst shows the model to each group in a series of meetings until all concerns are addressed. Several iterations are required, because one group might surface an issue that an analyst needs to confirm with a previous group. Back-room modeling also parades from group to group, but users don't see the model.

We initiate round-robin modeling with a seed model that is based on existing business documentation. We don't like to start with a blank sheet of paper, because it wastes time and tries the patience of users. In contrast, a seed model stimulates discussion. Users see the analyst as well prepared and can focus on deeper issues.

In the meetings, we tell the users that they are the business experts and that we need their help in capturing requirements; we are the computer experts, and they should let us handle the details. Generally, users heave a sigh of relief. We don't dwell on formalisms and explain notation as we go. Participants don't have a problem, because we continually explain the model.

Round-robin modeling has the following trade-offs.

- **Advantages.** It requires fewer meetings than back-room modeling. Because the model is prominent, an analyst can resolve some issues during meetings. In contrast, with back-room modeling, the analyst just takes notes and may overlook needed details.
- **Disadvantages.** It still requires several iterations, and it is inefficient to shuttle ideas across the user groups. If there is contention, it can be difficult to reach agreement. The analyst is in the uncomfortable position of being an intermediary among conflicting user groups. Back-room modeling also shares this flaw. Some users do not understand models or may fear them, so the analyst must take care to allay their concerns.

22.4.3 Live Modeling

Live modeling is appropriate for expert modelers. We arrange a meeting of 10–20 persons with a range of interests—developers, managers, and various kinds of business experts. During the meeting, we build a model on the fly, listening to suggestions, volunteering comments, resolving names, and agreeing on scope. Usually, we can keep pace with the dialogue; we pause a moment when we get overloaded. A projector displays the model, which we draw with a modeling tool. A typical session lasts about two hours, and three sessions can usually elicit 80 percent of the structure for a model with 50 classes. Large and complex models take additional sessions. In between sessions, the modeler cleans up diagrams, documents the model, and resolves open issues.

The process is stimulated by the size and variety of the group; reluctant participants see others react and want to get involved to air their point of view. Comments from one person tend to trigger comments from another. We have been especially successful with skeptics.

It is acceptable to start live modeling with a clean sheet of paper, but the analyst should prepare and learn about the application in advance. The ideas will come quickly, and the analyst must be ready. Sometimes we prepare a seed model if we have prior information. Normally, we request that our client prepare a requirements statement to stimulate discussion if there are any lulls.

We are active facilitators, not just passive recorders; we ask questions and probe the attendees when answers seem unsatisfactory. We make suggestions on the basis of our experience. Ultimately, business experts make the final decisions. Occasionally, we encounter a deep modeling issue that we defer until the next meeting.

Often there are animated discussions over names. These can be helpful. Good names avoid misunderstandings. Also the discussions stimulate related information. We press business experts to devise good names—names that are brief, crisp, and not subject to confounding interpretations.

Live modeling has the following trade-offs.

- **Advantages.** This is clearly the best way to obtain user input for proficient modelers. We practice live modeling all the time, and clients are delighted by the rapid progress. The participants have different areas of knowledge and different perceptions; by working together in the same meeting, they can reconcile their views.

A major side benefit is that the meetings induce the participants to talk to each other. Persons from different backgrounds who usually don't have the time or inclination are brought together and converse.

- **Disadvantages.** An analyst has to be highly confident of modeling, able to run a meeting, and adept with a modeling tool. Few developers have this combination of skills. Live modeling is good at eliciting structure—classes and relationships. It is less effective at finding attributes, because it is difficult to coordinate a large group for fine detail. Other input sources can provide attributes.

Live modeling is not suitable for difficult applications, such as applications with intense metamodeling. For these situations we recommend back-room modeling.

Table 22.1 summarizes the trade-offs for the three kinds of modeling sessions.

	Back-room modeling	Round-robin modeling	Live modeling
Explanation	Record user comments and build model offline	Show model to user groups, but still build it offline	Build model during a meeting with all the users
Required skill level	Low	Medium	Very high
Productivity (for a model with 50 classes)	Low (about 15 meetings, each 2 hours long)	Medium (about 12 meetings each 2 hours long)	Very high (about 3 meetings, each 2 hours long)
Net recommendation	Best for a novice modeler	Best for a modeler with some application experience	Best for a very experienced modeler

Table 22.1 Trade-offs for different approaches to modeling sessions. Consider different approaches to modeling and their trade-offs.

22.5 Organizing Personnel

As Figure 22.1 shows, a large organization can most effectively service demand by placing a few experts in a technology-oriented group that supports groups of developers organized by business area. Table 22.2 clarifies the respective roles of the technology and application groups.

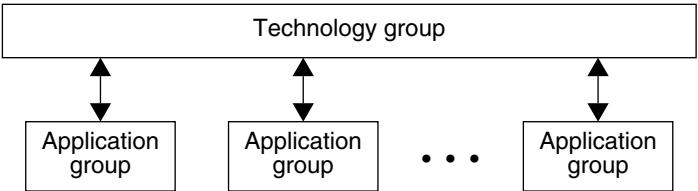


Figure 22.1 Corporate structure. A technology group can provide expertise for groups of developers organized by business area.

	Technology group	Application group
Perspective	The entire organization	A business area
Appropriate tasks	Promulgate standards and computing techniques; maintain enterprise models; support the application groups	Build applications; evaluate products for potential purchase
Required modeling skill	Expert	Fluent
Number of groups	One per organization	Many per organization
Size of group	Small to limit overhead	As many as needed to serve business area

Table 22.2 Technology vs. application groups. A large organization needs both kinds of groups.

The technology group takes the perspective of the entire organization. It promulgates standards and computing techniques and supports the application groups. You should place the best modelers and experts in the technology group, so that their skills are available to everyone. The technology group should not build applications; this is the purpose of the application groups. Rather, the technology group should be the custodian of advanced skills. Keep this group small to limit overhead.

Application groups have a different role. Their purpose is to learn about the business and transfer knowledge across related applications. Application group members should work closely with their business counterparts. Developers should be fluent with modeling, even though the best modelers belong to the technology group.

Some firms use a different organization; they place all modelers in a technology group and loan them out to perform modeling for application developers. We advise against this arrangement. Modeling is such a stimulus to insight and dialog that it should be dispersed across an entire computing organization. (For that matter, it is also beneficial if some business and marketing staff learn about models.) Modeling is the lingua franca for software development, and application developers should build models for themselves.

22.6 Learning Techniques

There are various actions that a person can take to learn about modeling. Some of these actions individuals can take on their own. Other actions require organizational support. This advice pertains to students in universities as well as practitioners in industry.

- **Training and mentoring.** Universities and commercial training houses both offer courses that explain modeling concepts and how to apply them. It is best to receive training shortly before (ideally a few weeks ahead of) its use on an actual project. Try not to receive training far in advance, or you will forget too much.

Reinforce training with mentoring. Developers will need active help as they seek to apply the training material. Novice modelers will lack confidence that they are modeling correctly and need experience on which to draw. It will not suffice to bring in outside resources to service a project. There must be a transition of knowledge from the outside resources to in-house developers.

- **Teaming.** Application models should be constructed by small teams that initially consist of developers, business experts, and external consultants. After several applications, an organization should no longer need the external consultants, and the best in-house modelers can provide expertise. The purpose of teaming is to disseminate knowledge within a firm about both computing technology and the business.
- **Seminars.** Periodic seminars provide cost-effective education. A firm should encourage developers to present technical seminars. Seminars get developers talking and exchanging ideas. They learn about the various projects and can leverage related efforts. Seminars provide peer support for dealing with the difficulties of modeling.
- **Continual learning.** Developers should strive to find new ideas and adopt the best practices of the larger software community. Periodic attendance at technical conferences and professional meetings is helpful. Books and magazines can provide useful ideas.
- **Technical reviews.** Formal technical reviews promote conversation and become a learning experience for both the presenters and the reviewers. The reviews provide a forum for technical staff members to help one another. (See Section 22.3.)

22.7 Teaching Techniques

There are different ways to teach modeling that have various trade-offs.

- **Induction.** A person learns to model only by *doing* it, not by *talking* about it. Over the years we have tried a number of techniques and have found induction to be the quickest way to start modeling. When we run modeling sessions (see Section 22.4), we forego the preamble of a modeling tutorial (even though many attendees have never experienced a model before) and jump right into modeling the application problem. After several sessions, attendees have a good start on their application and have at the same time started to learn about modeling.
- **Practice.** Students should receive extensive hands-on practice and exercises. They should solve problems from disparate domains, with different kinds of input, different levels of abstraction, and varying difficulty.
- **Correction.** Students often make modeling mistakes, and it is difficult to anticipate all the possibilities. Part of the process of learning is for students to make mistakes and receive correction. They can learn both from their mistakes and the mistakes of their peers—through joint work with other students and class presentations. An academic setting can coerce the presentations, but in industrial courses we normally make presentations voluntary.
- **Implementation.** It is important that students understand that models can be readily implemented. They must realize that any model they can express can be implemented in a robust, predictable, and efficient manner. However, once they grasp that point, they need to set aside implementation concerns and think directly in terms of models.
- **Apprenticeship.** Another way of teaching is individually rather than en masse with a class. A new modeler can learn by forging a close working relationship with a skilled modeler. This form of teaching is best for someone who has already started to learn.
- **Patterns.** When we model, we always think in terms of patterns (Chapter 14). We encounter application situations, recognize their abstract mathematical underpinnings, and then jump to a pattern. A pattern provides a tried and true solution to a standard problem that has been studied by experts and is known to work well. There are many kinds of patterns: analysis, architecture, design, and implementation. One problem with patterns is recognizing when to apply them. Also even though they are important, patterns cover only part of a model.

22.8 Tools

Any serious software development effort requires tools—tools for modeling, configuration management, code generation, simulation, compiling, debugging, performance profiling, and so forth. We do not attempt to cover tools completely here, because there are so many kinds and such a variety of products. Instead, we focus on tools directly relevant to modeling and mention some prominent vendors.

22.8.1 Modeling Tools

Large applications (50 classes or more) require a heavyweight modeling tool. The minor benefit of a tool is that it increases productivity. The major benefit is that it can deepen think-

ing. Tools help experts build models more quickly and organize information about classes in a form that is easy to search. Tools help novices observe syntax and avoid common mistakes. IBM Rational Rose XDE, Rhapsody, Magic Draw, Together/J, and Enterprise Architect are heavyweight tools for the UML notation.

Small applications are less demanding and do not absolutely require a heavyweight modeling tool. Nevertheless, it is still a good idea to use a tool for small applications—modeling is essential to clear thinking, and the use of a tool eases model construction. Given the availability of inexpensive tools, as well as site licenses, there is seldom a good reason for not using a modeling tool.

22.8.2 Configuration Management Tools

Serious applications involve a number of files—files for programs (source code, compiled code, and executable code), documentation (for users, administrators, and maintainers), and data (configuration data, metadata, and test data). In practice it is difficult to coordinate all these files, and that is the purpose of configuration management tools. The tools improve developer efficiency and reduce the risk of losing useful work.

[Pressman-97] lists five major tasks that constitute configuration management.

- **Identification.** A configuration management tool must provide a mechanism for identifying each configuration file and relating it to other files.
- **Version control.** An organization must be able to track copies of a file as it evolves over time. Sometimes it is necessary to revert to old files (such as when trying to find the cause of a bug).
- **Change control.** An organization must determine who can approve changes as well as synchronize the work of collaborating developers. Check-in and check-out of files is a popular protocol.
- **Auditing.** A configuration management tool keeps a log of access activity. Users can access the log and determine the precise revisions to particular files, who made the changes, and the date of changes.
- **Status accounting.** There must be a means for reporting changes to others.

You can get by without configuration management for models if a single person is doing modeling and that person is disciplined about backups. However, an ad-hoc approach becomes increasingly difficult as the number of models and modelers increases. When you must manage many models and coordinate multiple persons, you should use configuration management software.

Prominent configuration management tools include IBM Rational's ClearCase, Merant PVCS, Microsoft's Visual SourceSafe, and the public-domain tool CVS.

22.8.3 Code Generators

Many modeling tools can generate application code. The typical modeling tool can generate data declarations for a program and a database, if you are using one. Some tools can generate

algorithmic code, but that is more difficult. For example, a tool might generate programming code for a state diagram.

Regardless of the tools used, developers should be careful with generated code and spot-check it for correctness and efficiency. Some tools generate bad code with flaws that are subtle and difficult to catch. Also, to our surprise, some tools have had gross errors in their outputs. If your developers pay attention to tool output, they will better understand what the tool is doing.

22.8.4 Simulation Tools

Tools can also be used to predict the behavior and performance of a finished application. Some modeling tools can simulate the performance and behavior of the finished software in advance of building it. For example, i-Logix's Statemate can simulate state diagrams.

22.8.5 Repository

Repositories are also important to application development, because they store metadata that lets the various tools communicate. A repository sits at the center of tool usage. Because they involve metadata, repositories are difficult to deploy, but effective use of a repository can leverage your usage of the individual tools. Allen System Group, Computer Associates, IBM, and Microsoft have repository products.

22.9 Estimating Modeling Effort

Any software development effort is an economic proposition. Business people estimate the cost of building the software along with the resulting revenues and cost savings. Modeling is generally a small part (much less than 10%) of the overall application effort. The following factors affect modeling effort.

- **Application complexity.** Tangible applications are simpler; highly abstract applications take longer. For example, it is easier to build software for handling customer calls than to build a system for all kinds of customer interaction.
 - **Proficiency.** A skilled modeler can work an order of magnitude faster than an inexperienced one. In addition, a skilled modeler is more likely to produce a quality model with thoughtful abstractions.
 - **Tools.** It helps if the developer has access to a powerful modeling tool and is skilled with it.
 - **Model size.** The time to construct a model is not linear with its size. Modeling time is roughly proportional to the number of classes to the one-and-one-half power. Thus, construction for a model with 500 classes takes about 30 times longer than for one with 50.
 - **Reviews.** Thorough review reduces the number of iterations needed for a model.
- Given all these factors, most models require from two weeks to six months of effort.

22.10 Chapter Summary

This chapter has covered several topics to help an organization assimilate the technology of modeling.

In practice, there are several categories of models—application, meta, enterprise, and product assessment—with different motivations, characteristics, and content. Many developers overlook how to properly use modeling for the various categories. For example, an enterprise model cannot contain all the details of the covered applications, or it will become unwieldy. Models can also be used to guide purchase evaluations.

Models have many benefits, but like any technology they also entail risks. We identified some major risks and noted actions that an organization can take to mitigate them.

There are different ways that you can engage users in the process of modeling. We listed several kinds of interactions—back room, round robin, and live—along with their trade-offs.

There are various actions that a person can take to learn about modeling, including training, mentoring, teaming, seminars, continual learning, and technical reviews.

Similarly, there are different ways to teach modeling. One of the most successful is induction—a person learns to model only by doing it, not by talking about it. Teachers should give students extensive practice with models and give them the opportunity to have their mistakes corrected. Students must understand that models can be readily implemented but they should directly think in terms of models. Advanced modelers will recognize patterns and apply them.

Any serious software development effort requires tools. We listed tools relevant to modeling and gave some criteria for choosing tools. We also gave some guidelines for estimating modeling effort.

abstraction	estimation	peer support
application model	induction	product assessment
apprenticeship	live modeling	repository
back-room modeling	mentoring	round-robin modeling
code generation	metamodel	seminar
configuration management	model review	simulation tools
continuing education	modeling pitfall	teaming
documentation	modeling tool	technical review
enterprise model	pattern	training

Figure 22.2 Key concepts for Chapter 22

Bibliographic Notes

This book says little about management issues, such as project planning, project estimation, costing, metrics, personnel assignment, and team dynamics. These are all important topics, but other books cover them, such as [Pressman-97] and [Blaha-01].

In [Colwell-03] Bob Colwell recalls some of his first-hand experiences with design reviews and stresses their importance to high-quality work.

[Berndtsson-04] describes his experiences with teaching three configurations of OO analysis and design courses over a nine-year period. He presents data for the following conclusions.

- **Programming success is not an indicator of modeling success.** 65% of students who did well in an OO programming course performed poorly in an OO modeling course.
- **Modeling success is an indicator of programming success.** 84% of students who did well in an OO modeling course also did well in an OO programming course.
- **Abstraction is the key difficulty with modeling.** There is a strong correlation in grades between an OO modeling course (high abstraction) and a distributed systems course (also high abstraction).

[Box-00] concludes that OO technology involves more abstraction than the older technique of structured technology. The authors consider abstraction to be the difficult learning action.

References

- [Berndtsson-04] Mikael Berndtsson. Teaching object-oriented modeling and design. *Draft paper*, 2004.
- [Blaha-01] Michael R. Blaha. *A Manager's Guide to Database Technology: Building and Purchasing Better Applications*, Upper Saddle River, NJ: Prentice Hall, 2001.
- [Blaha-04] Michael Blaha. A copper bullet for software quality improvement. *IEEE Computer*, February 2004, 21–25.
- [Boehm-01] Barry Boehm and Victor R. Basili. Software defect reduction top 10 list. *IEEE Computer*, January 2001, 135–137.
- [Box-00] Roger Box and Michael Whitelaw. Experiences when migrating from structured analysis to object-oriented modeling. *Fourth Australasian Computing Education Conference*, Melbourne, Australia, December 4–6, 2000, 12–18.
- [Colwell-03] Bob Colwell. Design reviews. *IEEE Computer*, October 2003, 8–10.
- [Pressman-97] Roger S. Pressman. *Software Engineering: A Practitioner's Approach, Fourth Edition*. New York: McGraw-Hill, 1997.

Legacy Systems

Most development does not involve new applications but rather evolves existing ones. Rarely can you build an application completely from scratch. Even if you do get to build a new application, you will often need to gather information from existing applications and integrate with them. You can salvage requirements, ideas, data, and code.

It is difficult to modify an application if you don't understand its design. If an application was previously designed using OO models and they are accurate, you can use the models to understand and evolve the application. If the models are lacking or have been lost, you should start by building a model of the existing design.

23.1 Reverse Engineering

Reverse engineering is the process of examining implementation artifacts and inferring the underlying logical intent. Reverse engineering has its origins in the analysis of hardware—where the practice of deciphering designs from finished products is commonplace [Rekoff-85]. Models facilitate reverse engineering, because they can express both abstract concepts and implementation decisions.

When building new applications, the purpose of reverse engineering is to salvage useful information. It is not intended to perpetuate past flaws. The reverse engineer must determine what to preserve and what to discard. You should regard reverse-engineered models as merely one source of requirements for a new application.

Reverse engineering requires judgment—interpretative decisions by the developer—and cannot be fully automated. Tools can assist with the sheer volume of code to be reverse engineered and the rote. However, it is difficult for the current tools to accept human decisions. Many modeling tools can generate an initial model, but this model is little more than a visual representation of the program structure. The reverse engineer must overcome at least two problems with the program code: retrieving obscure or lost information and uncovering implicit behavior.

23.1.1 Reverse Engineering vs. Forward Engineering

As Table 23.1 shows, reverse engineering is the inverse to normal development (forward engineering); you start with the actual application and work backward to deduce the requirements that spawned the software.

Forward engineering	Reverse engineering
Given requirements, develop an application.	Given an application, deduce tentative requirements.
More certain. The developer has requirements and must deliver an application that implements them.	Less certain. An implementation can yield different requirements, depending on the reverse engineer's interpretation.
Prescriptive. Developers are told how to work.	Adaptive. The reverse engineer must find out what the developer actually did.
More mature. Skilled staff readily available.	Less mature. Skilled staff sparse.
Time consuming (months to years of work).	Can be performed 10 to 100 times faster than forward engineering (days to weeks of work).
The model must be correct and complete or the application will fail.	The model can be imperfect. Salvaging partial information is still useful.

Table 23.1 Forward engineering vs. reverse engineering. Reverse engineering is the opposite of forward engineering and requires a different mindset.

23.1.2 Inputs to Reverse Engineering

When performing reverse engineering, you must be resourceful and consider all inputs. The available information varies widely across problems.

- **Programming code.** The programming source code can be a rich information source. Tools can help you understand the flow of control and the data structure. Comments and suggestive names of variables, functions, and methods can deepen your understanding.
- **Database structure.** If the application has a database, you can also learn from it. The database specifies the data structure and many constraints—precisely and explicitly.
- **Data.** If data are available, you can discover much of the data structure. A thorough application program or disciplined users may yield data of better quality than the data structure enforces. For large systems, you may have to sample the data to reach tentative conclusions, and then explore further for verification. Examination cannot prove many propositions, but the more data you encounter, the more likely will be the conclusion.
- **Forms and reports.** Suggestive titles and layouts can clarify data structure and processing logic. Form and report definitions are especially helpful if their binding to variables

is available. An empirical approach is to enter known, unusual values to establish the binding between forms and the underlying variables.

- **Documentation.** Problems vary in their quality, quantity, and kind of documentation. Documentation provides context for reverse engineering. User manuals are especially helpful. Data dictionaries—lists of important entities and their definitions—may be available. Be careful with all documentation, because it can become stale and inconsistent with application code.
- **Application understanding.** If you understand an application well, you can make better inferences. Application experts may be available to answer questions and explain rationale. You may be able to leverage models from related applications.
- **Test cases.** Test cases are intended to exercise the normal flow of control and unusual situations. Sometimes they provide useful clues.

23.1.3 Outputs from Reverse Engineering

Reverse engineering has several useful outputs.

- **Models.** The model conveys the software's scope and intent. It provides a basis for understanding the original software and building any successor software.
- **Mappings.** You can tie model attributes to variables. Less precisely, you can bind programming code to state and interaction models.
- **Logs.** Reverse engineers should record their observations and pending questions. A log documents decisions and rationale.

23.2 Building the Class Model

Begin by constructing a class model of the application so that you can understand the classes and relationships. We suggest building the class model using three distinct phases: implementation recovery, design recovery, and analysis recovery.

23.2.1 Implementation Recovery

First quickly learn about the application and create an initial class model. If the program is written in an OO language, you can recover classes and generalizations directly. Otherwise you must study the data structures and operations and manually determine classes. The system may lack a proper design, so the result may not be pleasing. Try to avoid making any inferences other than determining classes at this point. It is helpful to have an initial model focused on the implementation.

23.2.2 Design Recovery

Next probe the application to recover associations. The typical implementation of an association is as a single pointer attribute. The multiplicity in the forward direction is usually clear.

In contrast, the multiplicity in the reverse direction is typically not declared, and you must determine it from general knowledge or examination of the code.

Many implementations use a collection of pointers to implement an association with “many” multiplicity. Then the initial generated model points to the collection class, rather than the class of the elements. You should move the association to the class of the elements and adjust the multiplicity accordingly. Collection classes are mechanisms and should not appear in most analysis or design models. You can mark them on associations as recommended implementation for a “many” direction.

Sometimes pointers will implement both association directions. In those cases, you must identify the matching pointers and consolidate them. You should suspect any two classes that have pointer attributes to each other. If you reverse engineer the initial model with a tool, it will have one association for each pointer. You should remove one association and move its information to the reverse direction of the other.

Multiplicity typically has a lower limit of 0 or 1. The lower limit is ‘0’ if the target object is initialized somewhere in the source code, but it is uncertain when initialization will happen. The lower limit is ‘1’ if the target object is initialized at object creation time, such as in the constructor or the class initialization block.

23.2.3 Analysis Recovery

Finally, you interpret the model, refine it, and make it more abstract. Remove any remaining design artifacts and eliminate any errors. Be sure to get rid of all redundant information or mark it as such. It’s also a good time to reconsider the model. Is it readable and coherent? Reconcile the reverse engineering results with models of other applications and documentation. Show your model to application experts and incorporate their advice.

If your source code is not object oriented, you will have to infer generalizations by recognizing similarities and differences in structure and behavior. Similarly, you will have to use your application understanding and careful study of the code to determine aggregations and compositions. For example, objects with coincident lifetimes suggest composition. You will also need to understand the application and code to determine qualifiers and association classes.

You can add packages to organize the classes, associations, and generalizations. You can combine the classes in several code files into one package or split a large code file across multiple packages.

23.3 Building the Interaction Model

The purpose of each method is usually clear enough, but the way that objects interact to carry out the purposes of the system is often hard to understand from the code. The problem is that code is inherently reductionist: it describes how each piece works by itself. But the meaning of the system as a whole is holistic: the emergent interactions among objects give it meaning. The interaction model can give you a broad understanding.

You can add methods to the class model by using slicing. A *slice* is a subset of a program that preserves a specified projection of its behavior [Weiser-84]. You can perform slicing by marking some initial code to retain. Then, recursively, mark all statements used by the retained code. The accumulated code lets you project an excerpt of behavior from the original program. Thus slicing provides a means for converting procedural code into an OO representation that is centered about objects.

Programmers naturally think in terms of slicing as [Weiser-82] notes. The power of slices comes from four facts: 1) they can be found automatically, 2) slices are generally smaller than the program from which they originated, 3) they execute independently of one another, and 4) each reproduces exactly a projection of the original program's behavior [Weiser-84].

You can use an activity diagram to represent an extracted method so that you can understand the sequence of processing and the flow of data to various objects. You can then construct sequence diagrams from the activity diagrams for simplification.

23.4 Building the State Model

If you are studying a user interface, a state model can be quite helpful. Otherwise, state models are not prominent in most other kinds of application code.

If you do need to construct a state model, you can proceed as follows. As an input, you have sequence diagrams from building the interaction model. You need to fold the various sequence diagrams for a class together, by sequencing events and adding conditionals and loops as Chapter 13 describes.

You can augment the information in the sequence diagrams by studying the code and doing dynamic testing. It is helpful to find all the possible states for each class that has a state model. Initiation and termination correspond to construction and destruction of objects.

23.5 Reverse Engineering Tips

As you perform reverse engineering and build class, interaction, and state models, it will help if you keep in mind the following tips.

- **Distinguish suppositions from facts.** Reverse engineering yields hypotheses. You must thoroughly understand the application before reaching firm conclusions. As reverse engineering proceeds, you may need to revisit some of your earlier decisions and change them.
- **Use a flexible process.** We adjust the reverse engineering process to fit the problem. Problem styles and the available inputs vary widely. You must use your wits for reverse engineering. You are solving a large puzzle.
- **Expect multiple interpretations.** There is no single answer as in forward engineering. Alternative interpretations can yield different models. The more information that is available, the less judgments should vary among reverse engineers.

- **Don't be discouraged by approximate results.** It is worth a modest amount of time to extract 80 percent of an application's meaning. You can use the typical forward engineering techniques (such as interviewing knowledgeable users) to obtain the remaining 20 percent. Many people find this lack of perfection uncomfortable, because it is a paradigm shift from forward engineering.
- **Expect odd constructs.** Developers, even the experts, occasionally use uncommon constructs. In some cases, you won't be able to produce a complete, accurate model, because that model never existed.
- **Watch for a consistent style.** Software is typically designed using a consistent strategy, including consistent violations of good design practice. Usually, you can look at an excerpt of the software and deduce the underlying strategy.

23.6 Wrapping

Some applications are brittle and poorly understood—they may have been written long ago, have missing documentation, and lack guidance from the original developers. Changes can threaten their viability and risk introducing bugs. Consequently, many organizations limit changes to such applications. They prefer to isolate the code and build a wrapper around it.

A *wrapper* is a collection of interfaces that control access to a system. It consists of a set of boundary classes that provide the interfaces, and it should be designed to follow good OO principles. The boundary classes' methods call the existing system using the existing operations. A boundary method may involve several existing operations and bundle data from several places. Often the calls to legacy code are messy, but the boundary classes hide the details from the outside. The source code can either be OO or non-OO. Wrapping preserves the form of legacy software and accesses its functionality.

Many existing applications have functionality that is confusing, unpredictable, or complex. Often it is possible to extract a core of functionality that is more fundamental, simpler to use, and better tested. In this case, a wrapper provides a clean interface for exposing the core functionality. Some features of the original application are lost, but they are usually the most dubious ones.

If you are adding new functionality, it can usually be added as a separate package. Design this package using good OO principles. Try to minimize interactions with the existing system, and keep them as uniform as possible.

For an example consider a Web application. The original code may be a legacy banking application written in Cobol and running on old hardware. Wrapping can expose the Cobol logic as OO methods that can then be attached to a modern Web interface. The legacy code still must be maintained, but its maintenance is little affected by the existence of the Web interface. The actual code that executes for the Web application is ugly, but it works as long as maintenance on the underlying Cobol logic does not disrupt the wrapper's methods.

Wrapping is usually just a temporary solution, because a wrapper is heavily constrained by the organization (often accidental) of the legacy software. Eventually the combination of

the original old code, the wrapper, and new code in a different format become so unwieldy that it must be rewritten.

Sneed suggests the use of XML as a gateway for communication between the legacy software on the inside and the modern world on the outside. Programmers with different levels of sophistication and modernity can interact via the intermediary of XML [Sneed-01]. He also observes that wrapping serves both a technological and a social purpose. The maintainers of the old code do not have their artifacts disrupted with wrapping. Wrapping lets program maintainers keep their mental models of the software intact. This helps with the day-to-day maintenance of the wrapped code.

23.7 Maintenance

Much of the software literature treats maintenance as being monolithic, but we think the viewpoint of Rajlich and Bennett is more perceptive. According to [Rajlich-00], software moves through five stages.

- **Initial development.** Developers create the software.
- **Evolution.** The software undergoes major changes in functionality and architecture. Refactoring can be used to maintain software quality.
- **Servicing.** The available technical talent has been reduced, either by circumstances or by deliberate decision. Software changes are limited to minor fixes and simple functionality changes. At this stage the software begins its inexorable slide to obsolescence. Wrapping becomes an appropriate technology during this stage.
- **Phaseout.** The vendor continues to receive revenue from the product but is now planning its demise.
- **Closedown.** The product is removed from the market, and customers are redirected to other software.

These five stages do not have a rigid wall between them, but there is a continual decline in the technical quality of the software as it proceeds through its lifetime. Also the authors note that individual versions of software can experience these life stages, only to be replaced by successor versions.

The software engineering goal is to slow the decline and keep software in the evolution and servicing stages as long as possible. In any case, management wants to avoid an accidental slippage of the software and only let the transitions proceed with forethought.

23.8 Chapter Summary

Most development does not involve new applications but rather evolves existing ones. Accordingly, as a software engineer you must be able to evolve existing applications and integrate with them. You can salvage requirements, ideas, data, and code from existing

applications. Reverse engineering is a critical technology when dealing with legacy applications.

The purpose of reverse engineering is to salvage information from old systems and carry it forward. Reverse engineering is *not* intended to perpetuate past flaws—you discard any flaws that you find. Reverse engineering provides merely one source of requirements for new applications, but it is an important source. There can be a variety of inputs to reverse engineering, all of which you should be prepared to exploit. The primary outputs from reverse engineering are models.

Begin by building the class model, emphasizing classes, associations, and generalizations. We suggest building the class model using three distinct phases—implementation recovery, design recovery, and analysis recovery—that involve increasing amounts of decisions and interpretation about the software.

Next build the interaction model, being sure to tie your understanding of behavior to the class model. You can start with procedural code and use slicing to extract portions of the logic that are centered about objects. Hence slicing provides a means for taking the content of procedural code and restructuring it as OO code. Ultimately you express the interaction model as a collection of activity and sequence diagrams.

Finally, if you need it, build the state model. The sequence diagrams provide a helpful intermediary to state diagrams.

Reverse engineering is much different than forward engineering and consequently requires a different mindset. We provided a number of tips to help with the changed mindset.

Wrapping is another technique for dealing with legacy applications. You can regard the legacy application as a black box and build interfaces around it. New applications then access the legacy logic via the intermediary of the wrapper.

forward engineering maintenance	re-engineering reverse engineering	slice wrapper
------------------------------------	---------------------------------------	------------------

Figure 23.1 Key concepts for Chapter 23

Bibliographic Notes

[Bachman-89] explains that most information systems build on past work and only the occasional project truly involves new work. We note that this claim applies not only to information systems but also to software in general.

[Chikofsky-90] has been influential in standardizing reverse engineering terminology. [Kollmann-01] explains how to recover a class model from source code and test executions. The test executions often do not prove hypotheses about the class model, but they can help understanding and are suggestive. [Sneed-96] presents a basic approach to taking a procedural COBOL program and converting it to an OO representation.

References

- [Bachman-89] Charles W. Bachman. A personal chronicle: Creating better information systems, with some guiding principles. *IEEE Transactions on Knowledge and Data Engineering* 1, 1 (March 1989), 17–32.
- [Chikofsky-90] Elliot J. Chikofsky and James H. Cross II. Reverse engineering and design recovery: A taxonomy. *IEEE Software*, January 1990, 13–17.
- [Kollmann-01] Ralf Kollmann and Martin Gogolla. Application of UML associations and their adornments in design recovery. *IEEE Eighth Working Conference on Reverse Engineering*, October 2001, Stuttgart, Germany, 81–90.
- [Rajlich-00] Vaclav T. Rajlich and Keith H. Bennett. A staged model for the software life cycle. *IEEE Computer*, July 2000, 66–71.
- [Rekoff-85] MG Rekoff, Jr. On Reverse Engineering. *IEEE Transactions on Systems, Man, and Cybernetics SMC-15*, 2 (March/April 1985), 244–252.
- [Sneed-96] Harry M. Sneed. Object-oriented COBOL recycling. *IEEE Third Working Conference on Reverse Engineering*, November 1996, Monterey, CA, 169–178.
- [Sneed-01] Harry M. Sneed. Wrapping legacy COBOL programs behind an XML interface. *IEEE Eighth Working Conference on Reverse Engineering*, October 2001, Stuttgart, Germany, 189–197.
- [Weiser-82] M. Weiser. Programmers use slices when debugging. *Communications of the ACM* 25, 7 (July 1982), 446–452.
- [Weiser-84] M. Weiser. Program slicing. *IEEE Transactions on Software Engineering* 10, 4 (July 1984), 352–357.

Appendix A

UML Graphical Notation

The inside covers of the book summarize the graphical notations for the class, state, and interaction models. You can use these four pages as a quick reference while constructing or reading diagrams. However, we must caution you that a novice cannot simply take these four pages and understand them. To understand the concepts represented by the notation, refer to the chapters of Part 1. To learn how to apply the notation and concepts within the software development life cycle, consult the chapters in Part 2 and 3. The index can also help you find relevant material in the book.

With the exception of the label for each construct and a few descriptive comments, all of the diagram elements, text names, and punctuation symbols shown are part of the notation. The names in these diagrams (such as *Class*, *attribute1*, *operation*, and *event2*) indicate what kind of element they are examples of. You may wish to modify the syntax of names and the declarations of attributes and signatures to make them consistent with the syntax of your implementation language.

Most of the items shown are optional, especially during early stages of modeling. Even in design, it is unwise to overspecify by including superfluous names and notations. For example, when an association is labeled by end names, it is usually not necessary to give the association itself a name. We have not indicated which elements are optional, because we wanted to show only the actual UML notation wherever possible, without obscuring it with an additional metanotation.

Please feel free to copy the notation summaries on the inside covers. You can obtain an electronic copy at www.modelsoftcorp.com.

Appendix B

Glossary

The following terms are used in OO modeling for analysis, design, and implementation.

abstract class a class that has no direct instances. The UML notation is to italicize an abstract class name or place the keyword *{abstract}* below or after the name. (Contrast with *concrete class*.)

abstract operation an operation that lacks an implementation. A concrete descendant class must provide a method to implement the operation. The UML notation is to italicize an abstract operation name or place the keyword *{abstract}* after its name.

abstraction the ability to focus on essential aspects of an application while ignoring details.

access modifier (in Java) the means of controlling access to methods and data via *public*, *private*, *protected*, and *package* visibility.

access specifier (in C++) the means of controlling access to methods and data via *public*, *private*, and *protected* visibility as well as a *friend* declaration.

activation the period of time for an object's execution. The UML notation is a thin rectangle. (Synonymous with *focus of control*.)

active object an object that has its own thread of control. (Contrast with *passive object*.)

activity a specification of executable behavior.

activity diagram a diagram that shows the sequence of steps that make up a complex process.

activity token a token that can be placed on an activity symbol to show the progress of an execution.

actor a direct external user of a system. The UML notation is a "stick man" icon.

aggregation a kind of association in which a whole, the assembly, is composed of parts. Aggregation is often called the "a-part-of" or "parts-explosion" relationship and may be

nested to an arbitrary number of levels. Aggregation bears the transitivity and antisymmetry properties. The UML notation is a small *hollow* diamond superimposed on the association end next to the assembly class. (Contrast with *composition*.)

analysis the development stage in which a real-world problem is examined to understand its requirements without planning the implementation.

ancestor class a class that is a direct or indirect superclass of a given class. (Contrast with *descendant class*.)

API (acronym) *application programming interface*.

application analysis the second substage of analysis that addresses the computer aspects of the application that are visible to users.

application programming interface a collection of methods that provide the functionality of an application.

architecture the high-level plan or strategy for solving an application problem.

assembly (for an aggregation) a class of objects that is composed of part objects.

association a description of a group of links with common structure and common semantics. The UML notation is a line between classes that may consist of several line segments.

association class an association that is also a class. Like the links of an association, the instances of an association class derive identity from instances of the related classes. Like a class, an association class can have attributes, operations, and participate in associations. The UML notation is a box (a class box) attached to the association by a dashed line.

association end an end of an association. A binary association has two ends, a ternary has three ends, and so forth.

attribute a named property of a class that describes a value held by each object of the class. The UML notation lists attributes in the second compartment of the class box.

automatic transition an unlabeled transition that automatically fires when the activity associated with the source state is completed.

bag an unordered collection of elements with duplicates allowed. The UML notation is to annotate an association end with *{bag}*.

base class (in C++) a superclass.

batch transformation (architectural style) a sequential input-to-output transformation, in which inputs are supplied at the start and the goal is to compute an answer. There is no ongoing interaction with the outside world. (Contrast with *continuous transformation*.)

boundary class a class of objects that provide a staging area for communications between a system and an external source.

call-by-reference (in a programming language) a mechanism that passes arguments to a method by passing the address of each argument. (Contrast with *call-by-value*.)

call-by-value (in a programming language) a mechanism that passes arguments to a method by passing a copy of the data values. If an argument is modified, the new value will not take effect outside of the method that modifies it. (Contrast with *call-by-reference*.)

candidate key (in a relational database) a combination of columns that uniquely identifies each row in a table. The combination must be minimal and include only those columns that are needed for unique identification. No column in a candidate key can be null.

cardinality the count of elements that are in a collection. (Contrast with *multiplicity*.)

change event an event that is caused by the satisfaction of a boolean expression. The intent of a change event is that the expression is continually tested—whenever the expression changes from false to true the event happens. The UML notation is the keyword *when* followed by a parenthesized boolean expression. (Contrast with *guard condition*.)

changeability an indication whether a property (such as an association end) can be modified after the initial value is created. The possibilities are *changeable* (can be updated) and *readonly* (can only be initialized).

class a description of a group of objects with similar properties (attributes), common behavior (operations and state diagrams), similar relationships to other objects, and common semantics. The UML notation is a box with the name in the top compartment.

class design the development stage for expanding and optimizing the analysis models so that they are amenable to implementation.

class diagram a graphic representation that describes classes and their relationships, thereby describing possible objects. (Contrast with *object diagram*.)

class model a description of the structure of the objects in a system including their identity, relationships to other objects, attributes, and operations.

classification a grouping of objects with the same data structure and behavior.

client a subsystem that requests services from another subsystem. (Contrast with *server*.)

coherence a property of an element, such as a class, an operation, or a package, such that it is organized on a consistent plan and all its parts fit together toward a common goal.

completion transition a transition that automatically fires when the activity associated with the source state is completed.

composite state a state that provides shared behavior for nested states. (Contrast with *nested state*.)

composition a form of aggregation with two additional constraints. A part can belong to at most one assembly. Furthermore, once a part has been assigned an assembly, it has a coincident lifetime with the assembly. The UML notation is a small *solid* diamond superimposed on the association end next to the assembly class. (Contrast with *aggregation*.)

concrete class a class that can have direct instances. (Contrast with *abstract class*.)

concurrent two or more activities or events whose execution may overlap in time.

condition (see *guard condition*).

constraint a boolean condition involving model elements such as objects, classes, attributes, associations, and generalization sets. The UML notation for simple constraints is a text string enclosed in braces or placed in a “dog-eared” comment box. For complex constraints, you can use the Object Constraint Language.

constructor (in C++ and Java) an operation that initializes a newly created instance of a class. (Contrast with *destructor*.)

container class a class of *container objects*. Examples include sets, arrays, dictionaries, and associations.

container object an object that stores a collection of other objects and provides various operations to access or iterate over its contents.

continuous transformation (architectural style) a system in which the outputs actively depend on changing inputs and must be periodically updated. (Contrast with *batch transformation*.)

control the aspect of a system that describes the sequences of operations that occur in response to stimuli.

controller an active object that manages control within an application.

database a permanent, self-descriptive store of data that is contained in one or more files. Self-description is what sets a database apart from ordinary files.

database management system the software for managing access to a database.

data dictionary the definition of all modeling elements (classes, associations, attributes, operations, and enumeration values) and an explanation of the rationale for key modeling decisions.

DBMS (acronym) *database management system*.

default value the value used to initialize an attribute or method argument.

delegation an implementation mechanism in which an object, responding to an operation on itself, forwards the operation to another object.

denormalization the violation of normal forms. Developers should violate normal forms only for good cause, such as to increase performance for a bottleneck. (See *normal form*.)

derived class (in C++) a subclass.

derived element (in UML) an element that is defined in terms of other elements. Classes, attributes, and associations can all be derived. Do not confuse the UML term *derived* with the C++ *derived class*. A C++ derived class refers to the subclass of a generalization and has nothing to do with UML’s meaning of derived element. The UML notation is a slash preceding the element name.

descendant class a class that is a direct or indirect subclass of a given class. (Contrast with *ancestor class*.)

destructor (in C++) an operation that cleans up an existing instance of a class that is no longer needed. (Contrast with *constructor*.)

development the construction of software.

development life cycle an approach for managing the process of building software.

development stage a step in the process of building software. This book covers the following sequence of development stages: system conception, domain analysis, application analysis, system design, class design, implementation modeling, and implementation. Even though the development stages are ordered, all portions of an application need not proceed in tandem. We do not mean to imply waterfall development.

dictionary an unordered collection of object pairs with duplicates allowed. Each pair binds a key to an element. You can then use the key to look up the element.

direction whether an argument to an operation/method is an input (*in*), output (*out*), or an input argument that can be modified (*inout*).

do-activity an activity that continues for an extended time. The UML notation is “*do /*” followed by the do-activity name.

domain analysis the first substage of analysis that focuses on modeling real-world things that carry the semantics of an application.

dynamic binding a form of method resolution that associates a method with an operation at run time, depending on the class of one or more target objects.

dynamic simulation (architectural style) a system that models or tracks objects in the real world.

effect a reference to a behavior that is executed in response to an event. The UML notation for an effect is a slash (“/”) followed by the activity name.

encapsulation the separation of external specification from internal implementation. (Synonymous with *information hiding*.)

enterprise model a model that describes an entire organization or some major aspect of an organization.

Entity-Relationship (ER) model a graphical approach to modeling originated by Peter Chen that shows entities and the relationships between them. The UML class model is based on the ER model.

entry activity an activity that is executed upon entry to a state. The UML notation is to list an entry activity within a state preceded by “*entry /*”. (Contrast with *exit* activity.)

enumeration a data type that has a finite set of values. The UML notation is the keyword «*enumeration*» above the enumeration name in the top section of a box. The second section lists the enumeration values.

ER (acronym) *Entity-Relationship model*.

event an occurrence at a point in time. (Contrast with *state*.)

- event-driven control** an approach in which control resides within a dispatcher or monitor that the language, subsystem, or operating system provides. Developers attach application methods to events, and the dispatcher calls the methods when the corresponding events occur (“callback”). (Contrast with *procedure-driven control*.)
- exit activity** an activity that is executed just before exit from a state. The UML notation is to list an exit activity within a state preceded by “*exit /*”. (Contrast with *entry* activity.)
- extend** (use case relationship) a relationship that adds incremental behavior to a use case. Note that the extension adds itself to the base; in contrast, for an *include* relationship the base explicitly incorporates the inclusion. The UML notation is a dashed arrow from the extension use case to the base use case. The keyword «*extend*» annotates the arrow. (Contrast with *include*.)
- extensibility** a property of software such that new kinds of objects or functionality can be added to it with little or no modification to existing code.
- extent** (of a class) the set of objects for a class.
- feature** an attribute or an operation.
- final** (for a Java class) a directive that prevents further subclassing.
- final** (for a Java method) a directive that prevents the method from being overridden.
- fire** to cause a transition to occur.
- focus of control** the period of time for an object’s execution. The UML notation is a thin rectangle. (Synonymous with *activation*.)
- foreign key** (in a relational database) a reference to a candidate key (normally a reference to a primary key). It is the glue that binds tables.
- forward engineering** the building of an application from general requirements through to an eventual implementation. (Contrast with *reverse engineering*.)
- fourth-generation language** a framework for straightforward database applications that provides screen layout, simple calculations, and reports.
- framework** a skeletal structure of a program that must be elaborated to build a complete application.
- friend** (in C++) a declaration that permits selective access to members. The class containing the *friend* declaration grants access to a named function, method, or class.
- garbage collection** (in a programming language) a mechanism for automatically deallocating data structures that can no longer be accessed and are therefore not needed.
- generalization** an organization of elements (such as classes, signals, or use cases) by their similarities and differences. The UML notation is a triangle with the apex next to the superelement. (Contrast with *specialization*.)
- generalization set name** an enumerated attribute that indicates which aspect of an object is being abstracted by a particular generalization.

- guard condition** a boolean expression that must be true in order for a transition to occur. A guard condition is checked only once, at the time the event occurs, and the transition fires if the condition is true. The UML notation is to list a guard condition in square brackets after an event.
- identifier** one or more attributes in an implementation that unambiguously differentiate an object from all others.
- identity** the inherent property of an object which distinguishes each object from all others.
- implementation** the development stage for translating a design into programming code and database structures.
- implementation inheritance** an abuse of inheritance that seeks to reuse existing code, but does so with an illogical application structure that can compromise future maintenance.
- implementation method** (style) a method that implements specific computations on fully specified arguments, but does not make context-dependent decisions. (Contrast with *policy method*.)
- implementation modeling** the development stage for adding fine details to a model that transcend languages. Implementation modeling is the immediate precursor to the actual implementation.
- include** (use case relationship) a relationship that incorporates one use case within the behavior sequence of another use case. The UML notation is a dashed arrow from the source (including) use case to the target (included) use case. The keyword «include» annotates the arrow. (Contrast with *exclude*.)
- index** a data structure that maps one or more attribute values into the objects or database table rows that hold the values. Indexes are used for optimization (to quickly locate objects and table rows) and to enforce uniqueness.
- information hiding** (see *encapsulation*)
- inheritance** the mechanism that implements the generalization relationship.
- integration testing** testing of code from multiple developers to determine how the classes and methods fit together. (Contrast with *unit testing* and *system testing*.)
- interaction model** the model that describes how objects collaborate to achieve results. It is a holistic view of behavior across many objects, whereas the state model is a reductionist view of behavior that examines each object individually.
- interactive interface** (architectural style) a system that is dominated by interactions between the system and agents, such as humans, devices, or other programs.
- interface** (in Java) an uninstantiable class specification the contains only constants and method declarations.
- iterative development** the development of a system by a process broken into a series of steps, or iterations, each of which provides a better approximation to the desired system than the previous iteration. (Contrast with *rapid prototyping* and *waterfall development*.)

- iterator** (in a programming language) a construct that controls iteration over a range of values or a collection of objects.
- layer** a subsystem that provides multiple services, all of which are at the same level of abstraction. A layer can be built on subsystems at a lower level of abstraction. (Contrast with *partition*.)
- leaf class** a class with no subclasses. It must be a concrete class. In Java, this is the same as a final class.
- library** a collection of classes that are reusable across applications.
- life cycle** (see *development life cycle*).
- lifeline** the period of time during which an object exists.
- link** a physical or conceptual connection among objects. A link is an instance of an association. The UML notation is a line between objects that may consist of several line segments.
- lock** a logical object associated with some defined subset of a resource that gives the lock holder the right to access the resource directly.
- member** (in C++) data or methods of a class.
- metaclass** a class describing other classes.
- metadata** data that describes other data.
- method** the implementation of an operation for a class. The UML notation lists methods in the third compartment of the class box. (Contrast with *operation*.)
- method caching** (in a programming language) an optimization of method searching in which the address of a method is found the first time an operation is applied to an object of a class and then stored in a table attached to the class.
- method resolution** (in a programming language) the process of matching an operation on an object to the method appropriate to the object's class.
- methodology** (in software engineering) a process for the organized production of software using a collection of predefined techniques and notational conventions.
- model** an abstraction of some aspect of a problem. We express models with various kinds of diagrams.
- modularity** the organization of a system into groups of closely related objects.
- multiple inheritance** a type of inheritance that permits a class to have more than one superclass and to inherit features from all ancestors. (Contrast with *single inheritance*.)
- multiplicity** (of an association end) the number of instances of one class that may relate to a single instance of an associated class. Multiplicity is a constraint on the size of a collection. The UML notation is a numeric interval or the special symbol "*" denoting "many" (zero or more). (Contrast with *cardinality*.)

multiplicity (of an attribute) the possible number of values for each instantiation of an attribute. The most common specifications are a mandatory single value [1], an optional single value [0..1], and many [*].

namespace (in C++) a means for providing a semantic scope for symbols to alleviate name conflicts.

***n*-ary association** an association involving three or more association ends. The UML symbol is a diamond with lines connecting to the related classes. If the association has a name, it is written in italics next to the diamond.

navigability the direction of traversal of a binary association in an implementation. The possibilities are none, either direction, or both directions. The UML shows navigability with an arrowhead on the association end attached to the target class.

navigation a traversal of associations and generalizations in a class model to go from source objects to target objects.

nested state a state that shares behavior from its composite state and adds additional behavior of its own. (Contrast with *composite state*.)

new (in C++ and Java) the operator to create objects.

normal form (in a relational database) a guideline for relational database design that increases data consistency.

***n*-tier architecture** an extension of the three-tier architecture, permitting any number of application layers. (Contrast with *three-tier architecture*.)

null a special value denoting that an attribute value is unknown or not applicable.

object a concept, abstraction, or thing that can be individually identified and has meaning for an application. An object is an instance of a class.

Object Constraint Language (OCL) a language for defining constraints that is part of the UML. You can also use the OCL to navigate class models.

object diagram a graphical representation that shows individual objects and their relationships. (Contrast with *class diagram*.)

object identity (in a relational database) the use of an artificial number to identify each record in a table. (Contrast with *value-based identity*.)

Object Management Group (OMG) a standards forum that is the owner of the UML.

object-orientation (OO) a strategy for organizing systems as collections of interacting objects that combine data and behavior.

OCL (acronym) *Object Constraint Language*.

OMG (acronym) *Object Management Group*.

OO (acronym) *object-oriented*.

OO database a database that is perceived as objects that mix data and behavior. (Contrast with relational database.)

- OO-DBMS** a DBMS that provides persistent objects in addition to the transient objects provided by OO programming languages. (Contrast with relational DBMS.)
- OO development** a software development technique that uses objects as a basis for the construction of software.
- OO programming language** a language that supports objects (combining identity, data, and operations), method resolution, and inheritance.
- operation** a function or procedure that may be applied to or by objects in a class. (Contrast with *method*.)
- ordered** a sorted collection of elements with no duplicates allowed. The UML notation is to annotate an association end with *{ordered}*. (Contrast with *sequence*.)
- origin class** the topmost class in an inheritance hierarchy that defines a feature.
- overloading** (in a programming language) binding the same name to multiple methods whose signatures differ in number or types of arguments. A call to an overloaded operation is resolved at compile time based on the types of the calling arguments.
- override** to define a method for an operation that replaces an inherited method for the same operation.
- package** (class modeling construct) a group of elements (classes, associations, generalizations, and lesser packages) with a common theme. The UML notation is a box with a tab; the package name is placed in the box.
- package** (referring to visibility) accessible by methods of classes in the same package as the containing class.
- partition** a subsystem that provides a particular kind of service in parallel to other subsystems. A partition may itself be built from lower-level subsystems. (Contrast with *layer*.)
- passive object** an object that does not have its own thread of control. (Contrast with *active object*.)
- pattern** a parameterized excerpt of a model that is important and recurring. It is mathematical in nature and worthy of reuse across multiple applications.
- peer** two or more subsystems that are mutually interdependent for services. (Contrast with *client* and *server*.)
- persistent object** an object that is stored in a database and can span multiple application executions. (Contrast with *transient object*.)
- policy method** (style) a method that makes context-dependent decisions but calls on implementation methods for detailed computations. (Contrast with *implementation method*.)
- polymorphism** takes on many forms; the property that an operation may behave differently on different classes.

primary key (in a relational database) a candidate key that is preferentially used to access the records in a table. A table can have at most one primary key; normally each table should have a primary key.

private (referring to visibility) accessible by methods of the containing class only.

procedure-driven control an approach in which control resides within the program code. Procedures request external input and then wait for it; when input arrives, control resumes within the procedure that made the call. The location of the program counter and the stack of procedure calls and local variables define the system state. (Contrast with *event-driven control*.)

programming-in-the-large the creation of large, complex programs with teams of programmers.

protected (referring to C++ visibility) accessible by methods of the containing class and any of its descendant classes.

protected (referring to Java visibility) accessible by methods of the containing class, any of its descendant classes, and classes in the same package as the containing class.

public (referring to visibility) accessible by methods of any class.

qualified association an association in which one or more attributes (called qualifiers) disambiguate the objects for a “many” association end. The UML notation is a small box on the end of the association line near the source class.

qualifier an attribute that distinguishes among the objects at a “many” association end. The UML notation is to place a qualifier in a small box on the end of the association line near the source class.

race condition a situation in which the order of receiving concurrent signals can affect the final state of an object.

rapid prototyping the quick development of a portion of a system for experimentation and evaluation. Prototyping is proof of concept and often throwaway by intent. (Contrast with *iterative development* and *waterfall development*.)

real-time system (architectural style) an interactive system for which time constraints on actions are particularly tight or in which the slightest timing failure cannot be tolerated.

refactoring changes to the internal structure of software to improve its design without altering its external functionality.

reference an attribute value in one object that refers to another object.

reflection a property of a system such that it can examine its own structure dynamically and reason about its own state.

region a portion of a state diagram.

reification the promotion of something that is not an object into an object.

relational database a database in which the data are perceived as tables. (Contrast with OO database.)

- relational DBMS** a DBMS that manages tables of data and associated structures that increase the functionality and performance of tables. (Contrast with OO-DBMS.)
- responsibility** something that an object knows or something it must do. A responsibility is not a precise concept; it is meant to get the thought process going.
- reverse engineering** the process of examining implementation artifacts and inferring the underlying logical intent. (Contrast with *forward engineering*.)
- robust** a property of software such that it does not fail catastrophically when some of its design assumptions are violated.
- scenario** a sequence of events that occur during one particular execution of a system.
- schema** the structure of the data in a database.
- scope** an indication if a feature applies to an object or a class. An underline distinguishes features with class scope (static) from those with object scope.
- sequence** a sorted collection of elements with duplicates allowed. The UML notation is to annotate an association end with *{sequence}*. (Contrast with *ordered*.)
- sequence diagram** a diagram that shows the participants in an interaction and the sequence of messages among them.
- server** a subsystem that provides a service to other subsystems. (Contrast with *client*.)
- service** a group of related functions or operations that share some common purpose.
- shopping-list operation** an operation that is meaningful in its own right. Bertrand Meyer coined the term *shopping list* because discovery of such an operation is driven by the intrinsic meaning of a class and not by the needs of a particular application. Sometimes the real-world behavior of classes suggests operations.
- signal** an explicit one-way transmission of information from one object to another. The UML notation is the keyword «*signal*» above the signal class name in the top section of a box. The second section lists the signal attributes.
- signal event** the event of sending or receiving a signal.
- signature** the number and types of the arguments for an operation and the type of its result.
- single inheritance** a type of inheritance in which a class may have only a single superclass. (Contrast with *multiple inheritance*.)
- software engineering** a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software.
- specialization** the refinement of a class into variants. Specialization has the same meaning as generalization but takes a top-down perspective. In contrast, generalization takes a bottom-up perspective. (Contrast with *generalization*.)
- SQL** the standard language for interacting with a relational database.
- state** an abstraction of the values and links of an object. The UML notation is a rounded box containing an optional state name. (Contrast with *event*.)

state diagram a graph whose nodes are states and whose directed arcs are transitions between states.

state model a description of those aspects of a system concerned with time and the sequencing of operations. The state model consists of multiple state diagrams, one state diagram for each class with important temporal behavior.

static (in C++ and Java) data and methods that belong not to an instance of a class, but to the class itself.

stored procedure (in a relational database) a method that is stored in a database.

strong typing (in a programming language) a requirement that the type of each variable must be declared. (Contrast with *weak typing*.)

subclass a class that adds specific attributes, operations, state diagrams, and associations for a generalization. (Contrast with *superclass*.)

submachine a state diagram that may be invoked as part of another state diagram. The UML notation for invoking a submachine is to list a local state name followed by a colon and the submachine name.

substate a state that expresses an aspect of concurrent behavior for a parent state.

subsystem a major piece of a system that is organized around some coherent theme. A system may be divided into subsystems using either *partitions* or *layers*.

superclass the class that holds common attributes, operations, state diagrams, and associations for a generalization. (Contrast with *subclass*.)

swimlane a column in an activity diagram that shows the person or organization who performs activities; a partition.

system an application that is the subject of interest.

system architecture (see *architecture*).

system boundary the outline of the scope of a system determining what the system includes and what the system omits.

system conception the development stage that deals with the genesis of an application.

system design the development stage during which the developer devises the architecture and establishes general design policies.

system testing the checking of an entire application. (Contrast with *unit testing* and *integration testing*.)

table (in a relational database) an organization of data that has a specific number of columns and an arbitrary number of rows.

ternary association an association among three association ends. The UML symbol is a diamond with lines connecting to the related classes. If the association has a name, it is written in italics next to the diamond.

this (in C++ and Java) the default name of the target object of a method.

thread of control a single path of execution through a program, a state model, or some other representation of control flow.

three-tier architecture an approach that separates data management, application functionality, and the user interface. The data management layer holds the database schema and data. The application layer holds the methods that embody the application logic. The user-interface layer manages the forms and reports that are presented to the user. (Contrast with *n-tier architecture*.)

time event an event caused by the occurrence of an absolute time or the elapse of a time interval. The UML notation for an absolute time is the keyword *when* followed by a parenthesized expression involving time. The notation for a time interval is the keyword *after* followed by a parenthesized expression that evaluates to a time duration.

transaction manager (architectural style) a database system whose main function is to store and access information.

transient object an object that exists only in memory and disappears when an application terminates execution. Thus a transient object is an ordinary programming object. (Contrast with *persistent object*.)

transition an instantaneous change from one state to another. The UML notation is a line (with possibly multiple line segments) from the origin state to the target state; an arrow-head points to the target state.

transitive closure (from graph theory) the set of nodes that are reachable by some sequence of edges.

UML (acronym, trademark of the OMG) *Unified Modeling Language*.

Unified Modeling Language (trademark of the OMG) a comprehensive suite of object-oriented models intended to represent software and other kinds of applications fully. The UML has been developed under the auspices of the OMG.

UML1 informal term for the first release of the UML approved in 1997.

UML2 informal term for the second release of the UML approved in 2004. This book is based on UML2.

unit testing testing by developers of their own code for classes and methods. (Contrast with *integration testing* and *system testing*.)

use case a coherent piece of functionality that a system can provide by interacting with actors. The UML notation is an ellipse with the use case name inside.

use case diagram a graphical notation for summarizing actors and use cases.

user interface an object or group of objects that provides the user of a system with a coherent way to access its domain objects, commands, and application options.

value a piece of data. A value is an instance of an attribute.

value-based identity (in a relational database) the use of some combination of real-world attributes to identify each record in a table. (Contrast with *existence-based identity*.)

view (in a relational database) a table that a relational DBMS dynamically computes.

virtual (in C++) an operation that can be overridden by a descendant class.

visibility the ability of a method to reference a feature from another class. The UML denotes visibility with the following prefixes. The possibilities are *public* (“+”), *protected* (“#”), *private* (“-”), and *package* (“~”).

waterfall development the development life cycle of performing the software development stages in a rigid linear sequence with no backtracking. (Contrast with *iterative development* and *rapid prototyping*.)

weak typing (in a programming language) the lack of a requirement that the type of each variable be declared. (Contrast with *strong typing*.)

wrapper a collection of interfaces that allow access into a system.